System Design, Step by Step

A free, diagram-first handbook. Every design starts simple, shows where it breaks, and fixes only that.

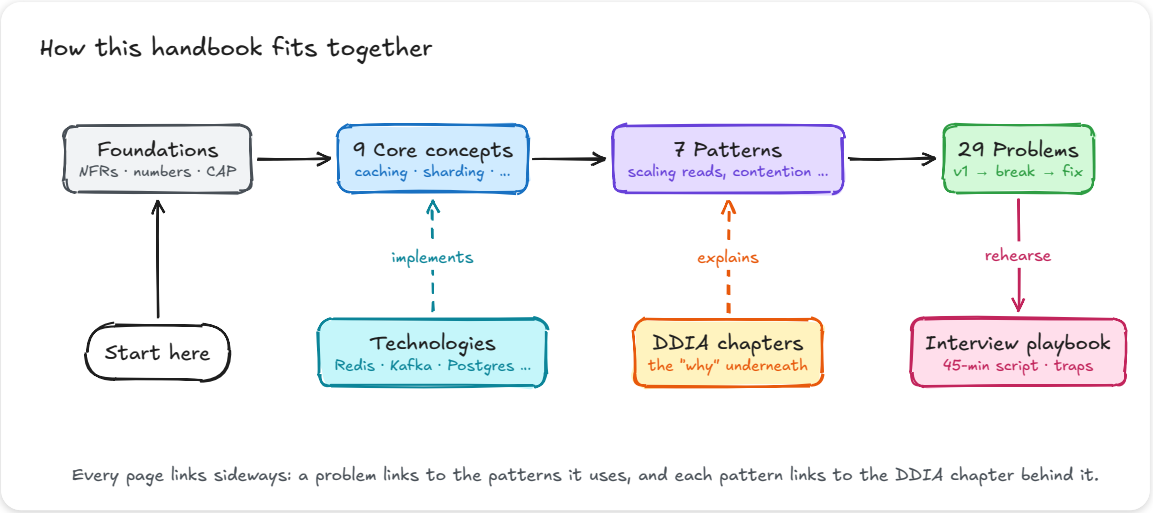
- Foundations
- Core concepts
- Patterns
- Technologies
- Problems
- DDIA
- Low-level design
- Interview playbook
- Slide decks

Content licensed CC BY-SA 4.0. Diagrams use Excalifont (SIL OFL 1.1).

System Design, Step by Step

A free, diagram-first handbook for system design interviews and for building real systems.

Most resources show you a finished architecture and expect you to memorize it. This one shows **how you get there**: start with the simplest design that works, calculate where it breaks, fix only that, and repeat. Every page has hand-drawn diagrams you can open and edit in Excalidraw.



What's inside

Section	What you get
Foundations	The method, non-functional requirements with SLOs, and the numbers to know
Core concepts	Networking, API design, data modeling, indexing, caching, sharding, consistent hashing, CAP
7 patterns	Scaling reads and writes, real-time updates, contention, multi-step processes, large blobs, long-running tasks
Technologies	Postgres, Redis, Kafka, Cassandra, DynamoDB, Elasticsearch, Flink, ZooKeeper, API gateways: when and when not
29 problems	URL shortener to payments, each from v1 to final design with an interview script

Section	What you get
DDIA, chapter by chapter	All 12 chapters of *Designing Data-Intensive Applications* as diagrams, tied to interview problems
Low-level design	File system, movie booking, logger, inventory: class diagrams and tested Java
Interview playbook	The 45-minute framework, what senior answers sound like, quick-fire practice
Slide decks	Three interactive decks for review and out-loud rehearsal

Pick your path

If you want to...	Do this
Learn system design	Foundations → Core concepts → Patterns → Problems. Each problem walks from a naive **v1** to the final design.
Prepare for an interview in two weeks	Read the playbook, then do two problems a day: read the page, then rehearse it with the interview deck.
Understand the theory	Read the DDIA pages alongside the book; every chapter ends with how it shows up in interviews.
Review the night before	Skim the TL;DRs and *Common mistakes* of each problem, then do the quick-fire questions.

Every problem page has the same shape

1. **TL;DR**: three lines you could say out loud.
2. **Requirements**: functional, then non-functional with numbers.
3. **Back-of-envelope numbers**: only the ones that change the design.
4. **Core entities and API**
5. **v1, and where it breaks**
6. **The final design**, step by step, with why each part exists
7. **Deep dives**: the questions interviewers push on
8. **Interview script**: what to do in each part of 45 minutes

9. **Common mistakes** and **related** pages

Formats

- **Website**: searchable, with dark mode and mobile layout.
- **GitHub**: every page is plain Markdown that renders in the repository.
- **Slides**: three interactive decks that work offline.
- **PDF**: the whole handbook in one file for printing or reading offline.
- **Editable diagrams**: every diagram has an `.excalidraw` source next to its `.svg`.

Contributing

Fixes and new pages are welcome. See CONTRIBUTING for the page template and how to edit diagrams.

Credits

Written in our own words, with our own diagrams. The DDIA pages are study notes on *Designing Data-Intensive Applications* by Martin Kleppmann; please read the book. Diagrams use the open-source Excalifont (SIL OFL 1.1). Problem names refer to well-known products only to describe the kind of system; this project is not affiliated with any company or course.

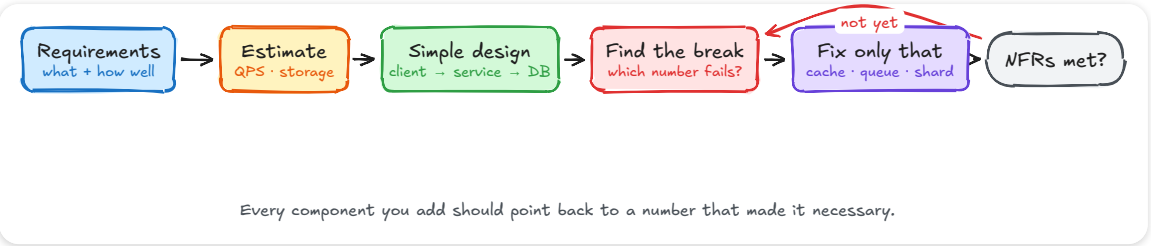
Foundations

Read these first. They give you the method that every other page uses.

1. How to approach any design: the loop of simple design → find the break → fix only that.
2. Non-functional requirements: the qualities that drive architecture, with SLOs and error budgets.
3. Numbers and estimation: what hardware can do, and how to estimate quickly.

How to approach any design

Good designs aren't memorized. They are **derived**: you start from what the system must do, put numbers on how well it must do it, and add complexity only when a number forces you to.



The loop

1. **Requirements.** Write down what users can do (functional) and how well it must work (non-functional: latency, availability, consistency, scale, durability). See NFRs.
2. **Estimate.** Turn the NFRs into a few numbers: requests per second, storage, fan-out, bandwidth. See Numbers and estimation.
3. **Simple design.** Client → service → one database. It is almost always the right starting point, and often more capable than people assume.
4. **Find the break.** Compare your numbers with what each component can handle. The first limit you exceed is the bottleneck.
5. **Fix only that.** Apply the pattern that removes that specific bottleneck: a cache for reads, a queue for spiky writes, a conditional update for contention.
6. **Repeat** until every NFR is met. Then stop.

Why this works in interviews

- It **shows reasoning**, which is what interviewers grade, rather than a recited architecture.
- It **keeps you out of trouble**: you never add Kafka, sharding or microservices without a reason you can state.
- It gives you **natural deep dives**: each "break" is a place the interviewer can push, and you've already thought about it.

> **Tip**
>
> **Say this in the interview:** "Let me start with the simplest design that meets the functional requirements, then use the numbers to find where it breaks and fix those points one at a time."

A worked mini-example

"Design a URL shortener for 100M daily users."

Step	What you say
Requirements	Create short links; redirect fast (p99 < 100 ms); links never collide.
Estimate	1B redirects/day ≈ 12k/s average, ~100k/s peak. Writes ≈ 12/s. 1B links × 500 B = 500 GB.
Simple design	API service + Postgres. 500 GB and 12 writes/s fit one database easily.
Find the break	100k reads/s on one Postgres is too much.
Fix only that	Redis cache in front; popular links dominate traffic, so most reads never reach Postgres.
Repeat	Codes must be unique: counter + base62. Done. No sharding needed.

The full version is the URL shortener page.

Where to go next

- Non-functional requirements: the numbers that drive every decision.
- Numbers and estimation: what modern hardware can actually do.
- The 7 patterns: the fixes you'll apply in step 5.

Non-functional requirements

Functional requirements say **what** the system does. Non-functional requirements (NFRs) say **how well**, and they decide the architecture. Two systems with identical features but different NFRs look nothing alike.

> **Tip**
>
> **Say this in the interview:** "Before designing, let me pin down the NFRs with numbers, because they drive every decision that follows."

The NFRs that matter, and how to quantify them

NFR	Question to ask	Say it as a number	What it changes in the design
Scale	How many users? Peak vs average?	"100M DAU, 50k reads/s at peak"	Caching, replicas, sharding
Latency	Which path must be fast?	"Feed p99 < 200 ms"	Caches, precomputation, CDN
Availability	What can be down, and for how long?	"99.99% for reads (≈ 4 min/month)"	Redundancy, failover, multi-AZ
Consistency	Where is stale data unacceptable?	"Booking: never oversell; browse: ≤ 5 s stale"	Transactions, read-from-primary, CP vs AP
Durability	Can we ever lose data?	"Payments: zero loss after ack"	Sync replication, write-ahead logs, backups
Throughput	Sustained work per second?	"10k events/s ingest"	Queues, batching, partitions
Read:write ratio	Which dominates?	"100:1"	Read path vs write path optimizations
Data size / growth	How much, how fast?	"500 GB now, +1 TB/year"	Storage engine, partitioning, tiering

NFR	Question to ask	Say it as a number	What it changes in the design
Freshness	How quickly must changes appear?	"New posts visible within 1 min"	Sync vs async pipelines
Security / privacy	Who can see what? PII?	"Card data never leaves the vault"	Tokenization, isolation, encryption
Cost	Any hard constraints?	"GPU capacity is fixed"	Queues, priorities, admission control

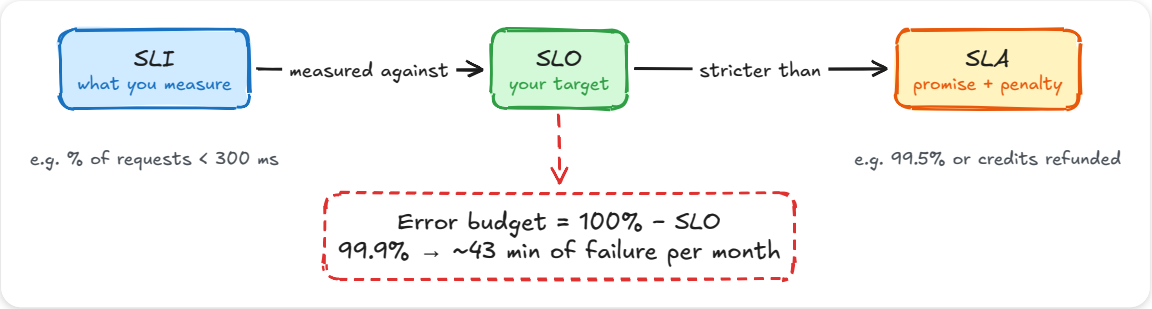
You rarely need all of them. Pick the **three to five** that matter for this problem and quantify those.

Latency: use percentiles, not averages

An average hides the slow requests users actually notice. Use **p50** (typical user), **p99** (one request in a hundred) and sometimes **p99.9**.

Tail latency compounds: if one page calls 10 services in parallel and each is slow 1% of the time, about **10%** of page loads hit at least one slow call ($1 - 0.99^{10} \approx 0.096$). Set targets on p99 and use timeouts.

SLIs, SLOs, SLAs and error budgets



Term	Meaning	Example
SLI (indicator)	What you measure	Share of requests served in < 300 ms
SLO (objective)	Your internal target for the SLI	99.9% of requests < 300 ms over 30 days

Term	Meaning	Example
SLA (agreement)	External promise, with penalties	99.5%, or customers get credits
Error budget	100% − SLO: how much failure you can "spend"	0.1% of a month ≈ 43 minutes

The error budget turns reliability into a decision rule: while budget remains, ship features; when it's spent, focus on reliability.

Availability in minutes

Availability	Downtime per month	Per year
99%	~7.3 hours	~3.65 days
99.9%	~43 minutes	~8.8 hours
99.99%	~4.3 minutes	~53 minutes
99.999%	~26 seconds	~5 minutes

Two components in series multiply: 99.9% × 99.9% ≈ 99.8%. Redundant components in parallel help: two independent 99% replicas give ≈ 99.99%.

Consistency is a per-feature choice

Don't declare a whole system "strongly consistent" or "eventually consistent". Decide feature by feature:

- **Strong (CP):** money, inventory, seat holds, unique usernames.
- **Eventual (AP):** feeds, search results, like counts, reviews, recommendations.

See CAP theorem and PACELC.

Common mistakes

- Listing NFRs without numbers ("it should be fast and scalable").
- Using averages instead of percentiles for latency.

- Asking for "five nines" everywhere. Each extra nine costs a lot; choose it where it matters.
- Forgetting that consistency can differ between the read path and the write path.

Numbers and estimation

Estimates exist for one reason: to decide whether you need a given component. Calculate only what changes a decision, round aggressively, and say the number out loud.

Conversions you'll use every time

Fact	Use it as
1 day ≈ 86,400 s ≈ 10^5 s	1M requests/day ≈ 12/s; 1B/day ≈ 12k/s
Peak ≈ 2–5 × average	Design for peak, quote both
1 KB × 1M = 1 GB; 1 KB × 1B = 1 TB	Storage per year: daily items × size × 365
1 Gbps ≈ 125 MB/s	Bandwidth: requests/s × response size

What one machine or service can handle

These are planning numbers for well-configured systems on modern hardware. Real numbers depend on payload size and query shape, so use them to find orders of magnitude, not exact limits.

Component	Rough capacity
Modern server	64–128 cores, 256 GB–1 TB RAM, 10–25 Gbps network
Postgres / MySQL primary	~10–50k simple transactions/s; multiple TB of data
Redis (one node)	~100k+ simple operations/s, sub-millisecond; data must fit in RAM
Kafka	~5–10k messages/s per partition is a comfortable plan; ~1M messages/s per broker
Cassandra node	~10k+ writes/s

Component	Rough capacity
Elasticsearch	Thousands of queries/s per cluster tier; new documents searchable in ~1 s
WebSocket server	~100k–1M concurrent connections when tuned
Stateless app server	Thousands of requests/s, depending on work per request

Latency ladder

Operation	Time
Read from RAM	~100 ns
Read from SSD	~100 µs
Round trip within a datacenter	~0.5 ms
Redis GET (same datacenter)	~0.5–1 ms
Indexed database read	~1–10 ms
Round trip across a continent	~50–100 ms
Round trip across the world	~150–250 ms

The method

1. **Users → requests:** DAU × actions per user per day ÷ 86,400 = average QPS. Multiply by 3 for peak.
2. **Split reads and writes:** apply the read:write ratio.
3. **Storage:** new items per day × size × retention.
4. **Compare** each number with the table above. If it fits one component with headroom, you don't need to scale that part yet.

Example: photo-sharing app

- 500M DAU × 20 feed views = 10B reads/day ≈ **115k reads/s** → needs caching (one Postgres can't serve this).
- 100M uploads/day ≈ **1.2k writes/s** → one primary could take the metadata, but...
- 100M × 2 MB = **200 TB of media per day** → object storage plus a CDN, never the database.

Three numbers, three decisions.

> **Tip**
>
> **Say this in the interview:** "1B requests a day is about 12k per second on average, maybe 40k at peak. A single Redis handles that; Postgres needs a cache in front."

Common mistakes

- Spending ten minutes on arithmetic that doesn't change any decision.
- Using outdated limits and sharding systems that fit on one modern machine.
- Forgetting peak traffic, or forgetting bandwidth for large responses.

Core concepts

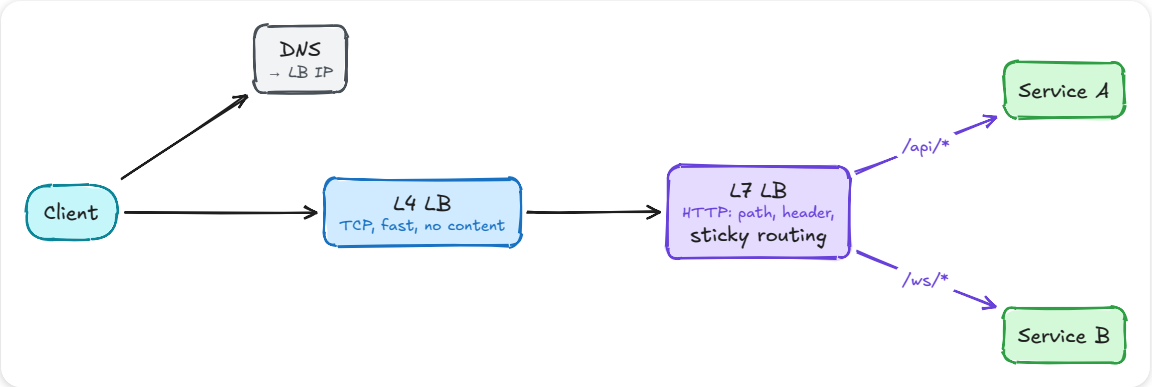
The vocabulary every design uses. Each page has a diagram, a checklist and a sentence you can say in the interview.

Concept	In one line
Networking essentials	Protocols, load balancers, and which to use when
API design	Resources, pagination, idempotency, auth
Data modeling	Start from the queries, then keys and indexes
Database indexing	B-tree, composite, geospatial, inverted
Caching	Cache-aside, stampedes, hot keys
Sharding	Shard keys, hot spots, and when not to shard
Consistent hashing	Add nodes without moving everything
CAP theorem and PACELC	Consistency vs availability, per feature

Also essential: Numbers and estimation.

Networking essentials

How a request travels from a user to your service, and which protocol to pick for which job.



1. DNS resolves the name; an **L4** load balancer forwards TCP connections without reading them. Very fast, good for WebSockets and raw TCP.
2. An **L7** load balancer reads HTTP and routes by path, header, or cookie; it terminates TLS and can do sticky sessions.

Protocol	Use
REST over HTTP	Default public API
gRPC (HTTP/2 + Protobuf)	Internal service-to-service, high QPS
SSE	Server→client stream (feeds, tokens, comments)
WebSocket	Bidirectional real-time (chat, docs, games)
Long polling	Fallback push when nothing else works

Checklist

1. Public API used by browsers and partners: REST over HTTPS.
2. High-volume calls between internal services: gRPC.
3. Server pushes a stream to the client: SSE.
4. Two-way, low-latency messages: WebSocket.

5. Routing by URL path, header or cookie: L7 load balancer. Raw TCP or very high throughput: L4.

In the interview

> **Tip**
>
> "Clients talk REST through an L7 load balancer; internal services use gRPC. Live updates go over SSE, which reconnects automatically."

Related

- Pattern: Real-time updates

API design

Spend five minutes, get the important details right, and move on to the design.

Rule	Example
Resources = core entities	GET /events/{id}, POST /bookings
Cursor pagination, not offset	GET /feed?cursor=(ts,id)&limit=20: stable when new items arrive
Idempotency on mutations	Idempotency-Key header + unique index → retries are safe
Auth from token, not body	user_id comes from the JWT; never trust {user_id} in the request
Async for long work	POST returns 202 + job_id; GET /jobs/{id}
Version	/v1/... or header; add optional fields only

Tip

Say this in the interview: "REST with resources from our entities: POST /bookings with an idempotency key, cursor-paginated GET /bookings. User identity comes from the auth token."

Warning

Don't over-invest: Spending 15 minutes on endpoint shapes is a classic way to run out of time for the deep dives.

Checklist

1. Name resources after your core entities.
2. Use cursor pagination, not offsets.
3. Accept an Idempotency-Key on every mutating call.
4. Take the user's identity from the auth token, never from the body.
5. Return 202 and a job ID for long work.

In the interview

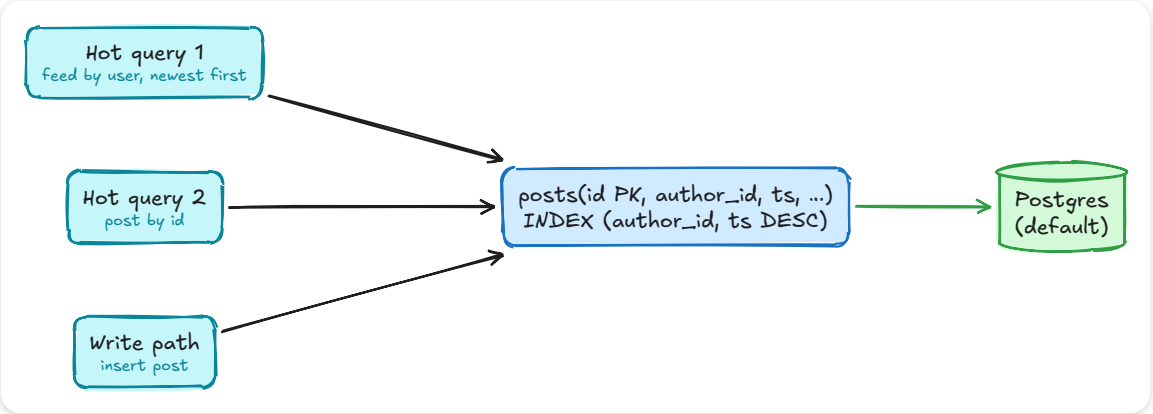
> **Tip**
>
> "Three endpoints cover the requirements; writes take an idempotency key, lists use cursor pagination, and identity comes from the token."

Related

- Pattern: Long-running tasks

Data modeling

Start from the queries the product runs, then choose keys and indexes that make each one cheap.



1. Start from the queries the product actually runs, not from the nouns.
2. Choose keys and indexes that make each hot query a single index lookup or range scan.
3. Default to Postgres; deviate only with a reason (write volume → Cassandra, flexible docs → DynamoDB/Mongo, search → Elasticsearch).

> **Note**
>
> **Normalize vs denormalize:** Normalize by default (one place to update). Denormalize deliberately for a hot read path, and say how the copy stays in sync (CDC, async job).

Checklist

1. List the two or three hottest queries first.
2. Make each one a key lookup or a single index range scan.
3. Default to Postgres; switch only with a reason you can state.
4. Normalize by default; denormalize deliberately for a hot read and say how the copy stays in sync.
5. Choose the partition key from the hottest query if you will shard.

In the interview

> **Tip**
>
> "The hot query is a user's posts, newest first, so the posts table gets an index on (author_id, created_at DESC)."

Related

- Pattern: Scaling reads

Database indexing

An index turns a scan of every row into a short walk down a tree. Pick the right kind for the query.

Index	Use for	Example
B-tree	Equality + range + ORDER BY	created_at > X; composite (user_id, ts)
Hash	Equality only	session lookup
Geospatial (R-tree, geohash)	Nearby queries	Yelp, Uber, Gopuff
Inverted	Full-text search	Post search, Elasticsearch
Composite	Multi-column filters (leftmost prefix rule)	WHERE user_id=? ORDER BY ts
Covering	Index-only scans	INCLUDE (title)

> **Tip**
>
> **Say this in the interview:** "A composite index on (user_id, created_at DESC) makes the feed query a single range scan."

> **Warning**
>
> **Cost:** Every index slows writes; index only the hot queries.

Checklist

1. Equality and ranges: B-tree (the default).
2. Several filters: a composite index, ordered by the leftmost-prefix rule.
3. Distance queries: a geospatial index (R-tree, geohash).
4. Full-text search: an inverted index.

5. Every index slows writes, so index only the queries you need.

In the interview

> **Tip**
>
> "A composite index on (user_id, created_at DESC) turns the feed query into one index range scan."

Related

- Pattern: Scaling reads

Caching

Used by: Ticketmaster · URL shortener · News feed · Local delivery · Distributed cache

TL;DR

- A cache keeps a copy of hot data in memory so you don't pay the full cost of reading it every time. A database read takes about 5–50 ms; a Redis read takes about 1 ms.
- The default pattern is **cache-aside**: read from the cache, and on a miss read the database and fill the cache. On a write, update the database and **delete** the cached key.
- Every cache is a trade. You get speed and less database load, and you pay with **stale data**, **stampedes** when hot keys expire, and **hot keys** that overload one node. Name all three before the interviewer does.

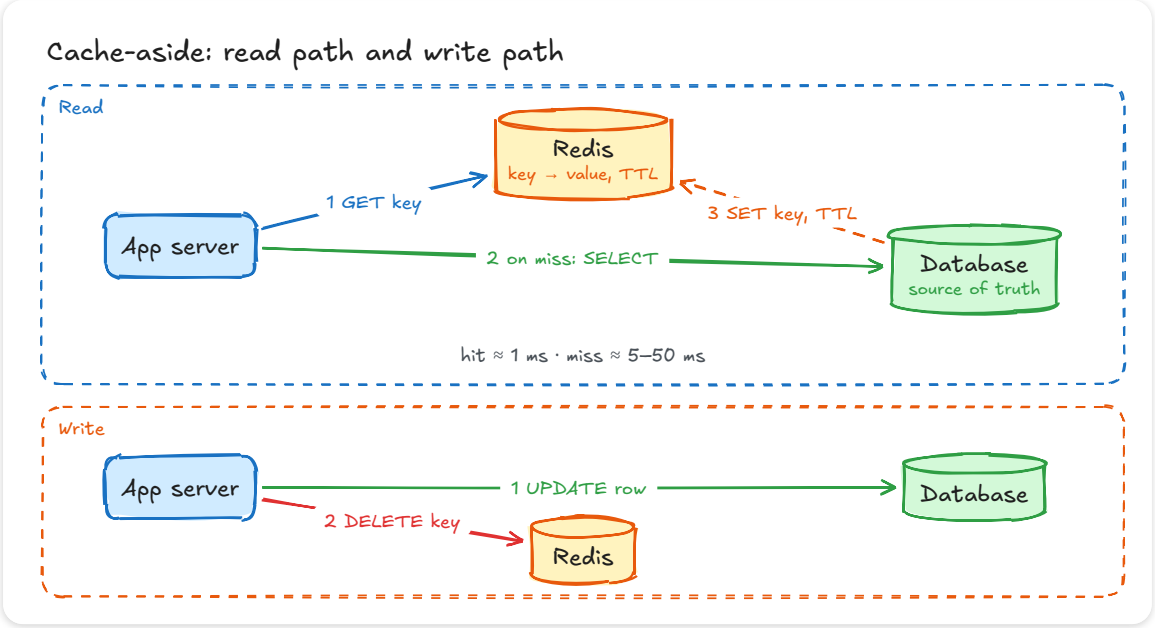
Why cache at all?

Two reasons, and say which one you mean:

1. **Latency.** Memory is orders of magnitude faster than a database query that touches disk or runs joins.
2. **Load.** Popularity is uneven. A small share of items gets most of the traffic, so caching that share removes most of the database load. A cache hit rate of 90% means the database sees one request in ten.

> **Note**
>
> **Rule of thumb:** add a cache when a calculation shows the database can't keep up (for example, ~50k reads/sec against one Postgres primary) or when a read path misses its latency target. Don't add one "because caches are good".

Cache-aside: the default



Read: `GET key` → hit? return it. Miss? `SELECT` from the database, `SET key value EX 300`, return it.

Write: `UPDATE` the database first, then `DELETE key`. The next read repopulates it.

Why delete instead of updating the cache? Two concurrent writers can update the cache in the wrong order and leave it permanently wrong. A delete can't be applied in the wrong order. The worst case is one extra cache miss.

Other strategies

Strategy	How it works	Choose it when	Cost
Cache-aside	App reads cache, falls back to DB, fills cache	Default for almost everything	Brief staleness after writes
Read-through	Cache library loads from DB itself on a miss	You want the logic in one place	Same staleness as cache-aside
Write-through	Every write goes to cache and DB together	Data is read right after it's written	Slower writes; cache fills with data nobody reads
Write-behind	Write to cache now, flush to DB later	Very high write rates you can afford to lose	Data loss if the cache dies before flushing

Strategy	How it works	Choose it when	Cost
CDN / edge cache	HTTP responses cached near users	Static or identical-for-everyone responses	Invalidation is slow, so use short TTLs or versioned URLs

What to cache, and for how long

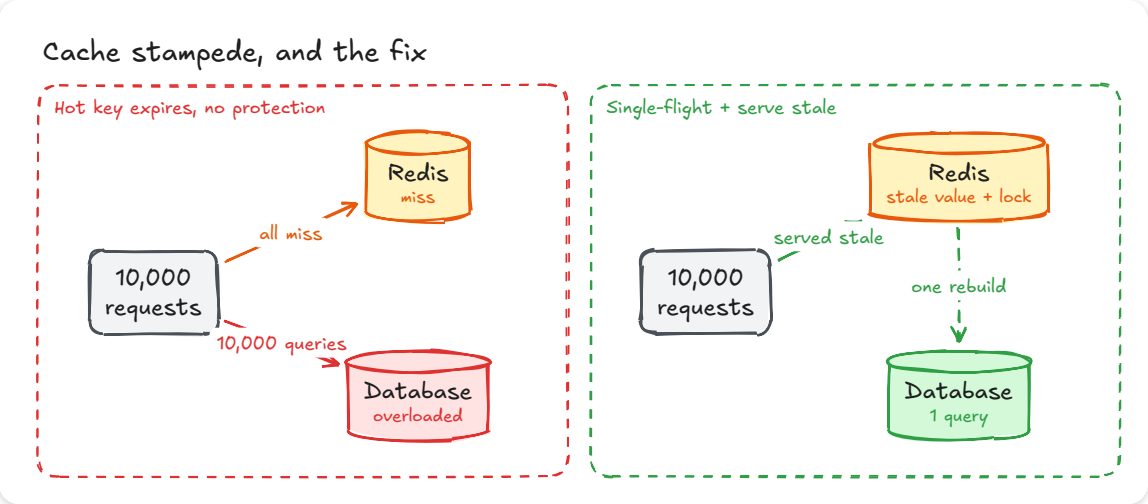
- **Cache:** things read far more often than written, such as event pages, user profiles, URL redirects and computed feeds.
- **Don't cache:** data that must never be stale (a seat's booking status, an account balance at checkout), or data that's read once.
- **TTL:** pick it from how stale the product can tolerate. "A price shown on a search page may be 60 seconds old" means a 60 s TTL. Always set one, so that bugs in invalidation heal themselves.
- **Eviction:** when memory is full, LRU (evict least recently used) is the usual default. LFU suits stable popularity.

The three ways caches hurt you

1. Stale data

Readers can see an old value between the database write and the cache delete, or until the TTL expires. Decide per feature whether that's acceptable. On the booking path of Ticketmaster it isn't, so that path reads the database. On the browse path it is.

2. Stampede (thundering herd)



A very popular key expires, and thousands of requests miss at the same instant. Every one of them queries the database.

Fixes, usually combined:

- **Single-flight / request coalescing:** only one request rebuilds the value, using a short lock such as `SET lock:key 1 NX EX 5`. The others wait briefly or get the old value.
- **Serve stale while revalidating:** keep the old value a little past its TTL and refresh it in the background.
- **Jittered TTLs:** add randomness so many keys don't expire together.
- **Pre-warm** known hot keys before a big event.

3. Hot keys

One key gets so much traffic that the single cache node holding it saturates, for example a celebrity's profile or a viral link. Fixes:

- **Replicate the key:** store copies as `key#0` … `key#9` on different nodes and read a random one.
- **Local in-process cache** with a few seconds' TTL in front of Redis.
- **Know your hot keys:** track them, because you can't fix what you can't see.

Numbers worth knowing

	Typical
Redis GET (same datacenter)	~0.5–1 ms round trip
Redis throughput, one node	~100k+ simple ops/sec
Postgres indexed read	~1–10 ms; complex queries 10–100 ms+
Memory per Redis node	tens to a few hundred GB

Interview phrases

> **Tip**
>
> - "Reads outnumber writes 100 to 1 and popularity is skewed, so a cache-aside Redis layer should take 80–90% of reads off Postgres."
> - "On write I update the database, then delete the key. Deleting is safe under concurrent writers."
> - "This key is hot, so I'll guard against a stampede with single-flight and jittered TTLs."
> - "The checkout path doesn't read from cache, because it must never be stale."

Common mistakes

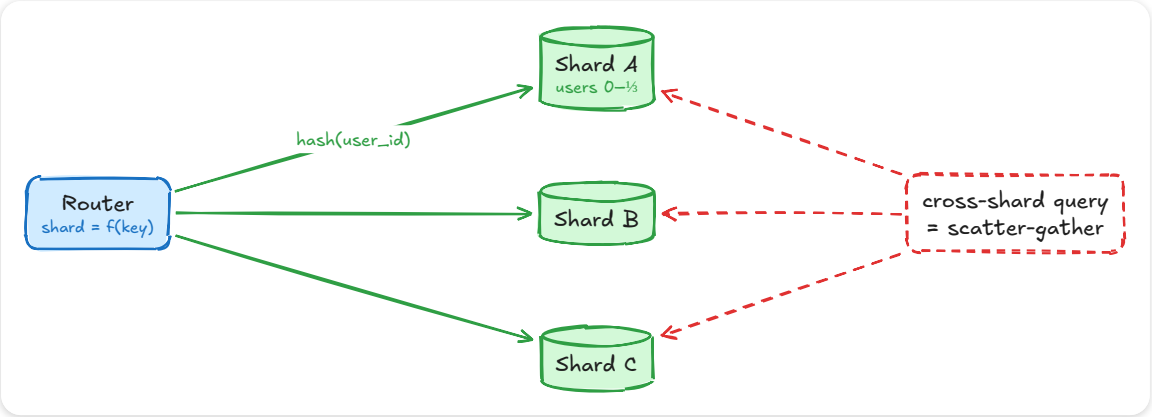
- Caching without a TTL, so one invalidation bug lives forever.
- Updating the cache on write instead of deleting it.
- Treating the cache as the source of truth. Redis can lose data, and the database must still be correct on its own.
- Forgetting the cold start: after a deploy or failover the cache is empty and the database takes the full load.

Related

- **Problems:** Ticketmaster · Distributed cache (design the cache itself)
- **DDIA:** Ch 5 Replication. Read replicas are the other way to scale reads, with the same staleness trade-off.
- **Patterns:** Scaling reads

Sharding

Split data across machines when one can't hold it or keep up with the writes, but only then.



1. Pick a **shard key** from the hottest access pattern so most queries hit one shard. Hash for even spread; range for range scans.
2. Queries that don't include the shard key hit every shard. That is the cost you accept.

> **Warning**
>
> **Biggest mistake: sharding too early:** One Postgres handles ~10–50k TPS and multiple TB. Shard when the calculation says so.

> **Note**
>
> **Hot spots:** Even with hashing, one hot key overloads one shard: key splitting, caching, or buffering.

Checklist

1. Has the calculation shown one primary isn't enough after caching and replicas?
2. Pick the shard key from the hottest query, so most requests hit one shard.
3. Hash for even spread; ranges only if you need range scans on the key.
4. Plan for hot keys and for rebalancing (many logical partitions, or consistent hashing).

5. Accept that queries without the shard key fan out to every shard.

In the interview

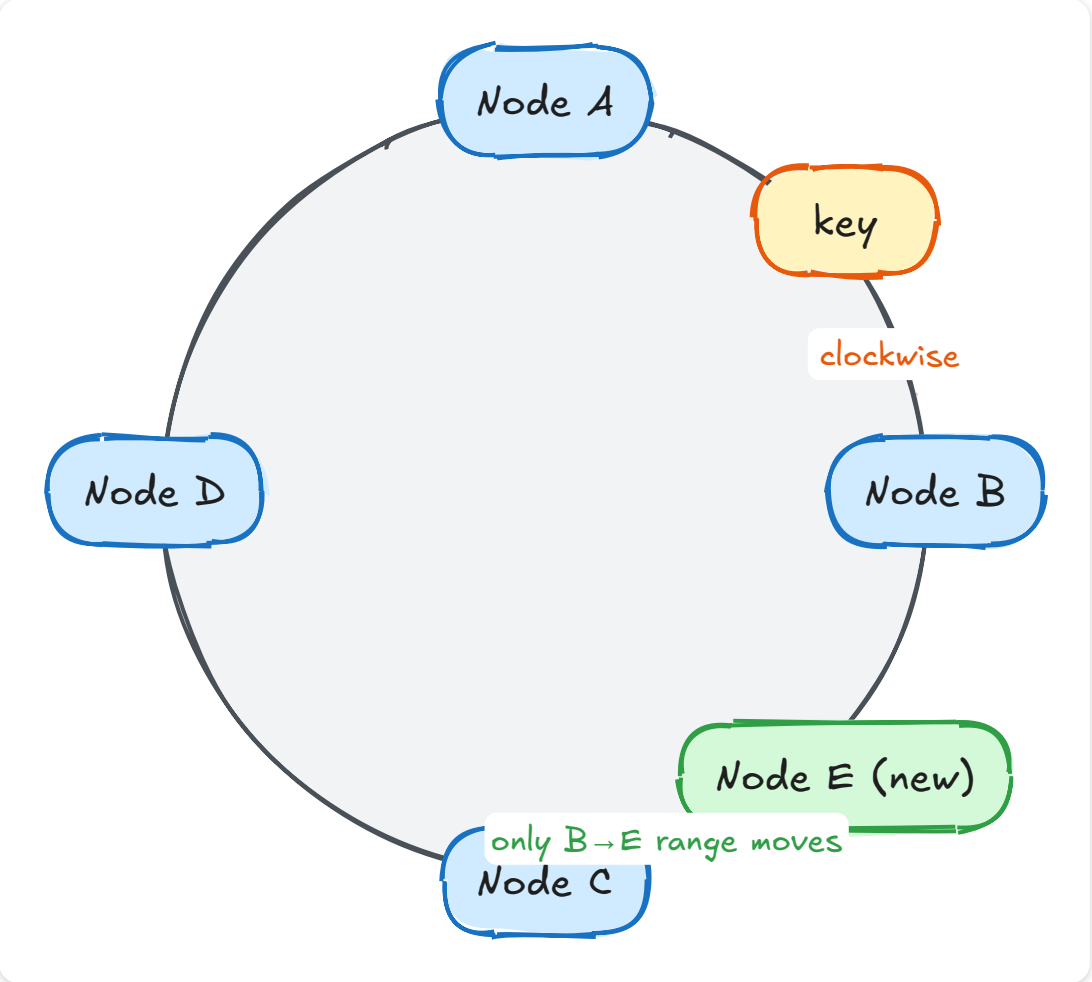
> **Tip**
>
> "Writes outgrow one primary, so I'll shard by hash of user_id: every hot query includes it. Cross-user analytics can scatter-gather or run on a warehouse."

Related

- Pattern: Scaling writes

Consistent hashing

Add or remove a node and move only a small share of the keys.



1. Nodes and keys hash onto a ring.
2. A key belongs to the next node clockwise.
3. Adding a node moves only the keys in one arc (~1/N), unlike hash % N which moves almost everything. Virtual nodes even out the load.

> **Note**
>
> **Used by:** Distributed caches, Cassandra, DynamoDB, L7 routing of video/doc IDs to servers.

Checklist

1. Place nodes and keys on the same hash ring.
2. A key belongs to the first node clockwise from it.
3. Use virtual nodes (many points per server) to even out load.
4. Adding a node moves roughly 1/N of keys; hash % N moves almost all of them.

In the interview

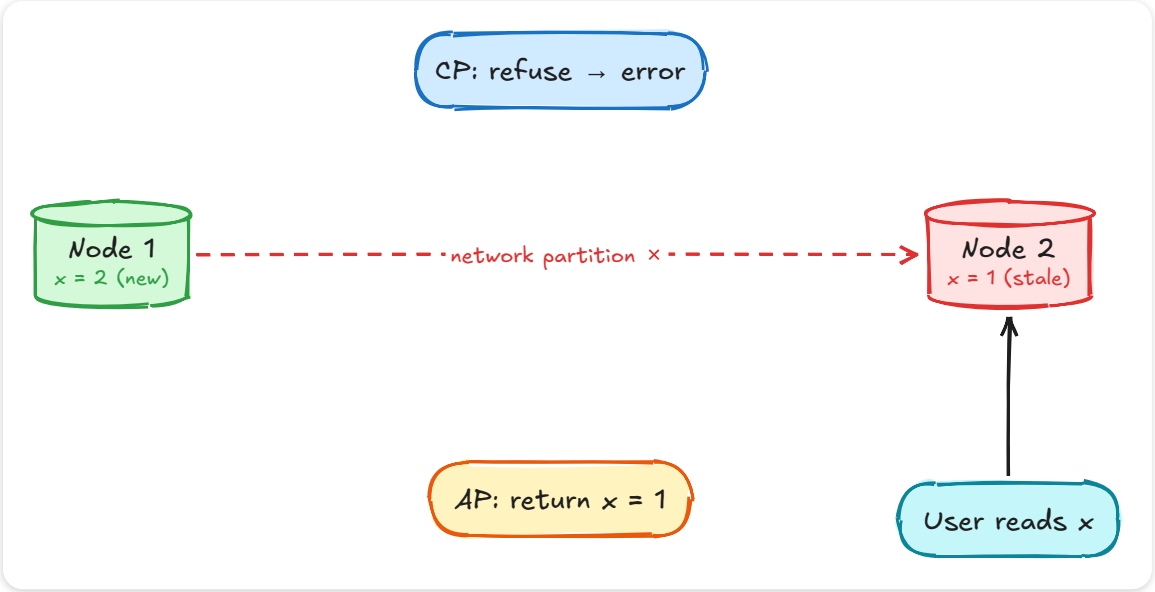
> **Tip**
>
> "Cache nodes sit on a consistent-hash ring with virtual nodes, so adding a node remaps about 1/N of keys instead of emptying the whole cache."

Related

- Pattern: Scaling reads

CAP theorem and PACELC

When the network splits, choose between refusing requests and answering with possibly stale data, separately for each feature.



1. The network between two replicas breaks. Partitions **will** happen, so P is not optional.
2. A user asks the node that can't see the latest write.
3. Either refuse (consistent, unavailable) or answer with stale data (available, inconsistent).

Feature	Choice
Booking, payments, inventory	CP: never oversell or double-charge
Search, feeds, reviews, likes	AP: stale for seconds is fine
Usernames / unique constraints	CP (linearizable)

Tip

Say this in the interview: "CAP is a per-feature choice: the booking path is CP, browsing is AP."

Checklist

1. Partitions will happen, so the real choice is C or A during one.
2. Choose per feature, not per system.
3. Money, inventory and uniqueness: consistency (CP).
4. Feeds, search, reviews, counters: availability (AP).
5. PACELC: even without partitions, stronger consistency costs latency.

In the interview

> **Tip**
>
> "Booking is CP: I'd rather reject than double-sell. Browsing is AP: a few seconds stale is fine."

Related

- Pattern: Dealing with contention

The 7 patterns

Almost every system design problem combines two or three of these. Learn each as a picture: what breaks, and what fixes it.

Pattern	In one line	Problems
Scaling reads	Most systems read far more than they write. Make the common read cheap, and the database only sees what the cache can't answer.	13
Scaling writes	When writes outgrow one machine, absorb the spikes, write in batches, split the data, and write less.	7
Real-time updates	Push changes to clients the moment they happen, even when the sender and receiver are connected to different servers.	7
Dealing with contention	Many users want the same thing at the same moment. Exactly one should win, every time.	3
Multi-step processes	A business action spans several services and any step can fail halfway. Make progress durable and failures reversible.	3
Handling large blobs	Files, photos and videos belong in object storage, and their bytes should never pass through your application servers.	3
Long-running tasks	If work takes longer than a request should, accept it, queue it, and let the client check back.	3

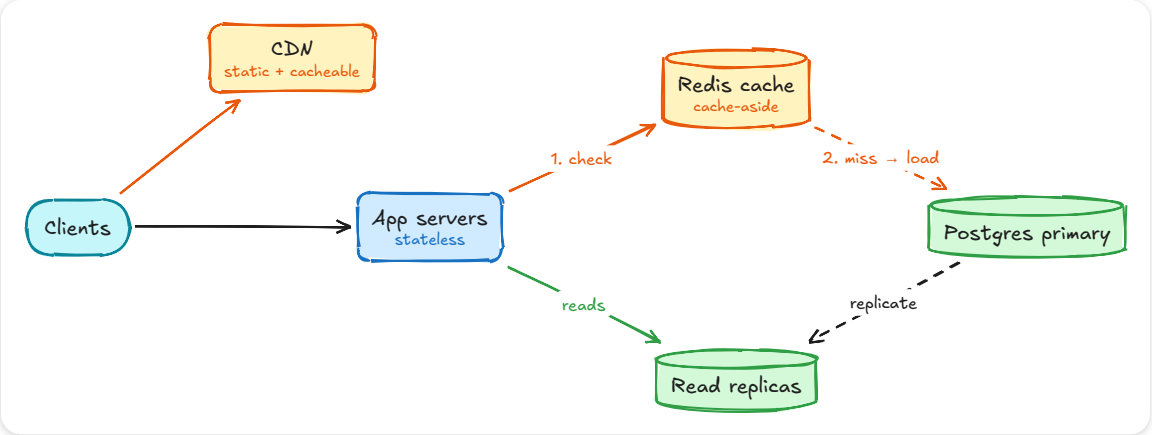
Scaling reads

Most systems read far more than they write. Make the common read cheap, and the database only sees what the cache can't answer.

When to reach for it

- Read:write ratio is 10:1 or higher.
- The same items are read over and over (popularity is skewed).
- A read path misses its latency target, or the calculated read QPS exceeds what one database serves (~10–50k/s).

The idea in one diagram



1. One database serves every read. At ~10k+ reads/sec it becomes the bottleneck.
2. **Cache-aside**: check Redis; on a miss, read the DB and populate the cache with a TTL. A hit takes <1ms vs ~5–50ms from the DB.
3. **Read replicas** spread the remaining reads (expect replication lag, DDIA Ch 5).
4. **CDN** serves static and shared responses from the edge, so origin never sees them.

> **Warning**
>
> **Cache problems to name: Stampede**: a hot key expires and 10k requests hit the DB (fix: lock/single-flight, jittered TTL). **Invalidation**: delete on write, short TTL. **Hot key**: replicate or use a local L1 cache.

Decision checklist

1. Is the response the same for many users? Put a CDN in front.
2. Is it per-user but read often? Cache-aside in Redis with a TTL.
3. Still too much load on the primary? Add read replicas and accept replication lag.
4. Is one key extremely hot? Replicate the key or add a short-lived local cache.
5. Which reads must never be stale? Send those to the primary.

In the interview

> **Tip**
>
> "Reads outnumber writes about 100 to 1 and popularity is skewed, so a cache-aside Redis layer should absorb most reads. The rest go to replicas; only checkout reads the primary."

Common mistakes

- Sharding the database before trying a cache and replicas.
- Caching data that must be exact (stock at checkout, balances).
- Forgetting cache stampedes on hot keys.

Problems that use it

- URL shortener (Bitly): It sounds like a toy, but it is the cleanest example of a read-heavy system: almost every request is a redirect.
- Rate limiter: Every public API needs one, and the obvious implementation has a race condition.
- Business discovery (Yelp): The trap is over-engineering: the whole dataset fits on one database server.

- News feed (Facebook): The canonical fan-out problem: build a personalised feed for billions of users in a few hundred milliseconds.
- News aggregator (Google News): A feed problem where the feed is the same for everyone in a region, which changes everything.
- Local delivery (Gopuff): It looks like an ordering system, but most of the load is people checking what is available near them.
- Dating app (Tinder): Swipes are a firehose of writes, and two people swiping right at the same moment must never miss their match.
- Distributed cache: Design Redis itself: data structures inside one node, then distribution across many.
- Post search (Facebook): Search over billions of posts, where like counts change constantly.
- Price tracker (CamelCamelCamel): Five hundred million products can't all be crawled often, so decide what is worth checking.
- Ticketmaster: Millions of people arrive for one on-sale, and no seat may ever be sold twice.
- Video platform (YouTube): Two hard problems: getting gigabyte uploads in reliably, and streaming smoothly on any connection.
- Photo sharing (Instagram): A news feed plus a media pipeline: two well-known problems combined.

The theory underneath

- DDIA Ch 5: Replication
- DDIA Ch 6: Partitioning

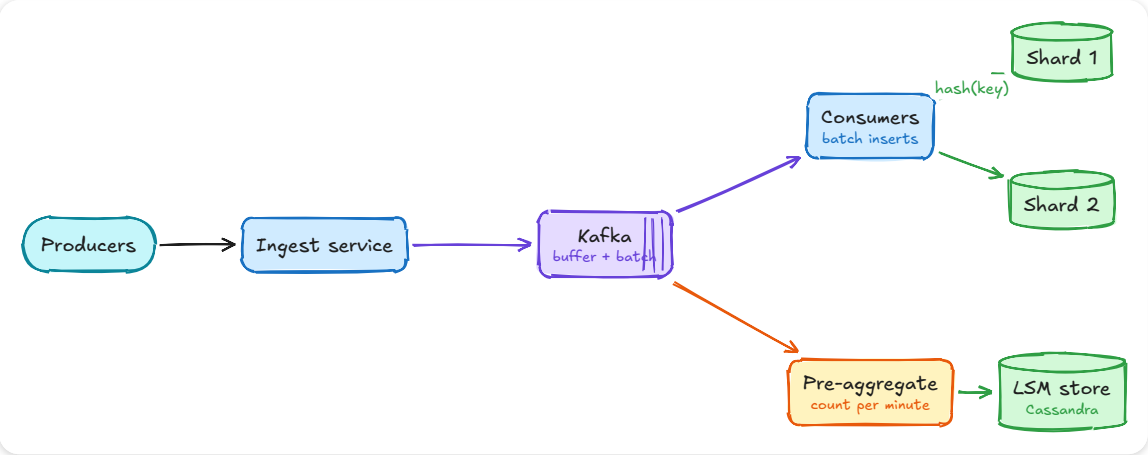
Scaling writes

When writes outgrow one machine, absorb the spikes, write in batches, split the data, and write less.

When to reach for it

- Sustained writes exceed what one primary handles (roughly 10k+ per second for simple rows).
- Write traffic is spiky: events, clicks, telemetry.
- Data is append-mostly and read in aggregate (counts, time series).

The idea in one diagram



1. Every event is an individual synchronous DB write. Spikes overwhelm the database.
2. A **queue/log** (Kafka) absorbs spikes and decouples producers from the DB. Consumers write in **batches**.
3. **Shard** by a key that spreads load evenly (hash of id); each shard takes a fraction of the writes.
4. **Reduce the writes**: pre-aggregate (10k clicks → 1 row per minute), and use write-optimized LSM storage.

> **Warning**
>
> **Hot partition:** One key (celebrity, viral ad) can still overload one shard: split the key with a suffix, or aggregate before writing.

Decision checklist

1. Can the client get a quick ack while the work happens later? Put a durable log (Kafka) in front.
2. Can writes be grouped? Batch inserts in consumers.
3. Can you store less? Pre-aggregate (one row per key per minute instead of one per event).
4. Still too much for one node? Shard by a key that spreads load evenly.
5. Write-heavy and simple key access? Use an LSM-based store such as Cassandra.

In the interview

> **Tip**
>
> "Clicks arrive at 10k/s with spikes, so they go to Kafka first. A stream job aggregates per ad per minute, which cuts the database writes by orders of magnitude."

Common mistakes

- A hot partition: one key still lands on one shard.
- Losing data by acknowledging before anything is durable.
- Using a B-tree OLTP database for a firehose of append-only events.

Problems that use it

- Top-K videos (YouTube): The design falls straight out of one number: hundreds of thousands of views per second.
- Ad click aggregator: Clicks are money, so the numbers must be fast for dashboards and exact for billing.

- Web crawler: Ten billion pages in five days is a pipeline problem, constrained by politeness and network bandwidth.
- Metrics monitoring (Datadog): Millions of data points per second arrive, and alerts must fire within a minute.
- Fitness tracking (Strava): The key insight is where the work happens: on the phone, not the server.
- Post search (Facebook): Search over billions of posts, where like counts change constantly.
- Price tracker (CamelCamelCamel): Five hundred million products can't all be crawled often, so decide what is worth checking.

The theory underneath

- DDIA Ch 3: Storage and retrieval
- DDIA Ch 6: Partitioning
- DDIA Ch 11: Stream processing

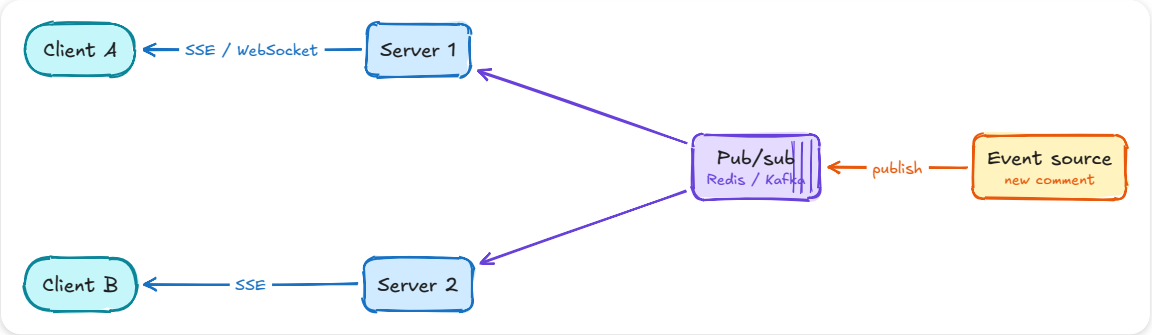
Real-time updates

Push changes to clients the moment they happen, even when the sender and receiver are connected to different servers.

When to reach for it

- Users expect to see changes within about a second: chat, live comments, prices, collaboration, ride tracking.
- Polling would be mostly empty responses or far too frequent.

The idea in one diagram



1. **Polling**: simple, but latency = interval and most requests return nothing. Fine for slow updates (every ~30s).
2. **Server push**: SSE (server→client, plain HTTP, auto-reconnect) or WebSocket (bidirectional). Long-polling is the fallback.
3. With many servers, the receiver's connection lives on **another server** than the one that got the event.
4. **Pub/sub** routes events to whichever servers hold the interested connections: channel per video, per user, or per symbol.

> **Note**
>
> **Choosing:** Updates rare → poll. Server→client stream → SSE. Chat, games, collaboration → WebSocket. Between servers → pub/sub.

Decision checklist

1. How fresh must it be? Every 30 s or slower: just poll.
2. Server-to-client only? Use Server-Sent Events (plain HTTP, auto-reconnect).
3. Both directions, low latency? Use WebSockets.
4. Many servers? Route events with pub/sub (Redis, Kafka) to whichever server holds the connection.
5. Can events be lost? If not, persist first and let clients resume from an offset after reconnecting.

In the interview

> **Tip**
>
> "Viewers only receive, so SSE is enough. Each comment is published once to a channel per video; every server holding viewers of that video is subscribed and pushes it to them."

Common mistakes

- Polling at large scale.
- Assuming sender and receiver share a server.
- No plan for reconnects and missed messages.

Problems that use it

- Ride sharing (Uber): Four problems in one: a flood of location updates, matching, locking drivers, and a multi-step trip lifecycle.
- Live comments (Facebook Live): One popular stream can mean a million viewers each expecting new comments within a fraction of a second.
- Collaborative editor (Google Docs): The challenge is correctness, not scale: everyone must end up with the same document.
- Stock trading (Robinhood): Two separate problems: pushing live prices to millions, and never losing or duplicating an order.
- Online auction: Many people bid on one row at the same time, and exactly one of them must win each race.

- LLM chat (ChatGPT): GPUs are the scarce resource, so every queueing decision is really a cost decision.
- Chat app (WhatsApp): Chat looks like CRUD until you notice the two users are on different servers and one of them is offline.

The theory underneath

- DDIA Ch 11: Stream processing

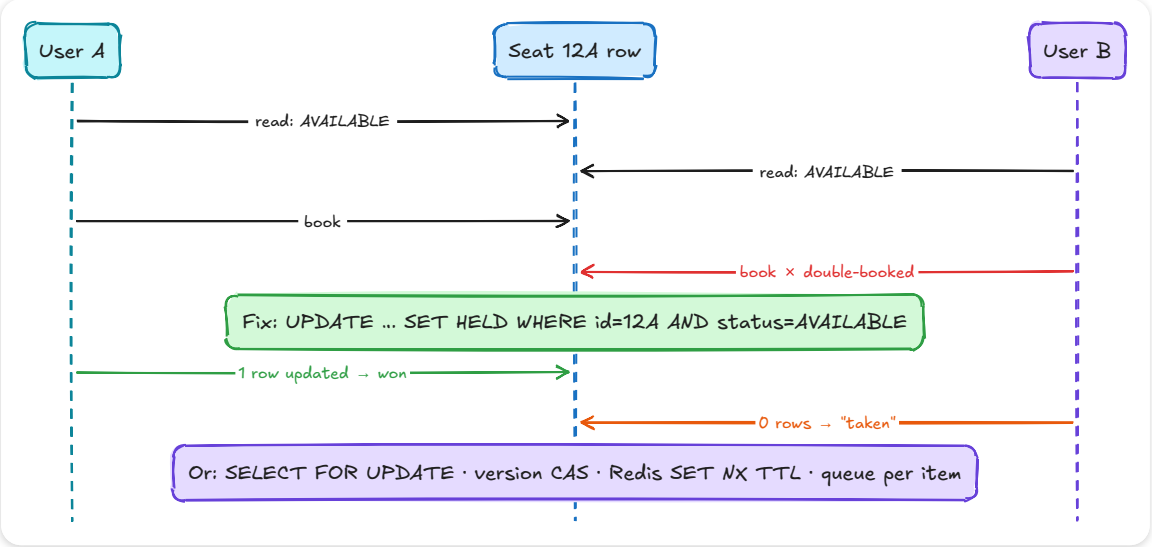
Dealing with contention

Many users want the same thing at the same moment. Exactly one should win, every time.

When to reach for it

- Limited resources: seats, stock, a username, a driver, the highest bid.
- Any "check, then act" sequence that two requests can run at the same time.

The idea in one diagram



1. Check-then-act from two users: both see AVAILABLE, both book. A race condition (DDIA Ch 7 write skew).
2. Make the check and the write **one atomic statement**. The database guarantees only one succeeds.
3. Other tools: pessimistic lock (high contention), optimistic version check (low contention), distributed lock with TTL (holds across services), or a per-item queue (extreme contention).

Tool	Use when
Atomic conditional UPDATE	Single row, single DB: the default

Tool	Use when
SELECT ... FOR UPDATE	Multi-step logic on few rows, high contention
Optimistic (version column)	Low contention; retry on conflict
Redis lock SET NX PX	Temporary holds across services (seat hold, driver lock)
Queue per resource	Extreme hot spots (auction last seconds)

Decision checklist

1. Can the check and the write be one statement? Use a conditional UPDATE (... WHERE status = AVAILABLE). This is the default.
2. Multi-step logic on a few rows with high contention? SELECT ... FOR UPDATE, locking rows in a fixed order.
3. Low contention? Optimistic concurrency with a version column; retry on conflict.
4. A temporary hold that spans services? A lock with a TTL (Redis SET NX EX), plus a fencing token if correctness depends on it.
5. Extremely hot single item? Serialize requests through a queue partitioned by item.

In the interview

> **Tip**
>
> "The check and the write must be atomic, so the hold is a single conditional UPDATE. One row updated means you got the seat; zero means someone else did."

Common mistakes

- Check-then-act in application code.
- Holding database locks while waiting on a human (payment, form).
- Trusting a distributed lock without fencing when a paused process could wake up later.

Problems that use it

- Rate limiter: Every public API needs one, and the obvious implementation has a race condition.
- Online auction: Many people bid on one row at the same time, and exactly one of them must win each race.
- Ticketmaster: Millions of people arrive for one on-sale, and no seat may ever be sold twice.

The theory underneath

- DDIA Ch 7: Transactions
- DDIA Ch 8: Trouble with distributed systems

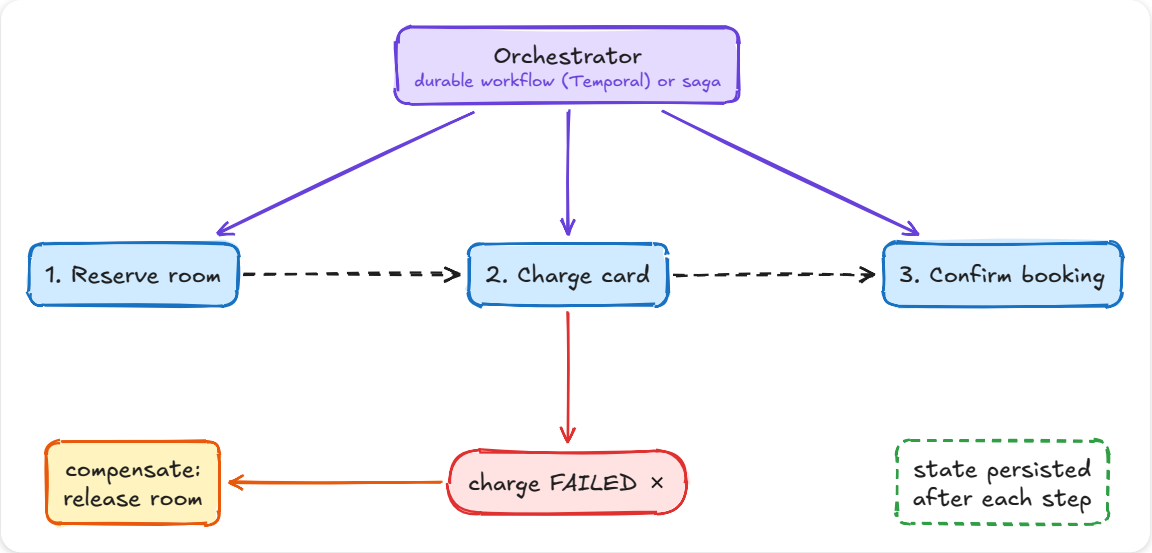
Multi-step processes

A business action spans several services and any step can fail halfway. Make progress durable and failures reversible.

When to reach for it

- One user action touches several services or external APIs: booking + payment + confirmation, ride lifecycle, order fulfilment.
- Steps can take seconds to days, or depend on outside callbacks.

The idea in one diagram



1. A business action spans several services, and no distributed ACID transaction is available.
2. Any step can fail after earlier steps succeeded.
3. A **saga** runs a **compensating action** for each completed step (refund, release, cancel), in reverse order.
4. A durable orchestrator records progress after each step, so a crash resumes where it stopped instead of starting over or forgetting.

> **Note**
>
> **Building blocks:** Outbox pattern for reliable events · idempotent steps (retries are safe) · timeouts with unknown-outcome handling.

Decision checklist

1. Is there a cross-service transaction? Replace it with a saga: a compensating action for each step.
2. Who drives the steps? An orchestrator (Temporal, Step Functions) is easier to reason about than event choreography for complex flows.
3. Are all steps idempotent so retries are safe?
4. Do events leave the database reliably? Use the outbox pattern.
5. What happens on timeouts with unknown outcomes? Query status; don't blindly retry side effects.

In the interview

> **Tip**
>
> "There is no distributed transaction across booking and payment, so I'll run a saga: hold the room, charge, confirm, and if the charge fails, release the hold. A workflow engine persists progress so a crash resumes instead of restarting."

Common mistakes

- Proposing two-phase commit across microservices.
- Non-idempotent steps that double-charge on retry.
- Dual writes to a database and a queue without an outbox.

Problems that use it

- Payment system (Stripe): Throughput is moderate. The hard part is staying correct when a call to the bank has an unknown outcome.

- Online judge (LeetCode): You are running strangers' code on your servers, so isolation comes first.
- Job scheduler: Ten thousand jobs per second, each within two seconds of its scheduled time.

The theory underneath

- DDIA Ch 9: Consistency and consensus
- DDIA Ch 11: Stream processing
- DDIA Ch 12: The future of data systems

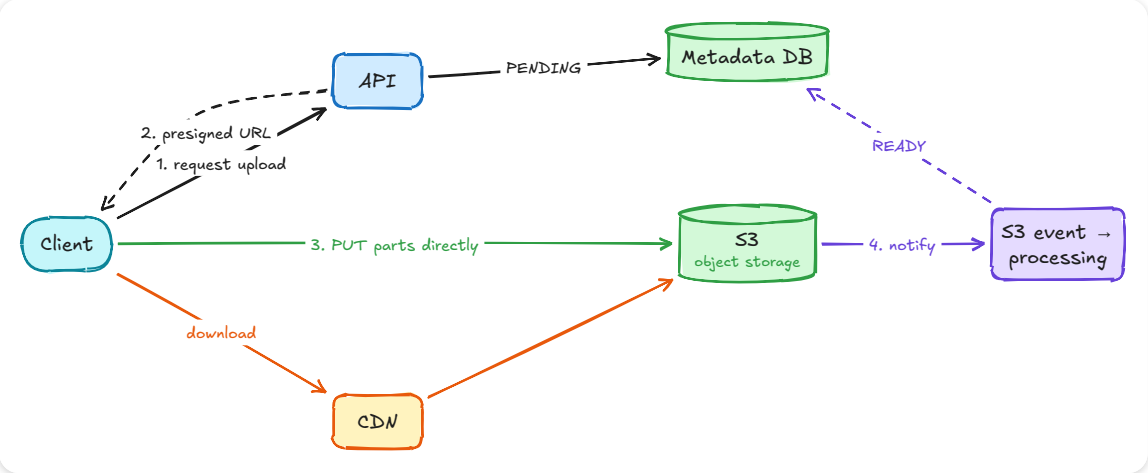
Handling large blobs

Files, photos and videos belong in object storage, and their bytes should never pass through your application servers.

When to reach for it

- Users upload or download files larger than a few megabytes.
- Media needs processing after upload (thumbnails, transcoding).

The idea in one diagram



1. Pushing gigabytes through your API servers wastes their bandwidth and memory and fails on long uploads.
2. The API hands out a **presigned URL**; the client uploads directly to S3 (**multipart**: parallel, resumable chunks). Metadata is recorded separately.
3. S3 **event notifications** trigger processing (thumbnails, transcoding) and flip status to READY. Don't trust the client to say it finished.
4. Downloads come from a **CDN** (or presigned GET URLs), never through the app servers.

Decision checklist

1. Upload with presigned URLs directly to object storage.
2. Use multipart upload for anything big: parallel, resumable chunks.

3. Store metadata (owner, status, size, checksum) in a database, separate from the bytes.
4. React to storage events to start processing; mark the item ready only when done.
5. Serve downloads through a CDN or presigned GET URLs.

In the interview

> **Tip**
>
> "The API only hands out a presigned multipart URL; the client uploads straight to S3. An S3 event triggers processing, and the file shows as ready when processing finishes."

Common mistakes

- Streaming file bytes through API servers.
- Storing blobs in the database.
- Trusting the client to report that an upload completed.

Problems that use it

- Video platform (YouTube): Two hard problems: getting gigabyte uploads in reliably, and streaming smoothly on any connection.
- File storage and sync (Dropbox): Syncing a 50 GB file after a one-byte edit should not mean uploading 50 GB again.
- Photo sharing (Instagram): A news feed plus a media pipeline: two well-known problems combined.

The theory underneath

- DDIA Ch 10: Batch processing

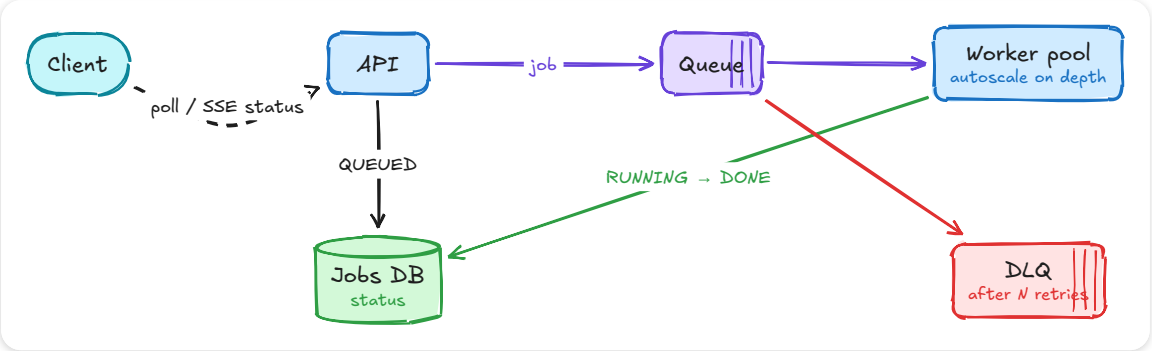
Long-running tasks

If work takes longer than a request should, accept it, queue it, and let the client check back.

When to reach for it

- Work takes more than a second or two: transcoding, report generation, code execution, model inference.
- Work is bursty and expensive, so it needs backpressure.

The idea in one diagram



1. Doing heavy work inside the request: timeouts, retried requests duplicate work, and no backpressure.
2. Return a job ID immediately; enqueue the work. Workers scale independently of the API.
3. Workers update the job status; the client polls or subscribes (SSE/WebSocket) for progress.
4. Failures retry with backoff (visibility timeout); poison jobs go to a dead-letter queue. Jobs must be idempotent.

Decision checklist

1. Return a job ID immediately (HTTP 202) and enqueue the work.
2. Workers scale on queue depth, independently of the API.
3. Store status (queued, running, done, failed); clients poll or subscribe.

4. Retry with backoff using a visibility timeout; send poison jobs to a dead-letter queue.
5. Make jobs idempotent: a retried job must not duplicate side effects.

In the interview

> **Tip**
>
> "Submission returns a job ID right away. The job goes on a queue; autoscaled workers run it and update its status, and the client polls every second."

Common mistakes

- Doing heavy work inside the HTTP request.
- No dead-letter queue, so one bad job blocks or loops forever.
- Non-idempotent jobs combined with at-least-once delivery.

Problems that use it

- Distributed cache: Design Redis itself: data structures inside one node, then distribution across many.
- Online judge (LeetCode): You are running strangers' code on your servers, so isolation comes first.
- Job scheduler: Ten thousand jobs per second, each within two seconds of its scheduled time.

The theory underneath

- DDIA Ch 11: Stream processing
- DDIA Ch 8: Trouble with distributed systems

Choosing technologies

You need fewer technologies than you think. Start with **Postgres**, add **Redis** for speed and **Kafka** for streams, and reach for the others only when a requirement asks for them.

Need	Default choice
Source of truth with transactions	Postgres
Cache, counters, locks, leaderboards, geo, pub/sub	Redis
Durable event stream, buffering writes, change data capture	Kafka
Huge write volume with simple key access	Cassandra (or DynamoDB on AWS)
Full-text and faceted search	Elasticsearch (fed from the source of truth)
Windowed aggregation over streams	Flink
Leader election, locks with fencing, cluster metadata	ZooKeeper or etcd
Single entry point: auth, rate limits, routing	API gateway

Postgres

What it is: a relational database with ACID transactions, rich indexing (B-tree, GIN, GiST/PostGIS) and JSON support.

- **Use when:** you need transactions, relations, constraints, or you're not sure yet. It is the right default.
- **Avoid when:** writes exceed what one primary can take even after batching (tens of thousands per second sustained), or data is tens of TB of append-only events.
- **Numbers:** ~10–50k simple transactions/s per primary; multiple TB per node; read replicas for read scaling.

- **Gotcha:** the default isolation (Read Committed) does not prevent write skew. Use conditional updates or `SELECT … FOR UPDATE` for bookings and inventory.

> **Tip**
>
> "Postgres is the source of truth. I'll only move data elsewhere when a specific number says it doesn't fit."

Redis

What it is: an in-memory data-structure server: strings, hashes, sorted sets, streams, geo indexes, pub/sub. Commands run on a single thread, which makes each one atomic.

- **Use when:** caching, rate limiting (atomic Lua scripts), counters, leaderboards (sorted sets), short-lived locks (`SET NX EX`), geo lookups, fan-out of messages between servers.
- **Avoid when:** the data must survive anything (it's not your durable record), or it doesn't fit in memory.
- **Numbers:** ~100k+ operations/s per node, sub-millisecond latency.
- **Gotcha:** a lock in Redis isn't safe on its own if a process can pause and wake up after the lock expired. Use fencing tokens, or make the database enforce the rule.

Kafka

What it is: a partitioned, replicated, append-only log. Consumers track their own offsets, so many independent consumers can read the same data and replay it.

- **Use when:** buffering write spikes, event-driven pipelines, change data capture, stream processing, several consumers of the same events.
- **Avoid when:** you need per-message work queues with individual retries and delays (use SQS/RabbitMQ), or request/response.
- **Numbers:** plan ~5–10k messages/s per partition; ~1M messages/s per broker; 7-day default retention.
- **Gotcha:** ordering is guaranteed only **within a partition**, and parallelism is capped by the number of partitions. Choose the partition key deliberately (e.g. `booking_id`).

Cassandra

What it is: a leaderless, wide-column database built on LSM trees. Very fast writes, tunable consistency.

- **Use when:** massive write volume with simple access by key: chat history, metrics, activity logs, swipes.
- **Avoid when:** you need joins, ad-hoc queries or multi-row transactions.
- **Numbers:** ~10k+ writes/s per node; scales linearly by adding nodes.
- **Gotcha:** you model tables **per query**; a query without the partition key is expensive. Concurrent writes resolve by last-write-wins.

DynamoDB

What it is: a managed key-value and document store on AWS with automatic scaling and single-digit-millisecond latency.

- **Use when:** you're on AWS and want predictable key-based access without running a cluster.
- **Avoid when:** access patterns are unknown or relational.
- **Gotcha:** one hot partition key throttles; global secondary indexes are eventually consistent.

Elasticsearch

What it is: a distributed search engine built on inverted indexes, with relevance ranking, fuzzy matching, facets and geo queries.

- **Use when:** full-text search, faceted filtering, log search.
- **Avoid when:** you'd make it the source of truth, or `pg_trgm` / Postgres full-text already covers the need.
- **Numbers:** new documents become searchable after a refresh interval (~1 s).
- **Gotcha:** feed it from the database through change data capture. Never write to both the database and the index from application code.

Flink

What it is: a stateful stream processor with event-time windows, checkpoints and exactly-once state.

- **Use when:** real-time aggregation (clicks per minute, top-K), alerting on streams, complex event patterns.
- **Avoid when:** a simple consumer writing to a database is enough.
- **Gotcha:** "exactly once" means effectively once. Sinks must be idempotent (upserts) or transactional.

ZooKeeper / etcd

What it is: a small, strongly consistent key-value store built on a consensus protocol.

- **Use when:** electing a leader, holding locks with fencing tokens, storing cluster membership and configuration that clients watch.
- **Avoid when:** storing application data. It's built for small metadata.
- **Gotcha:** every write needs a majority round trip, so keep traffic low.

API gateway

What it is: the single entry point in front of your services.

- **Use when:** you need authentication, rate limiting, TLS termination, routing and request logging in one place.
- **Avoid:** putting business logic in it.

Related

- The 7 patterns: when each technology earns its place.
- Numbers and estimation

29 problems

Every page follows the same template: requirements, numbers, a **v1** design and where it breaks, the final design and why each part exists, deep dives, a minute-by-minute interview script, and common mistakes.

New to this? Start with URL shortener, Rate limiter and Ticketmaster.

Scaling reads

Pattern: Scaling reads

Problem	Difficulty	What makes it interesting
URL shortener (Bitly)	Easy	It sounds like a toy, but it is the cleanest example of a read-heavy system: almost every request is a redirect.
Rate limiter	Medium	Every public API needs one, and the obvious implementation has a race condition.
Business discovery (Yelp)	Medium	The trap is over-engineering: the whole dataset fits on one database server.
News feed (Facebook)	Medium	The canonical fan-out problem: build a personalised feed for billions of users in a few hundred milliseconds.
News aggregator (Google News)	Easy	A feed problem where the feed is the same for everyone in a region, which changes everything.
Local delivery (Gopuff)	Easy	It looks like an ordering system, but most of the load is people checking what is available near them.
Dating app (Tinder)	Medium	Swipes are a firehose of writes, and two people swiping right at the same moment must never miss their match.
Distributed cache	Medium	Design Redis itself: data structures inside one node, then distribution across many.

Scaling writes

Pattern: Scaling writes

Problem	Difficulty	What makes it interesting
Top-K videos (YouTube)	Hard	The design falls straight out of one number: hundreds of thousands of views per second.
Ad click aggregator	Hard	Clicks are money, so the numbers must be fast for dashboards and exact for billing.
Web crawler	Hard	Ten billion pages in five days is a pipeline problem, constrained by politeness and network bandwidth.
Metrics monitoring (Datadog)	Hard	Millions of data points per second arrive, and alerts must fire within a minute.
Fitness tracking (Strava)	Medium	The key insight is where the work happens: on the phone, not the server.
Post search (Facebook)	Medium	Search over billions of posts, where like counts change constantly.
Price tracker (CamelCamelCamel)	Medium	Five hundred million products can't all be crawled often, so decide what is worth checking.

Real-time updates

Pattern: Real-time updates

Problem	Difficulty	What makes it interesting
Ride sharing (Uber)	Hard	Four problems in one: a flood of location updates, matching, locking drivers, and a multi-step trip lifecycle.
Live comments (Facebook Live)	Medium	One popular stream can mean a million viewers each expecting new comments within a fraction of a second.
Collaborative editor (Google Docs)	Hard	The challenge is correctness, not scale: everyone must end up with the same document.
Stock trading (Robinhood)	Hard	Two separate problems: pushing live prices to millions, and never losing or duplicating an order.

Problem	Difficulty	What makes it interesting
Online auction	Medium	Many people bid on one row at the same time, and exactly one of them must win each race.
LLM chat (ChatGPT)	Hard	GPUs are the scarce resource, so every queueing decision is really a cost decision.
Chat app (WhatsApp)	Medium	Chat looks like CRUD until you notice the two users are on different servers and one of them is offline.

Dealing with contention

Pattern: Dealing with contention

Problem	Difficulty	What makes it interesting
Ticketmaster	Hard	Millions of people arrive for one on-sale, and no seat may ever be sold twice.

Multi-step processes

Pattern: Multi-step processes

Problem	Difficulty	What makes it interesting
Payment system (Stripe)	Hard	Throughput is moderate. The hard part is staying correct when a call to the bank has an unknown outcome.
Online judge (LeetCode)	Medium	You are running strangers' code on your servers, so isolation comes first.
Job scheduler	Hard	Ten thousand jobs per second, each within two seconds of its scheduled time.

Handling large blobs

Pattern: Handling large blobs

Problem	Difficulty	What makes it interesting
Video platform (YouTube)	Hard	Two hard problems: getting gigabyte uploads in reliably, and streaming smoothly on any connection.
File storage and sync (Dropbox)	Easy	Syncing a 50 GB file after a one-byte edit should not mean uploading 50 GB again.
Photo sharing (Instagram)	Medium	A news feed plus a media pipeline: two well-known problems combined.

Long-running tasks

Pattern: Long-running tasks

Problem	Difficulty	What makes it interesting

URL shortener (Bitly)

Difficulty: Easy · **Patterns:** Scaling reads · **DDIA:** Ch 5 replicas · Ch 6 hash sharding

It sounds like a toy, but it is the cleanest example of a read-heavy system: almost every request is a redirect.

TL;DR

- Redirects outnumber new links about 100 to 1, so the design is mostly about serving reads fast.
- Generate short codes from a counter encoded in base62: unique by construction, no collision checks.
- Put a cache in front of the database; a small share of links gets most clicks, so the hit rate is high.

1. Requirements

Functional

1. Create a short URL for a long URL (optional custom alias, expiry)
2. Redirect short URL → original URL

Non-functional

- Redirect p99 < 100ms
- 99.99% available for redirects
- Short codes globally unique
- Read:write ≈ 100:1

2. Back-of-envelope numbers

100M DAU × 10 clicks = 1B redirects/day ≈ **12k/s avg, ~100k/s peak**. Writes ~1M/day ≈ 12/s. 1B URLs × 500B = **500GB**, which fits one Postgres.

3. Core entities and API

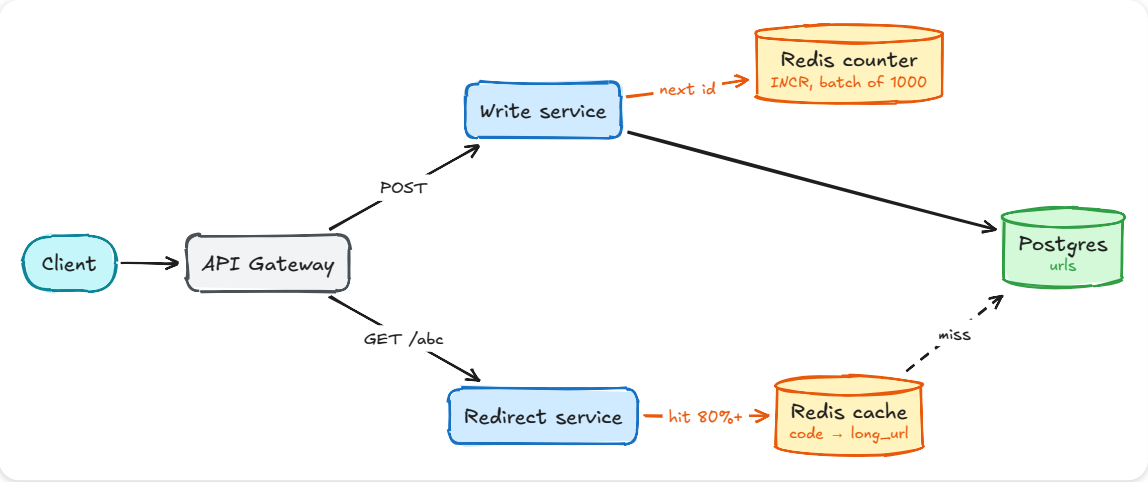
```
urls(code PK, long_url, user_id, created_at, expires_at)
```

```
POST /urls {long_url, alias?, expires_at?} → short_url
GET /{code} → 302 Location: long_url
```

4. v1: the simplest design, and where it breaks

Start simple: a write service creates a code and stores it in Postgres. ~12 writes/sec is nothing.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Start simple: a write service creates a code and stores it in Postgres. ~12 writes/sec is nothing.
2. Redirects are 100× more frequent. 100k reads/sec straight to Postgres is what breaks first.
3. Add a Redis cache (code → URL). Clicks follow Pareto, so 80%+ hit the cache in under 1ms. Postgres only sees misses; add replicas if needed.
4. Generate codes from a global counter: Redis INCR + base62. 7 chars = 3.5 trillion codes, no collisions. Servers grab IDs in batches of 1,000.

6. Deep dives

How do you guarantee unique codes?

Counter + base62 is unique by construction. Hashing (MD5 prefix) needs a unique constraint plus retry on collision. Counters are guessable, so add a reversible shuffle if that matters.

301 or 302?

302: every click reaches you (analytics, expiry, editing). 301 lets browsers cache it, which cuts load but loses analytics.

What if the counter node dies?

Redis with AOF + replica, or servers lease ranges from a Postgres sequence. A crash wastes a range; gaps are harmless.

Custom aliases?

Same table with a unique constraint; the insert fails if taken. Keep namespaces apart (e.g., counter codes are 7 chars, aliases ≥ 8).

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Create a short URL for a long URL (optional custom alias, expiry); Redirect short URL → original URL
5–8	Non-functional requirements	Redirect p99 < 100ms; 99.99% available for redirects; Short codes globally unique; Read:write ≈ 100:1
8–10	Numbers that drive the design	100M DAU × 10 clicks = 1B redirects/day ≈ **12k/s avg, ~100k/s peak**. Writes ~1M/day ≈ 12/s. 1B URLs × 500B = **500GB**, which fits one Postgres.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: Redirect service → Redis cache → Redis counter. Justify each addition with a number.
25–40	Deep dives	How do you guarantee unique codes? · 301 or 302? · What if the counter node dies? · Custom aliases?

Time	What you do	Content
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- Sharding 500GB at 12 writes/sec. It fits one machine; scale reads with cache and replicas.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Scaling reads
- **DDIA:** Ch 5 replicas · Ch 6 hash sharding
- **Similar problems:** Rate limiter · Business discovery (Yelp) · News feed (Facebook) · News aggregator (Google News)

Rate limiter

Difficulty: Medium · **Patterns:** Scaling reads · Dealing with contention · **DDIA:** Ch 7 lost update · Ch 6 shard by key

Every public API needs one, and the obvious implementation has a race condition.

TL;DR

- Counting per server gives each client N times its limit; the counter must be shared, usually in Redis.
- Read-then-write on a shared counter races. Do the refill, check and decrement atomically in one Lua script.
- Decide what happens when Redis is down: allow traffic (fail-open) or block it (fail-closed), per endpoint.

1. Requirements

Functional

1. Limit requests per client (user, IP, API key) by configurable rules
2. Return 429 with Retry-After and rate-limit headers

Non-functional

- Adds < 10ms latency
- Accurate across many API servers
- Highly available (choose fail-open vs fail-closed)
- 1M requests/sec total

2. Back-of-envelope numbers

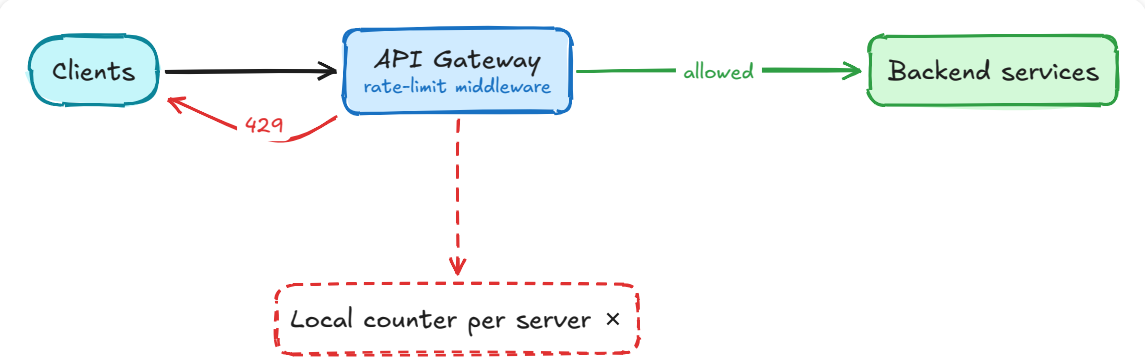
1M req/s = 1M Redis ops/s; one Redis node does ~100k ops/s, so a **~10–20 shard Redis Cluster** keyed by client. 10M active keys × 100B ≈ **1GB** of memory.

3. Core entities and API

```
rules(endpoint, limit, window)
bucket:{client}:{rule} → {tokens, last_refill}
```

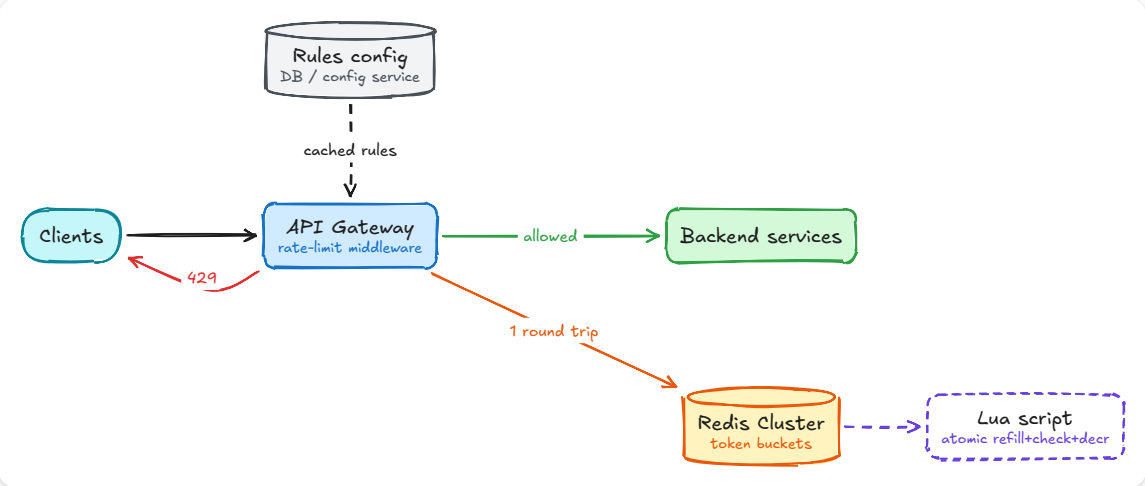
```
isAllowed(client_key, rule) → {allowed, remaining, reset_at}
Response headers: X-RateLimit-Limit / Remaining / Reset
```

4. v1: the simplest design, and where it breaks



Naive: each API server counts in local memory. With 50 servers, a client gets 50× the limit.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Naive: each API server counts in local memory. With 50 servers, a client gets 50× the limit.
2. Centralize counters in Redis. But GET then SET from two servers races: both see 99/100 and both allow.

3. A Lua script runs refill + check + decrement atomically inside single-threaded Redis, in one round trip, with no race.
4. Rules come from a config store cached in each gateway. Decide failure mode: fail-open for normal APIs, fail-closed for login and payments.

6. Deep dives

Which algorithm?

Token bucket (capacity = burst, refill = sustained rate) is the industry default. Fixed window allows 2× bursts at boundaries; a sliding-window log is exact but stores every timestamp; a sliding-window counter is a good approximation.

Race between servers?

Read-modify-write is a lost update (DDIA Ch 7). Atomic Lua or INCR + EXPIRE inside Redis fixes it.

Redis is down?

Fall back to local limits (limit ÷ N servers). Fail-open vs fail-closed is a product decision per endpoint.

Very hot client?

One key lands on one shard. Give each server a local allowance and sync periodically, trading accuracy for load.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Limit requests per client (user, IP, API key) by configurable rules; Return 429 with Retry-After and rate-limit headers
5–8	Non-functional requirements	Adds < 10ms latency; Accurate across many API servers; Highly available (choose fail-open vs fail-closed); 1M requests/sec total

Time	What you do	Content
8–10	Numbers that drive the design	1M req/s = 1M Redis ops/s; one Redis node does ~100k ops/s, so a **~10–20 shard Redis Cluster** keyed by client. 10M active keys × 100B ≈ **1GB** of memory.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: Redis Cluster → Lua script → Rules config. Justify each addition with a number.
25–40	Deep dives	Which algorithm? · Race between servers? · Redis is down? · Very hot client?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- Counters in Postgres: 1M writes/sec of throwaway data.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Scaling reads · Dealing with contention
- **DDIA:** Ch 7 lost update · Ch 6 shard by key
- **Similar problems:** URL shortener (Bitly) · Business discovery (Yelp) · News feed (Facebook) · News aggregator (Google News)

Business discovery (Yelp)

Difficulty: Medium · **Patterns:** Scaling reads · **DDIA:** Ch 3 geo indexes · Ch 7 atomic updates · Ch 5 read replicas

The trap is over-engineering: the whole dataset fits on one database server.

TL;DR

- Ten million businesses at ~1 KB each is about 10 GB, small enough for one Postgres instance.
- Two-dimensional "near me" queries need a geospatial index (PostGIS, geohash), not two B-tree indexes.
- Keep the average rating correct under concurrent reviews by updating it atomically in the same transaction.

1. Requirements

Functional

1. Search businesses by location + radius, category, text
2. View business details and reviews
3. Leave one review per user per business; see average rating

Non-functional

- Search p99 < 500ms
- 100M DAU, read-heavy
- Ratings can lag slightly (eventual)
- Reviews durable

2. Back-of-envelope numbers

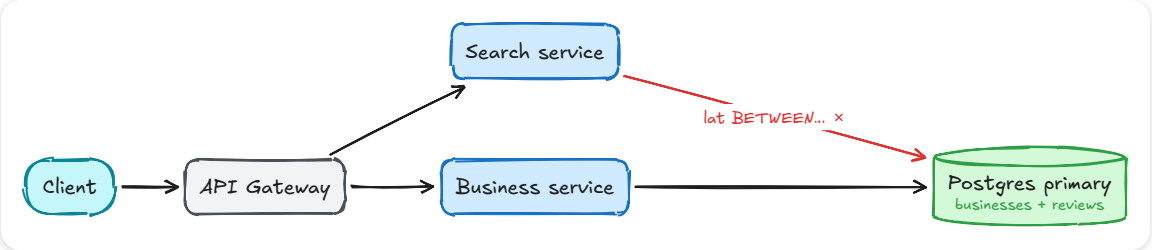
10M businesses × 1KB = **10GB**, which fits in memory. Search: 100M × 5 / 86,400 ≈ **6k/s, ~20k/s peak**. Reviews ~12/s.

3. Core entities and API

```
businesses(id, name, location, category, avg_rating, num_ratings)
reviews(id, business_id, user_id, rating, text, UNIQUE(user_id, business_id))
```

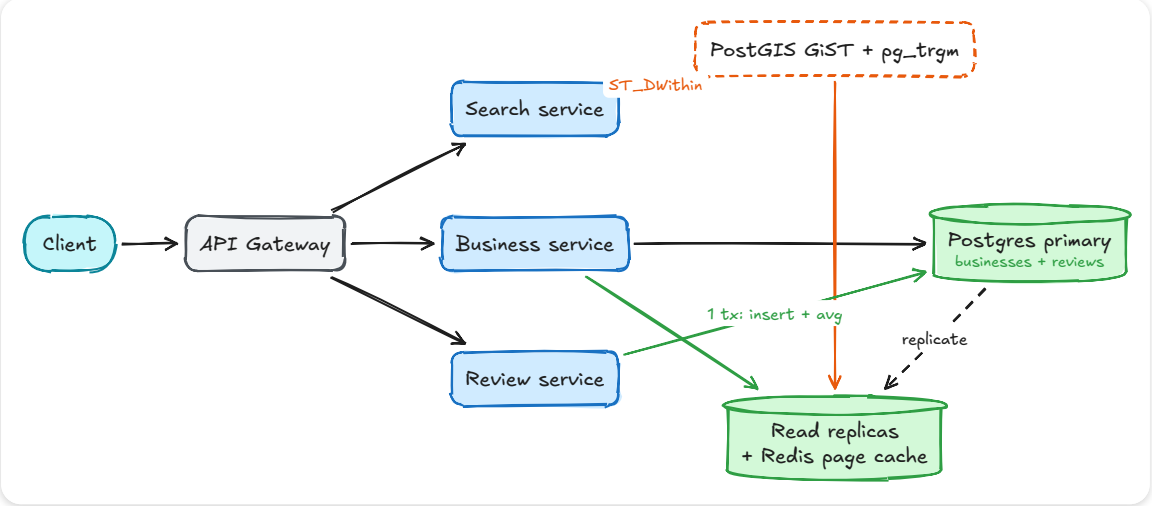
```
GET /businesses?lat&lng&radius&category&q&cursor
GET /businesses/{id}
POST /businesses/{id}/reviews {rating, text}
```

4. v1: the simplest design, and where it breaks



Naive: WHERE lat BETWEEN ... AND lng BETWEEN .... Two 1-D B-tree indexes can't do a 2-D range efficiently.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Naive: WHERE lat BETWEEN ... AND lng BETWEEN .... Two 1-D B-tree indexes can't do a 2-D range efficiently.
2. Add a geospatial index (PostGIS GiST, ST_DWithin) and pg_trgm for name search. **10GB fits one Postgres**, so no extra search cluster yet.

3. ~20k searches/sec: serve reads from replicas and cache hot business pages in Redis. The primary only takes writes.
4. Reviews: a unique (user, business) constraint, and the average updated in the same transaction as the insert, with no lost updates.

6. Deep dives

Which geo index?

Geohash: prefix = area, B-tree friendly; query the cell plus 8 neighbours to handle edges. **Quadtree**: adapts to density. **R-tree / PostGIS**: the production default.

Average rating with concurrent reviews?

Store sum and count; UPDATE ... SET sum=sum+:r, count=count+1 in the review's transaction. That is an atomic operation, so no lost update.

One review per user?

A DB unique constraint, not an application check (check-then-insert races).

When would you add Elasticsearch?

Only if search needs relevance ranking, fuzzy matching, and facets beyond pg_trgm, or QPS outgrows replicas. Then feed it from Postgres via CDC; never dual-write.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Search businesses by location + radius, category, text; View business details and reviews; Leave one review per user per business; see average rating
5–8	Non-functional requirements	Search p99 < 500ms; 100M DAU, read-heavy; Ratings can lag slightly (eventual); Reviews durable
8–10	Numbers that drive the design	10M businesses × 1KB = **10GB**, which fits in memory. Search: 100M × 5 / 86,400 ≈ **6k/s, ~20k/s peak**. Reviews ~12/s.

Time	What you do	Content
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: PostGIS GiST + pg_trgm → Read replicas → Review service. Justify each addition with a number.
25–40	Deep dives	Which geo index? · Average rating with concurrent reviews? · One review per user? · When would you add Elasticsearch?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- A geo-sharded cluster for 10GB of data. One Postgres with PostGIS plus replicas is enough.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Scaling reads
- **DDIA:** Ch 3 geo indexes · Ch 7 atomic updates · Ch 5 read replicas
- **Similar problems:** URL shortener (Bitly) · Rate limiter · News feed (Facebook) · News aggregator (Google News)

News feed (Facebook)

Difficulty: Medium · **Patterns:** Scaling reads · **DDIA:** Ch 1 Twitter load · Ch 11 materialized views · Ch 6 hot keys

The canonical fan-out problem: build a personalised feed for billions of users in a few hundred milliseconds.

TL;DR

- Computing the feed on every read means hundreds of queries per page view, so precompute it when people post.
- Push new post IDs into each follower's feed list in Redis (fan-out on write); a feed read becomes one lookup.
- Accounts with millions of followers break fan-out on write, so pull their posts at read time instead (hybrid).

1. Requirements

Functional

1. Create posts
2. Follow users
3. View feed of followed users' posts, newest first, paginated

Non-functional

- Feed p99 < 500ms
- 2B users; eventual consistency (post appears within ~1 min)
- Handle accounts with millions of followers
- Highly available

2. Back-of-envelope numbers

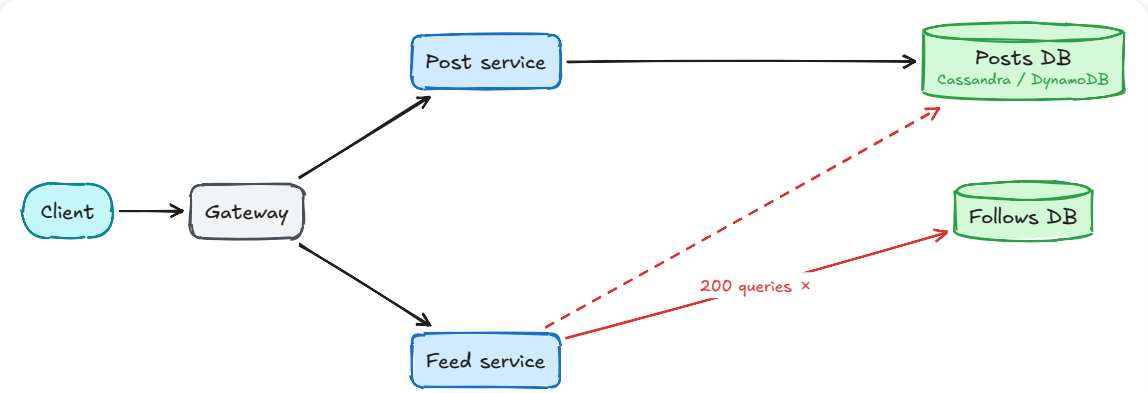
1B DAU × 10 feed loads = **~115k feed reads/s**. ~500M posts/day ≈ 6k/s. With ~200 followers each, fan-out on write ≈ **1.2M feed inserts/s**.

3. Core entities and API

```
posts(id, author_id, content, ts) partitioned by author
follows(follower_id, followee_id) + reverse index
feed:{user} = Redis ZSET(ts, post_id), capped at 500
```

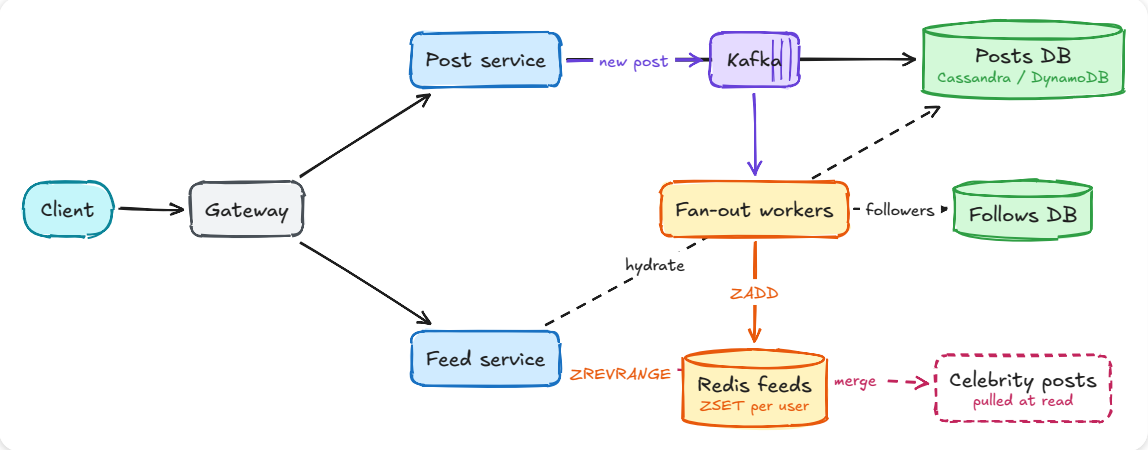
```
POST /posts {content}
PUT /users/{id}/followers
GET /feed?cursor=(ts,post_id)
```

4. v1: the simplest design, and where it breaks



Fan-out on **read**: look up 200 followees, then their posts, on every feed load. At 115k feeds/sec that is millions of queries/sec.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Fan-out on **read**: look up 200 followees, then their posts, on every feed load. At 115k feeds/sec that is millions of queries/sec.

2. Fan-out on **write**: a new post goes to Kafka; workers push its ID into every follower's Redis sorted set. A feed load becomes one ZREVRANGE.
3. Celebrity problem: one post to 100M followers means 100M writes. Hybrid: skip fan-out above ~100k followers and merge their recent posts at read time.
4. Hydrate post IDs into content from the posts store, with a post cache. Cursor pagination on (ts, post_id).

6. Deep dives

Celebrity with 100M followers?

Hybrid. Don't fan out; followers pull celebrity posts at read time and merge. Threshold ~100k–1M followers.

How much memory for feeds?

Only active users, only IDs, capped at 500: 300M × 500 × ~8B ≈ 1.2TB across a Redis cluster. Inactive users get rebuilt on demand.

Pagination?

Cursor = (ts, post_id) of the last item. Offsets shift as new posts arrive.

User follows someone new?

An async job backfills the followee's recent posts into the follower's feed.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Create posts; Follow users; View feed of followed users' posts, newest first, paginated
5–8	Non-functional requirements	Feed p99 < 500ms; 2B users; eventual consistency (post appears within ~1 min); Handle accounts with millions of followers; Highly available

Time	What you do	Content
8–10	Numbers that drive the design	1B DAU × 10 feed loads = **~115k feed reads/s**. ~500M posts/day ≈ 6k/s. With ~200 followers each, fan-out on write ≈ **1.2M feed inserts/s**.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: Kafka → Fan-out workers → Redis feeds → Celebrity posts. Justify each addition with a number.
25–40	Deep dives	Celebrity with 100M followers? · How much memory for feeds? · Pagination? · User follows someone new?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- Fan-out on write for everyone: one celebrity post means 100M writes and workers hours behind.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Scaling reads
- **DDIA:** Ch 1 Twitter load · Ch 11 materialized views · Ch 6 hot keys
- **Similar problems:** URL shortener (Bitly) · Rate limiter · Business discovery (Yelp) · News aggregator (Google News)

News aggregator (Google News)

Difficulty: Easy · **Patterns:** Scaling reads · **DDIA:** Ch 1 load parameters · Ch 10 ingest pipelines

A feed problem where the feed is the same for everyone in a region, which changes everything.

TL;DR

- Writes are tiny (new articles); reads spike hard when big news breaks.
- Precompute one list per region and category at ingest time instead of querying on every request.
- Because the feed is not per-user, a CDN can cache it for a minute and absorb almost all traffic.

1. Requirements

Functional

1. Feed of recent articles from thousands of publishers, by region and category
2. Click through to the publisher; infinite scroll

Non-functional

- Feed p99 < 200ms
- 100M DAU with breaking-news spikes
- New articles visible within ~30 min
- Availability over consistency

2. Back-of-envelope numbers

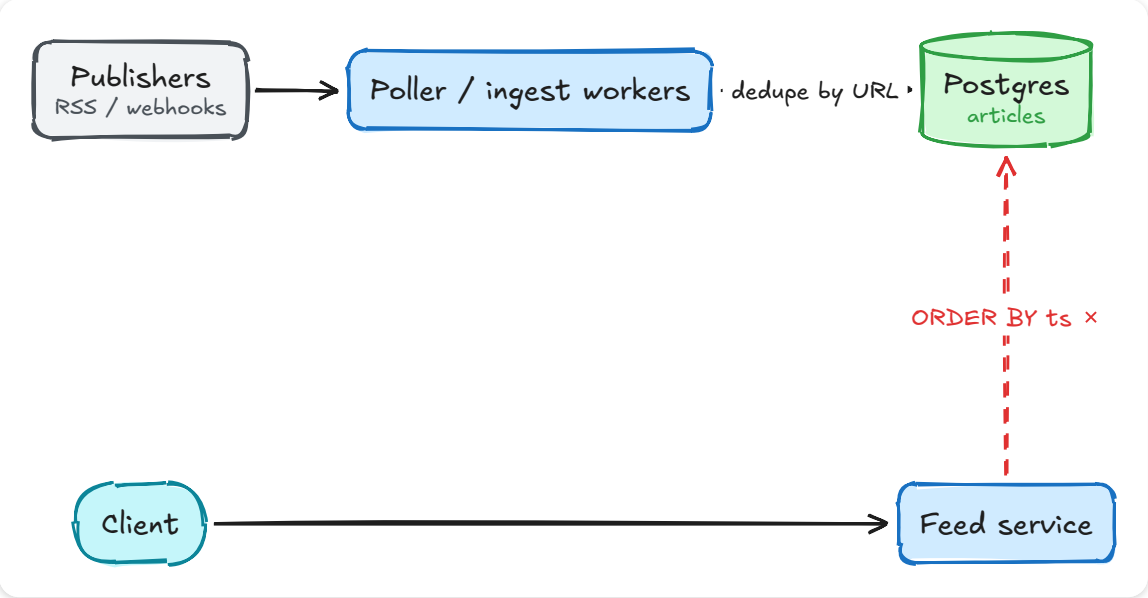
100M × 10 = 1B feed reads/day ≈ **12k/s, 50k+/s spikes**. Writes: ~100k articles/day ≈ 1/s.

3. Core entities and API

```
articles(id, canonical_url UNIQUE, title, summary, thumbnail_url, publisher_id, region, category, published_at)
```

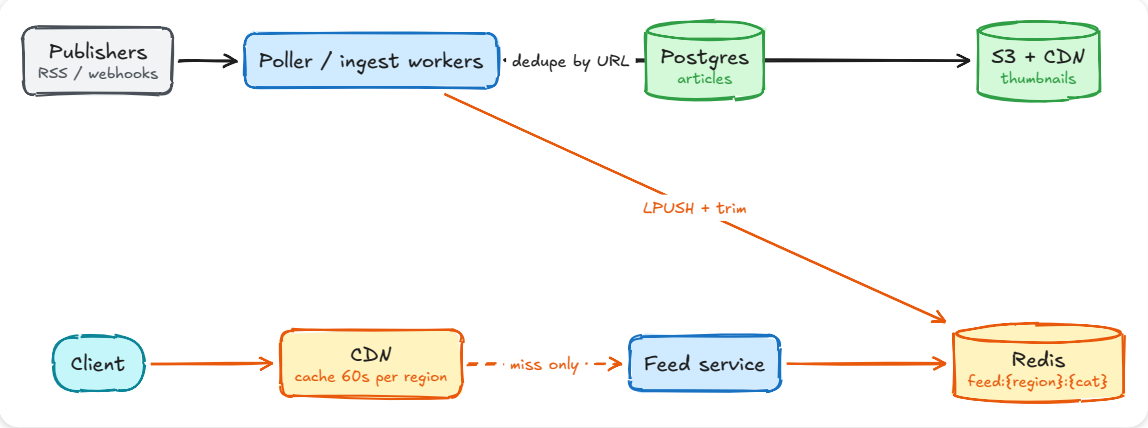
```
GET /feed?region&category&cursor → [{title, summary, thumbnail, url, publisher}]
```

4. v1: the simplest design, and where it breaks



Pull articles from publishers on a schedule and query Postgres for each feed. Fine at low traffic.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Pull articles from publishers on a schedule and query Postgres for each feed. Fine at low traffic.

2. Precompute one list per (region, category) in Redis at ingest time. Reads become a single list read.
3. The feed is **not personalized**, so a CDN can cache it for 60s per region. A 50k/s spike becomes a few origin requests.
4. Thumbnails are copied to S3 and served by CDN; publishers that support webhooks push instead of being polled.

6. Deep dives

Why can a CDN cache the feed?

Everyone in a region gets the same response. 60s of staleness is fine for news (AP).

Polling cost vs freshness?

Adaptive polling: busy publishers every few minutes, quiet ones hourly. Use webhooks where offered.

Duplicate stories?

Unique canonical URL; near-duplicates clustered with SimHash/MinHash on title and body.

Pagination?

Cursor on (published_at, id). Offsets break when new articles arrive.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Feed of recent articles from thousands of publishers, by region and category; Click through to the publisher; infinite scroll
5–8	Non-functional requirements	Feed p99 < 200ms; 100M DAU with breaking-news spikes; New articles visible within ~30 min; Availability over consistency

Time	What you do	Content
8–10	Numbers that drive the design	100M × 10 = 1B feed reads/day ≈ **12k/s, 50k+/s spikes**. Writes: ~100k articles/day ≈ 1/s.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: Redis → CDN → S3 + CDN. Justify each addition with a number.
25–40	Deep dives	Why can a CDN cache the feed? · Polling cost vs freshness? · Duplicate stories? · Pagination?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- Per-user fan-out like a social feed when the feed is per region and category: build it once per region and let the CDN cache it.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Scaling reads
- **DDIA:** Ch 1 load parameters · Ch 10 ingest pipelines
- **Similar problems:** URL shortener (Bitly) · Rate limiter · Business discovery (Yelp) · News feed (Facebook)

Local delivery (Gopuff)

Difficulty: Easy · **Patterns:** Scaling reads · **DDIA:** Ch 7 conditional updates · Ch 5 replica lag

It looks like an ordering system, but most of the load is people checking what is available near them.

TL;DR

- Browsing availability is far more frequent than ordering, so cache it and serve it from replicas.
- Find nearby warehouses with an in-memory list and a cheap distance filter before calling any travel-time API.
- Prevent overselling with conditional stock decrements for every item in one transaction.

1. Requirements

Functional

1. Check availability of items deliverable to a location (DCs within 1 hour)
2. Order multiple items; never sell stock you don't have

Non-functional

- Availability p99 < 100ms
- Orders strongly consistent (no overselling)
- 10k distribution centres (DCs), ~100k SKUs
- Browse load ≫ order load

2. Back-of-envelope numbers

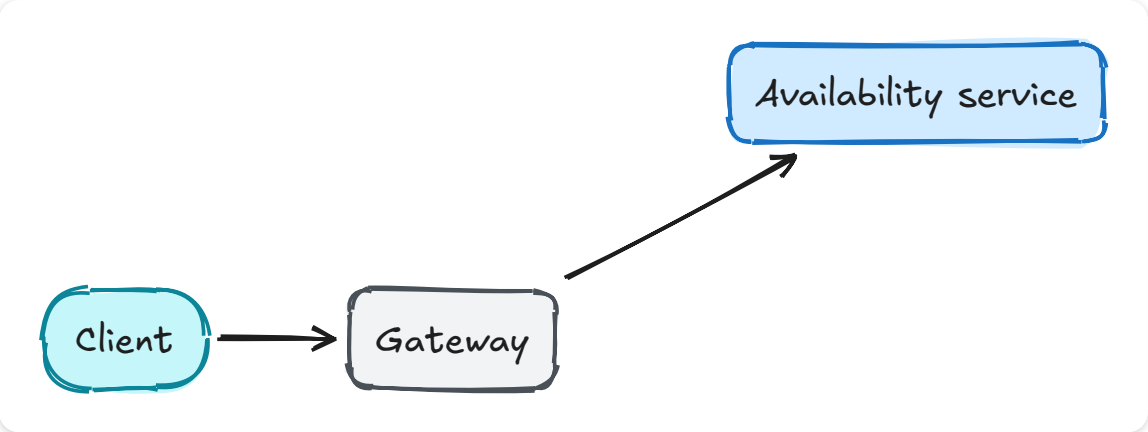
Browse ~10M × 10 / 86,400 ≈ **1.2k/s, ~10k/s peak**. Orders ~1M/day ≈ 12/s. The DC list (10k rows) fits in memory.

3. Core entities and API

```
dcs(id, lat, lng, region)
inventory(dc_id, item_id, qty) PK(dc_id, item_id)
orders, order_items
```

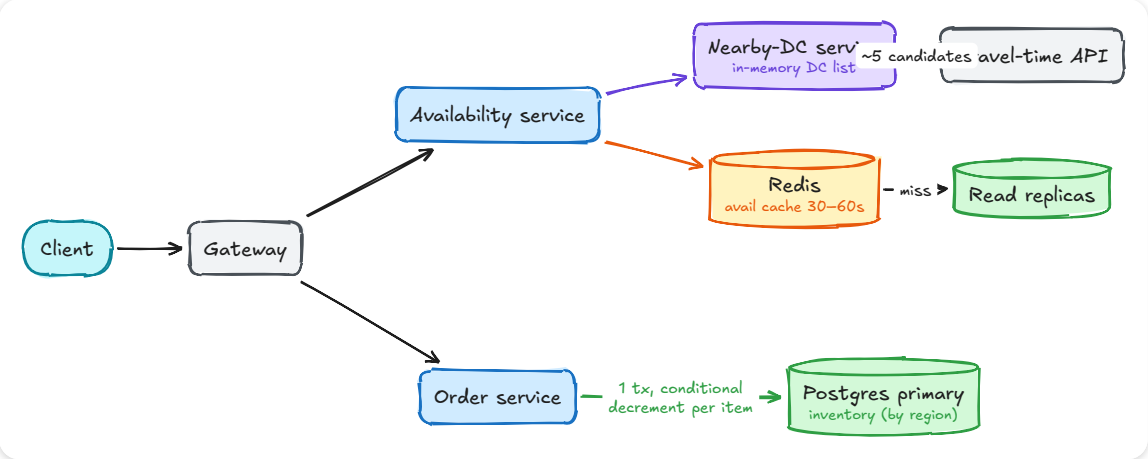
```
GET /availability?lat&lng&item_ids
POST /orders {items:[{id, qty}], lat, lng}
```

4. v1: the simplest design, and where it breaks



Naive: call a travel-time API for every DC. 10k calls per page view is far too slow.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Naive: call a travel-time API for every DC. 10k calls per page view is far too slow.
2. Keep the small DC list in memory, prefilter by straight-line distance, then ask travel time for only a handful of candidates.

3. Browse reads come from a short-TTL cache and read replicas. Slightly stale is fine here (AP).
4. Ordering hits the primary: one transaction with UPDATE … SET qty = qty − n WHERE qty ≥ n per item. Any zero-row update means ROLLBACK, so there is no overselling (CP).

6. Deep dives

How do you prevent overselling?

A conditional decrement per item inside one transaction; row locks serialize concurrent buyers. 0 rows updated means rollback and out of stock.

Stale availability vs orders?

Different consistency per path: browse is cached and eventually consistent; checkout reads and writes the primary.

Scale orders?

Partition inventory by region; an order only touches nearby DCs, so there are no cross-shard transactions.

Hot item (flash sale)?

Row contention on one inventory row: queue orders per item, or split stock into sub-buckets.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Check availability of items deliverable to a location (DCs within 1 hour); Order multiple items; never sell stock you don't have
5–8	Non-functional requirements	Availability p99 < 100ms; Orders strongly consistent (no overselling); 10k distribution centres (DCs), ~100k SKUs; Browse load ≫ order load

Time	What you do	Content
8–10	Numbers that drive the design	Browse ~10M × 10 / 86,400 ≈ **1.2k/s, ~10k/s peak**. Orders ~1M/day ≈ 12/s. The DC list (10k rows) fits in memory.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: Nearby-DC service → Travel-time API → Redis → Read replicas → Order service → Postgres primary. Justify each addition with a number.
25–40	Deep dives	How do you prevent overselling? · Stale availability vs orders? · Scale orders? · Hot item (flash sale)?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- Redis as the inventory source of truth, with no multi-item transaction or durability.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Scaling reads
- **DDIA:** Ch 7 conditional updates · Ch 5 replica lag
- **Similar problems:** URL shortener (Bitly) · Rate limiter · Business discovery (Yelp) · News feed (Facebook)

Dating app (Tinder)

Difficulty: Medium · **Patterns:** Scaling reads · **DDIA:** Ch 7 write skew · Ch 3 Bloom filters, LSM

Swipes are a firehose of writes, and two people swiping right at the same moment must never miss their match.

TL;DR

- Precompute each user's stack of nearby candidates and hide already-swiped profiles with a Bloom filter.
- Swipes are write-heavy and append-only, a good fit for an LSM store like Cassandra.
- Detect matches atomically on one key per pair of users, so simultaneous swipes can't both miss each other.

1. Requirements

Functional

1. Profile with preferences (age, distance)
2. Get a stack of nearby candidates
3. Swipe left/right; get notified on a mutual match

Non-functional

- Stack load p99 < 300ms
- No missed matches (strong consistency on swipes)
- Don't re-show swiped profiles
- 20M DAU × 100 swipes

2. Back-of-envelope numbers

2B swipes/day ≈ **25k/s avg, ~100k/s peak**, which is write-heavy. Stacks precomputed: 20M × 200 IDs × 8B ≈ 32GB.

3. Core entities and API

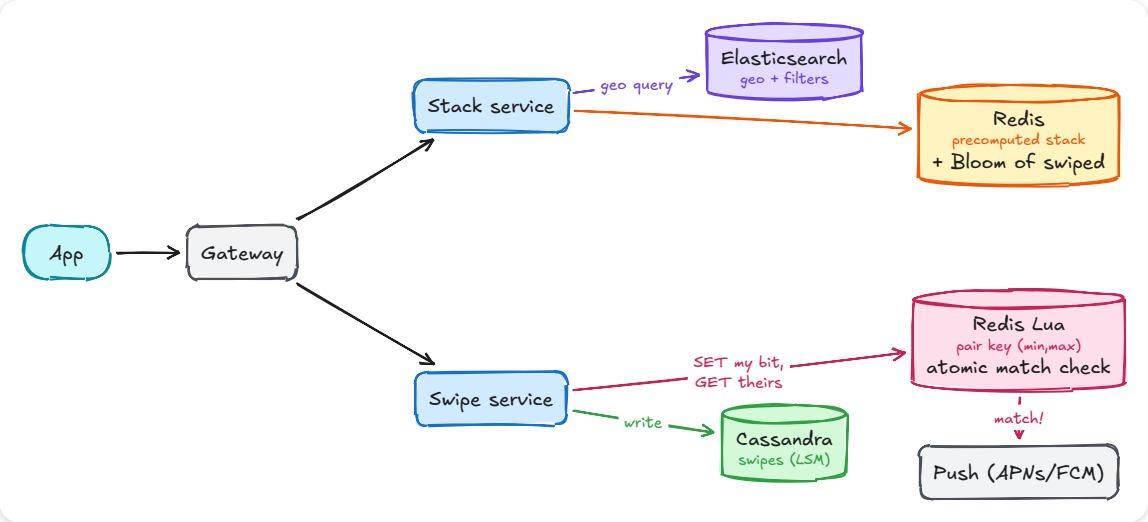
```
profiles
swipes(swiper_id, swipee_id, decision) partitioned by swiper
matches(a, b, ts)
```

```
GET /feed?lat&lng → [profiles]
POST /swipes {target_id, decision}
push: match notification
```

4. v1: the simplest design, and where it breaks

Build each stack with a geo + preferences query in Elasticsearch. That works, but is slow on every open.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Build each stack with a geo + preferences query in Elasticsearch. That works, but is slow on every open.
2. Precompute the next ~200 candidates per user in Redis and refresh in the background. A per-user **Bloom filter** hides profiles already swiped.
3. Swipes are write-heavy (100k/s) and append-only, so an LSM store like Cassandra fits.
4. Match race: A and B swipe right at the same moment, and each check misses the other. Fix: one key per pair (min, max) and a Lua script that sets my decision and reads theirs atomically.

6. Deep dives

Both swipe at once: why can a match be missed?

Two writes followed by two reads: each read runs before the other's write lands. That is write skew (Ch 7). An atomic op on a single pair key makes exactly one side see "both".

Hide already-swiped profiles?

A Bloom filter per user. A false positive skips an unseen profile, which is acceptable; no false negatives.

Location changes?

Invalidate and recompute the stack when the user moves more than X km.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Profile with preferences (age, distance); Get a stack of nearby candidates; Swipe left/right; get notified on a mutual match
5–8	Non-functional requirements	Stack load p99 < 300ms; No missed matches (strong consistency on swipes); Don't re-show swiped profiles; 20M DAU × 100 swipes
8–10	Numbers that drive the design	2B swipes/day ≈ **25k/s avg, ~100k/s peak**, which is write-heavy. Stacks precomputed: 20M × 200 IDs × 8B ≈ 32GB.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: Redis → Swipe service → Cassandra → Redis Lua → Push (APNs/FCM). Justify each addition with a number.
25–40	Deep dives	Both swipe at once: why can a match be missed? · Hide already-swiped profiles? · Location changes?

Time	What you do	Content
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- Separate "write swipe" and "check other swipe" calls without atomicity, so matches are lost.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Scaling reads
- **DDIA:** Ch 7 write skew · Ch 3 Bloom filters, LSM
- **Similar problems:** URL shortener (Bitly) · Rate limiter · Business discovery (Yelp) · News feed (Facebook)

Distributed cache

Difficulty: Medium · **Patterns:** Scaling reads · Long-running tasks · **DDIA:** Ch 6 consistent hashing · Ch 5 async replication

Design Redis itself: data structures inside one node, then distribution across many.

TL;DR

- One node is a hash map plus a doubly linked list for O(1) least-recently-used eviction.
- Spread keys with consistent hashing and virtual nodes, so adding a node moves only a small share of keys.
- Replicate asynchronously for failover, and spread hot keys over several nodes.

1. Requirements

Functional

1. get(key), set(key, value, ttl), delete(key)
2. LRU eviction when full

Non-functional

- p99 ~1ms
- ~1TB data, 100k+ ops/s
- Survive node failure
- Eventual consistency acceptable

2. Back-of-envelope numbers

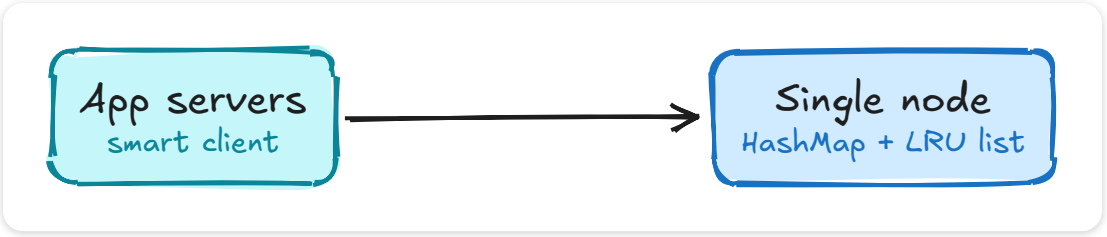
1TB ÷ ~32GB usable per node ≈ **32+ nodes** (+ replicas). Each node handles ≫ 100k ops/s, so memory, not CPU, sets the node count.

3. Core entities and API

```
per node: HashMap key → node in a doubly linked list (LRU order) + expiry
```

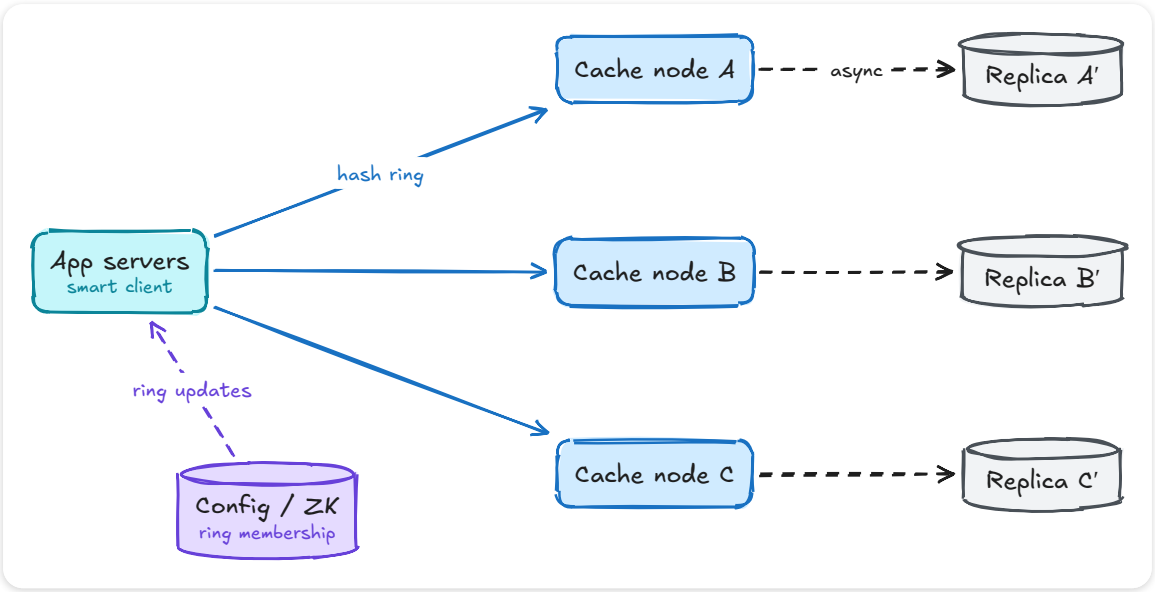
```
GET /cache/{key}
PUT /cache/{key} {value, ttl}
DELETE /cache/{key}
```

4. v1: the simplest design, and where it breaks



One node: a HashMap for O(1) lookup plus a doubly linked list for LRU order (move to head on access, evict the tail). TTL by lazy check plus a background sweep.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. One node: a HashMap for O(1) lookup plus a doubly linked list for LRU order (move to head on access, evict the tail). TTL by lazy check plus a background sweep.
2. Shard across nodes with **consistent hashing** (with virtual nodes). The client computes the owner directly, so there is no extra hop.
3. Each primary replicates asynchronously to a replica; on failure the replica is promoted. Some recent writes may be lost, which is acceptable for a cache.

4. Ring membership lives in a config service; clients watch it. Hot keys get replicated across several nodes or a short-TTL local cache.

6. Deep dives

LRU in O(1)?

HashMap → linked-list node; get = move to head; evict = remove tail.

Add a node?

Consistent hashing moves ~1/N of keys; vnodes keep load even; warm gradually.

Hot key?

Copies key#0..k on several nodes with random reads, or a client-side L1 cache.

Why not hash % N?

Every scale event remaps ~all keys, and the cold cache stampedes the database.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	get(key), set(key, value, ttl), delete(key); LRU eviction when full
5–8	Non-functional requirements	p99 ~1ms; ~1TB data, 100k+ ops/s; Survive node failure; Eventual consistency acceptable
8–10	Numbers that drive the design	1TB ÷ ~32GB usable per node ≈ **32+ nodes** (+ replicas). Each node handles ≫ 100k ops/s, so memory, not CPU, sets the node count.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: Cache node A → Cache node B → Cache node C → Replica A′ → Replica B′ → Replica C′ → Config / ZK. Justify each addition with a number.

Time	What you do	Content
25–40	Deep dives	LRU in O(1)? · Add a node? · Hot key? · Why not hash % N?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- hash(key) % N sharding.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Scaling reads · Long-running tasks
- **DDIA:** Ch 6 consistent hashing · Ch 5 async replication
- **Similar problems:** URL shortener (Bitly) · Rate limiter · Business discovery (Yelp) · News feed (Facebook)

Top-K videos (YouTube)

Difficulty: Hard · **Patterns:** Scaling writes · **DDIA:** Ch 11 windows, partitioned streams · Ch 6 partition by key

The design falls straight out of one number: hundreds of thousands of views per second.

TL;DR

- A counter row per video in a database can't take ~800k updates per second.
- Stream views through Kafka partitioned by video ID; each consumer counts its videos in memory and keeps a local top-K.
- Merge the per-partition top-K lists; the result is exact because each video lives in exactly one partition.

1. Requirements

Functional

1. Top K most-viewed videos for the last hour, day, month, and all time

Non-functional

- Query latency < 100ms
- Freshness ~1 minute
- Accurate counts (billing-grade not required, but no big drift)
- Massive view volume

2. Back-of-envelope numbers

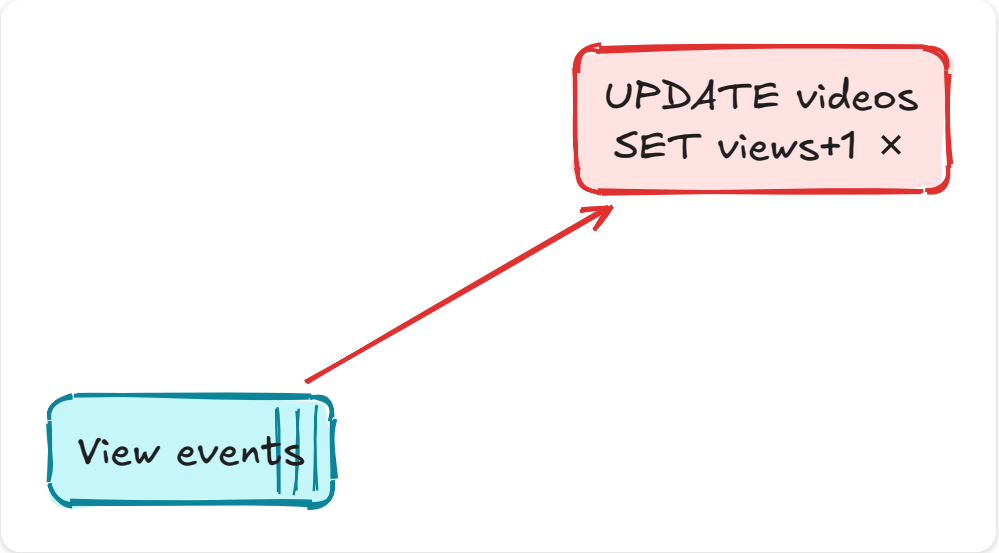
70B views/day ≈ **~800k views/s**. 4B videos × 8B counters ≈ 32GB of state, spread over partitions.

3. Core entities and API

```
view events (video_id, ts) → per-minute counts per video → per-window top-K heap
```

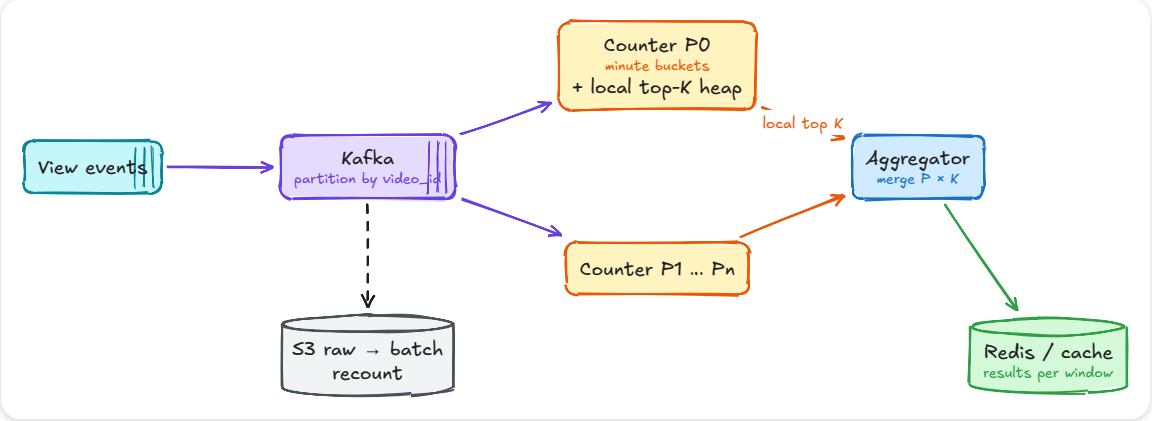
```
GET /topk?window=hour|day|month|all&k=100
```

4. v1: the simplest design, and where it breaks



Naive: increment a counter row per view and ORDER BY views. That is 800k UPDATEs/sec on hot rows, then a full sort.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Naive: increment a counter row per view and ORDER BY views. That is 800k UPDATEs/sec on hot rows, then a full sort.
2. Stream views into Kafka partitioned by **video_id**, so every view of a video lands on the same consumer.

3. Each consumer counts in memory in per-minute buckets and keeps a local top-K heap. The aggregator merges P×K candidates, which is exact because each video lives in exactly one partition.
4. Windows: hour = sum of the last 60 minute-buckets (subtract the expiring one). Results are cached; raw events go to S3 for batch correction.

6. Deep dives

Why is the merge exact?

A video's full count lives in one partition, so its global rank can only come from its local rank. The global top K ⊆ union of local top Ks.

Sliding windows?

A ring buffer of minute counts per video; for months use hourly buckets. Rolling sums avoid rescanning.

Very hot video?

Pre-aggregate on producers (send counts per second, not individual views).

Approximate is OK?

Count-Min Sketch + heap uses fixed memory and is good for trending; exact needs partitioned counting.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Top K most-viewed videos for the last hour, day, month, and all time
5–8	Non-functional requirements	Query latency < 100ms; Freshness ~1 minute; Accurate counts (billing-grade not required, but no big drift); Massive view volume
8–10	Numbers that drive the design	70B views/day ≈ **~800k views/s**. 4B videos × 8B counters ≈ 32GB of state, spread over partitions.

Time	What you do	Content
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: Kafka → Counter P0 → Counter P1 … Pn → Aggregator → Redis / cache → S3 raw → batch. Justify each addition with a number.
25–40	Deep dives	Why is the merge exact? · Sliding windows? · Very hot video? · Approximate is OK?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- A counter row per video in the DB: 800k writes/sec on hot rows.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Scaling writes
- **DDIA:** Ch 11 windows, partitioned streams · Ch 6 partition by key
- **Similar problems:** Ad click aggregator · Web crawler · Metrics monitoring (Datadog) · Fitness tracking (Strava)

Ad click aggregator

Difficulty: Hard · **Patterns:** Scaling writes · **DDIA:** Ch 11 windows, exactly-once · Ch 12 Lambda · Ch 3 column stores

Clicks are money, so the numbers must be fast for dashboards and exact for billing.

TL;DR

- Redirect immediately and publish the click to Kafka; never aggregate raw rows at query time.
- A stream processor counts clicks per ad per minute and writes them to a column store for fast queries.
- Deduplicate by a signed impression ID, and recount from raw logs in a nightly batch job for billing accuracy.

1. Requirements

Functional

1. Click on an ad redirects the user to the advertiser
2. Advertisers query click metrics over time (minute granularity), with filters

Non-functional

- 10k clicks/s peak, 100M/day
- Never lose or double-count clicks (it is billing)
- Metrics queries < 1s
- Freshness ~1 minute

2. Back-of-envelope numbers

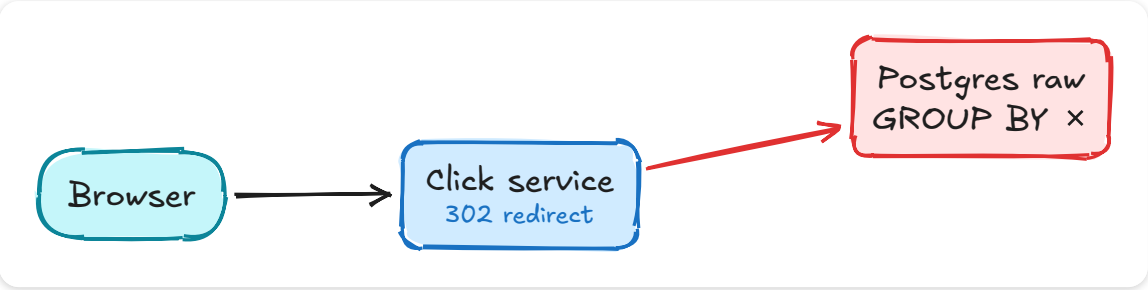
100M clicks × 100B ≈ **10GB/day raw**. Aggregated rows: ads × minutes, which is small. A 10k/s peak is easy for Kafka.

3. Core entities and API

```
click(impression_id, ad_id, ts, user) → agg(ad_id, minute, count)
```

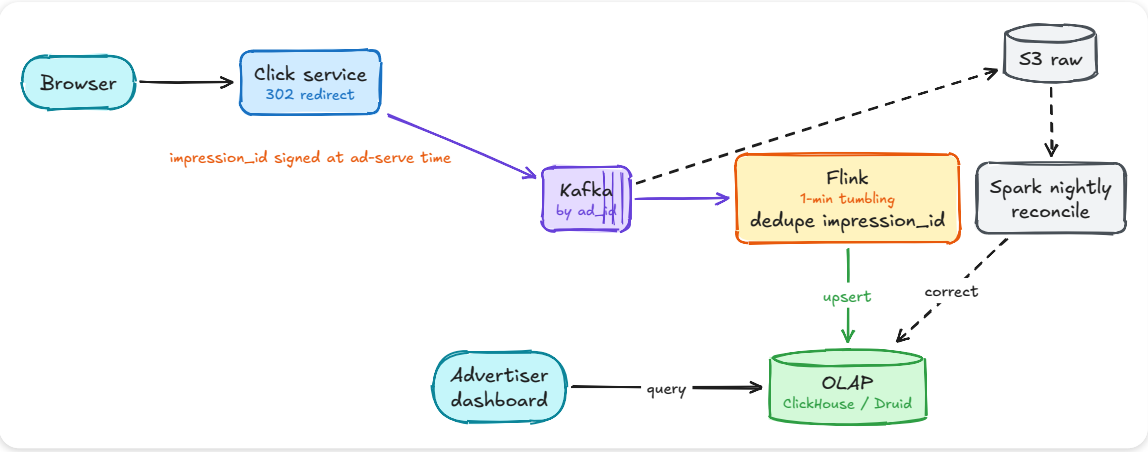
```
GET /click?ad_id&impression_id → 302 advertiser_url
GET /metrics?ad_id&from&to&granularity
```

4. v1: the simplest design, and where it breaks



Naive: insert each click into Postgres and GROUP BY at query time. Slow queries, and writes compete with reads.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Naive: insert each click into Postgres and GROUP BY at query time. Slow queries, and writes compete with reads.
2. Click service redirects immediately and publishes to Kafka. Flink aggregates per (ad, minute) on event time and upserts into an OLAP column store.
3. Idempotency: every impression gets a **signed unique ID** when the ad is served; Flink dedupes on it, so double-clicks and retries count once and forged clicks are rejected.

4. Lambda-style safety net: raw clicks to S3; a nightly batch recount corrects aggregates, because billing must be exact.

6. Deep dives

Flink crashes?

Checkpoints + Kafka offsets give replay; the sink upserts (ad_id, minute), so replays overwrite instead of double-counting.

Hot ad (Super Bowl)?

Key on ad_id:random(0..N) to spread across partitions, then aggregate twice.

Why also batch?

Streams give fast numbers; batch from immutable raw data gives exact billing numbers.

Why OLAP?

Column store and pre-aggregates make time-range queries over billions of rows sub-second.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Click on an ad redirects the user to the advertiser; Advertisers query click metrics over time (minute granularity), with filters
5–8	Non-functional requirements	10k clicks/s peak, 100M/day; Never lose or double-count clicks (it is billing); Metrics queries < 1s; Freshness ~1 minute
8–10	Numbers that drive the design	100M clicks × 100B ≈ **10GB/day raw**. Aggregated rows: ads × minutes, which is small. A 10k/s peak is easy for Kafka.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.

Time	What you do	Content
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: Kafka → Flink → OLAP → Advertiser → S3 raw → Spark nightly. Justify each addition with a number.
25–40	Deep dives	Flink crashes? · Hot ad (Super Bowl)? · Why also batch? · Why OLAP?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- Aggregating raw rows at query time in the OLTP database.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Scaling writes
- **DDIA:** Ch 11 windows, exactly-once · Ch 12 Lambda · Ch 3 column stores
- **Similar problems:** Top-K videos (YouTube) · Web crawler · Metrics monitoring (Datadog) · Fitness tracking (Strava)

Web crawler

Difficulty: Hard · **Patterns:** Scaling writes · **DDIA:** Ch 10 batch pipelines · Ch 3 Bloom filters · Ch 11 queues

Ten billion pages in five days is a pipeline problem, constrained by politeness and network bandwidth.

TL;DR

- About 23k pages per second; size the fleet by network bandwidth, not CPU.
- Split fetching and parsing into separate stages with queues and stored raw pages, so each scales and retries on its own.
- Respect per-domain rate limits and robots.txt, and skip already-seen URLs and duplicate content.

1. Requirements

Functional

1. Crawl from seed URLs, fetch pages, extract text, follow links
2. Store text for LLM training: 10B pages in 5 days

Non-functional

- Politeness: robots.txt, per-domain rate limits
- Fault tolerant: resume without losing progress
- No duplicate URLs or content
- Efficient (network-bound)

2. Back-of-envelope numbers

10B ÷ 5 days ≈ **23k pages/s**. × ~2MB ≈ 46GB/s, so size the fleet by **network bandwidth**, not CPU. Raw storage ~20PB → S3.

3. Core entities and API

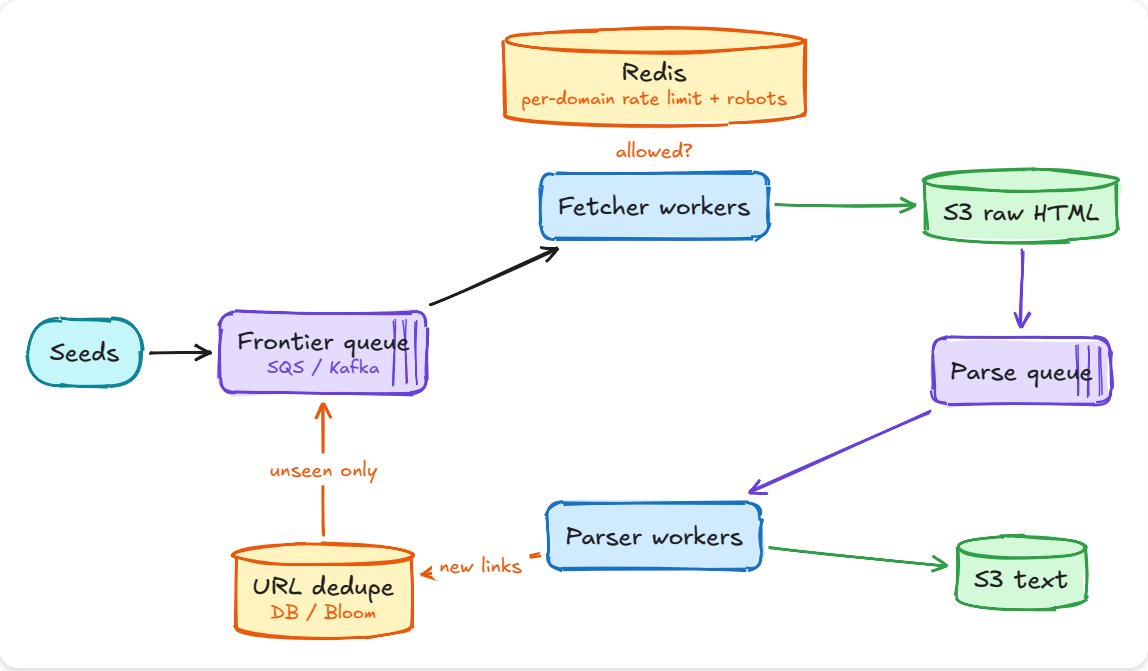
```
urls(url_hash PK, url, status, last_crawled)
domains(domain, robots, crawl_delay, next_allowed_at)
```

```
internal: frontier queue messages {url, depth}
S3: raw/{url_hash}, text/{url_hash}
```

4. v1: the simplest design, and where it breaks

Start with a frontier queue of URLs and fetcher workers pulling from it.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Start with a frontier queue of URLs and fetcher workers pulling from it.
2. Split **fetch** (network-bound) from **parse** (CPU-bound) with S3 + a second queue. Each scales independently, and parsing can be re-run from stored HTML.
3. Politeness: per-domain token buckets and cached robots.txt in Redis. Not allowed yet? Re-queue with a delay (visibility timeout).
4. Extracted links pass a dedupe check (URL hash in DB or Bloom filter) before entering the frontier. Content hashes catch mirrored pages.

6. Deep dives

Why separate stages?

Different bottlenecks, independent scaling and retries, and you can re-parse 20PB without re-crawling.

Crash mid-crawl?

A message is acked only after the raw page is in S3: at-least-once, idempotent by URL hash.

Crawler traps?

Max depth per domain, URL normalization, content-hash dedupe.

DNS bottleneck?

Cache DNS per worker; multiple resolvers.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Crawl from seed URLs, fetch pages, extract text, follow links; Store text for LLM training: 10B pages in 5 days
5–8	Non-functional requirements	Politeness: robots.txt, per-domain rate limits; Fault tolerant: resume without losing progress; No duplicate URLs or content; Efficient (network-bound)
8–10	Numbers that drive the design	10B ÷ 5 days ≈ **23k pages/s**. × ~2MB ≈ 46GB/s, so size the fleet by **network bandwidth**, not CPU. Raw storage ~20PB → S3.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: S3 raw HTML → Parse queue → Parser workers → S3 text → Redis → URL dedupe. Justify each addition with a number.
25–40	Deep dives	Why separate stages? · Crash mid-crawl? · Crawler traps? · DNS bottleneck?

Time	What you do	Content
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- One monolithic fetch→parse loop: a slow parser stalls fetching and a crash loses in-flight work.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Scaling writes
- **DDIA:** Ch 10 batch pipelines · Ch 3 Bloom filters · Ch 11 queues
- **Similar problems:** Top-K videos (YouTube) · Ad click aggregator · Metrics monitoring (Datadog) · Fitness tracking (Strava)

Metrics monitoring (Datadog)

Difficulty: Hard · **Patterns:** Scaling writes · **DDIA:** Ch 3 LSM, compression · Ch 11 stream alerting

Millions of data points per second arrive, and alerts must fire within a minute.

TL;DR

- Buffer the write firehose through Kafka into a time-series database built for appends and compression.
- Downsample old data and move it to cheap storage; dashboards read rollups.
- Evaluate alert rules on the stream itself instead of polling the database.

1. Requirements

Functional

1. Ingest metrics from 500k servers
2. Query dashboards over time ranges
3. Alert rules with notifications

Non-functional

- 5M datapoints/s ingest
- Recent-data queries < 1s
- Alerts within ~1 minute
- Late/out-of-order tolerant, durable

2. Back-of-envelope numbers

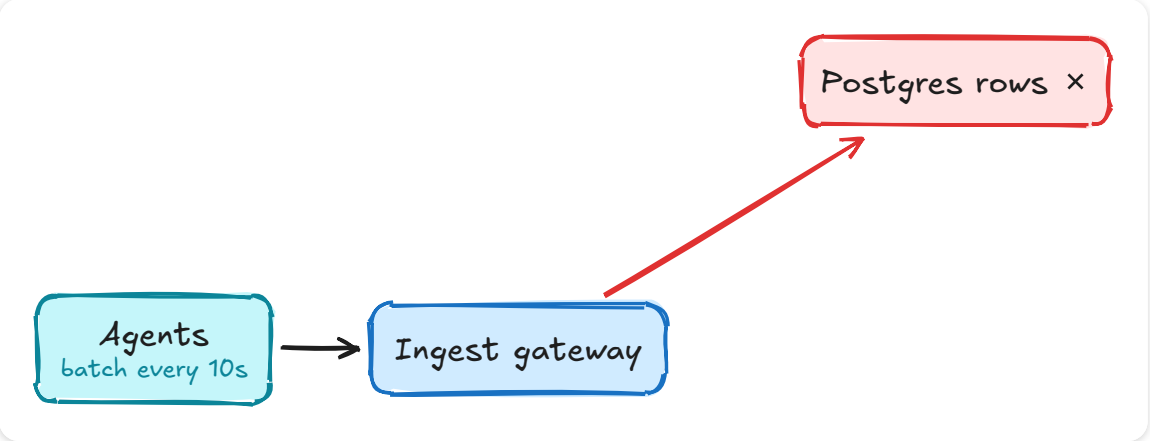
500k hosts × 100 metrics ÷ 10s = **5M points/s**. Gorilla compression ≈ 1.4B/point → ~600GB/day; downsample after 7 days.

3. Core entities and API

```
series(metric, tags) → (ts, value) blocks
rollups 1m / 1h
alert_rules
```

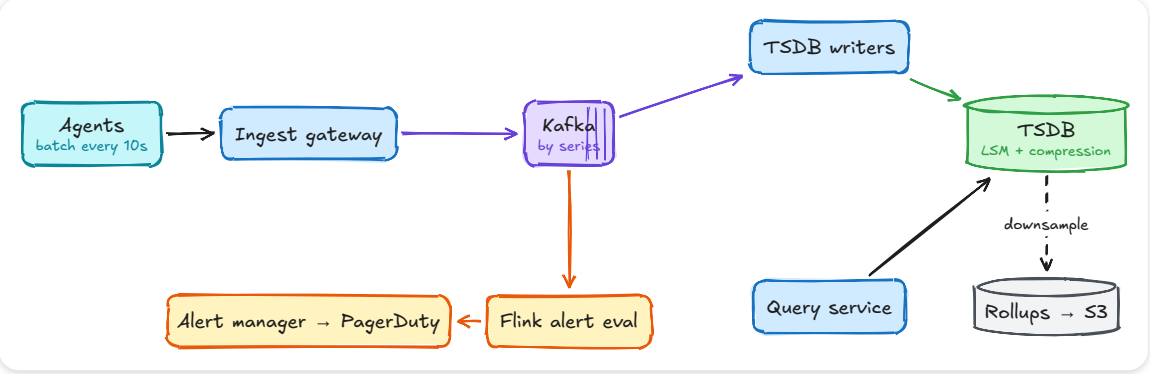
```
POST /ingest (batched points from agents)
GET /query?metric&tags&from&to&step
POST /alerts {rule, threshold, channel}
```

4. v1: the simplest design, and where it breaks



Naive: one Postgres row per datapoint with a B-tree index. It falls over around ~50k inserts/sec.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Naive: one Postgres row per datapoint with a B-tree index. It falls over around ~50k inserts/sec.
2. Buffer through Kafka into a **time-series DB**: append-only LSM writes, series-by-time blocks, delta-of-delta + XOR compression.

3. Queries hit recent blocks; older data is downsampled (1s → 1m → 1h) and tiered to S3. Dashboards use rollups.
4. Alerts are evaluated **on the stream** (Flink windows over Kafka), not by polling the TSDB with thousands of queries.

6. Deep dives

Why a TSDB?

Time-ordered appends suit LSM; compression gives ~10×; queries are range scans per series.

High cardinality?

A user_id tag explodes series count. Cap or reject high-cardinality labels.

Late data?

Event-time windows with watermarks; allowed lateness for corrections.

Alert flapping?

Require N consecutive windows; dedupe and group in the alert manager.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Ingest metrics from 500k servers; Query dashboards over time ranges; Alert rules with notifications
5–8	Non-functional requirements	5M datapoints/s ingest; Recent-data queries < 1s; Alerts within ~1 minute; Late/out-of-order tolerant, durable
8–10	Numbers that drive the design	500k hosts × 100 metrics ÷ 10s = **5M points/s**. Gorilla compression ≈ 1.4B/point → ~600GB/day; downsample after 7 days.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.

Time	What you do	Content
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: Kafka → TSDB writers → TSDB → Query service → Rollups → S3 → Flink alert eval → Alert manager → PagerDuty. Justify each addition with a number.
25–40	Deep dives	Why a TSDB? · High cardinality? · Late data? · Alert flapping?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- Row-per-point in Postgres, and alerting by polling queries.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Scaling writes
- **DDIA:** Ch 3 LSM, compression · Ch 11 stream alerting
- **Similar problems:** Top-K videos (YouTube) · Ad click aggregator · Web crawler · Fitness tracking (Strava)

Fitness tracking (Strava)

Difficulty: Medium · **Patterns:** Scaling writes · **DDIA:** Ch 11 event time · Ch 12 idempotent upload

The key insight is where the work happens: on the phone, not the server.

TL;DR

- Record GPS on the device and upload the whole activity when it ends; no need to stream every point.
- Store the track in object storage and the metadata in a database, with an idempotent, resumable upload.
- Compute stats and leaderboards asynchronously from a queue; leaderboards are Redis sorted sets.

1. Requirements

Functional

1. Start/pause/stop recording an activity (GPS track)
2. View activity stats and maps
3. Share with friends; see friends' activities; leaderboards

Non-functional

- Works offline; never lose an activity
- 10M DAU
- Feed eventually consistent
- Uploads resumable

2. Back-of-envelope numbers

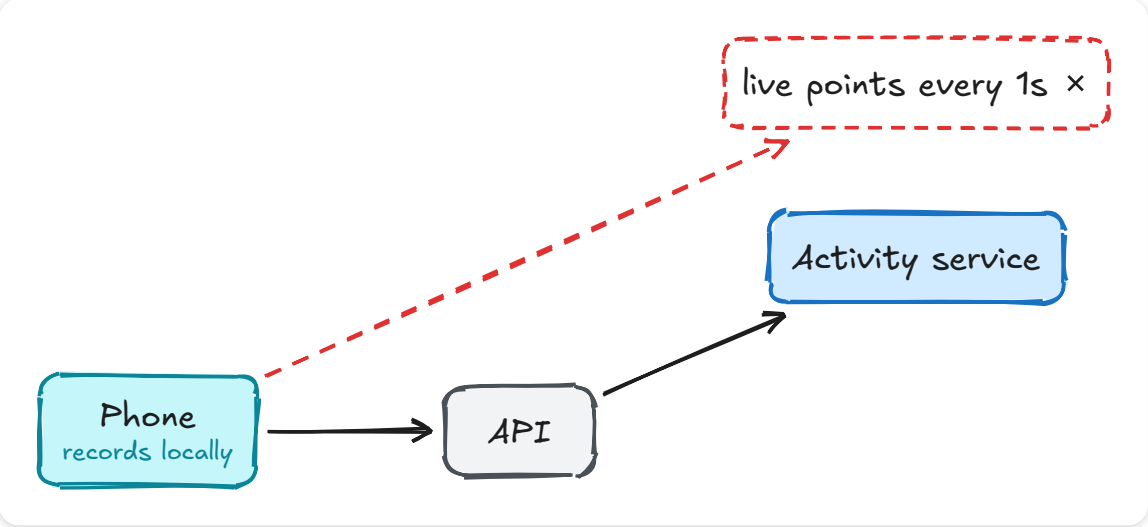
10M activities/day; 1hr at 1 point/5s = 720 points ≈ 15–100KB per track → **~1TB/day** of track blobs.

3. Core entities and API

```
activities(id, user_id, type, start, distance, duration, track_s3_key)
follows
leaderboard:{segment} ZSET
```

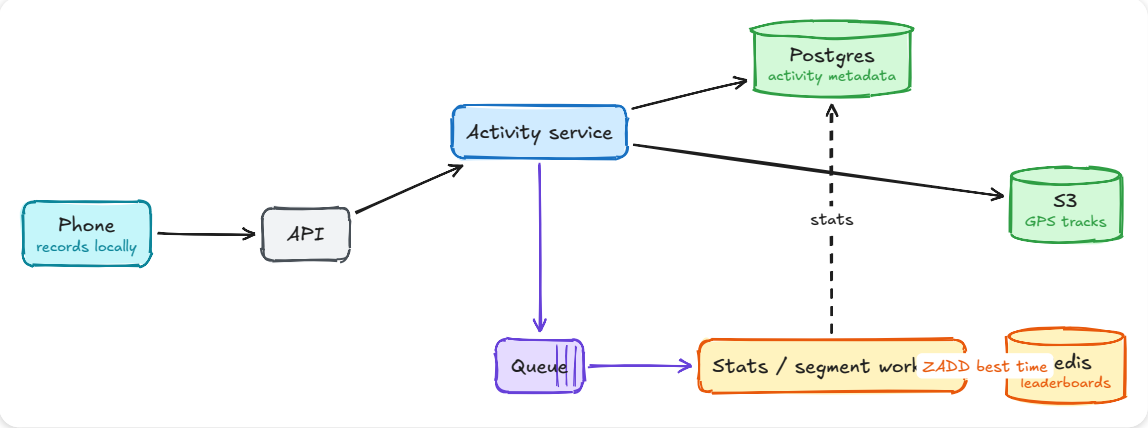
```
POST /activities (complete track, idempotency key)
GET /activities/{id}
GET /feed?cursor
```

4. v1: the simplest design, and where it breaks



Naive: stream every GPS point to the server in real time. 10M devices means millions of writes/sec, for no user value.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Naive: stream every GPS point to the server in real time. 10M devices means millions of writes/sec, for no user value.
2. Record on the device (works in tunnels and mountains). Upload the whole activity when finished, with an idempotency key and resumable upload.
3. Track blob to S3, metadata to Postgres.
4. Heavy processing (stats, segment matching) runs asynchronously from a queue; leaderboards are Redis sorted sets per segment.

6. Deep dives

Why device-first?

Signal is unreliable; users need it to work offline. Only optional live tracking sends periodic points (best effort).

Upload reliability?

Idempotency key = activity_id; resumable chunks; retry until acked.

Leaderboards?

ZADD segment:{id} with the time as score; ZRANGE for top N; snapshot to DB.

Time correctness?

Use device timestamps (event time), not upload time.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Start/pause/stop recording an activity (GPS track); View activity stats and maps; Share with friends; see friends' activities; leaderboards
5–8	Non-functional requirements	Works offline; never lose an activity; 10M DAU; Feed eventually consistent; Uploads resumable
8–10	Numbers that drive the design	10M activities/day; 1hr at 1 point/5s = 720 points ≈ 15–100KB per track → **~1TB/day** of track blobs.

Time	What you do	Content
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: Postgres → S3 → Queue → Stats / segment workers → Redis. Justify each addition with a number.
25–40	Deep dives	Why device-first? · Upload reliability? · Leaderboards? · Time correctness?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- Server-side real-time ingestion of every GPS point.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Scaling writes
- **DDIA:** Ch 11 event time · Ch 12 idempotent upload
- **Similar problems:** Top-K videos (YouTube) · Ad click aggregator · Web crawler · Metrics monitoring (Datadog)

Post search (Facebook)

Difficulty: Medium · **Patterns:** Scaling writes · Scaling reads · **DDIA:** Ch 3 inverted index · Ch 6 term-partitioned index · Ch 11 stream indexing

Search over billions of posts, where like counts change constantly.

TL;DR

- Build an inverted index (term → post IDs) from the stream of new posts, sharded by term.
- Keep two orderings per term, recency and likes, and update like-based ranking only at thresholds or in batches.
- Keep hot postings in memory, move old ones to cheaper storage, and cache popular queries.

1. Requirements

Functional

1. Search posts by keyword
2. Sort results by recency or by likes
3. (Inputs: create posts, like posts)

Non-functional

- Search < 500ms
- New posts searchable within ~1 minute
- Huge write volume (posts and likes)
- Read volume ~10k searches/s

2. Back-of-envelope numbers

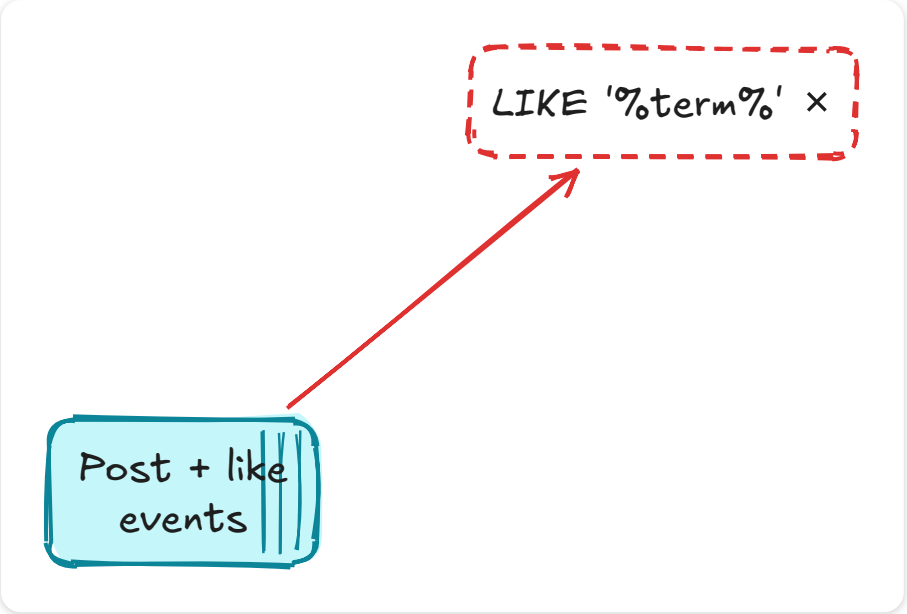
~1B posts/day ≈ 12k/s and ~10× that in likes. Indexing every like individually is far too much write load, so batch it.

3. Core entities and API

```
inverted index: term → post IDs (recency list) and term → ZSET(post_id, like_bucket)
```

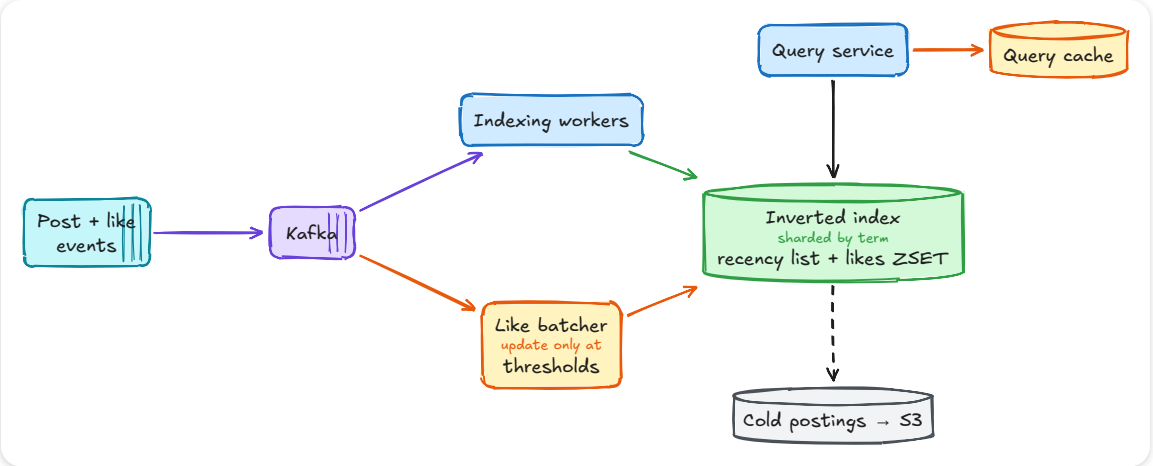
```
GET /search?q&sort=recency|likes&cursor
```

4. v1: the simplest design, and where it breaks



Naive: LIKE '%term%' on the posts table, a full scan at any real scale.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Naive: LIKE '%term%' on the posts table, a full scan at any real scale.

2. Build an **inverted index** (term → post IDs) from the event stream, sharded by term. New posts are searchable within seconds.
3. Two orderings: a recency list and a likes-sorted set. Don't reindex on every like; update the score only when the count crosses thresholds (powers of 2) or in per-minute batches.
4. Keep hot postings in memory and move older ones to blob storage; cache popular queries; intersect posting lists for multi-word queries.

6. Deep dives

Why not just Elasticsearch?

In a real design it is a fine answer; know its sharding (document-partitioned, scatter-gather) and refresh interval. This problem tests whether you understand the inverted index underneath.

Likes change constantly?

Threshold or batched updates to the ranking score; exact counts live elsewhere.

Multi-keyword?

Intersect posting lists, smallest first; consider bigram indexes.

Hot term?

Term-partitioned index means one shard per term, so replicate hot terms.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Search posts by keyword; Sort results by recency or by likes; (Inputs: create posts, like posts)
5–8	Non-functional requirements	Search < 500ms; New posts searchable within ~1 minute; Huge write volume (posts and likes); Read volume ~10k searches/s

Time	What you do	Content
8–10	Numbers that drive the design	~1B posts/day ≈ 12k/s and ~10× that in likes. Indexing every like individually is far too much write load, so batch it.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: Kafka → Indexing workers → Inverted index → Query service → Like batcher → Cold postings → S3 → Query cache. Justify each addition with a number.
25–40	Deep dives	Why not just Elasticsearch? · Likes change constantly? · Multi-keyword? · Hot term?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- A Postgres LIKE scan, or re-indexing on every like.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Scaling writes · Scaling reads
- **DDIA:** Ch 3 inverted index · Ch 6 term-partitioned index · Ch 11 stream indexing
- **Similar problems:** Top-K videos (YouTube) · Ad click aggregator · Web crawler · Metrics monitoring (Datadog)

Price tracker (CamelCamelCamel)

Difficulty: Medium · **Patterns:** Scaling writes · Scaling reads · **DDIA:** Ch 11 event-driven · Ch 3 time-series storage

Five hundred million products can't all be crawled often, so decide what is worth checking.

TL;DR

- Prioritise crawling by how many people track a product and how often its price changes.
- Add crowd-sourced prices from a browser extension, validated before they trigger alerts.
- Make alerts event-driven: when a price drops, look up subscriptions for that product only.

1. Requirements

Functional

1. Track price history for products
2. Users subscribe to alerts at a target price
3. View price charts

Non-functional

- 500M products
- Alert within ~1 hour of a drop
- Respect site rate limits (crawl budget)
- Charts load fast

2. Back-of-envelope numbers

500M products crawled daily = **~6k pages/s**, which runs into rate limits. Subscriptions are concentrated on a small fraction of products, so prioritize those.

3. Core entities and API

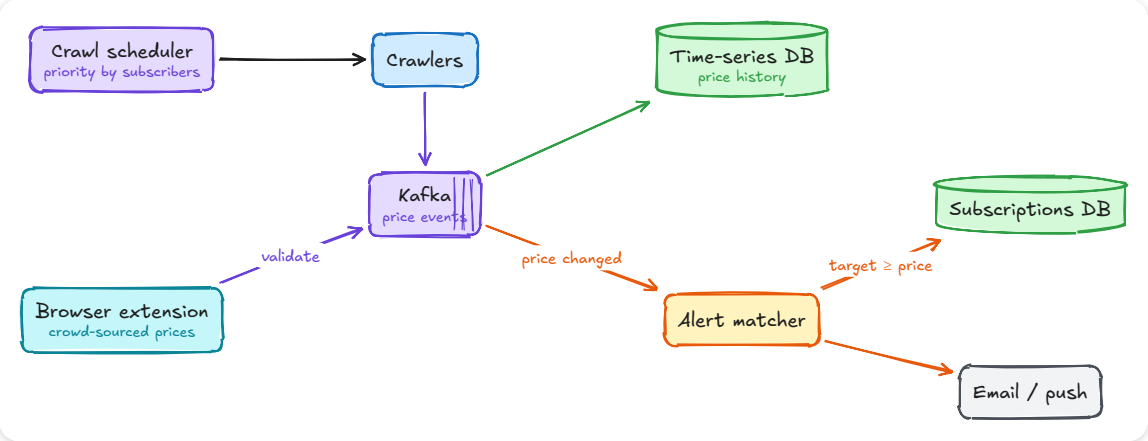
```
price_history(product_id, ts, price) time-series
subscriptions(product_id, user_id, target_price) indexed (product_id, target_price)
```

```
POST /subscriptions {product_id, target_price}
GET /products/{id}/history?range
```

4. v1: the simplest design, and where it breaks

A scheduler decides what to crawl: products with many subscribers or volatile prices are checked more often; the long tail rarely.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. A scheduler decides what to crawl: products with many subscribers or volatile prices are checked more often; the long tail rarely.
2. A browser extension reports prices users actually see, which adds coverage for free. Validate it (N agreeing reports or a crawler re-check) before trusting it.
3. Every price change becomes an event; history goes to a time-series table (downsampled for charts).
4. Alerts are **event-driven**: on a drop, query subscriptions WHERE product_id=? AND target_price ≥ new_price and notify, deduped per user and product.

6. Deep dives

Crawl budget?

Prioritize by subscribers and volatility; adaptive frequency; respect robots and rate limits.

Bad extension data?

Require agreement or confirm with a crawl before alerting.

Matching efficiently?

Index on (product_id, target_price); react to events, never scan all subscriptions.

Charts?

Daily aggregates, cached.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Track price history for products; Users subscribe to alerts at a target price; View price charts
5–8	Non-functional requirements	500M products; Alert within ~1 hour of a drop; Respect site rate limits (crawl budget); Charts load fast
8–10	Numbers that drive the design	500M products crawled daily = **~6k pages/s**, which runs into rate limits. Subscriptions are concentrated on a small fraction of products, so prioritize those.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: Browser extension → Kafka → Time-series DB → Alert matcher → Subscriptions DB → Email / push. Justify each addition with a number.
25–40	Deep dives	Crawl budget? · Bad extension data? · Matching efficiently? · Charts?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- Re-scanning all subscriptions on a schedule instead of reacting to price changes.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Scaling writes · Scaling reads
- **DDIA:** Ch 11 event-driven · Ch 3 time-series storage
- **Similar problems:** Top-K videos (YouTube) · Ad click aggregator · Web crawler · Metrics monitoring (Datadog)

Ride sharing (Uber)

Difficulty: Hard · **Patterns:** Real-time updates · **DDIA:** Ch 8 leases · Ch 3 geo index · Ch 6 partition by region

Four problems in one: a flood of location updates, matching, locking drivers, and a multi-step trip lifecycle.

TL;DR

- Driver locations are ephemeral and frequent, so keep them in Redis's geospatial index, not a database.
- Offer a ride to one driver at a time and lock that driver with a short TTL so they can't get two offers.
- Run ride requests through a durable, per-region queue so a crashed matcher loses nothing.

1. Requirements

Functional

1. Rider requests a ride (pickup, destination) and sees a fare estimate
2. Match to a nearby available driver; driver accepts or declines
3. Live location of driver during the trip

Non-functional

- Match within ~1 minute
- A driver is never assigned two rides (consistency)
- High availability for location updates
- Handle surge peaks per city

2. Back-of-envelope numbers

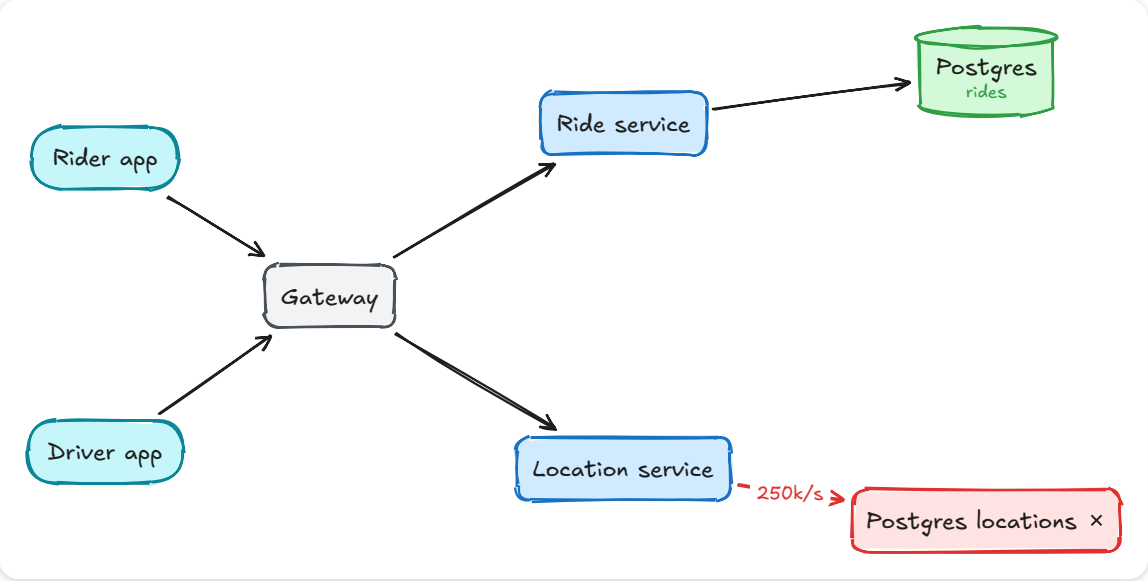
1M active drivers × 1 update/4s = **250k location writes/s**, ephemeral (only the latest matters). Ride requests ~100/s avg.

3. Core entities and API

```
rides(id, rider, driver, status, pickup, dest, fare)
driver_location in Redis GEO (TTL)
```

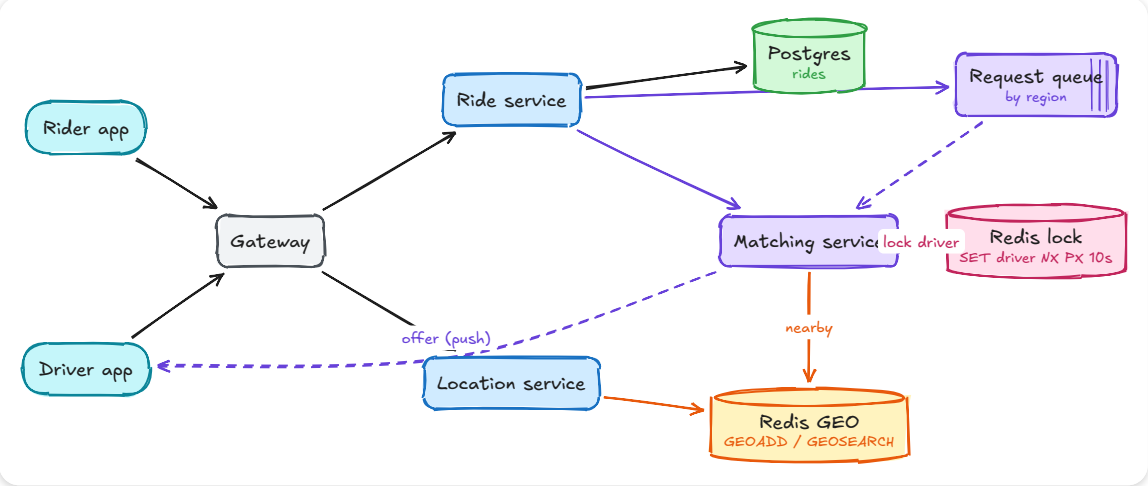
```
POST /rides/estimate
POST /rides {estimate_id}
POST /drivers/location {lat, lng}
PATCH /rides/{id} {accept|decline}
```

4. v1: the simplest design, and where it breaks



Naive: drivers write locations to Postgres. 250k writes/sec of throwaway data breaks it.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Naive: drivers write locations to Postgres. 250k writes/sec of throwaway data breaks it.
2. Locations go to **Redis GEO**: in memory, latest point only, with TTL. GEOSEARCH finds drivers within radius.
3. Matching offers the ride to **one driver at a time** and takes a lock on that driver (SET NX with a 10s TTL), so they can't be offered two rides. Ride assignment is a conditional UPDATE WHERE driver IS NULL.
4. Requests go through a durable queue partitioned by region, so a crashed matcher doesn't lose rides and surges scale per city.

6. Deep dives

Driver double-assigned?

Lock per driver with TTL = accept window, plus conditional assignment in Postgres. Lock expiry = automatic decline.

Reduce location load?

Adaptive update frequency (less when stationary), client batching.

Why not query Postgres + PostGIS?

Fine for reads, but 250k updates/sec of ephemeral data belongs in memory.

Matcher crash mid-match?

Durable workflow/queue; the request is re-processed and idempotent by ride_id.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Rider requests a ride (pickup, destination) and sees a fare estimate; Match to a nearby available driver; driver accepts or declines; Live location of driver during the trip

Time	What you do	Content
5–8	Non-functional requirements	Match within ~1 minute; A driver is never assigned two rides (consistency); High availability for location updates; Handle surge peaks per city
8–10	Numbers that drive the design	1M active drivers × 1 update/4s = **250k location writes/s**, ephemeral (only the latest matters). Ride requests ~100/s avg.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: Redis GEO → Matching service → Redis lock → Request queue. Justify each addition with a number.
25–40	Deep dives	Driver double-assigned? · Reduce location load? · Why not query Postgres + PostGIS? · Matcher crash mid-match?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- Offering a ride to many drivers at once without locks, so two drivers accept the same ride.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Real-time updates
- **DDIA:** Ch 8 leases · Ch 3 geo index · Ch 6 partition by region
- **Similar problems:** Live comments (Facebook Live) · Collaborative editor (Google Docs) · Stock trading (Robinhood) · Online auction

Live comments (Facebook Live)

Difficulty: Medium · **Patterns:** Real-time updates · **DDIA:** Ch 11 pub/sub · Ch 6 partition by video

One popular stream can mean a million viewers each expecting new comments within a fraction of a second.

TL;DR

- Polling at this scale is a self-inflicted denial of service; push with Server-Sent Events.
- Write each comment once and publish it once; pub/sub fans it out to every server holding viewers of that video.
- Route viewers of the same video to the same servers so each server subscribes to few channels.

1. Requirements

Functional

1. Post a comment on a live video
2. Viewers see new comments in near real time
3. Late joiners can scroll back through earlier comments

Non-functional

- Delivery p99 < 200ms
- Millions of concurrent viewers; mega-streams with 1M+ viewers
- Eventual consistency and ordering are acceptable

2. Back-of-envelope numbers

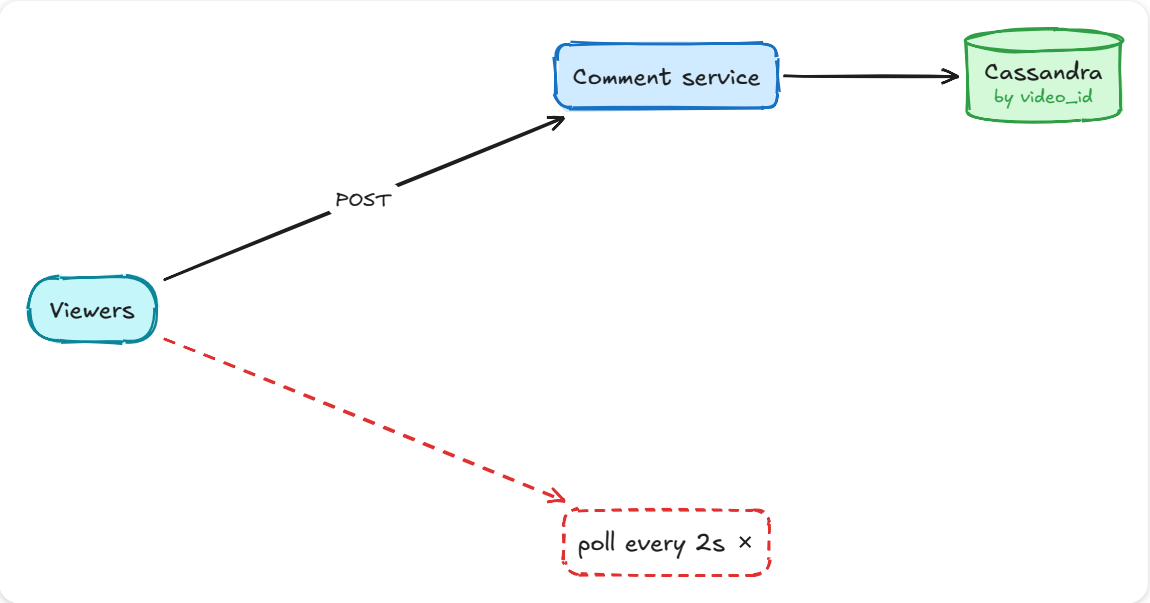
Hot stream: 1M viewers × 1k comments/s = **1B deliveries/s** for one video. Polling every 2s = 500k req/s from that one stream.

3. Core entities and API

```
comments(video_id, comment_id, user_id, text, ts) partitioned by video_id, clustered by ts
```

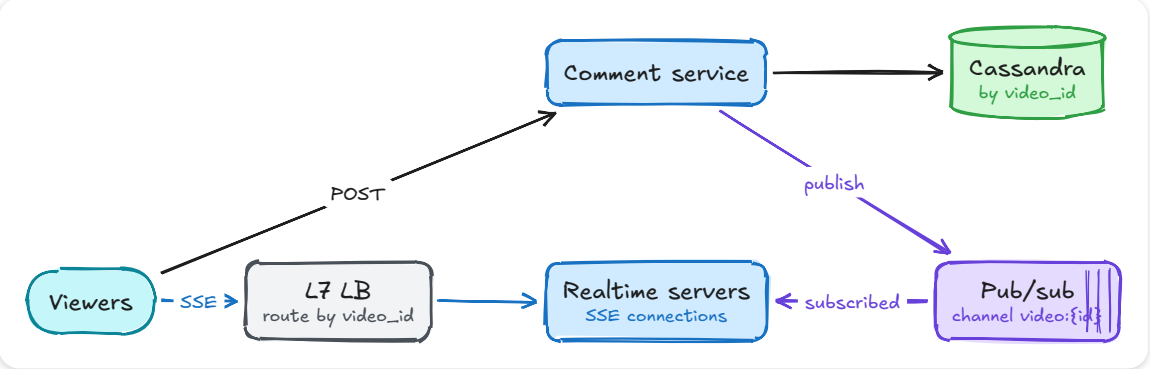
```
POST /videos/{id}/comments
GET /videos/{id}/comments?cursor (history)
GET /videos/{id}/comments/stream (SSE)
```

4. v1: the simplest design, and where it breaks



Naive: viewers poll for new comments. Latency = poll interval, and 1M viewers create a self-inflicted DDoS.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Naive: viewers poll for new comments. Latency = poll interval, and 1M viewers create a self-inflicted DDoS.
2. Server push over **SSE**: one long-lived HTTP connection per viewer. Posting a comment stays a normal POST.
3. A comment is written once to Cassandra and published once to a pub/sub channel for that video. Every realtime server subscribed to the channel pushes it to its local viewers.
4. An L7 load balancer routes viewers of the same video to the same servers (consistent hashing on video_id), so each server subscribes to few channels.

6. Deep dives

SSE or WebSocket?

Traffic is server→client. SSE is plain HTTP with auto-reconnect; WebSocket is only needed for heavy bidirectional traffic.

How does one comment reach 1,000 servers?

Pub/sub fan-out: publish once; each server is subscribed to the channels of videos its viewers watch.

Mega-streams?

Sample or rate-limit the comment stream per viewer (nobody can read 1k/s); prioritize friends' comments.

Late join?

Page the last N from Cassandra by cursor, then attach to the stream.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Post a comment on a live video; Viewers see new comments in near real time; Late joiners can scroll back through earlier comments

Time	What you do	Content
5–8	Non-functional requirements	Delivery p99 < 200ms; Millions of concurrent viewers; mega-streams with 1M+ viewers; Eventual consistency and ordering are acceptable
8–10	Numbers that drive the design	Hot stream: 1M viewers × 1k comments/s = **1B deliveries/s** for one video. Polling every 2s = 500k req/s from that one stream.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: Realtime servers → Pub/sub → L7 LB. Justify each addition with a number.
25–40	Deep dives	SSE or WebSocket? · How does one comment reach 1,000 servers? · Mega-streams? · Late join?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- Polling, which gives high latency and huge wasted load.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Real-time updates
- **DDIA:** Ch 11 pub/sub · Ch 6 partition by video
- **Similar problems:** Ride sharing (Uber) · Collaborative editor (Google Docs) · Stock trading (Robinhood) · Online auction

Collaborative editor (Google Docs)

Difficulty: Hard · **Patterns:** Real-time updates · **DDIA:** Ch 5 conflict resolution · Ch 9 total order · Ch 11 event log

The challenge is correctness, not scale: everyone must end up with the same document.

TL;DR

- Send small operations, not whole documents; last-write-wins on documents loses people's work.
- One server per document orders operations and transforms concurrent ones (OT); CRDTs are the serverless alternative.
- Append every operation to a log before acknowledging it, and snapshot periodically for fast loading.

1. Requirements

Functional

1. Create and open documents
2. Many users edit concurrently and see each other's changes live
3. See collaborators' cursors and presence

Non-functional

- Remote edits visible < 100ms
- All replicas converge to the same document
- No lost edits (durable)
- ~100 concurrent editors per doc; millions of docs

2. Back-of-envelope numbers

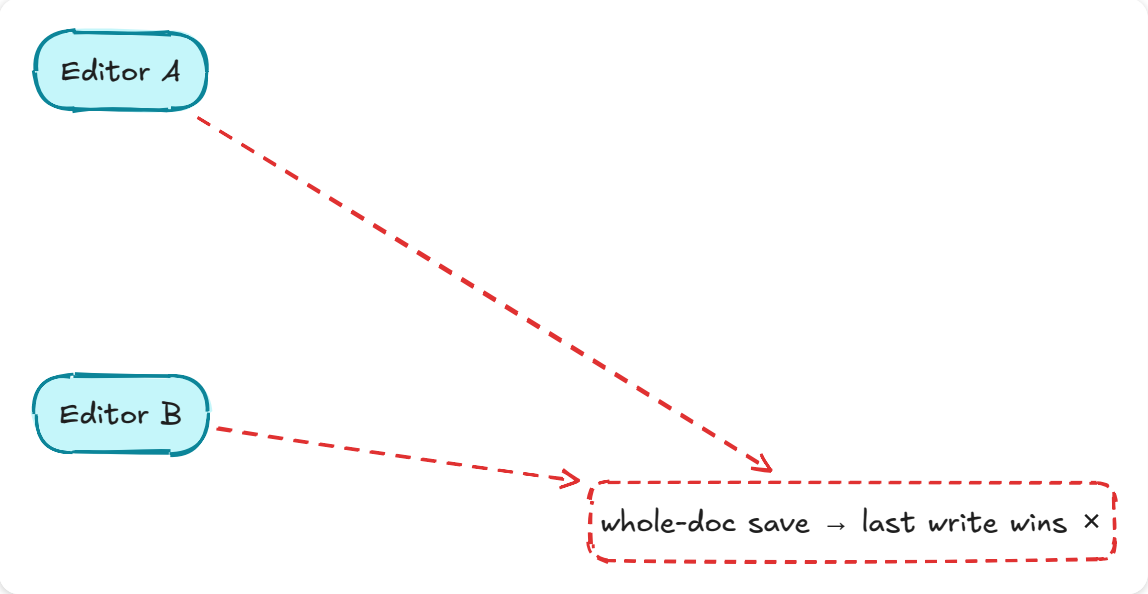
Each keystroke is an op (~100B). 100 editors × 5 ops/s = 500 ops/s per hot doc, which one server handles easily. The challenge is correctness, not scale.

3. Core entities and API

```
doc_ops(doc_id, version, op) append-only
doc_snapshots(doc_id, version, content)
```

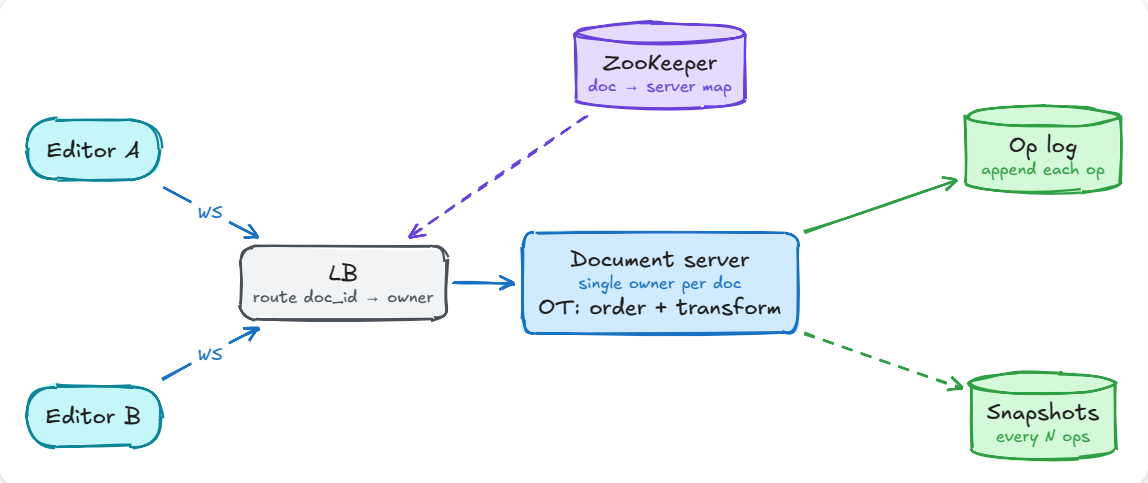
```
WS /docs/{id}: send op {type, pos, char, base_version}; receive transformed ops
GET /docs/{id} (snapshot + version)
```

4. v1: the simplest design, and where it breaks



Naive: send the whole document on save. Last write wins, and someone's work disappears.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Naive: send the whole document on save. Last write wins, and someone's work disappears.
2. Send **operations** (insert "x" at 5). Concurrent ops shift positions, so a single server per doc orders them and **transforms** each against concurrent ones (Operational Transformation), then broadcasts.
3. Every op is appended to the op log before it is acknowledged; periodic snapshots make loading fast (snapshot + tail).
4. All editors of a doc must reach the same owner server: consistent hashing on doc_id, with ownership coordinated through ZooKeeper.

6. Deep dives

OT or CRDT?

OT: a central server orders and transforms; compact; needs one coordinator per doc (Google Docs). **CRDT**: ops commute, no server needed, good for offline and P2P; bigger metadata (Figma, Yjs).

Why one server per doc?

OT needs a single total order of ops (DDIA Ch 9).

Offline edits?

Client buffers ops tagged with a base version; on reconnect the server transforms them against everything since that version.

Cursors?

Ephemeral presence over the same socket; not persisted.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Create and open documents; Many users edit concurrently and see each other's changes live; See collaborators' cursors and presence
5–8	Non-functional requirements	Remote edits visible < 100ms; All replicas converge to the same document; No lost edits (durable); ~100 concurrent editors per doc; millions of docs
8–10	Numbers that drive the design	Each keystroke is an op (~100B). 100 editors × 5 ops/s = 500 ops/s per hot doc, which one server handles easily. The challenge is correctness, not scale.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: Document server → Op log → Snapshots → LB → ZooKeeper. Justify each addition with a number.
25–40	Deep dives	OT or CRDT? · Why one server per doc? · Offline edits? · Cursors?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- Locking the document or paragraph, which defeats real-time collaboration.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Real-time updates
- **DDIA:** Ch 5 conflict resolution · Ch 9 total order · Ch 11 event log
- **Similar problems:** Ride sharing (Uber) · Live comments (Facebook Live) · Stock trading (Robinhood) · Online auction

Stock trading (Robinhood)

Difficulty: Hard · **Patterns:** Real-time updates · **DDIA:** Ch 8 unknown outcomes · Ch 12 idempotency · Ch 11 pub/sub

Two separate problems: pushing live prices to millions, and never losing or duplicating an order.

TL;DR

- Prices: one consumer of the exchange feed publishes per symbol; push servers subscribe and stream to clients.
- Orders: persist the order as pending with your own ID before sending it to the exchange.
- Treat exchange calls as having unknown outcomes; reconcile by your order ID instead of resubmitting.

1. Requirements

Functional

1. See live stock prices for watched symbols
2. Place market and limit orders; cancel orders
3. See order status and fills

Non-functional

- Price updates < 200ms
- Orders never lost or duplicated
- Strong consistency on order state
- 20M users; market-open spikes

2. Back-of-envelope numbers

Exchange feed ~ thousands of updates/s for ~10k symbols. 20M users polling = millions of req/s, so push instead. Orders ~ thousands/s at peak.

3. Core entities and API

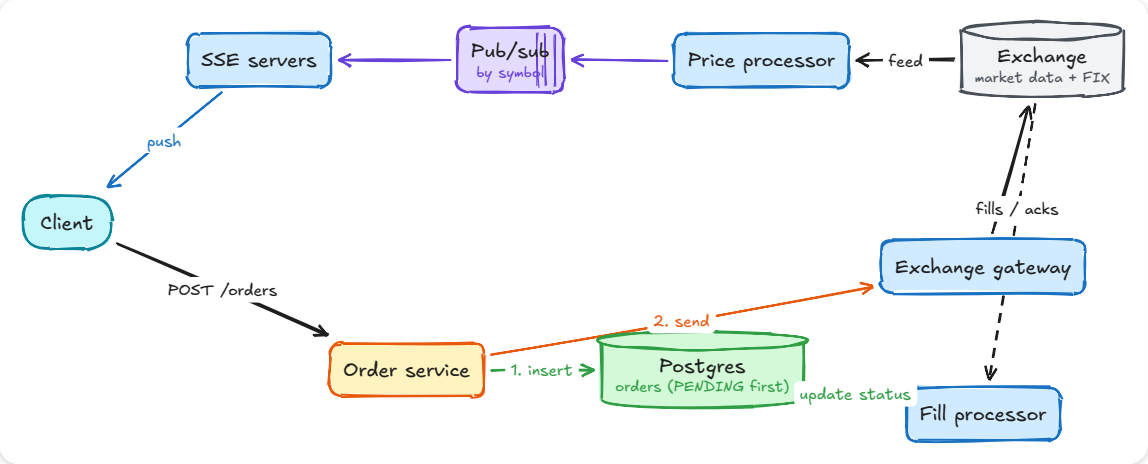
```
orders(id, user_id, symbol, status PENDING|SUBMITTED|FILLED|CANCELLED, exchange_order_id) partitioned
by user
```

```
GET /prices/stream?symbols (SSE)
POST /orders {symbol, side, qty, type, limit?} + Idempotency-Key
DELETE /orders/{id}
```

4. v1: the simplest design, and where it breaks

Order and price paths are separate problems: one is real-time fan-out, the other is correctness.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Order and price paths are separate problems: one is real-time fan-out, the other is correctness.
2. Prices: one consumer reads the exchange feed and publishes per symbol; SSE servers subscribe to the symbols their clients watch. Throttle to ~1 update/s per symbol for the app.
3. Orders: **persist PENDING first** with our own order ID, then send to the exchange using that ID as clientOrderId. If we crash in between, we can look it up and reconcile.
4. Fills and acks come back asynchronously and update order status; a reconciliation job compares against exchange reports daily.

6. Deep dives

Write DB before or after the exchange call?

Before. Otherwise a crash after sending leaves a live order you don't know about. The exchange call has an unknown outcome on timeout (DDIA Ch 8).

Duplicate submits?

Client Idempotency-Key with a unique index; the exchange also dedupes on clientOrderId.

Cancel races with a fill?

Cancel is a request; status changes only on the exchange ack. Handle partial fills.

Market-open spike?

Autoscale SSE servers; per-user rate limits on orders.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	See live stock prices for watched symbols; Place market and limit orders; cancel orders; See order status and fills
5–8	Non-functional requirements	Price updates < 200ms; Orders never lost or duplicated; Strong consistency on order state; 20M users; market-open spikes
8–10	Numbers that drive the design	Exchange feed ~ thousands of updates/s for ~10k symbols. 20M users polling = millions of req/s, so push instead. Orders ~ thousands/s at peak.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: Price processor → Pub/sub → SSE servers → Order service → Postgres → Exchange gateway → Fill processor. Justify each addition with a number.

Time	What you do	Content
25–40	Deep dives	Write DB before or after the exchange call? · Duplicate submits? · Cancel races with a fill? · Market-open spike?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- Treating the exchange call as a local function call, so timeouts create ghost or duplicate orders.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Real-time updates
- **DDIA:** Ch 8 unknown outcomes · Ch 12 idempotency · Ch 11 pub/sub
- **Similar problems:** Ride sharing (Uber) · Live comments (Facebook Live) · Collaborative editor (Google Docs) · Online auction

Online auction

Difficulty: Medium · **Patterns:** Real-time updates · Dealing with contention · **DDIA:** Ch 7 lost update · Ch 11 partitioned log · Ch 8 clocks

Many people bid on one row at the same time, and exactly one of them must win each race.

TL;DR

- Comparing bids in application code loses updates under concurrency.
- Accept a bid with one conditional update: set the new maximum only if it beats the current one and the auction is open.
- Push accepted bids to watchers in real time; for very hot auctions, serialize bids through a partitioned log.

1. Requirements

Functional

1. Create an auction (item, starting price, end time)
2. Place bids that must beat the current highest
3. Watch the highest bid update live; winner decided at the end

Non-functional

- Bids strongly consistent (never accept a stale "highest")
- Live updates < 1s
- No lost bids
- Hot auctions spike in the final seconds

2. Back-of-envelope numbers

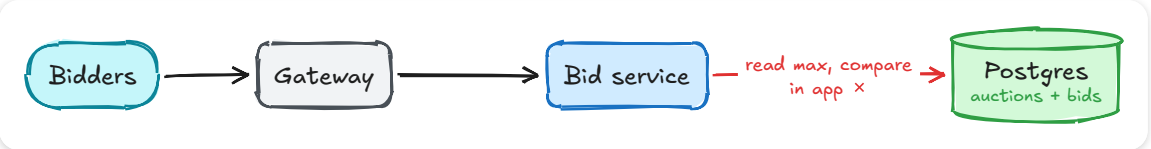
10M auctions, ~100 bids each over a week ≈ low average QPS, but a hot auction can see **~1k bids/s** on one row near the end.

3. Core entities and API

```
auctions(id, max_bid, max_bidder, end_time, status)
bids(auction_id, bid_id, user_id, amount, ts) append-only
```

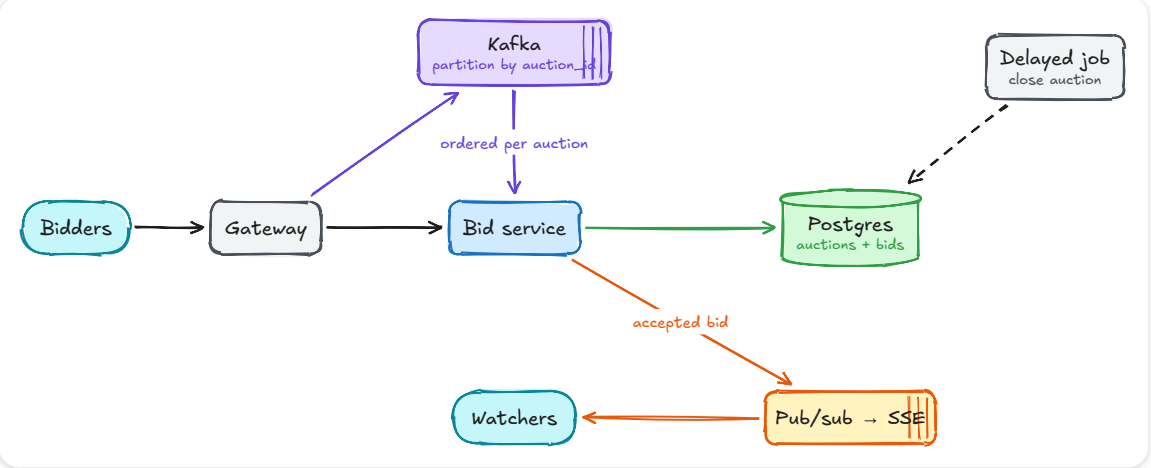
```
POST /auctions
POST /auctions/{id}/bids {amount} + Idempotency-Key
GET /auctions/{id}/stream (SSE)
```

4. v1: the simplest design, and where it breaks



Naive: read max_bid, compare in application code, insert. Two concurrent bids both "win", a lost update.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Naive: read max_bid, compare in application code, insert. Two concurrent bids both "win", a lost update.
2. Make it atomic in the DB: UPDATE auctions SET max_bid=:amt, max_bidder=:u WHERE id=:a AND max_bid < :amt AND end_time > now(). 1 row = accepted, 0 = outbid. Bids are also appended for history.
3. Accepted bids are published to pub/sub and pushed over SSE to everyone watching.

4. For very hot auctions, a Kafka partition per auction_id serializes bids before the DB (durable and ordered). A delayed job closes the auction at end time.

6. Deep dives

Two bids at the same instant?

Conditional update (or SELECT FOR UPDATE) on the auction row. That is the lost-update fix from DDIA Ch 7.

Clock fairness at the deadline?

Use the DB server time inside the same conditional update, never client time.

Durability?

Acknowledge only after commit (or Kafka acks=all).

Anti-sniping?

Extend end_time by N seconds on late bids, inside the same update.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Create an auction (item, starting price, end time); Place bids that must beat the current highest; Watch the highest bid update live; winner decided at the end
5–8	Non-functional requirements	Bids strongly consistent (never accept a stale "highest"); Live updates < 1s; No lost bids; Hot auctions spike in the final seconds
8–10	Numbers that drive the design	10M auctions, ~100 bids each over a week ≈ low average QPS, but a hot auction can see **~1k bids/s** on one row near the end.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.

Time	What you do	Content
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: Pub/sub → SSE → Watchers → Kafka → Delayed job. Justify each addition with a number.
25–40	Deep dives	Two bids at the same instant? · Clock fairness at the deadline? · Durability? · Anti-sniping?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- Comparing bids in a Redis cache without atomicity.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Real-time updates · Dealing with contention
- **DDIA:** Ch 7 lost update · Ch 11 partitioned log · Ch 8 clocks
- **Similar problems:** Ride sharing (Uber) · Live comments (Facebook Live) · Collaborative editor (Google Docs) · Stock trading (Robinhood)

LLM chat (ChatGPT)

Difficulty: Hard · **Patterns:** Real-time updates · **DDIA:** Ch 11 logs and streams · Ch 1 tail latency

GPUs are the scarce resource, so every queueing decision is really a cost decision.

TL;DR

- Stream tokens to the client as they are generated so the first words appear in under a second.
- Put a queue with priorities in front of the GPU pool for backpressure, fairness and batching.
- Buffer tokens per message so a client that reconnects can resume where it left off.

1. Requirements

Functional

1. Send a message in a conversation
2. Stream the model's answer token by token
3. View conversation history

Non-functional

- Time to first token < 1s
- GPU capacity is the scarce resource
- Conversations durable
- 100M DAU; bursty

2. Back-of-envelope numbers

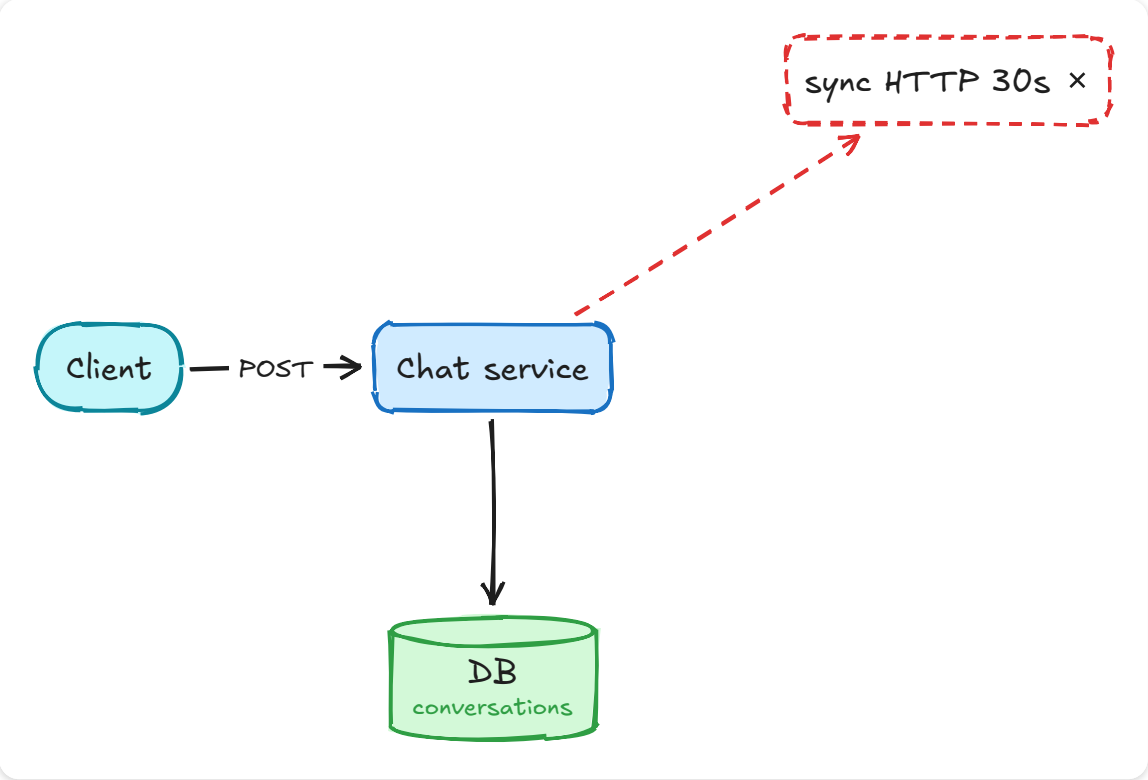
100M DAU × 10 msgs = 1B/day ≈ **12k/s** generations. Each holds a GPU for seconds, so GPUs are the bottleneck and every queueing decision is about them.

3. Core entities and API

```
conversations(id, user_id)
messages(conversation_id, id, role, content, status)
```

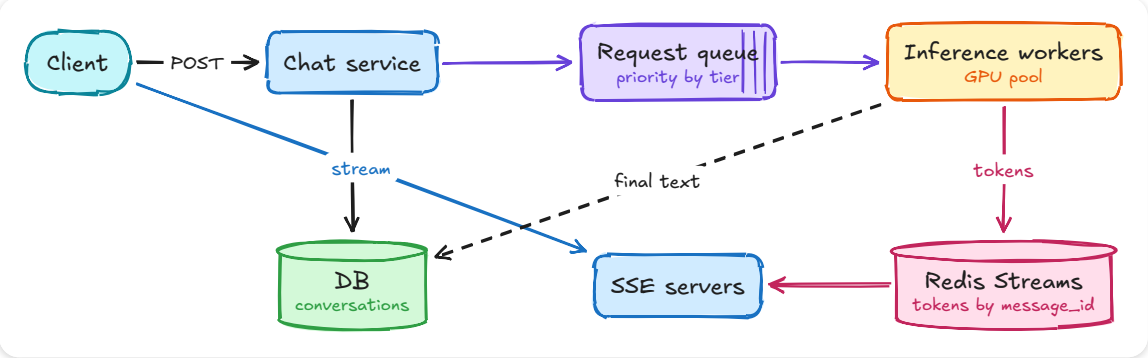
```
POST /conversations/{id}/messages → message_id
GET /messages/{id}/stream (SSE, Last-Event-ID resume)
GET /conversations/{id}
```

4. v1: the simplest design, and where it breaks



Naive: one synchronous request waits ~30s for the full answer. Timeouts, no progress shown, no backpressure.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Naive: one synchronous request waits ~30s for the full answer. Timeouts, no progress shown, no backpressure.
2. Stream tokens to the client over **SSE** as they are generated, so time-to-first-token stays low.
3. Put a **queue** in front of the GPU pool: absorbs bursts, enables priority for paid tiers, batching, and admission control (429 when full).
4. Workers write tokens into a Redis Stream keyed by message_id; SSE servers read from it. A client that reconnects resumes from its last token; the final text is persisted.

6. Deep dives

Why a queue before GPUs?

Scarce and expensive capacity: queueing gives backpressure, priority, fairness, and batching.

Client disconnects?

Generation continues; tokens are buffered in the stream; resume with Last-Event-ID.

Long context?

Load recent messages; summarize or truncate to fit the window; cache hot conversations.

Cost control?

Per-user rate limits and token quotas at the gateway.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Send a message in a conversation; Stream the model's answer token by token; View conversation history

Time	What you do	Content
5–8	Non-functional requirements	Time to first token < 1s; GPU capacity is the scarce resource; Conversations durable; 100M DAU; bursty
8–10	Numbers that drive the design	100M DAU × 10 msgs = 1B/day ≈ **12k/s** generations. Each holds a GPU for seconds, so GPUs are the bottleneck and every queueing decision is about them.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: SSE servers → Request queue → Inference workers → Redis Streams. Justify each addition with a number.
25–40	Deep dives	Why a queue before GPUs? · Client disconnects? · Long context? · Cost control?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- Holding a synchronous connection through the load balancer straight to a GPU.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Real-time updates
- **DDIA:** Ch 11 logs and streams · Ch 1 tail latency
- **Similar problems:** Ride sharing (Uber) · Live comments (Facebook Live) · Collaborative editor (Google Docs) · Stock trading (Robinhood)

Chat app (WhatsApp)

Difficulty: Medium · **Patterns:** Real-time updates · **DDIA:** Ch 5 consistent prefix · Ch 11 at-least-once + idempotency

Chat looks like CRUD until you notice the two users are on different servers and one of them is offline.

TL;DR

- Keep a persistent WebSocket per device; route messages between chat servers with a registry and per-user pub/sub.
- Persist each message and add it to the recipient's inbox before acknowledging the sender, so nothing is lost.
- Partition by conversation and number messages within it so every device shows the same order.

1. Requirements

Functional

1. 1:1 and group chats (up to ~100 members)
2. Send and receive messages in real time
3. Offline users get messages when they reconnect
4. Delivery and read receipts; media

Non-functional

- Delivery < 500ms when online
- Messages never lost
- Ordered within a chat
- 1B DAU

2. Back-of-envelope numbers

50B messages/day ≈ **~600k msgs/s**. Hundreds of millions of concurrent connections, at ~1M connections per tuned server → hundreds of chat servers.

3. Core entities and API

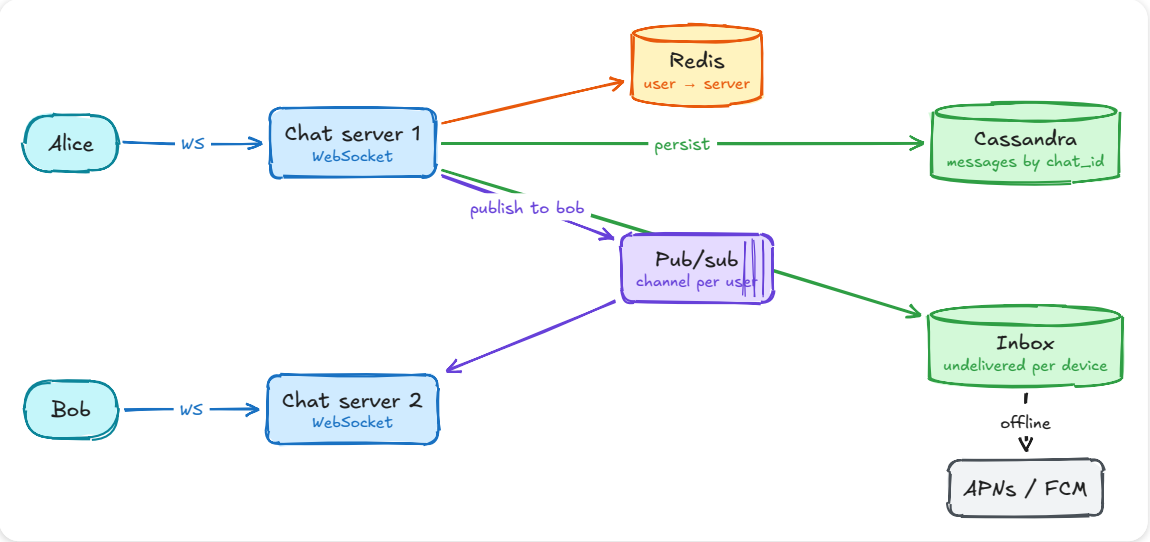
```
messages(chat_id, seq, msg_id, sender, body, ts) partitioned by chat_id
inbox(device_id, msg_id) until acked
chat_members
```

```
WS connect → send {chat_id, client_msg_id, body}; receive {msg, seq}; ack {msg_id}
POST /media → presigned upload URL
```

4. v1: the simplest design, and where it breaks

Persistent **WebSocket** connections for both directions, so there is no polling.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Persistent **WebSocket** connections for both directions, so there is no polling.
2. Alice and Bob are usually connected to **different** servers, so you need routing between them.
3. A registry (user → server) and a pub/sub channel per user: server 1 publishes to Bob's channel; server 2, which holds Bob's connection, delivers.
4. Durability first: persist to the messages table and the recipient's inbox **before** acknowledging Alice. Bob's device acks on receipt and the inbox entry is deleted; offline devices get a push and sync on reconnect.

6. Deep dives

Never lose a message?

Persist before ack; at-least-once delivery from the inbox; client dedupes by msg_id.

Ordering?

Per-chat sequence numbers, with chat_id as the partition key so one writer orders them (DDIA Ch 5 consistent prefix).

Groups?

Fan-out per member at send time (≤ ~100 members) into inboxes and channels.

Media?

Upload to S3 via presigned URL; the message carries only the object key.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	1:1 and group chats (up to ~100 members); Send and receive messages in real time; Offline users get messages when they reconnect; Delivery and read receipts; media
5–8	Non-functional requirements	Delivery < 500ms when online; Messages never lost; Ordered within a chat; 1B DAU
8–10	Numbers that drive the design	50B messages/day ≈ **~600k msgs/s**. Hundreds of millions of concurrent connections, at ~1M connections per tuned server → hundreds of chat servers.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: Chat server 2 → Redis → Pub/sub → Cassandra → Inbox → APNs / FCM. Justify each addition with a number.
25–40	Deep dives	Never lose a message? · Ordering? · Groups? · Media?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- Assuming both users share a server or are always online.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Real-time updates
- **DDIA:** Ch 5 consistent prefix · Ch 11 at-least-once + idempotency
- **Similar problems:** Ride sharing (Uber) · Live comments (Facebook Live) · Collaborative editor (Google Docs) · Stock trading (Robinhood)

Ticketmaster: selling seats without selling any seat twice

Difficulty: Medium · **Patterns:** Dealing with contention · Scaling reads · **Core concept:** Caching · **DDIA:** Ch 5 Replication, Ch 7 Transactions

TL;DR

- Two very different paths share one system. **Browsing** events is enormous, read-only traffic that can be slightly stale, so cache it hard. **Booking** is small but must be exactly right.
- The core bug is *check, then book*: two people both see seat 12A as free and both buy it. Fix it by making the check and the write **one atomic statement** in the database.
- For a huge on-sale, don't let 10 million people hit the booking path at once. A **virtual waiting room** admits them at the rate the system can actually handle.

1. Requirements

Functional (what users can do)

1. View an event and its seat map.
2. Search for events by name, date and location.
3. Book tickets: pick seats, hold them while paying, then confirm.

Out of scope for this page: resale, dynamic pricing, recommendations.

Non-functional (how well it must work; these drive the design)

NFR	Target	Why it matters here
Consistency (booking)	A seat is never sold twice	This is the whole product. Choose **consistency over availability** on this path.
Availability (browse)	Event pages and search stay up during spikes	Choose **availability**; a seat map that's a few seconds stale is fine.

NFR	Target	Why it matters here
Latency	Event page p99 < 300 ms, search p99 < 500 ms	Fans refresh constantly during an on-sale.
Scale	~10M users arrive for one popular on-sale	Traffic is extremely spiky, not steady.
Hold time	A held seat is reserved ~10 minutes during checkout	Long enough to pay, short enough not to block others.

2. Back-of-envelope numbers

- A stadium has roughly **50k–100k seats**. That is the ceiling on successful bookings for one event, however many people show up.
- 10M fans refreshing every few seconds is **millions of reads/sec**, but nearly all of them are identical page views. That makes them ideal for a CDN or cache.
- So the read path needs to absorb millions of requests/sec, while the write path handles at most ~100k bookings spread over minutes.

> **Tip**
>
> **Say this in the interview:** "Reads and writes differ by orders of magnitude here, so I'll design two paths: a cached, eventually consistent browse path and a small, strongly consistent booking path."

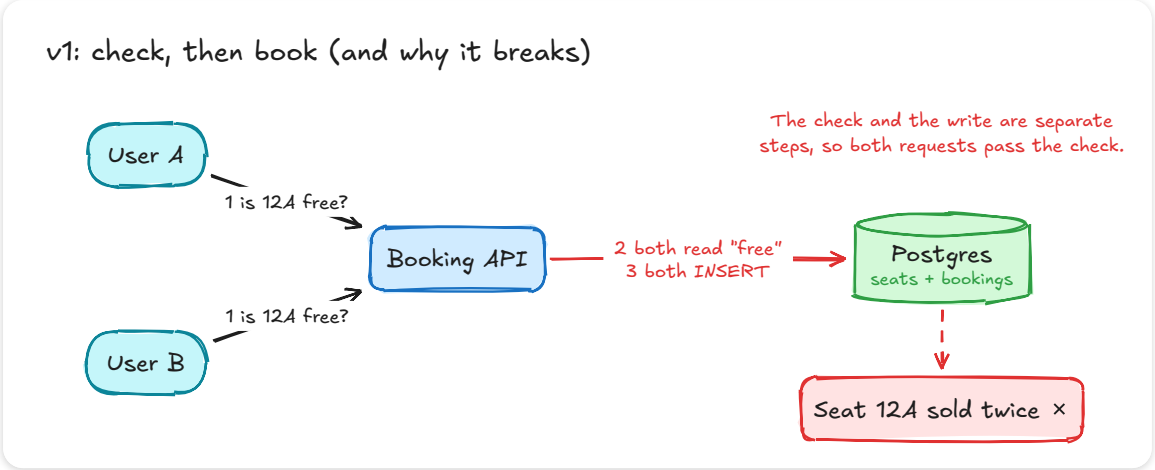
3. Core entities and API

Entity	Key fields
`Event`	id, venue_id, name, date
`Ticket`	id, event_id, seat, price, **status** (`AVAILABLE` / `HELD` / `BOOKED`), **hold_until**, user_id
`Booking`	id, user_id, ticket_ids, payment_status

```
GET  /events/{id}                    → event details + seat map
GET  /search?q=&date=&location=      → events
POST /bookings/hold    {ticket_ids}  → booking_id, hold_until
POST /bookings/{id}/confirm          → called after payment succeeds
```

Every `POST` carries an `Idempotency-Key` header so a retry never creates a second booking. The user id comes from the auth token, never from the request body.

4. v1: the simplest design, and where it breaks

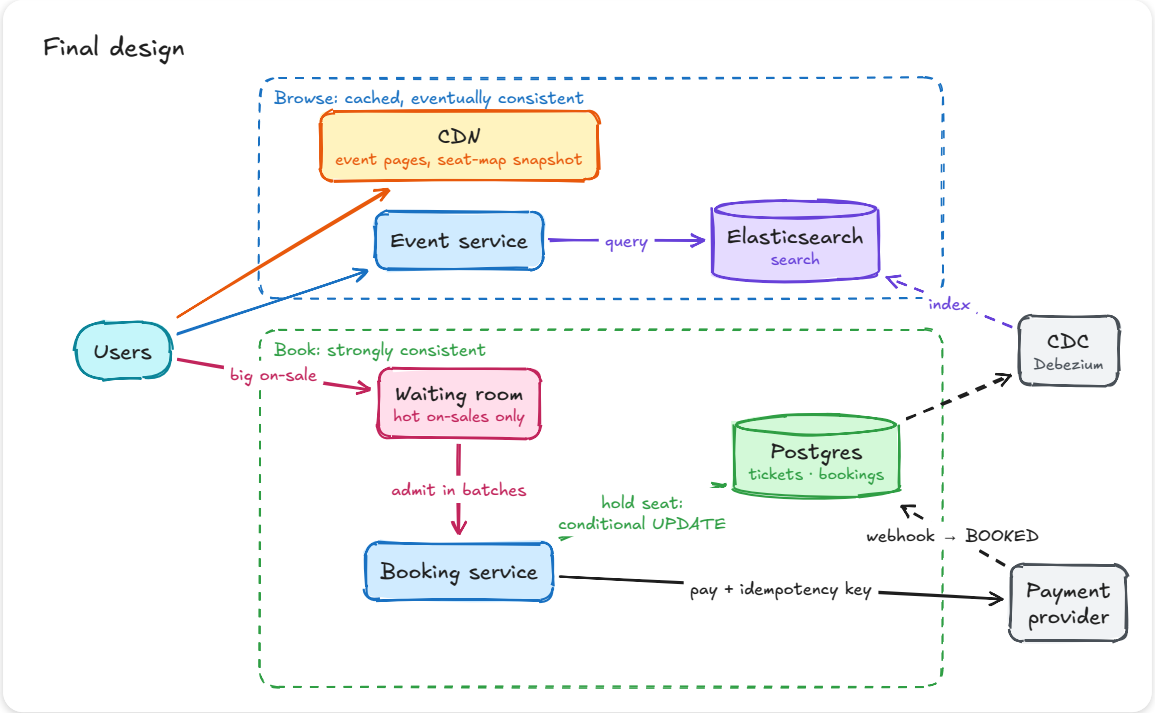


One API server and one Postgres database. To book, the API **reads** the seat, sees `AVAILABLE`, then **inserts** a booking.

What breaks: the read and the write are two separate steps. Two requests that arrive within milliseconds both read "free" before either has written, and both succeed. Adding more servers makes this *more* likely, not less.

This is a *write skew*: each transaction is fine on its own, but together they break the rule "one seat, one buyer". Most databases don't prevent it at their default isolation level. Postgres defaults to Read Committed.

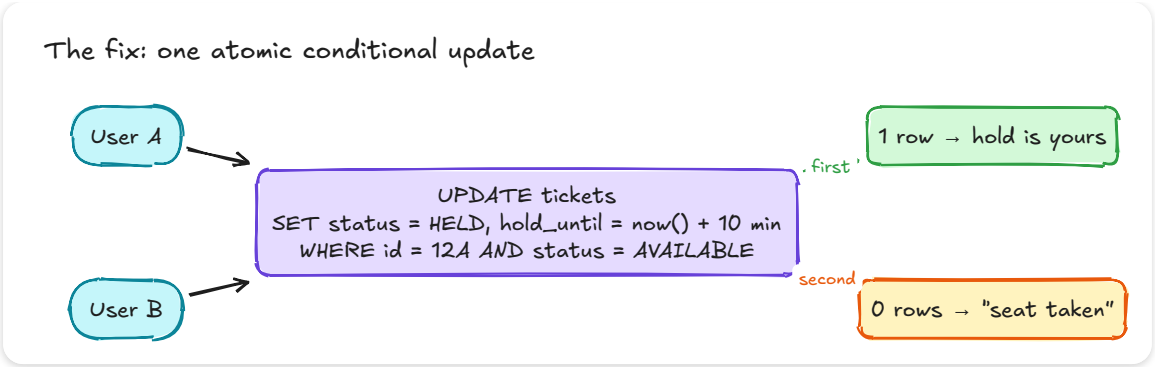
5. Final design



Component	Why it exists
CDN	Serves event pages and a seat-map snapshot from the edge, so millions of identical reads never reach your servers.
Event service + Elasticsearch	Search by text, date and location. Elasticsearch is a *derived copy*, kept in sync from Postgres by change data capture (CDC), never written to directly.
Waiting room	Only switched on for hot on-sales. Users join a queue and are admitted in batches, so the booking path sees steady, survivable load.
Booking service	Holds seats with one atomic conditional `UPDATE` (see the deep dive), then confirms after payment.
Postgres	The single source of truth for tickets and bookings. One primary is enough, because booking volume is capped by seat count.
Payment provider	Called with an idempotency key. Its webhook flips the booking from `HELD` to `BOOKED`.

6. Deep dives

6.1 How exactly do we stop double booking?



Put the check *inside* the write:

```
UPDATE tickets
SET    status = 'HELD', user_id = :user, hold_until = now() + interval '10 minutes'
WHERE  id = :ticket
  AND (status = 'AVAILABLE' OR (status = 'HELD' AND hold_until < now()));
```

The database locks the row while it applies the update, so concurrent requests are serialized. **1 row updated** means you hold the seat. **0 rows** means someone else got there first.

Notice what we *didn't* need:

- **No background job to release abandoned holds.** An expired hold simply satisfies the `WHERE` clause again.
- **No long transaction while the user pays.** The hold is a row state, not an open lock, so the database never waits on a human.

Alternatives, and when they fit:

Option	Good when	Watch out for
Conditional `UPDATE` (above)	Default. Single database.	None for this problem.
`SELECT … FOR UPDATE`	Multi-step logic on a few rows	Holding locks too long; deadlocks across several seats. Lock seats in a fixed order.
Redis `SET seat:12A user NX EX 600`	Holds shared across services; very hot seats	Redis is not the source of truth, so still write the final booking to Postgres. Locks need fencing if a process can pause.

6.2 Ten million people at 10:00:00

Without protection, everyone hits *Book* in the same second. Even a correct database melts, and users see errors instead of "sold out".

Virtual waiting room:

1. Users who arrive before the on-sale get a queue position (Redis sorted set, score = arrival time).
2. Every few seconds, admit the next *N* users. *N* is chosen from what the booking path can sustain.
3. Push "it's your turn" over SSE. Admitted users get a short-lived token that the booking service checks.

The booking service only ever sees the load you planned for, and users get a fair, honest wait instead of errors.

6.3 The seat map is stale. Is that a bug?

No, as long as you're deliberate about it. The seat map is a snapshot served from cache or CDN and refreshed every few seconds. A user might click a seat that was taken a second ago, and the conditional update simply tells them so. **The browse path is allowed to be wrong; the booking path never is.**

6.4 What if payment times out?

The outcome is unknown: the card may or may not have been charged. Keep the booking `HELD` and retry with the **same idempotency key**, or ask the provider for the status of that key. Never start a second charge. The webhook is the source of truth for the final state, and it must be idempotent too, because providers deliver webhooks more than once.

7. Interview script (45 minutes)

Time	What you do	What you say
0–5	Functional requirements	"Users can view events, search, and book seats with a short hold during checkout."
5–8	NFRs	"No seat sold twice: that path is CP. Browsing must survive huge spikes: that path is AP."
8–10	Numbers	"Bookings are capped by seat count (~100k); reads are millions per second and identical."

Time	What you do	What you say
10–13	Entities + API	Ticket with status and hold_until; hold and confirm endpoints with idempotency keys.
13–22	v1 and its bug	Draw v1, then show the check-then-book race with two users.
22–30	Fix	Conditional `UPDATE`; why no cleanup job is needed; why no long transaction.
30–40	Scale	CDN + cache for browse; Elasticsearch via CDC for search; waiting room for on-sales.
40–45	Wrap-up	Payment timeouts and idempotency; what you'd monitor (hold success rate, queue length, p99).

8. Common mistakes

- **Check, then insert** in application code. Correct-looking, wrong under concurrency.
- **Holding a database transaction open while the user pays.** Locks pile up and throughput collapses.
- **Using Redis as the only record of a booking.** Redis is a fine place for holds, but the booking must land in the durable database.
- **Treating browse and book the same.** Making browsing strongly consistent wastes money; making booking eventually consistent sells seats twice.
- **Sharding the bookings database "for scale".** Booking volume is capped by seat count; one primary handles it.

Related

- **Patterns:** Dealing with contention · Scaling reads
- **Core concepts:** Caching
- **DDIA:** Ch 5 Replication for why replicas can serve a stale seat map; Ch 7 Transactions for write skew and why the default isolation level doesn't save you.
- **Similar problems:** Online auction (contention on one row) · Local delivery (decrement stock without overselling) · LLD: Movie ticket booking

Payment system (Stripe)

Difficulty: Hard · **Patterns:** Multi-step processes · **DDIA:** Ch 8 unknown outcomes · Ch 12 end-to-end idempotency, integrity · Ch 11 outbox

Throughput is moderate. The hard part is staying correct when a call to the bank has an unknown outcome.

TL;DR

- Record the payment as pending before calling the processor, and pass your ID as the idempotency key.
- On a timeout, query by that key or retry it; never create a second charge.
- Publish status changes through an outbox, keep an append-only double-entry ledger, and reconcile daily.

1. Requirements

Functional

1. Merchants create payment intents; customers pay by card
2. Merchants receive status via webhooks
3. Refunds

Non-functional

- Never double-charge (exactly-once effect)
- Durable and auditable; strongly consistent ledger
- ~10k TPS
- PCI: raw card data isolated

2. Back-of-envelope numbers

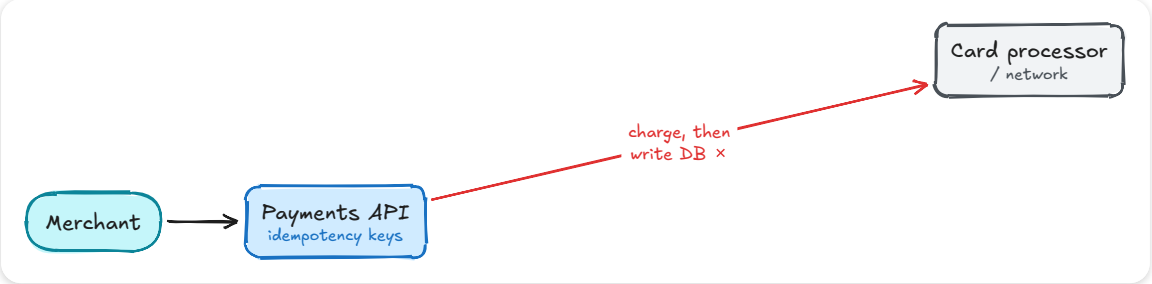
10k TPS is moderate for a well-indexed Postgres cluster. The hard part is **correctness under unknown outcomes**, not throughput.

3. Core entities and API

```
payment_intents(id, merchant, amount, status state machine)
ledger_entries(id, txn_id, account, debit, credit) append-only
outbox
```

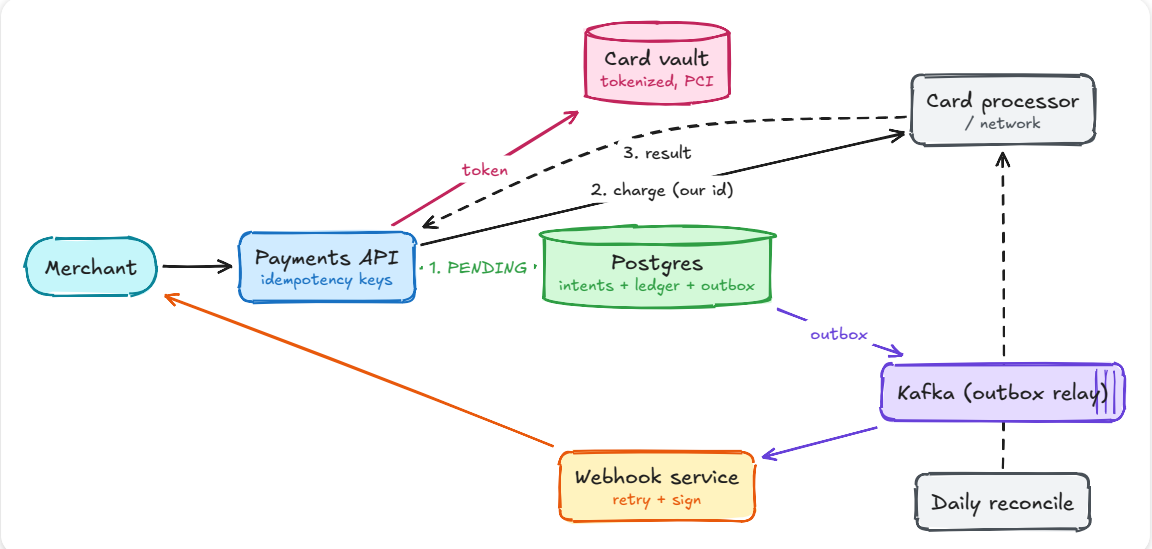
```
POST /payment_intents {amount, currency} + Idempotency-Key
POST /payment_intents/{id}/confirm {payment_method_token}
webhook → merchant: payment.succeeded / failed
```

4. v1: the simplest design, and where it breaks



Naive: call the processor, then write the DB. A crash in between charges the card with no record of it.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Naive: call the processor, then write the DB. A crash in between charges the card with no record of it.

2. Write the intent as **PENDING** first, call the processor with our intent ID as the idempotency key, then record the result. On timeout, query by our ID; never create a new charge.
3. Merchants send an Idempotency-Key on every mutating call (stored with a unique index). Card numbers are tokenized into an isolated vault; the rest of the system sees tokens.
4. Status changes write an **outbox** row in the same transaction; a relay publishes and webhooks are delivered with retries and signatures. Money moves as append-only double-entry ledger rows, reconciled daily against processor settlement files.

6. Deep dives

Processor times out?

Unknown outcome (DDIA Ch 8): keep PENDING, retry with the same idempotency key or query status by reference.

Webhook delivery?

Outbox in the same transaction; exponential backoff; merchants dedupe by event_id.

Why double-entry?

Every movement is a debit + credit summing to zero, so balances are derivable and auditable; never UPDATE amounts.

Distributed transaction with the bank?

Impossible. Design a state machine with compensation (refund) and reconciliation.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Merchants create payment intents; customers pay by card; Merchants receive status via webhooks; Refunds
5–8	Non-functional requirements	Never double-charge (exactly-once effect); Durable and auditable; strongly consistent ledger; ~10k TPS; PCI: raw card data isolated

Time	What you do	Content
8–10	Numbers that drive the design	10k TPS is moderate for a well-indexed Postgres cluster. The hard part is **correctness under unknown outcomes**, not throughput.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: Postgres → Card vault → Kafka (outbox relay) → Webhook service → Daily reconcile. Justify each addition with a number.
25–40	Deep dives	Processor times out? · Webhook delivery? · Why double-entry? · Distributed transaction with the bank?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- Using 2PC or assuming the processor call succeeds or fails cleanly.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Multi-step processes
- **DDIA:** Ch 8 unknown outcomes · Ch 12 end-to-end idempotency, integrity · Ch 11 outbox
- **Similar problems:** Online judge (LeetCode) · Job scheduler

Online judge (LeetCode)

Difficulty: Medium · **Patterns:** Multi-step processes · Long-running tasks · **DDIA:** Ch 11 queues · Ch 1 bursty load

You are running strangers' code on your servers, so isolation comes first.

TL;DR

- Run every submission in a locked-down sandbox: no network, read-only filesystem, strict CPU, memory and time limits.
- Queue submissions and return immediately; workers autoscale on queue depth and the client polls for results.
- Keep contest leaderboards in a Redis sorted set.

1. Requirements

Functional

1. Browse problems
2. Submit code in several languages; get pass/fail per test
3. Contest leaderboard

Non-functional

- Untrusted code fully isolated
- Result within ~5s
- Contest spikes (100k users submitting)
- Scalable

2. Back-of-envelope numbers

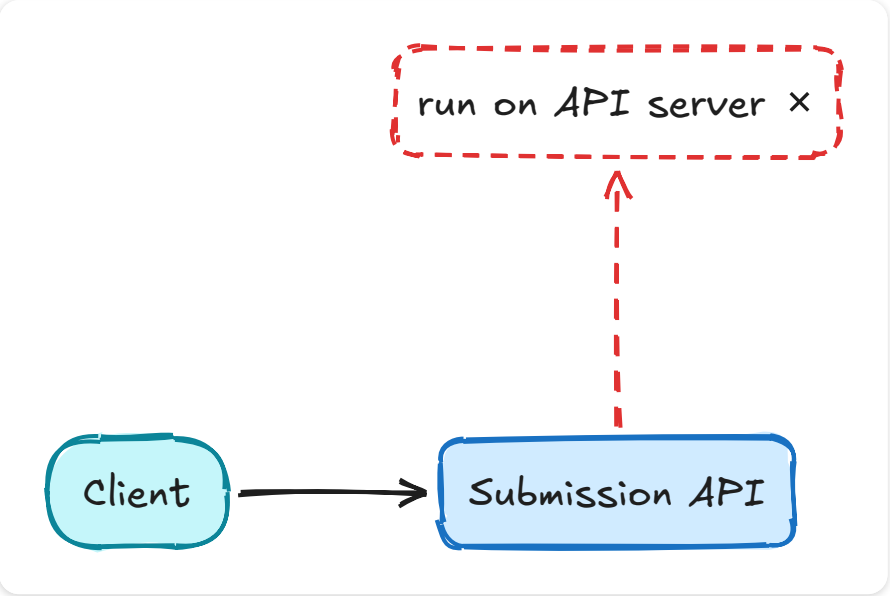
Contest: 100k users × ~5 submissions in 90 min ≈ **~100 submissions/s** peak; each run takes 1–5s of sandbox CPU, so ~500 concurrent sandboxes.

3. Core entities and API

```
problems
test_cases (S3)
submissions(id, user, problem, status, results)
leaderboard:{contest} ZSET
```

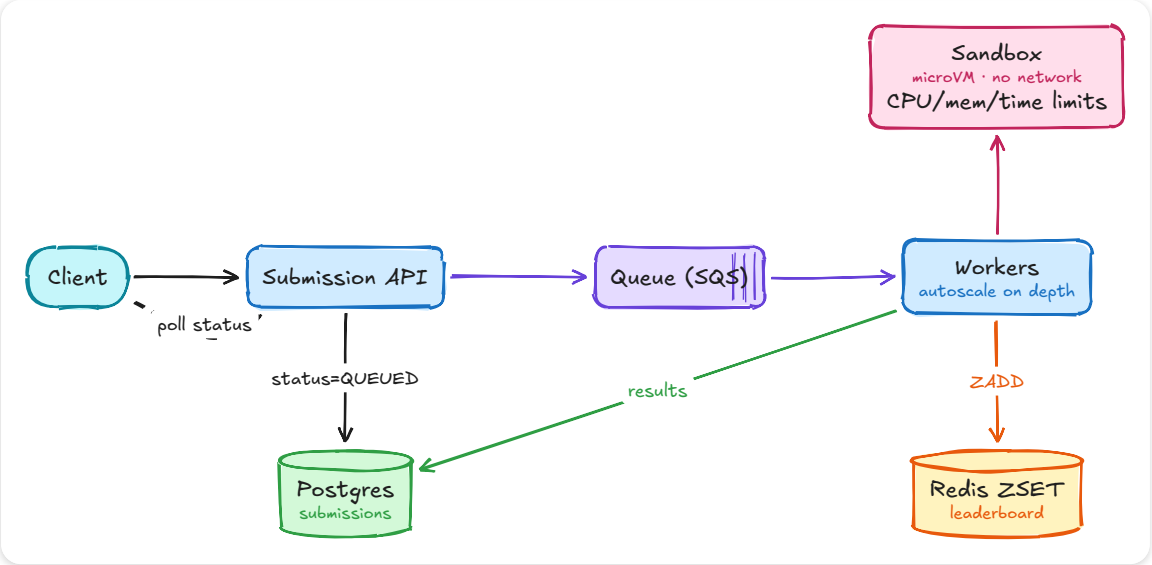
```
POST /problems/{id}/submissions {lang, code} → submission_id
GET /submissions/{id} (poll status)
GET /contests/{id}/leaderboard
```

4. v1: the simplest design, and where it breaks



Naive: run user code on the API servers. It is a security hole, and slow code blocks everything.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Naive: run user code on the API servers. It is a security hole, and slow code blocks everything.
2. Run every submission in an isolated sandbox (Firecracker/gVisor): read-only filesystem, no network, seccomp, strict CPU/memory/time limits.
3. Decouple with a queue: the API records QUEUED and returns immediately; autoscaled workers run tests and write results; the client polls status every second (the long-running-task pattern).
4. Contest leaderboard in a Redis sorted set; the score encodes points and time penalty.

6. Deep dives

Security layers?

MicroVM or gVisor, no network, read-only FS, seccomp filters, resource limits, kill on timeout.

Contest spike?

The queue absorbs it; scale workers on queue depth; pre-warm containers per language.

Polling vs push?

Polling every 1s is simple and fine at this scale; SSE is optional.

Leaderboard ties?

score = points × 1e6 − penalty_seconds.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Browse problems; Submit code in several languages; get pass/fail per test; Contest leaderboard
5–8	Non-functional requirements	Untrusted code fully isolated; Result within ~5s; Contest spikes (100k users submitting); Scalable
8–10	Numbers that drive the design	Contest: 100k users × ~5 submissions in 90 min ≈ **~100 submissions/s** peak; each run takes 1–5s of sandbox CPU, so ~500 concurrent sandboxes.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: Sandbox → Postgres → Queue (SQS) → Workers → Redis ZSET. Justify each addition with a number.
25–40	Deep dives	Security layers? · Contest spike? · Polling vs push? · Leaderboard ties?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- Executing user code with network access or on shared API hosts.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Multi-step processes · Long-running tasks
- **DDIA:** Ch 11 queues · Ch 1 bursty load

- **Similar problems:** Payment system (Stripe) · Job scheduler

Job scheduler

Difficulty: Hard · **Patterns:** Multi-step processes · Long-running tasks · **DDIA:** Ch 9 leader election · Ch 8 leases / fencing · Ch 6 time partitioning

Ten thousand jobs per second, each within two seconds of its scheduled time.

TL;DR

- Cron on one machine is a single point of failure that can't scale.
- Store executions partitioned by time; a poller moves soon-due jobs into a delay queue that releases each on time.
- Workers lease jobs with a visibility timeout, so crashed jobs are retried; jobs must be idempotent.

1. Requirements

Functional

1. Schedule jobs: one-off at time T, or cron
2. Execute jobs with a retry policy
3. View job status and history

Non-functional

- 10k job executions/s
- Start within 2s of scheduled time
- At-least-once; jobs idempotent
- Never lose a job; billions of jobs stored

2. Back-of-envelope numbers

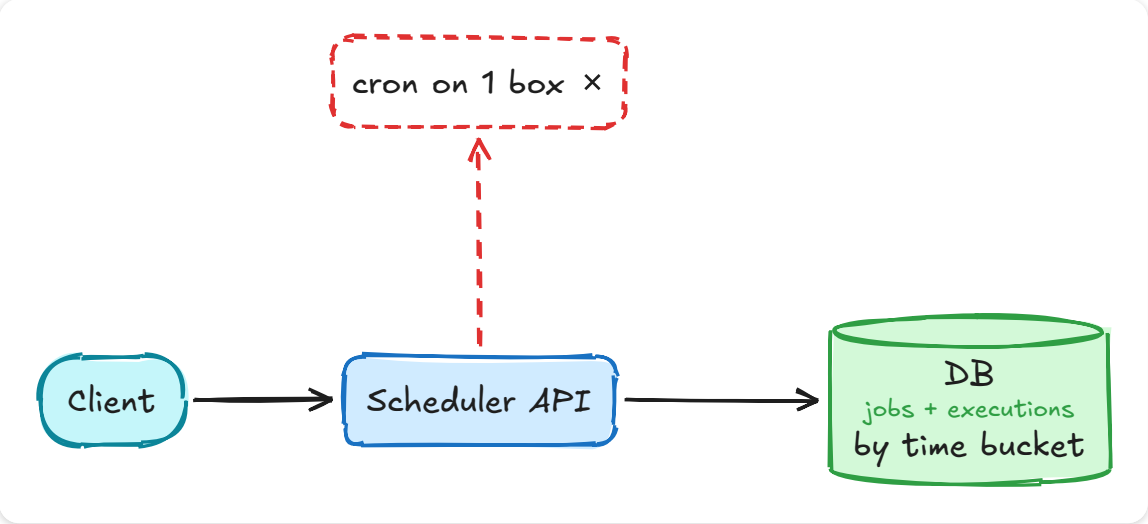
10k executions/s = 864M/day. A DB poll every second for "due now" across billions of rows will not hold, so use a two-layer design.

3. Core entities and API

```
jobs(id, user, schedule, payload)
executions(time_bucket, execution_id, job_id, scheduled_at, status, attempt) partitioned by time
bucket
```

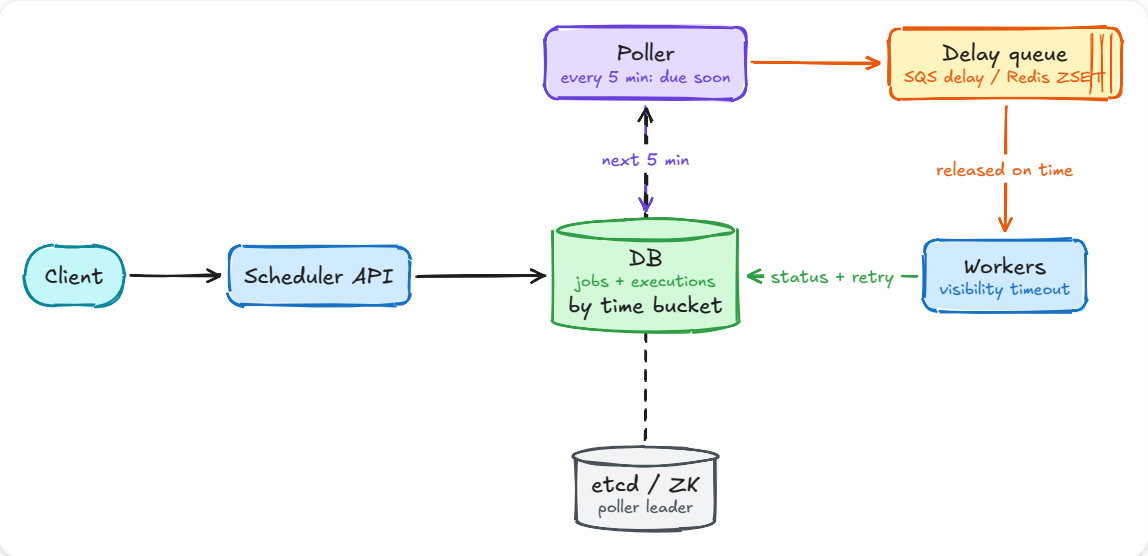
```
POST /jobs {schedule (cron | at), payload, retry_policy}
GET /jobs/{id}/executions
```

4. v1: the simplest design, and where it breaks



Naive: cron on one server. A single point of failure that can't scale.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Naive: cron on one server. A single point of failure that can't scale.

2. Store executions in a DB partitioned by time bucket; a poller fetches executions due in the next few minutes.
3. Two layers: the poller hands near-due jobs to a **delay queue** that releases each one at its exact time, giving second-level precision without hammering the DB.
4. Workers take jobs with a visibility timeout (lease). A crash means the job reappears and is retried, so jobs must be idempotent. Pollers are partitioned or leader-elected via etcd/ZooKeeper.

6. Deep dives

How do you hit 2s precision?

DB scan every few minutes → delay queue (SQS DelaySeconds ≤ 15 min, or a Redis ZSET by timestamp) releases on time.

Worker dies mid-job?

Lease expires and the job is retried. Heartbeats extend leases for long jobs; fencing token on completion writes.

Duplicate enqueues?

Dedupe by execution_id at the queue or worker.

User history queries?

Secondary index on (user_id, scheduled_at).

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Schedule jobs: one-off at time T, or cron; Execute jobs with a retry policy; View job status and history
5–8	Non-functional requirements	10k job executions/s; Start within 2s of scheduled time; At-least-once; jobs idempotent; Never lose a job; billions of jobs stored

Time	What you do	Content
8–10	Numbers that drive the design	10k executions/s = 864M/day. A DB poll every second for "due now" across billions of rows will not hold, so use a two-layer design.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: Poller → Delay queue → Workers → etcd / ZK. Justify each addition with a number.
25–40	Deep dives	How do you hit 2s precision? · Worker dies mid-job? · Duplicate enqueues? · User history queries?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- Polling the entire jobs table every second.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Multi-step processes · Long-running tasks
- **DDIA:** Ch 9 leader election · Ch 8 leases / fencing · Ch 6 time partitioning
- **Similar problems:** Payment system (Stripe) · Online judge (LeetCode)

Video platform (YouTube)

Difficulty: Hard · **Patterns:** Handling large blobs · Scaling reads · **DDIA:** Ch 10 batch DAGs · Ch 1 bandwidth math

Two hard problems: getting gigabyte uploads in reliably, and streaming smoothly on any connection.

TL;DR

- Upload directly to object storage with presigned, multipart, resumable URLs; bytes never touch your servers.
- Split videos into segments and transcode them in parallel into several resolutions.
- Serve segments from a CDN and let the player switch quality per segment (adaptive bitrate).

1. Requirements

Functional

1. Upload videos (up to many GB)
2. Watch videos with smooth playback on any network

Non-functional

- Resumable uploads
- Fast start, adaptive quality
- ~1M uploads/day; billions of views
- Metadata availability over consistency

2. Back-of-envelope numbers

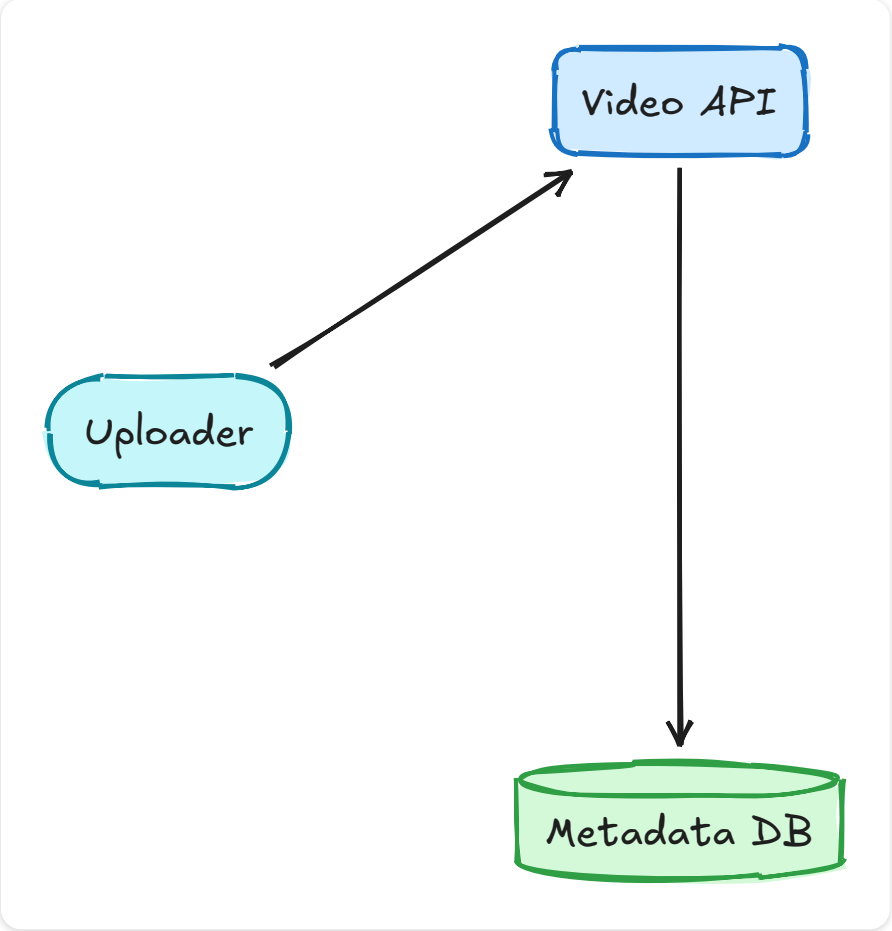
1M uploads/day × 1GB raw ≈ **1PB/day** raw, plus several renditions. Views are mostly served from the **CDN**, not origin.

3. Core entities and API

```
videos(id, uploader, title, status UPLOADING|PROCESSING|READY, manifest_url)
S3 raw/ + processed/{rendition}/{segment}
```

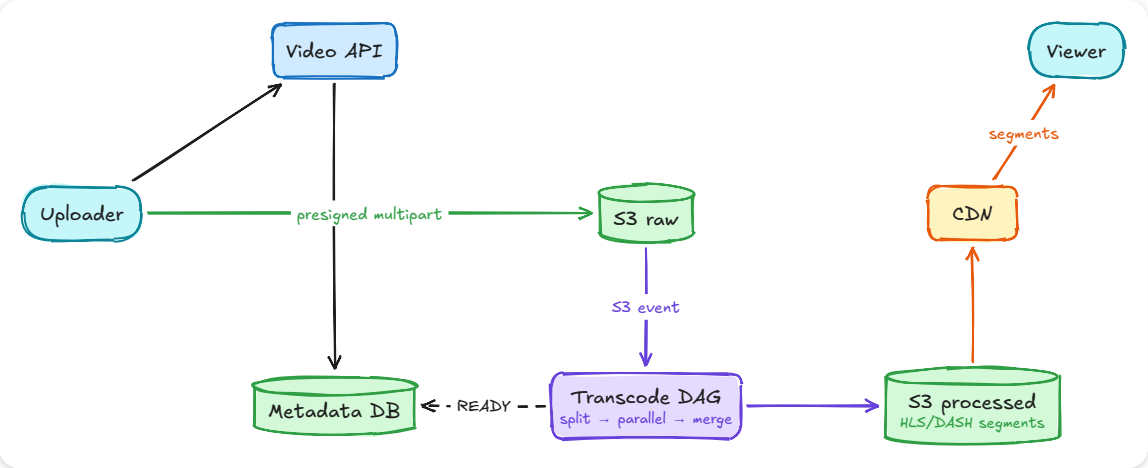
```
POST /videos → {video_id, presigned multipart URLs}
GET /videos/{id} → manifest URL
PUT parts directly to S3
```

4. v1: the simplest design, and where it breaks



Naive: upload gigabytes through your API servers. Slow, fragile, and expensive.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Naive: upload gigabytes through your API servers. Slow, fragile, and expensive.
2. The API returns **presigned multipart URLs**; the client uploads parts directly to S3 in parallel and resumes failed parts.
3. An S3 event starts a transcoding DAG: split into short segments, transcode each to several resolutions in parallel, write HLS/DASH manifests, and mark the video READY.
4. Viewers fetch the manifest and segments from the CDN; the player picks the quality per segment from measured bandwidth (adaptive bitrate).

6. Deep dives

Why presigned URLs?

Bytes never touch your servers; S3 handles scale; multipart gives parallel, resumable parts.

Fast transcoding?

Segment-level parallelism across many workers; the DAG orchestrator tracks completion.

Adaptive bitrate?

The manifest lists renditions; the player switches per segment.

Cost?

Transcode popular formats eagerly; long-tail renditions lazily.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Upload videos (up to many GB); Watch videos with smooth playback on any network
5–8	Non-functional requirements	Resumable uploads; Fast start, adaptive quality; ~1M uploads/day; billions of views; Metadata availability over consistency
8–10	Numbers that drive the design	1M uploads/day × 1GB raw ≈ **1PB/day** raw, plus several renditions. Views are mostly served from the **CDN**, not origin.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: S3 raw → Transcode DAG → S3 processed → CDN → Viewer. Justify each addition with a number.
25–40	Deep dives	Why presigned URLs? · Fast transcoding? · Adaptive bitrate? · Cost?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- Serving or uploading whole files through app servers.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Handling large blobs · Scaling reads
- **DDIA:** Ch 10 batch DAGs · Ch 1 bandwidth math
- **Similar problems:** File storage and sync (Dropbox) · Photo sharing (Instagram)

File storage and sync (Dropbox)

Difficulty: Easy · **Patterns:** Handling large blobs · **DDIA:** Ch 5 conflicts · Ch 11 change log / cursor

Syncing a 50 GB file after a one-byte edit should not mean uploading 50 GB again.

TL;DR

- Split files into chunks fingerprinted by hash, and upload only the chunks the server doesn't already have.
- Upload and download directly to object storage with presigned URLs.
- Other devices follow a per-user change log with a cursor and fetch only what changed.

1. Requirements

Functional

1. Upload and download files (up to 50GB)
2. Automatic sync across a user's devices
3. Share files and folders

Non-functional

- Resumable uploads
- Changes visible on other devices within seconds
- Durable (S3 11 nines)
- Bandwidth-efficient sync

2. Back-of-envelope numbers

Large files chunked at ~5MB: a 50GB file = 10k chunks. Only changed chunks are re-uploaded, so a 1-byte edit sends 5MB, not 50GB.

3. Core entities and API

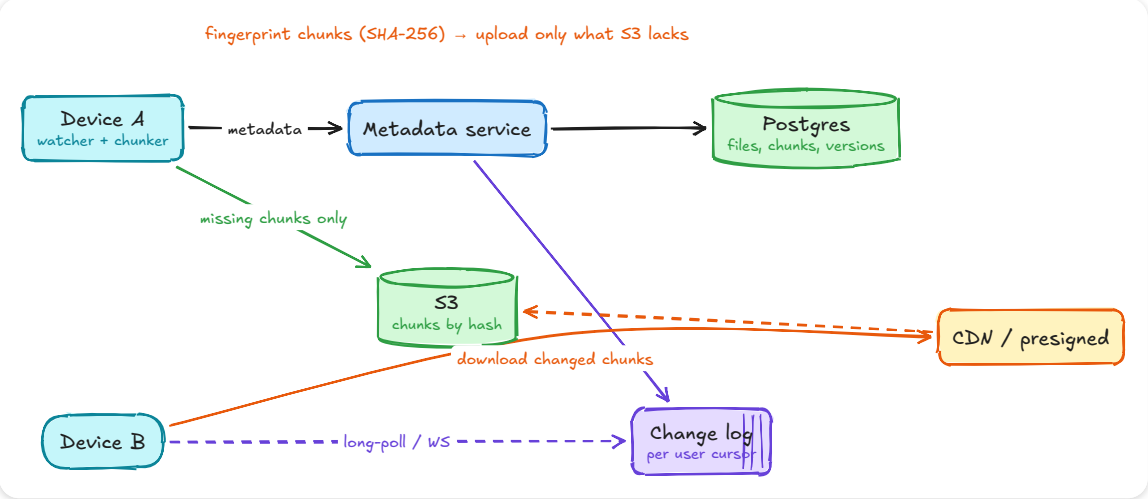
```
files(id, owner, path, version)
file_chunks(file_id, idx, sha256)
chunks(sha256, s3_key)
shares(file_id, user, perm)
```

```
POST /files {name, size, chunk_hashes[]} → missing chunks + presigned URLs
GET /changes?cursor → file metadata diffs
GET /files/{id} → chunk list + presigned download URLs
```

4. v1: the simplest design, and where it breaks

Naive: upload the whole file through the API. A 50GB upload dies at 99% and starts over.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Naive: upload the whole file through the API. A 50GB upload dies at 99% and starts over.
2. Split into ~5MB chunks; upload directly to S3 via presigned URLs, which is resumable and parallel.
3. Fingerprint each chunk, using **content-defined chunking** so an insert near the start doesn't shift every boundary. The server returns only missing chunks. Edits re-upload changed chunks, and identical chunks are stored once (dedupe).
4. Other devices follow a per-user change log with a cursor (long-poll or WebSocket), fetch the metadata diff, and download only the changed chunks.

6. Deep dives

Resume after failure?

The server knows which chunk hashes exist; the client re-sends only missing ones.

How do devices learn about changes?

Change log + cursor per device; long-poll or WebSocket notifications.

Edits on two offline devices?

Upload carries a base version; if stale, save a "conflicted copy" (DDIA Ch 5 multi-leader conflict).

Trust the client said "done"?

Verify via S3 event or ETags before marking the file version committed.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Upload and download files (up to 50GB); Automatic sync across a user's devices; Share files and folders
5–8	Non-functional requirements	Resumable uploads; Changes visible on other devices within seconds; Durable (S3 11 nines); Bandwidth-efficient sync
8–10	Numbers that drive the design	Large files chunked at ~5MB: a 50GB file = 10k chunks. Only changed chunks are re-uploaded, so a 1-byte edit sends 5MB, not 50GB.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: S3 → Change log → CDN / presigned. Justify each addition with a number.
25–40	Deep dives	Resume after failure? · How do devices learn about changes? · Edits on two offline devices? · Trust the client said "done"?

Time	What you do	Content
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

- Proxying file bytes through API servers, and re-uploading whole files on small edits.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Handling large blobs
- **DDIA:** Ch 5 conflicts · Ch 11 change log / cursor
- **Similar problems:** Video platform (YouTube) · Photo sharing (Instagram)

Photo sharing (Instagram)

Difficulty: Medium · **Patterns:** Handling large blobs · Scaling reads · **DDIA:** Ch 1 fan-out · Ch 11 async pipelines

A news feed plus a media pipeline: two well-known problems combined.

TL;DR

- Upload media straight to object storage; process thumbnails and sizes asynchronously; serve them from a CDN.
- Build feeds with fan-out on write into Redis, and pull celebrity posts at read time.
- Show a post only once its media has finished processing.

1. Requirements

Functional

1. Post photos and videos with captions
2. Follow users
3. View a feed of followed users' posts

Non-functional

- Feed p99 < 500ms
- Media upload and delivery fast globally
- 500M DAU
- Eventual consistency for the feed

2. Back-of-envelope numbers

500M DAU × 20 feed loads ≈ **~115k feed reads/s**. 100M posts/day; media ~2MB avg → 200TB/day to S3, served by CDN.

3. Core entities and API

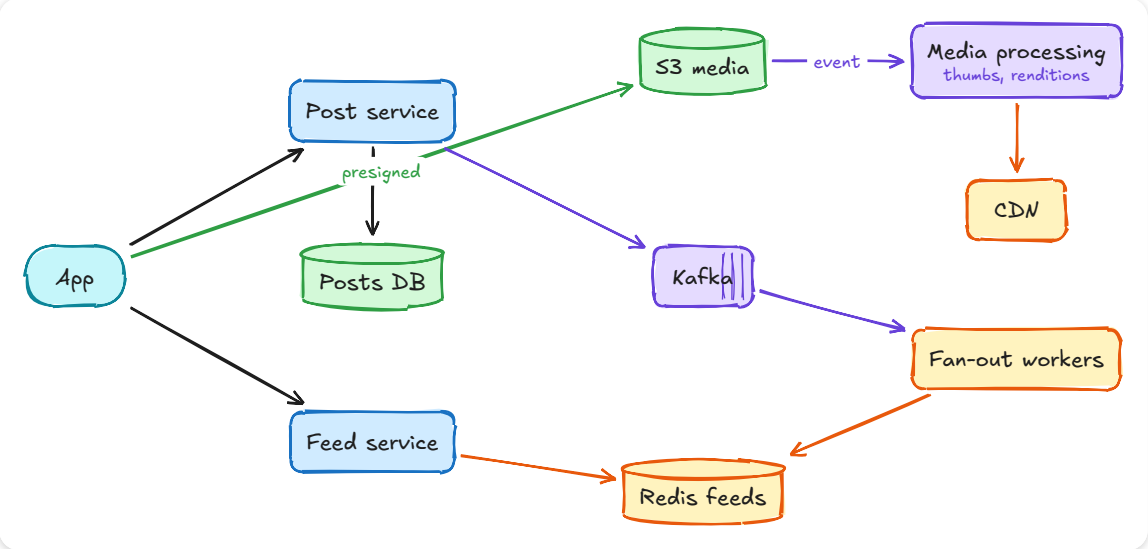
```
posts(id, author, caption, media_keys, status)
follows
feed:{user} ZSET
```

```
POST /posts {caption} → {post_id, presigned upload URL}
GET /feed?cursor
POST /users/{id}/follow
```

4. v1: the simplest design, and where it breaks

Naive: media through the API and a fan-out-on-read feed. Both break at this scale.

5. Evolving to the final design



Each step fixes one specific problem with the step before it:

1. Naive: media through the API and a fan-out-on-read feed. Both break at this scale.
2. Media: the client uploads directly to S3 with a presigned URL; an S3 event triggers thumbnails and renditions; the CDN serves them.
3. Feed: same as News Feed. New posts go to Kafka, workers push post IDs into followers' Redis feeds, and celebrities are pulled at read time (hybrid).

6. Deep dives

Post visible before media is processed?

Keep status PROCESSING; show it once renditions are ready (or show to the author only).

Celebrities?

Hybrid fan-out as in News Feed.

Like counts?

Redis INCR or sharded counters, flushed asynchronously to the DB.

Global latency?

CDN for media; regional read replicas or caches for the feed.

7. Interview script (45 minutes)

Time	What you do	Content
0–5	Functional requirements	Post photos and videos with captions; Follow users; View a feed of followed users' posts
5–8	Non-functional requirements	Feed p99 < 500ms; Media upload and delivery fast globally; 500M DAU; Eventual consistency for the feed
8–10	Numbers that drive the design	500M DAU × 20 feed loads ≈ **~115k feed reads/s**. 100M posts/day; media ~2MB avg → 200TB/day to S3, served by CDN.
10–13	Entities and API	Keep it short: the endpoints in §3, idempotency keys on writes.
13–25	v1, then evolve it	Draw v1 and show where it breaks. Then add, in order: S3 media → Media processing → CDN → Kafka → Fan-out workers → Redis feeds. Justify each addition with a number.
25–40	Deep dives	Post visible before media is processed? · Celebrities? · Like counts? · Global latency?
40–45	Wrap-up	Remaining bottlenecks, failure modes, what you would monitor.

8. Common mistakes

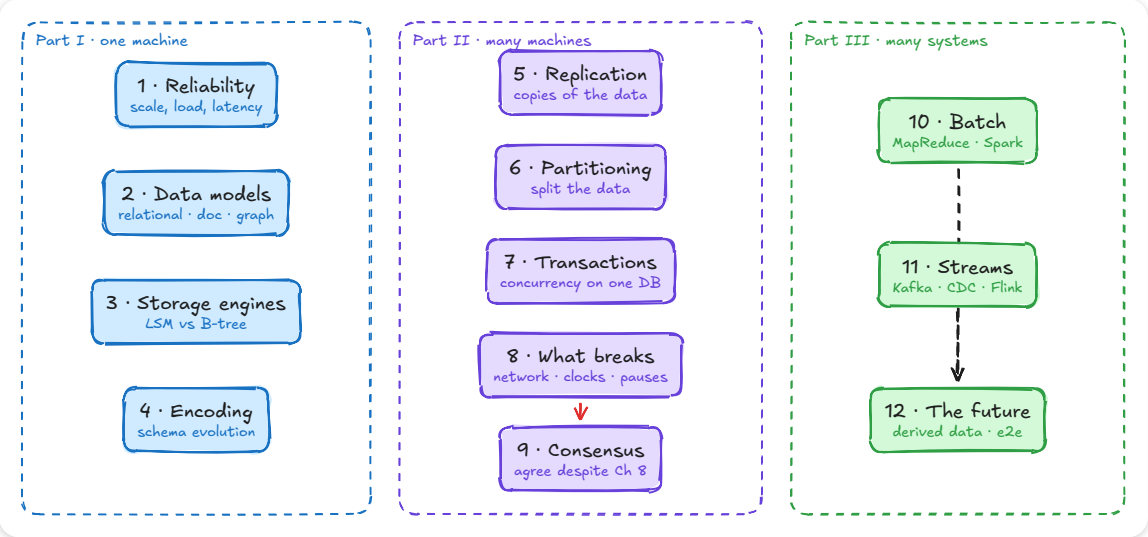
- Serving images from app servers or storing blobs in the database.
- Adding components before showing which number makes them necessary.
- Skipping failure handling: what happens on timeouts, retries and crashes.

Related

- **Patterns:** Handling large blobs · Scaling reads
- **DDIA:** Ch 1 fan-out · Ch 11 async pipelines
- **Similar problems:** Video platform (YouTube) · File storage and sync (Dropbox)

DDIA, chapter by chapter

> These DDIA pages are study notes in our own words and diagrams, connecting the book to interview problems. They don't replace the book; *Designing Data-Intensive Applications* by Martin Kleppmann is worth reading in full.



1. **Part I** is about one database on one machine: what "good" means, how data is shaped, stored, and encoded.
2. **Part II** spreads data across machines. Replication copies it, partitioning splits it. Ch 8 lists everything that breaks and Ch 9 shows how nodes still agree.
3. **Part III** connects many systems: the source of truth plus derived data (caches, search indexes, analytics) kept in sync with batch and stream processing.

Chapters

#	Chapter	Read it for
1	Reliable, scalable, maintainable	faults vs failures, load parameters, p99 latency, tail amplification
2	Data models	relational, document, graph, normalization
3	Storage and retrieval	append-only log, hash index, SSTable, LSM-tree
4	Encoding and evolution	backward compat, forward compat, Protobuf field tags, Avro schemas

#	Chapter	Read it for
5	Replication	leader / follower, sync vs async, failover, replication lag
6	Partitioning	key-range, hash partitioning, hot spots, secondary indexes
7	Transactions	ACID, dirty reads, read skew, MVCC
8	Trouble with distributed systems	partial failure, timeouts, clock skew, LWW danger
9	Consistency and consensus	linearizability, CAP, causality, Lamport clocks
10	Batch processing	Unix philosophy, MapReduce, HDFS, shuffle
11	Stream processing	message brokers, Kafka partitions, consumer groups, CDC
12	The future of data systems	derived data, unbundling the DB, Lambda vs Kappa, end-to-end argument

Which chapter shows up in which problem

Chapter	Concept	Shows up in
1 Reliability / load	Fan-out load parameter, p99	News Feed, Instagram, Twitter-style feeds
2 Data models	Document vs relational	Yelp, Ticketmaster, Dropbox metadata
3 Storage	LSM vs B-tree, column stores, geo / inverted index	Metrics (Datadog), Ad clicks, Yelp, Post Search
4 Encoding	Schema evolution, gRPC vs REST	Any microservice design; Kafka schemas
5 Replication	Lag, read-your-writes, failover, quorum	Bitly, Distributed Cache, WhatsApp, Google Docs
6 Partitioning	Shard key, hot keys, consistent hashing	Distributed Cache, Live Comments, Top-K, Rate Limiter

Chapter	Concept	Shows up in
7 Transactions	Lost update, write skew, row locks	Ticketmaster, Auction, Payments, Robinhood, Gopuff inventory
8 Distributed trouble	Timeouts, idempotency, fencing	Payments, Job Scheduler, Uber matching locks
9 Consensus	Leader election, linearizable CAS, sagas vs 2PC	Job Scheduler, Payments, unique usernames
10 Batch	MapReduce, joins, rebuildable outputs	Web Crawler, Top-K reconciliation, recommendations
11 Streams	Kafka, CDC, windows, exactly-once	Ad Clicks, Top-K, Metrics, Strava, Price Tracking
12 Future	Derived data, end-to-end idempotency	Payments, every system with search + cache

DDIA Chapter 1: Reliable, Scalable, Maintainable

> These DDIA pages are study notes in our own words and diagrams, connecting the book to interview problems. They don't replace the book; *Designing Data-Intensive Applications* by Martin Kleppmann is worth reading in full.

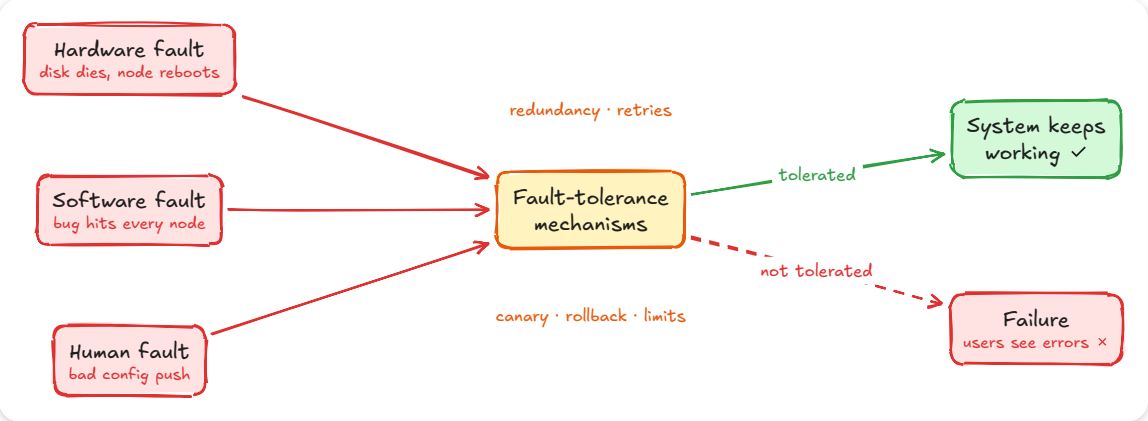
TL;DR

The three words every NFR discussion comes back to. This chapter gives them precise meaning: reliability means tolerating faults, scalability means describing load and performance with numbers, and maintainability means keeping the system easy to run and change.

Key ideas: faults vs failures · load parameters · p99 latency · tail amplification · scale up vs out

Fault ≠ failure

Reliability



1. A **fault** is one component going wrong. Faults are normal and constant at scale.
2. The design goal is mechanisms that absorb faults: replicas, retries, timeouts, canary deploys.
3. If the fault is absorbed, the user never notices. That is reliability.
4. A **failure** is when the whole system stops serving users. You design so faults don't become failures.

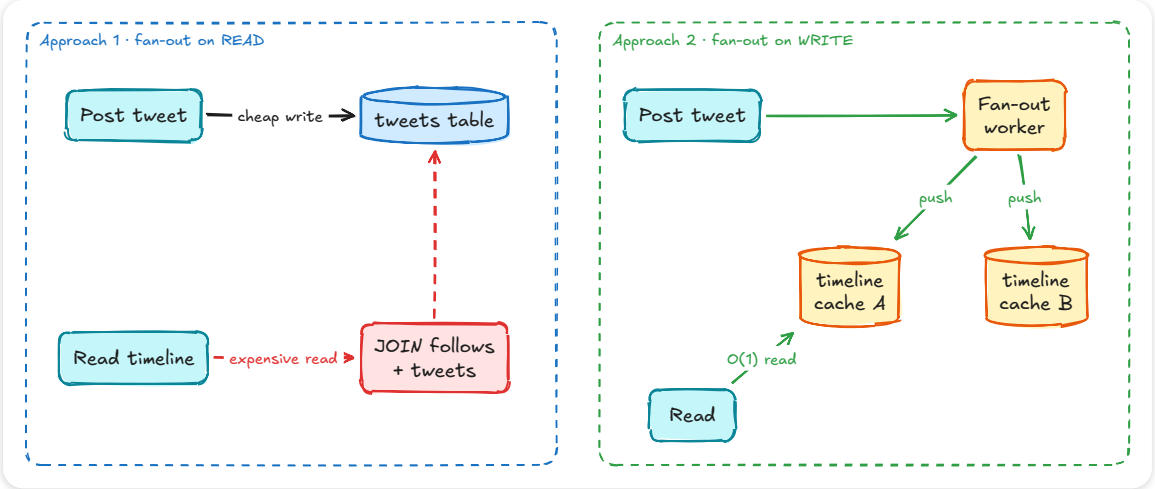
> **Note**
>
> **Scale makes faults routine:** Disks fail at roughly 2–5% per year. With 10,000 disks, expect **about one disk to die every day**.

> **Warning**
>
> **Most dangerous fault:** A **software** fault is correlated: the same bug hits every replica at once (leap second, bad deploy). Redundancy doesn't help, so you need staged rollouts.

Describing load: the Twitter timeline

Scalability · load parameters

Load is not "lots of users". Load is a specific number, such as reads per second or the fan-out per post. Twitter's key number: 4.6k tweets/sec written, 300k timeline reads/sec.

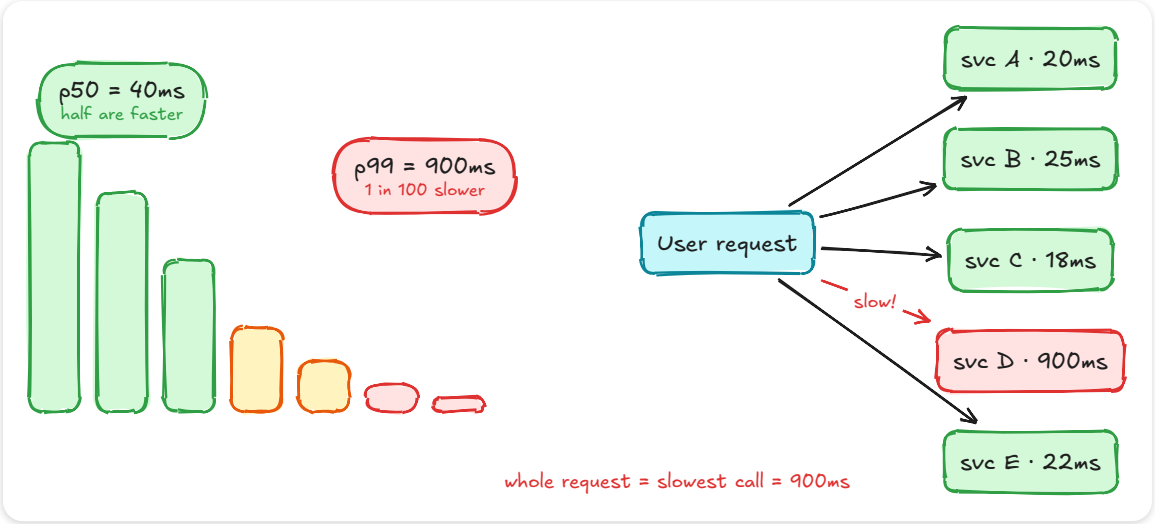


1. Approach 1: writes are cheap, just insert the tweet.
2. But every timeline read joins follows × tweets. At 300k reads/sec the database can't keep up.
3. Approach 2: when someone tweets, a worker pushes the tweet ID into each follower's timeline cache.
4. Reads become one cache lookup. The cost moved to write time. A celebrity with 30M followers means 30M pushes, so Twitter uses a **hybrid**: celebrities are fanned out on read.

> **Tip**
>
> **Say this in the interview:** "The load parameter that matters here is fan-out per post, not total users. That number decides push vs pull."

Describing performance: percentiles and tail latency

Scalability · performance



1. Most requests are fast, with a long tail of slow ones. Averages hide that tail.
2. Use percentiles. p50 is the typical user; p99 is your heaviest users, who often have the most data and are the most valuable.
3. **Tail latency amplification**: one user request fans out to many backends in parallel.
4. The user waits for the **slowest** backend. With 100 backends at 1% slow each, 63% of user requests hit at least one slow call.

> **Note**
>
> **The math:** P(no slow call) = 0.99^100 ≈ 0.37 → **63% of requests are slow**. This is why p99 of each service matters more than its average.

> **Tip**
>
> **Say this in the interview:** "I'll set the SLO on p99, not average. Because the page fans out to several services, the tail compounds, so I'll add timeouts and hedged requests."

Scaling up vs scaling out, and maintainability

Scalability · maintainability

Choice	What it means	When it wins
Scale up (vertical)	Bigger machine: more cores, RAM, NVMe	Simple, no distribution bugs. A single Postgres box handles 10–50k TPS. Start here.
Scale out (horizontal)	More machines, shared-nothing	When one box is not enough or you need fault tolerance. Stateless services scale out easily; stateful ones (databases) are hard.
Elastic	Add or remove capacity automatically	Unpredictable load. Manual scaling is simpler and has fewer surprises.
Operability	Easy for ops to keep running	Monitoring, good defaults, predictable behaviour, runbooks.
Simplicity	Remove accidental complexity	Good abstractions, for example SQL hiding storage and concurrency.
Evolvability	Easy to change later	Loose coupling, schema evolution (Ch 4), tests.

> **Warning**
>
> **Over-scaling:** There is no "magic scaling sauce". An architecture for 100k req/sec of 1KB reads looks nothing like one for 3 req/min of 2GB uploads. **Design for the load parameters you calculated.**

In the interview

> **Important**
>
> **Interviewer asks:** "How will this scale?"
> → Name the **load parameter** (reads/sec, fan-out, data size), calculate it, then say which component breaks first.

Important

Interviewer asks: "What's your latency target?"
→ **p99 < 200ms** for reads, not averages. Mention tail amplification if requests fan out.

Important

Interviewer asks: "What if a node dies?"
→ Separate fault from failure: "A node dying is a fault. Replicas plus health checks mean it never becomes a failure."

Tip

Say this in the interview: "Before choosing components, let me pin down the load parameters: DAU, read:write ratio, and peak QPS, because they decide whether we need any distribution at all."

Ch 2: Data models →

DDIA Chapter 2: Data Models & Query Languages

> These DDIA pages are study notes in our own words and diagrams, connecting the book to interview problems. They don't replace the book; *Designing Data-Intensive Applications* by Martin Kleppmann is worth reading in full.

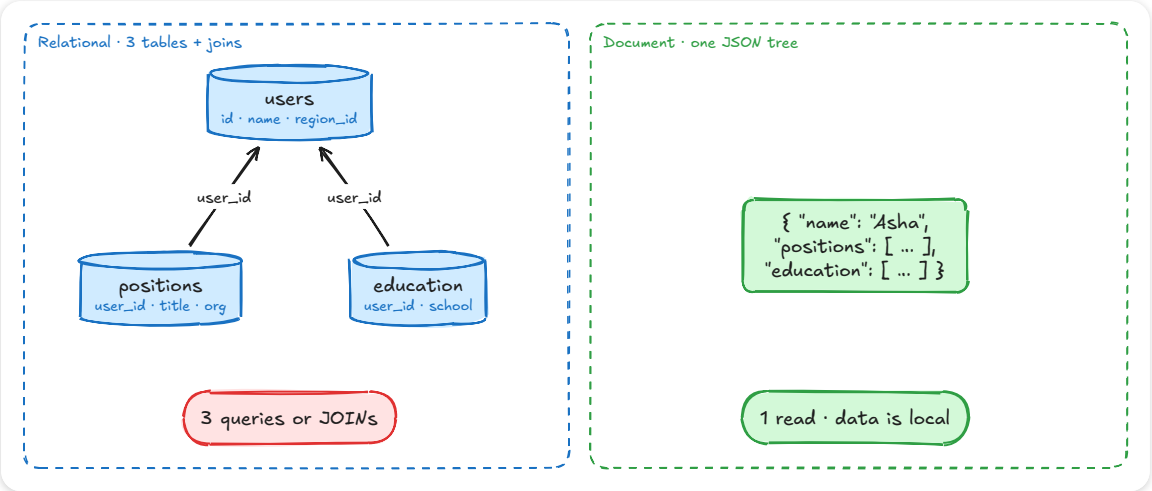
TL;DR

The shape of your data decides which database fits. Trees of data that are read together (one-to-many) suit documents. Data full of references (many-to-many) suits relational. Data where relationships are the point suits graphs.

Key ideas: relational · document · graph · normalization · schema-on-read · locality

One LinkedIn profile, two models

Relational vs document

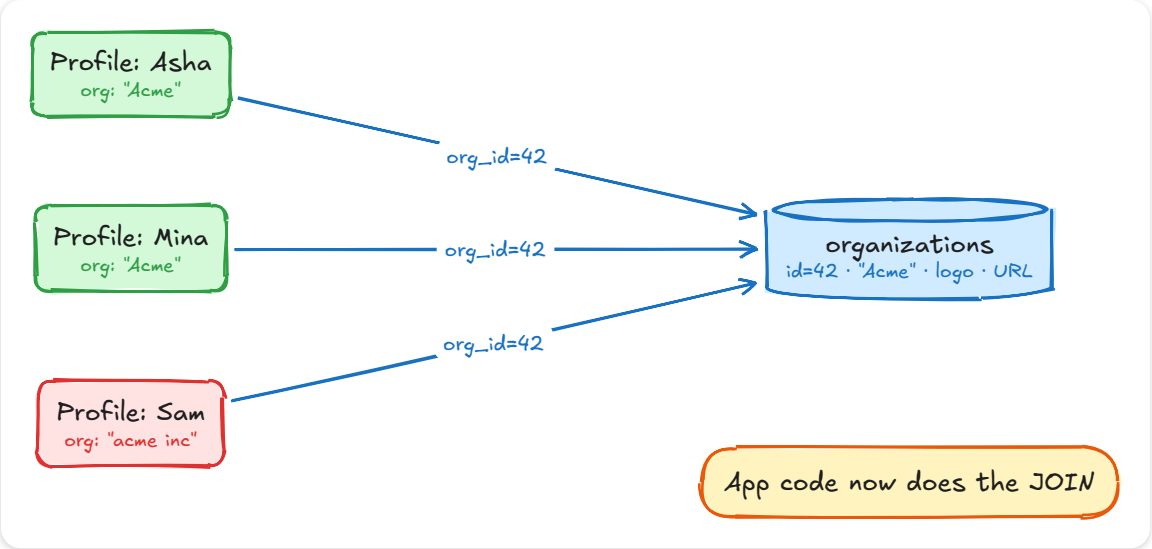


1. Relational: a profile is split across tables and reassembled with joins. Mapping objects to tables like this is called **impedance mismatch**.
2. Document: a profile is a self-contained tree. One read fetches it all because the data has **locality**. This works when the data is one-to-many and read as a whole.

> **Note**
>
> **When document wins:** Data forms a tree, you always load the whole tree, and there are few cross-references. Examples: a resume, a product page, an order with its line items.

Where documents break: many-to-one and many-to-many

Normalization



1. Storing the org name as text in every profile duplicates it.
2. Duplicates drift: a typo, a rename, or a logo change means updating thousands of documents.
3. Normalize: store an **ID** and keep the org in one place. That is a many-to-one relationship.
4. Document databases have weak join support, so the join moves into application code. As data gains references, the relational model wins again.

> **Tip**
>
> **Say this in the interview:** "Bookings reference hotels, users, and rooms, which is many-to-many, so I'll use Postgres. Each hotel's static content is one tree read as a whole, so it can live in a document store or as cached JSON."

Schema-on-write vs schema-on-read

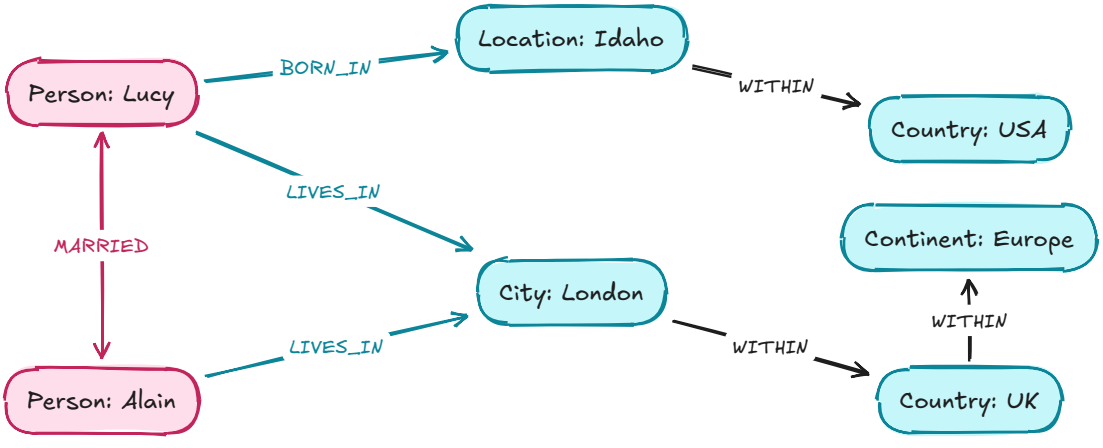
Schema flexibility

	Schema-on-write (relational)	Schema-on-read (document)
Analogy	Static typing: checked at compile time	Dynamic typing: checked at run time
Change a field	ALTER TABLE + migration (fast in Postgres; MySQL used to copy the whole table)	Just write new docs; code handles both old and new shapes
Good for	Uniform records with stable structure	Heterogeneous items: different shapes per type, or external data you don't control
Locality	Rows are spread across tables (but see Postgres JSONB, Spanner interleaving)	Whole doc stored together; rewrites the whole doc on update, so keep docs small
Joins	Native, optimized by query planner	Weak or none; done in application

> ⚠ **Warning**
>
> **"NoSQL is schemaless":** There is always a schema, it's just **implicit** in the reading code. You still pay for migrations, just later and in more places.

Graph model: when relationships are the data

Property graphs



1. Vertices (people, places) and edges (relationships), each with properties.
2. Edges chain at different depths: Idaho is in the USA, but a city might be in a county, in a state, in a country.
3. Anything can connect to anything. Adding a new kind of relationship needs no schema change.
4. A recursive query ("WITHIN, any number of hops") is one line in Cypher and painful in SQL (recursive CTE).

> **Note**
>
> **In interviews:** Fraud rings, "people you may know", knowledge graphs. Most interview problems don't need a graph DB; adjacency lists in Postgres or Redis are usually enough.

In the interview

Data shape	Pick	Interview example
Lots of cross-references, need transactions	Relational (Postgres)	Bookings, payments, inventory, users
Self-contained tree, read whole	Document (Mongo, DynamoDB item, JSONB)	Hotel content, product catalog page, chat message
Relationships are the query	Graph (Neo4j) or adjacency list	Social graph, recommendations, fraud

Data shape	Pick	Interview example
Huge write volume, simple key access	Wide-column (Cassandra)	Chat history, metrics, activity logs

> **Tip**
>
> **Say this in the interview:** "I start from access patterns. The two hot queries are X and Y, and both are key lookups with one join, so Postgres with the right indexes covers it. I'd only move to Cassandra if write volume passes about 50k/sec."

← Ch 1: Reliable, scalable, maintainable · Ch 3: Storage and retrieval →

DDIA Chapter 3: Storage & Retrieval

> These DDIA pages are study notes in our own words and diagrams, connecting the book to interview problems. They don't replace the book; *Designing Data-Intensive Applications* by Martin Kleppmann is worth reading in full.

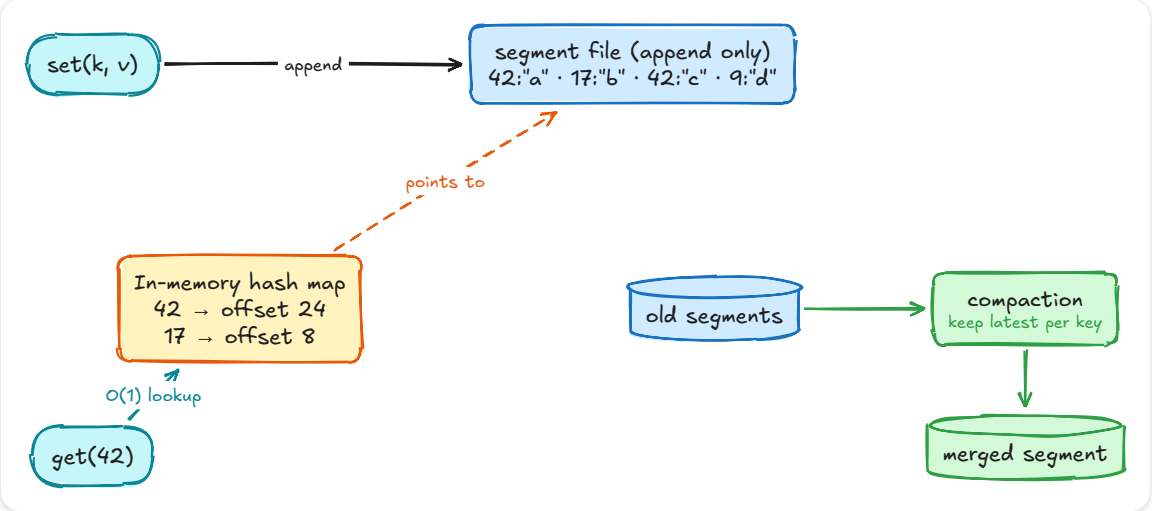
TL;DR

How a database actually stores bytes. Two families power almost everything: **log-structured** engines (LSM trees in Cassandra, RocksDB, Kafka) that append, and **page-oriented** engines (B-trees in Postgres, MySQL) that update in place. Plus why analytics needs column storage.

Key ideas: append-only log · hash index · SSTable · LSM-tree · B-tree · WAL · column store · star schema

The simplest database: append-only log + hash index

Log-structured storage (Bitcask)

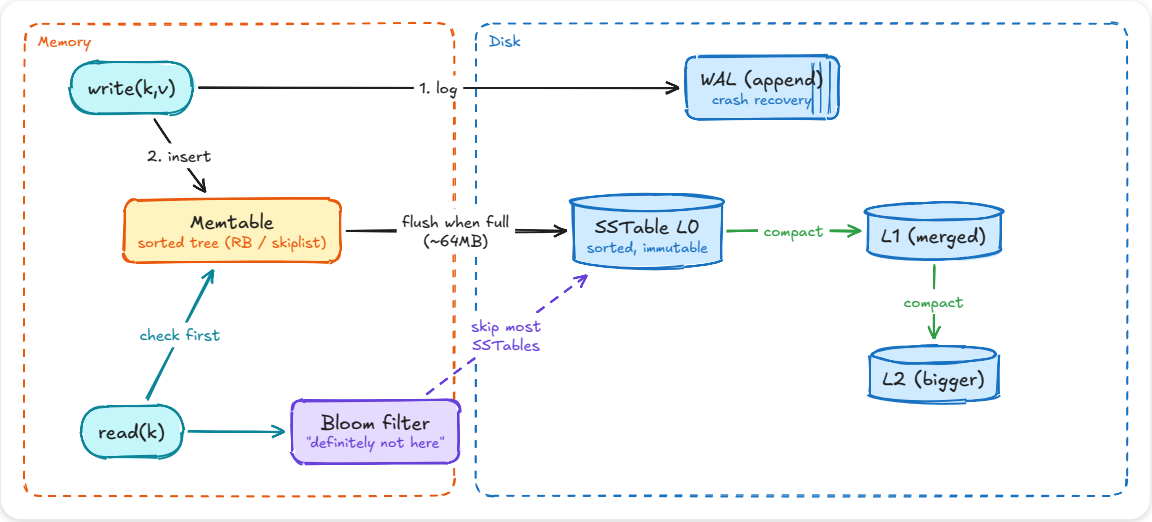


1. Every write is **appended** to the end of a file. Sequential writes are the fastest thing a disk can do.
2. An in-memory hash map stores, for each key, the byte offset of its latest value.
3. A read is one hash lookup plus one disk seek.
4. Old values pile up, so background **compaction** merges segments and keeps only the latest value per key.

> **Warning**
>
> **Limits:** All keys must fit in RAM, and range queries ("all keys between A and F") are impossible. SSTables fix both.

LSM-tree: how Cassandra and RocksDB write so fast

SSTables + LSM

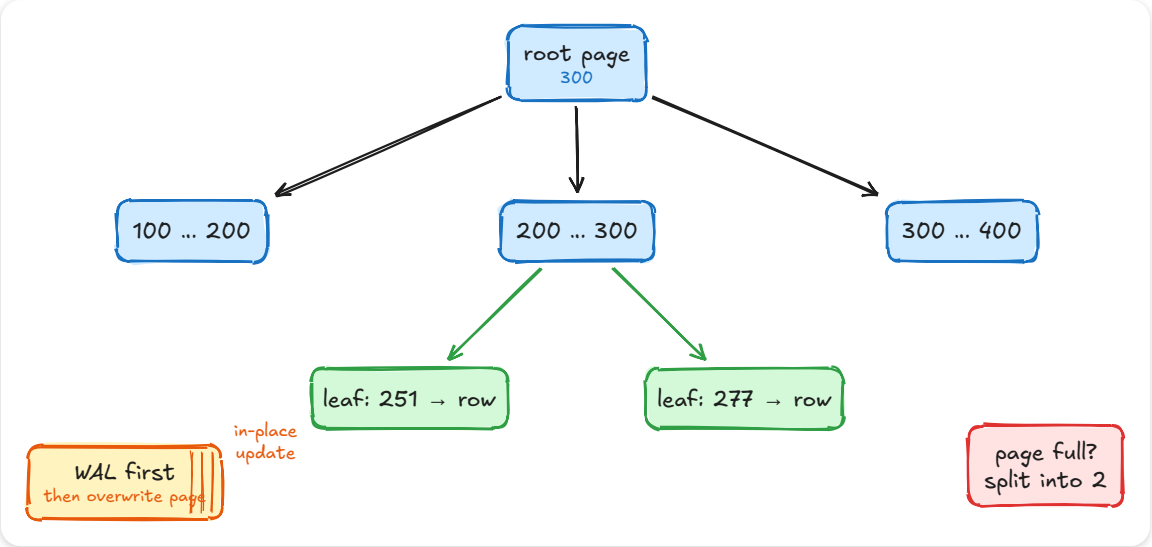


1. A write goes to a WAL (for durability) and an in-memory sorted **memtable**. No random disk I/O.
2. When the memtable is full it is flushed as an **SSTable**, a sorted immutable file, with one sequential write.
3. Background **compaction** merges SSTables into bigger levels and drops overwritten or deleted keys (tombstones).
4. A read checks the memtable, then SSTables from newest to oldest. **Bloom filters** skip files that can't contain the key, so reads cost more than writes.

> **Note**
>
> **Why it matters:** Sequential writes only, so you get very high write throughput. Cassandra handles about **10k+ writes/sec per node** easily. That makes LSM the default for metrics, logs, chat, and events.

B-tree: how Postgres and MySQL store data

Page-oriented storage



1. Data lives in fixed-size **pages** (4–16KB). The root page holds key ranges that point to child pages.
2. Each level narrows the range. A branching factor of about 500 means **4 levels hold 256TB**.
3. The leaf page holds the row (or a pointer to it). A lookup is O(log n) page reads, usually 3–4.
4. Updates overwrite the page **in place**, after writing to the WAL first so a crash mid-write can be repaired. A full page splits in two.

> **Note**
>
> **Why it matters:** Predictable reads, and each key exists in exactly one place, which makes row locks and transactions simple. That is why OLTP and ACID databases are B-tree based.

LSM vs B-tree: the trade-off you will be asked about

Comparison

	LSM-tree (Cassandra, RocksDB, HBase)	B-tree (Postgres, MySQL InnoDB)
Writes	Faster: sequential appends, less write amplification	Slower: random page writes + WAL (each write ≥ 2 writes)

	LSM-tree (Cassandra, RocksDB, HBase)	B-tree (Postgres, MySQL InnoDB)
Reads	Slower: may check several SSTables (Bloom filters help)	Faster, predictable: one path root → leaf
Space	Compresses well, little fragmentation	Pages partly empty after splits
Weak spot	Compaction competes with live traffic, so p99 spikes	Write-heavy workloads hit I/O limits
Transactions	Key may exist in many files, so locking is harder	Key in one place: easy range locks
Pick when	Write-heavy: metrics, logs, chat, IoT, events	Read-heavy, transactional: bookings, payments, users

Tip

Say this in the interview: "Ad clicks are write-heavy and append-only, around 10k/sec, so an LSM-based store like Cassandra fits. Bookings need transactions and point reads, so a B-tree store like Postgres fits."

Index types you should name in interviews

Secondary indexes

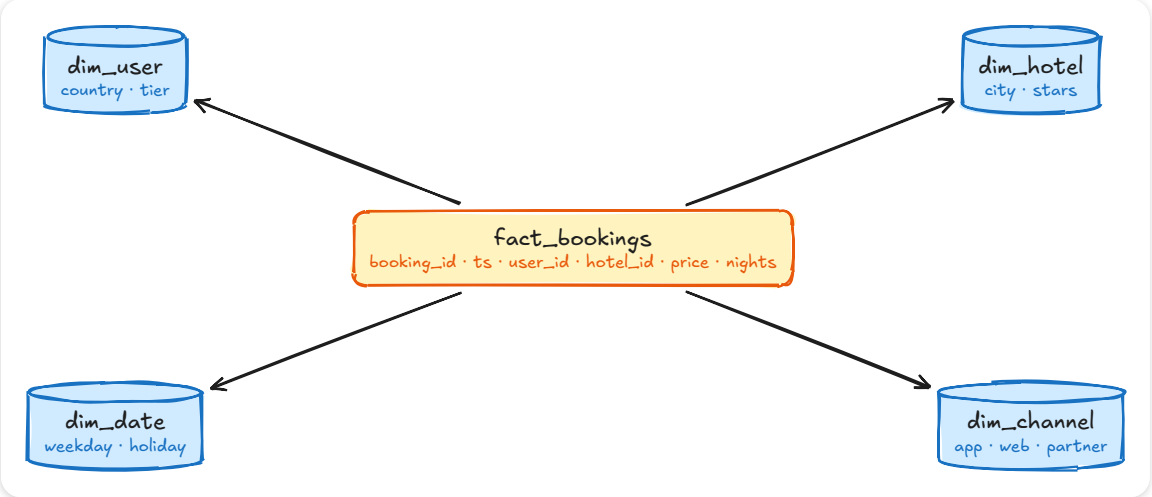
Index	How	Use for
B-tree (default)	Sorted tree on column(s)	Equality + range: WHERE created_at > X, ORDER BY
Composite	Sorted by (a, b): leftmost prefix rule	WHERE user_id=? ORDER BY ts DESC: one index scan
Covering / INCLUDE	Index stores extra columns	Index-only scans: no heap lookup
Clustered	Rows stored in index order	InnoDB primary key; range scans on PK are cheap
Hash	Hash map	Equality only; no ranges

Index	How	Use for
Geospatial	R-tree, geohash, quadtree (PostGIS)	"Hotels within 5km": Yelp, Uber, Gopuff
Inverted	term → list of doc IDs	Full-text search: Elasticsearch, Postgres GIN
Bloom filter	Probabilistic set membership	"Seen this URL?" in a web crawler, LSM reads

> ⚠ **Warning**
>
> **Indexes are not free:** Each index slows down every write and uses disk. Index for the **hot read queries** you named, not for every column.

OLTP vs OLAP, and the star schema

Analytics



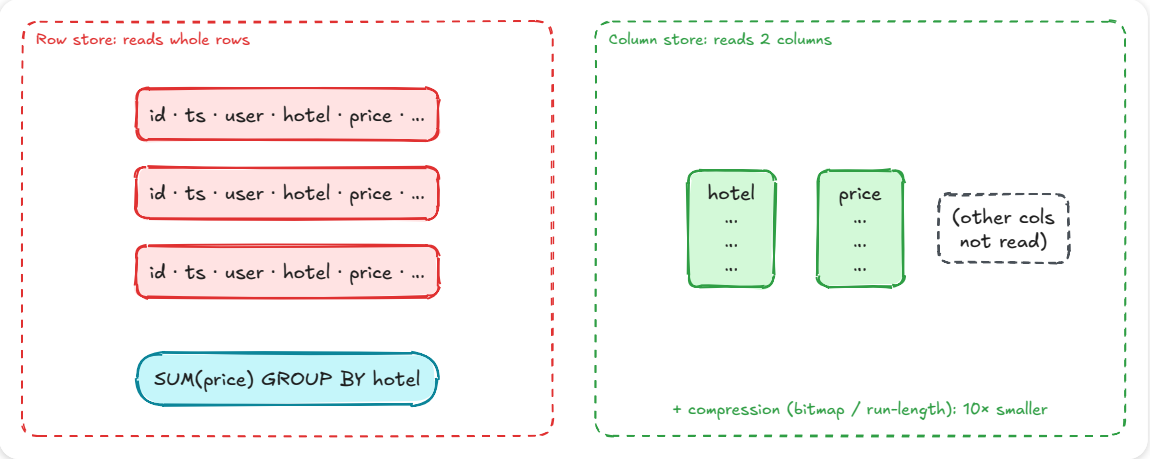
1. The **fact table** holds one row per event (each booking). It has billions of rows and is narrow but very long.
2. **Dimension tables** hold the who, what, where, and when. Queries filter on dimensions and aggregate facts: "revenue by city by weekday".

	OLTP	OLAP
Pattern	Few rows by key	Scan millions, aggregate

	OLTP	OLAP
Users	App / customers	Analysts, dashboards
Size	GB – TB	TB – PB
Engine	Postgres, MySQL	Snowflake, BigQuery, ClickHouse, Druid

Column-oriented storage: why analytics is fast

Column stores



1. A row store reads every column of every row from disk, even though the query needs two.
2. A column store keeps each column in its own file and reads only **hotel** and **price**, so it touches far less disk.
3. Values in one column look alike, so they compress very well. Vectorized CPU processing on top makes these scans 10–100× faster.

> **Tip**
>
> **Say this in the interview:** "The ad-click dashboard is an OLAP query, so I won't run it on the OLTP Postgres. Clicks flow through Kafka into a column store like ClickHouse or Druid for aggregation."

In the interview

Important

Interviewer asks: "Why Cassandra for this?"
→ "LSM-tree storage: writes are sequential appends to memtable + commit log, so write throughput is very high. Reads are slower, which is fine for this append-heavy workload."

Important

Interviewer asks: "How do you make this query fast?"
→ Name the **exact index**: "a composite index on (user_id, created_at DESC) turns it into a single index range scan."

Important

Interviewer asks: "Where do analytics queries run?"
→ Never on the OLTP primary. Use CDC or Kafka into a column store (ClickHouse, BigQuery, Druid).

Important

Interviewer asks: "What does a WAL do?"
→ Log the change before applying it. After a crash, replay the log. The same idea powers replication (Ch 5) and CDC (Ch 11).

← Ch 2: Data models · Ch 4: Encoding and evolution →

DDIA Chapter 4: Encoding & Evolution

> These DDIA pages are study notes in our own words and diagrams, connecting the book to interview problems. They don't replace the book; *Designing Data-Intensive Applications* by Martin Kleppmann is worth reading in full.

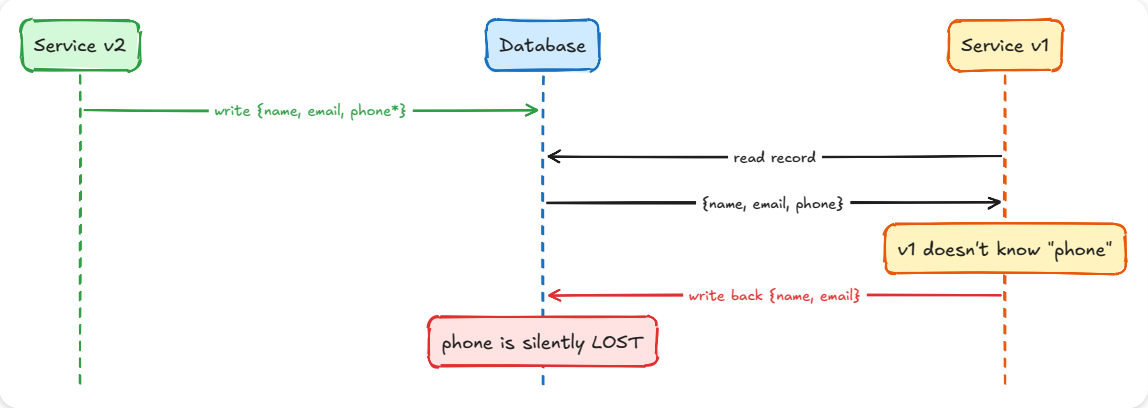
TL;DR

Data outlives code. During a rolling deploy, old and new versions of your service run at the same time, reading each other's data. Encoding formats such as Protobuf, Avro, and JSON decide whether that works or silently corrupts data.

Key ideas: backward compat · forward compat · Protobuf field tags · Avro schemas · schema registry · REST / RPC / messages

Rolling upgrade: old and new code at the same time

Why compatibility matters

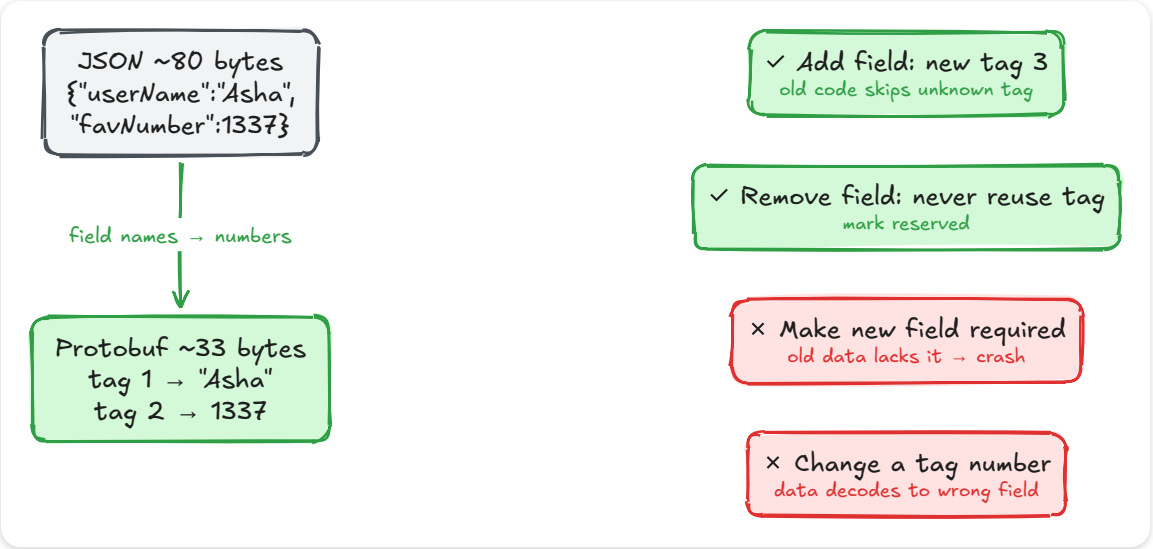


1. New code (v2) adds a field, **phone**, and writes it.
2. Old code (v1), still running on other nodes, reads that record.
3. v1 decodes it into its own model class, which has no phone field.
4. v1 updates the name and writes the whole object back, and phone is gone. Old code must **preserve unknown fields**.

> **Note**
>
> **The two directions: Backward compatible**: new code reads old data (easy).
> **Forward compatible**: old code reads new data (hard, which is this slide).

Protobuf / Thrift: field tags make evolution safe

Binary encoding

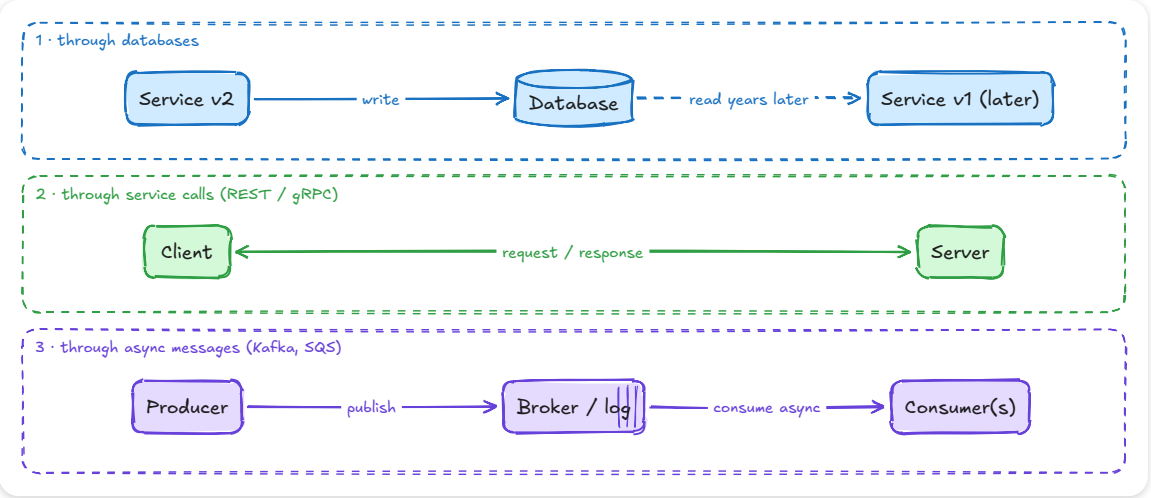


1. JSON repeats field names in every record and has no schema. Large, and number types are ambiguous (precision is lost above 2^{53}).
2. Protobuf replaces names with **tag numbers** plus a schema file, so records are about 2–3× smaller and faster to parse.
3. Rules that keep compatibility: add new **optional** fields with new tags; never reuse a removed tag.
4. Breaking changes: required fields, renumbering tags, or changing a type in an incompatible way.

> **Note**
>
> **Avro alternative:** Avro has no tags. The reader resolves the **writer's schema** against its own **reader schema** by field name, so a **schema registry** stores every version. This is common with Kafka.

Three ways data flows between processes

Modes of dataflow



1. **Database**: like sending a message to your future self. Data written years ago must still decode, so "data outlives code".
2. **Service calls**: a network call is not a local function call. It can time out, be retried and run twice, or have unknown outcome. Version your APIs.
3. **Messages**: the sender doesn't wait. A broker buffers, retries, and fans out. Producer and consumer evolve independently, so a schema registry keeps them compatible.

> **Warning**
>
> **RPC illusion:** "Just call it like a function" hides timeouts, retries, and partial failure. Design every remote call as **idempotent** with a timeout.

In the interview

> **Important**
>
> **Interviewer asks:** "How do you deploy schema changes with zero downtime?"
> → **Expand → migrate → contract**: add the nullable column, deploy code that writes both, backfill, switch reads, and only then drop the old column.

Important

Interviewer asks: "How do Kafka producers and consumers evolve?"
→ Avro or Protobuf with a **schema registry** enforcing backward-compatible changes. Consumers upgrade on their own schedule.

Important

Interviewer asks: "REST or gRPC between services?"
→ gRPC (Protobuf, HTTP/2) for internal high-QPS calls; REST/JSON for public APIs and browsers.

Tip

Say this in the interview: "Every message carries a schema version, and we only add optional fields. That lets the booking service deploy without coordinating with its consumers."

← Ch 3: Storage and retrieval · Ch 5: Replication →

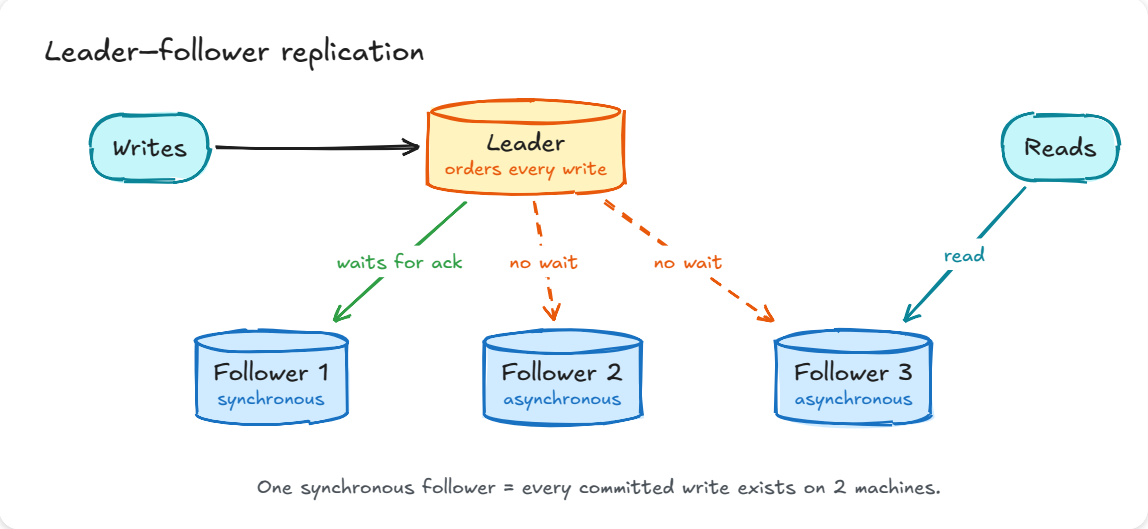
DDIA Chapter 5: Replication

> These DDIA pages are study notes in our own words and diagrams, connecting the book to interview problems. They don't replace the book; *Designing Data-Intensive Applications* by Martin Kleppmann is worth reading in full.

TL;DR

- Replication means keeping copies of the same data on several machines: to **survive failures**, to **serve more reads**, and to be **closer to users**.
- Copying data is easy. The hard part is **changes**: who accepts writes, how changes reach the copies, and what readers see while the copies disagree.
- Three designs: **single leader** (the default), **multi-leader** (writes in several regions), and **leaderless** (any replica accepts writes, using quorums).
- Most interview bugs here are **replication lag** bugs: a user writes something, reads it back from a replica, and it isn't there yet.

1. Leader–follower (single-leader) replication



- All writes go to **one leader**, which decides their order.
- The leader sends a stream of changes (its replication log) to every follower, and each follower applies them in the same order.
- Reads can go to followers. That spreads the load, but followers may be slightly behind.

Postgres, MySQL, MongoDB, Redis and Kafka (per partition) all work this way.

Synchronous or asynchronous?

	Synchronous follower	**Asynchronous follower**
Leader waits for it before confirming a write	Yes	No
Write latency	Higher; a slow follower slows every write	Unaffected
If the leader dies right after confirming	The write survives on the follower	Recent writes may be **lost**

The common setup is **semi-synchronous**: one synchronous follower and the rest asynchronous. Every confirmed write then exists on at least two machines, without one slow follower blocking everything.

When the leader dies (failover)

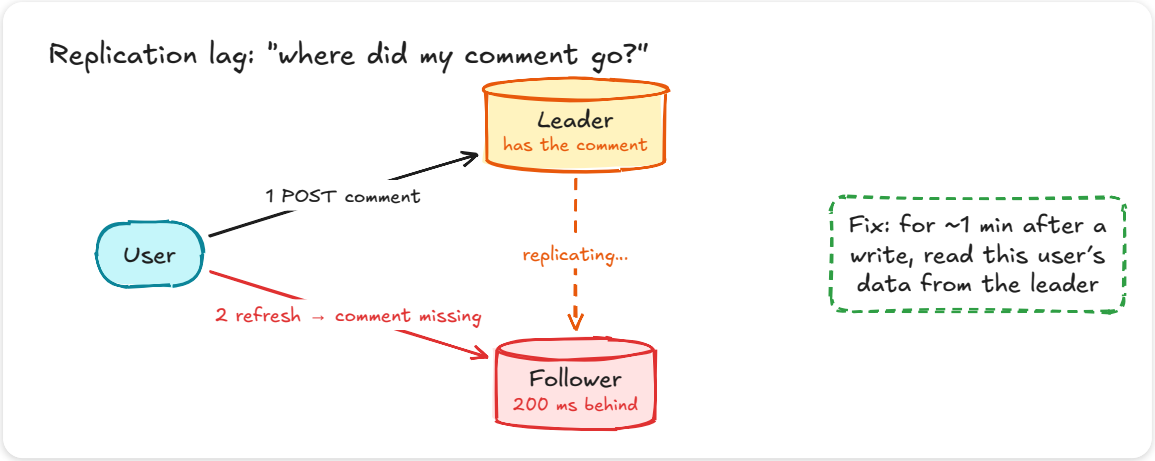
1. **Detect** it, usually because it stops answering for N seconds. Too short a timeout causes needless failovers; too long means more downtime.
2. **Promote** the most up-to-date follower.
3. **Reconfigure** clients and the remaining followers to use the new leader.

What goes wrong:

- **Lost writes:** with asynchronous replication, writes the old leader confirmed but never sent are gone.
- **Split brain:** the old leader comes back and still believes it's the leader, so two nodes accept writes. You need **fencing**: shut it out, or reject anything stamped with its old term number.

> **Tip**
>
> **Say this in the interview:** "Failover is automatic, but with async replication we can lose the last moment of writes. For payments I'd require a synchronous replica before confirming."

2. Replication lag: the bugs users actually see



Followers are usually milliseconds behind, but under load it can be seconds or more. Three guarantees you may need:

Guarantee	The bug without it	Common fix
Read-your-writes	"I posted a comment, refreshed, and it's gone."	Read the user's own data from the leader for a short window after they write, or until a replica has caught up to their last write.
Monotonic reads	"I saw 3 comments, refreshed, and now there are 2."	Pin each user to one replica, for example by a hash of their user id.
Consistent prefix	"I saw the reply before the question."	Keep related writes in the same partition, for example chat messages by conversation id.

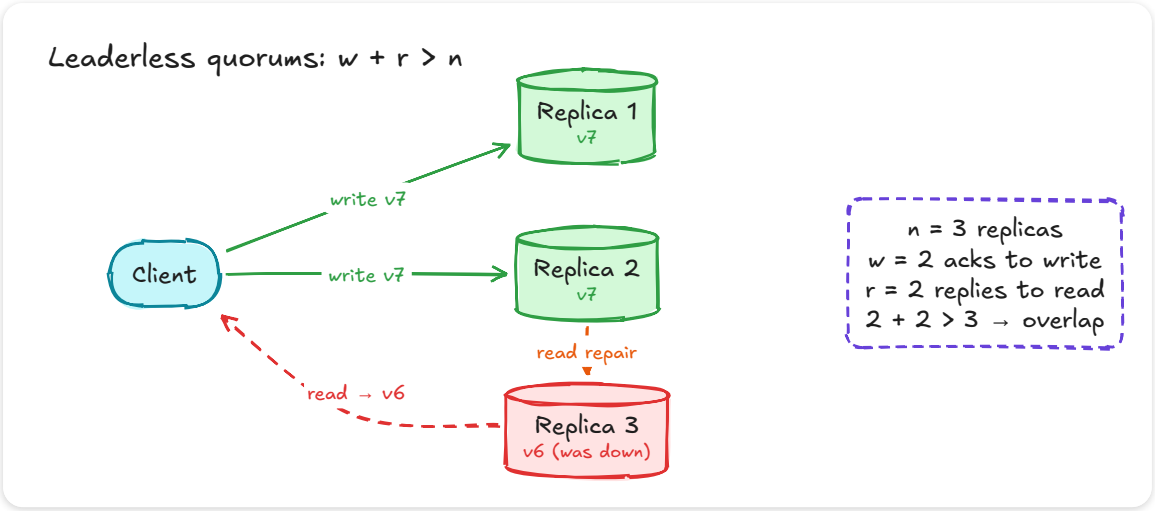
3. Multi-leader replication

Each region has its own leader that accepts local writes and syncs with the other regions asynchronously. Writes are fast everywhere and survive a region outage, **but two regions can change the same record at the same time**, and that conflict must be resolved somehow:

- **Last write wins:** simple, but it silently throws one write away.
- **Merge** the values, or keep both and ask the user, as file-sync tools do with "conflicted copy".
- **Conflict-free data types (CRDTs):** counters, sets and text editing that merge automatically.

The best strategy is usually to **avoid conflicts**: give each user or record a home region, so all its writes go to one leader.

4. Leaderless replication and quorums



There's no leader. The client (or a coordinator) sends each write to all *n* replicas and waits for *w* acknowledgements, and reads from *r* replicas.

- If **w + r > n**, the set of replicas you read from overlaps the set that took the latest write, so at least one reply is up to date. Pick the newest version.
- **Read repair** writes the newest value back to stale replicas you happened to read. A background anti-entropy process fixes the rest.
- Common setting: n = 3, w = 2, r = 2, which tolerates one replica down for both reads and writes.

Cassandra and DynamoDB-style systems use this. The catch: concurrent writes to the same key still need conflict handling, and "last write wins" by timestamp can drop data when clocks disagree.

5. How this shows up in interviews

Interviewer asks	Answer from this chapter
"Your primary database dies. What happens?"	Automatic failover to a replica in tens of seconds. With async replication the last writes can be lost, so use a synchronous replica where that's unacceptable.

Interviewer asks	Answer from this chapter
"A user updates a price, refreshes, and sees the old one."	Replication lag. Read-your-writes: route that user to the primary briefly.
"Active-active across regions?"	Multi-leader with conflicts. Prefer a home region per user or entity, and CRDTs for counters.
"Why Cassandra rather than Postgres replicas?"	Leaderless: no failover pause and writes keep working with a node down. The price is conflict handling and weaker consistency.

Related

- **Problems:** Ticketmaster: the seat map can come from a lagging replica; the booking must not.
- **Core concepts:** Caching has the same staleness trade-off as reading from replicas.
- **Pattern:** Scaling reads

← Ch 4: Encoding and evolution · Ch 6: Partitioning →

DDIA Chapter 6: Partitioning (Sharding)

> These DDIA pages are study notes in our own words and diagrams, connecting the book to interview problems. They don't replace the book; *Designing Data-Intensive Applications* by Martin Kleppmann is worth reading in full.

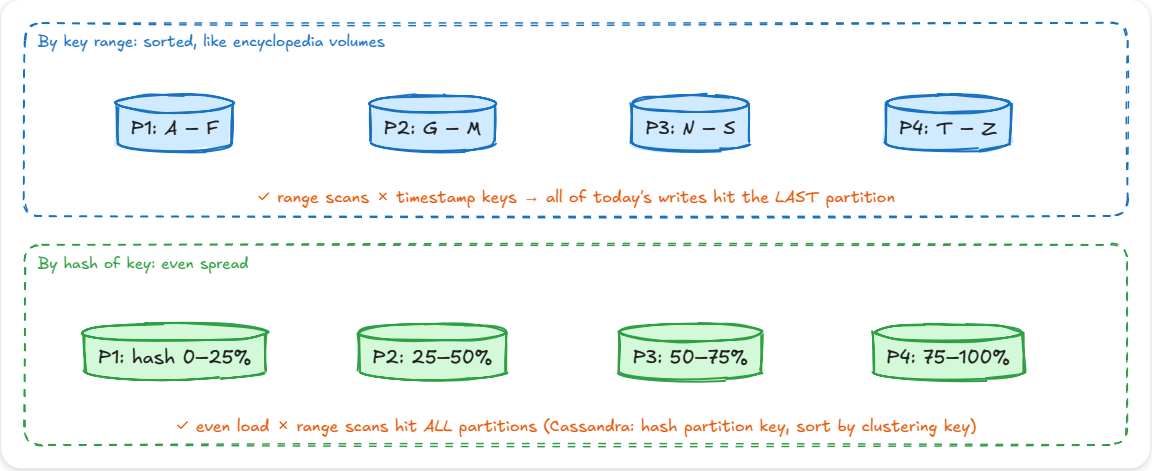
TL;DR

When data or write load is too big for one machine, split it. Each record belongs to exactly one partition. The goals are to spread data and load evenly, avoid **hot spots**, and move as little data as possible when nodes are added.

Key ideas: key-range · hash partitioning · hot spots · secondary indexes · rebalancing · consistent hashing · request routing

Key-range vs hash partitioning

How to split

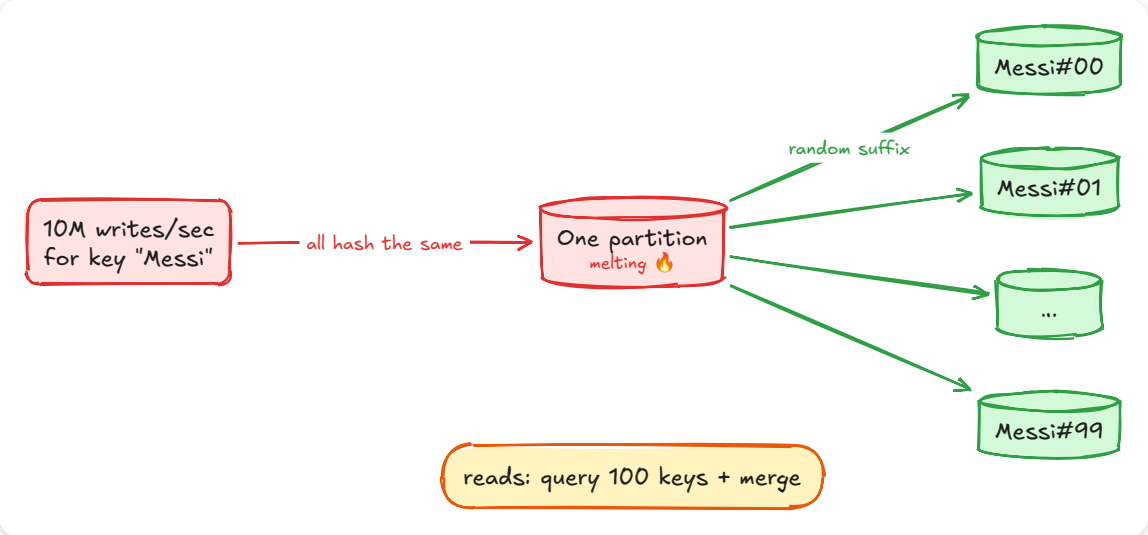


1. **Key range**: efficient range queries. But sequential keys (timestamps, auto-increment IDs) send all writes to one partition, a hot spot.
2. **Hash**: hash(key) spreads keys evenly, so load is uniform. You lose range queries on the key. The compound-key trick: hash the first part (user_id), sort within it by the second (timestamp).

> **Tip**
>
> **Say this in the interview:** "I'll partition messages by hash(conversation_id) and cluster by message timestamp, so one conversation's history is a single-partition range scan."

Skew and hot spots: the celebrity problem

Hot keys

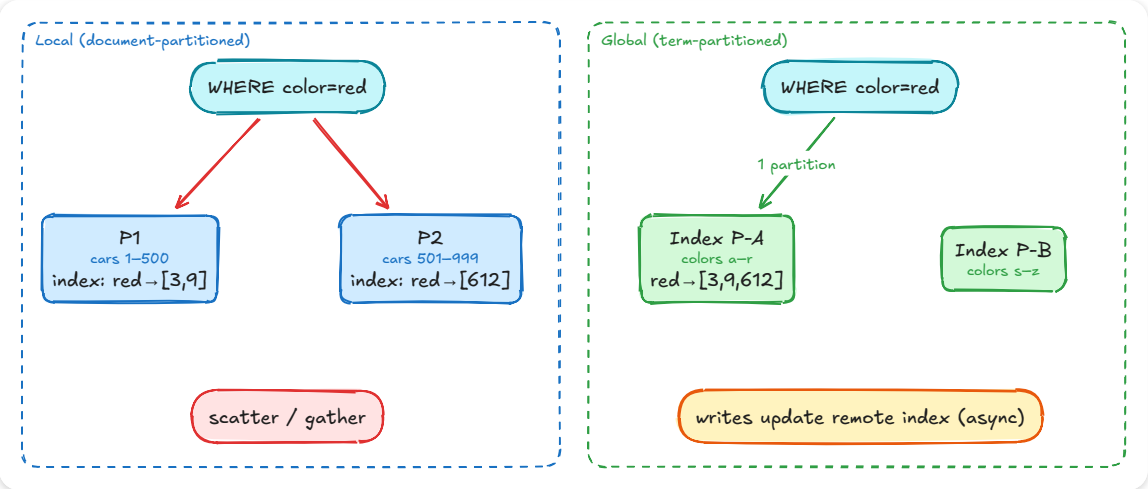


1. Hashing can't fix a single hot **key**. Every write for "Messi" goes to the same partition.
2. Split the key: append a random suffix 00–99 so writes spread over 100 partitions.
3. The cost moves to reads, which must fetch all 100 sub-keys and merge. Only do this for keys you **know** are hot, and track which ones those are.

> **Note**
>
> **Interview examples:** Live comments on a celebrity post, Ticketmaster's Taylor Swift on-sale, YouTube view counters, an ad campaign's click counter. All use **key splitting or write buffering**.

Secondary indexes: local vs global

Partitioning + indexes

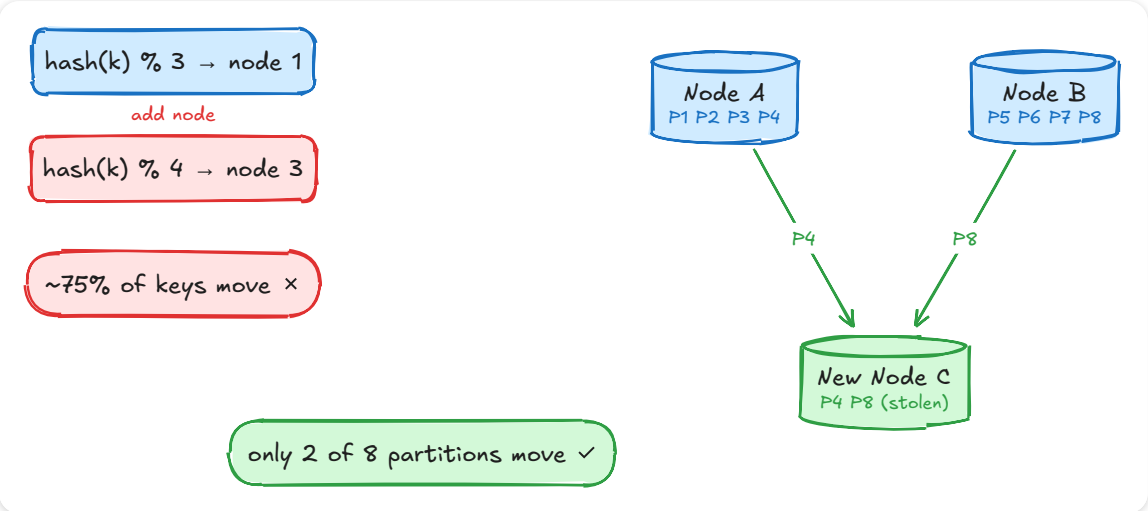


1. **Local**: each partition indexes only its own documents. Writes are simple (one partition), but a query by color must ask **every** partition and merge, so tail latency compounds. Used by MongoDB, Cassandra, and Elasticsearch shards.
2. **Global**: the index itself is partitioned by term. A read hits one index partition, but a write must update index partitions on other nodes, usually **asynchronously**, so the index lags. Used by DynamoDB GSIs.

> **Warning**
>
> **DynamoDB GSI:** Global secondary indexes are **eventually consistent**. A read right after a write via the GSI may miss it.

Rebalancing: never use hash mod N

Adding nodes

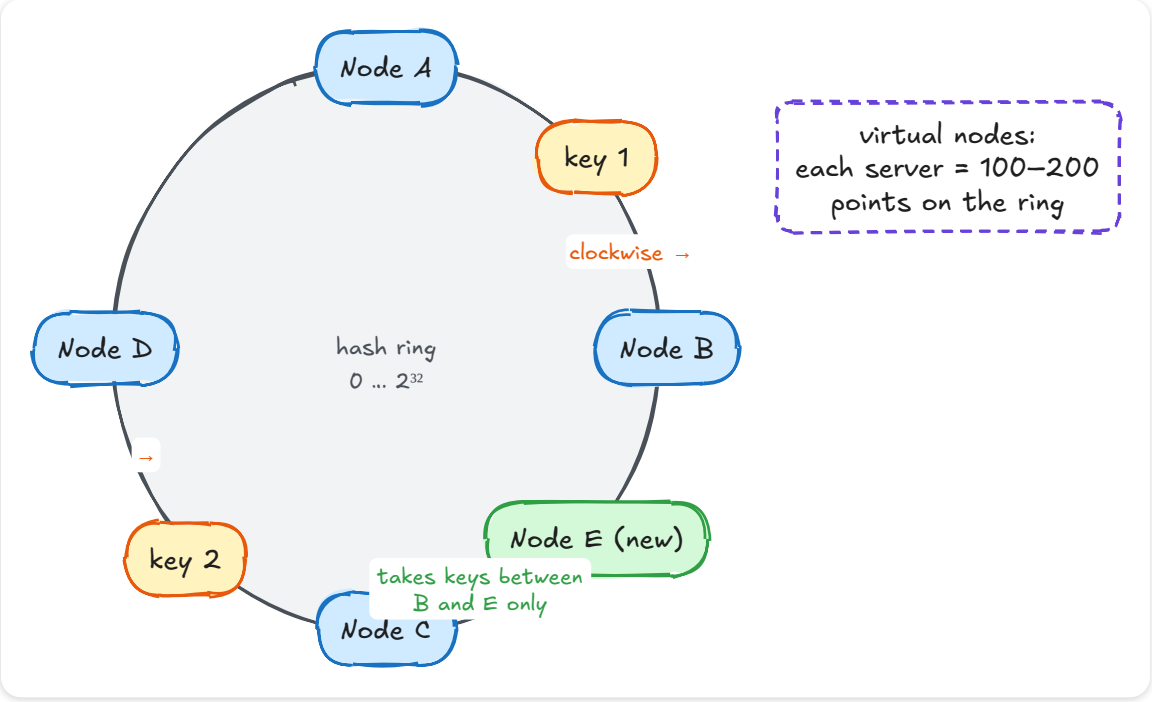


1. With mod N, changing N from 3 to 4 changes the answer for about 75% of keys, so almost all data moves.
2. Better: create **many more partitions than nodes** up front (for example 1,000 partitions on 10 nodes). Key→partition never changes; only partition→node does.
3. A new node **steals a few whole partitions** from each existing node. Used by Elasticsearch, Kafka, Riak, and Couchbase. Consistent hashing (next slide) is the ring version of this idea.

> **Note**
>
> **Other strategies: Dynamic**: split a partition when it grows past ~10GB (HBase, MongoDB). **Per-node**: a fixed number of partitions per node (Cassandra vnodes).

Consistent hashing: the ring

Consistent hashing

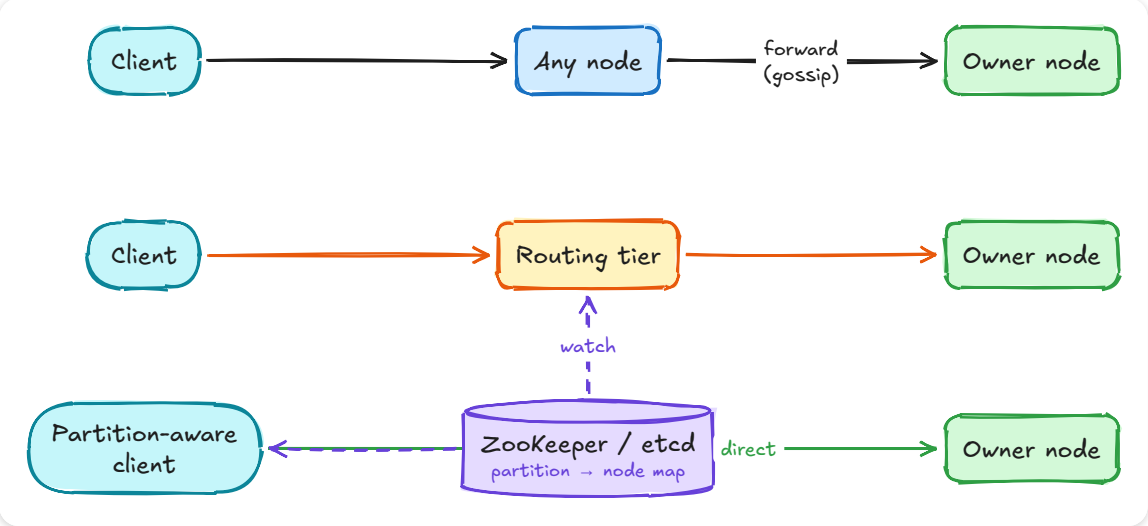


1. Hash both nodes and keys onto the same circle.
2. A key belongs to the **first node clockwise** from it.
3. Add node E: only keys between B and E move (from C to E). That is about 1/N of the data, not 75%.
4. With few nodes, arcs are uneven. **Virtual nodes** place each server at many points so load evens out, and a bigger server gets more points.

> **Tip**
>
> **Say this in the interview:** "Cache keys are placed on a consistent-hash ring with virtual nodes, so adding a cache node remaps only about 1/N of keys and avoids a cold-cache stampede on the database."

Request routing: who knows where key K lives?

Service discovery



1. ① Contact any node, which forwards the request if needed. Nodes learn the map via **gossip**. Used by Cassandra and Riak.
2. ② A partition-aware **routing tier** forwards the request. Used by MongoDB mongos and Vitess VTGate.
3. ③ A **smart client** knows the map and goes straight to the owner. Used by Kafka clients, Redis Cluster clients, and Cassandra drivers.
4. The map itself lives in a coordination service (ZooKeeper, etcd, KRaft), and routers or clients **watch** it for changes.

In the interview

> **Important**
>
> **Interviewer asks:** "How would you shard this?"
> → Name the **shard key** from the hottest query: "hash(user_id), because every hot query filters by user". Then name the cost: cross-user queries scatter.

Important

Interviewer asks: "What about the celebrity or hot hotel?"
→ Key splitting with a suffix, write buffering in Redis or Kafka, and a cache in front for reads.

Important

Interviewer asks: "Adding shards later?"
→ Pre-split into many logical partitions (e.g., 1024) mapped to few physical nodes, or consistent hashing. Never mod N.

Warning

Sharding too early: A single Postgres handles ~10–50k TPS and multiple TB. Shard only when the **calculation** says one primary can't take the write load.

← Ch 5: Replication · Ch 7: Transactions →

DDIA Chapter 7: Transactions

> These DDIA pages are study notes in our own words and diagrams, connecting the book to interview problems. They don't replace the book; *Designing Data-Intensive Applications* by Martin Kleppmann is worth reading in full.

TL;DR

A transaction groups reads and writes into one unit: all of it happens or none of it does, and concurrent transactions don't see each other's half-finished work. Most interview "contention" questions, such as seat booking, inventory, payments, and auctions, come down to which **isolation anomaly** you are defending against.

Key ideas: ACID · dirty reads · read skew · MVCC · lost update · write skew · phantoms · 2PL · SSI

What ACID really means

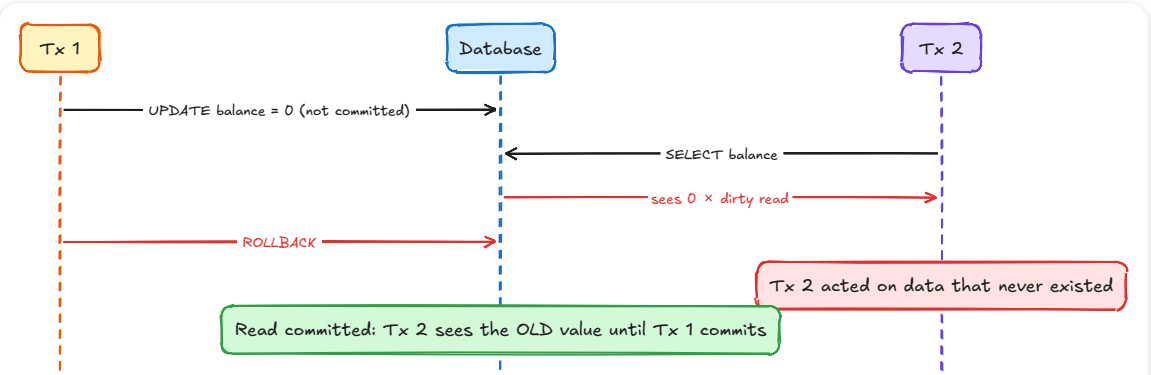
ACID

Letter	Common belief	What it actually guarantees
A · Atomicity	"Atomic like threads"	**Abortability**: on any error mid-transaction, all writes are rolled back. Safe to retry.
C · Consistency	"DB keeps data consistent"	Your **application's invariants** (debits = credits). This one is mostly the app's job, not the DB's.
I · Isolation	"Transactions are serial"	Concurrent transactions don't interfere. The strength depends on the **isolation level**, and most DBs default to weaker than serializable.
D · Durability	"Never lost"	Committed data survives crashes (WAL + fsync, or replication). Nothing is 100%.

> **Warning**
>
> **The default is not serializable:** Postgres defaults to **Read Committed**. MySQL InnoDB defaults to Repeatable Read. Neither prevents write skew, so your booking code can double-book unless you add locks.

Read committed: no dirty reads, no dirty writes

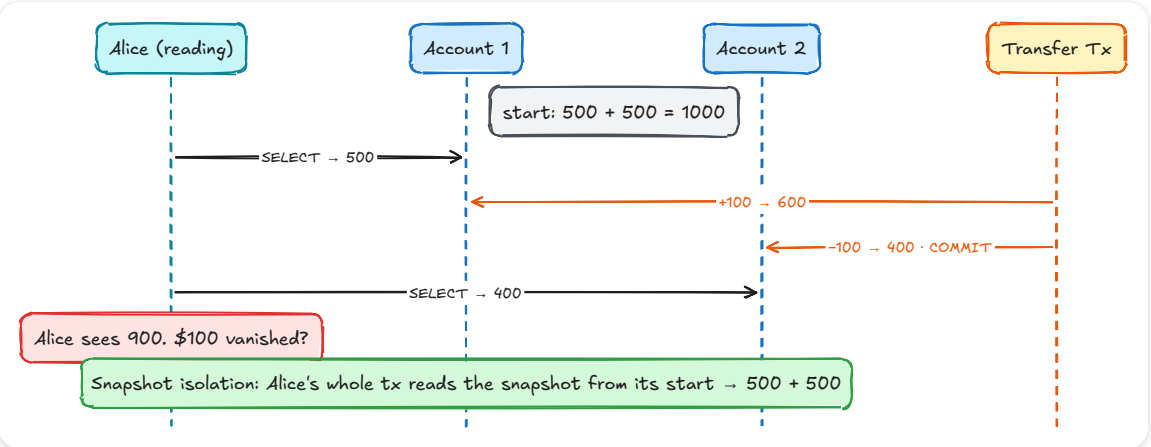
Weakest useful level



1. Tx 1 writes but hasn't committed.
2. Without isolation, Tx 2 reads the uncommitted value: a **dirty read**.
3. Tx 1 rolls back. Tx 2 made decisions based on a value that never existed.
4. **Read committed** prevents dirty reads (readers see the last committed value) and dirty writes (row locks until commit). It is the Postgres default.

Read skew → snapshot isolation (MVCC)

Repeatable read

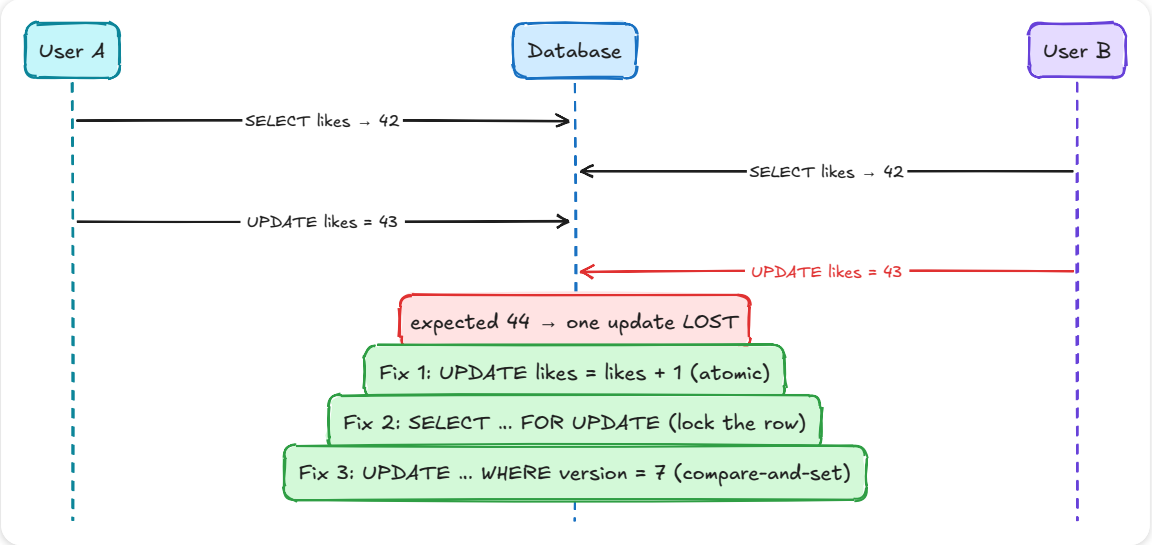


1. Alice reads account 1 (500) in the middle of a transfer.
2. The transfer commits between her two reads.
3. She reads account 2 after the commit: 500 + 400 = 900. This is **read skew**, and it breaks backups, analytics, and integrity checks.
4. **Snapshot isolation**: each transaction sees a consistent snapshot as of its start. It is implemented with **MVCC**: rows keep multiple versions tagged with created_by/deleted_by transaction IDs. Readers never block writers.

> **Note**
>
> **MVCC in one line:** UPDATE = mark old version deleted by tx 13 + insert new version created by tx 13. Your snapshot ignores versions created after you started. VACUUM cleans up old versions later.

Lost update: read-modify-write races

Concurrent writes



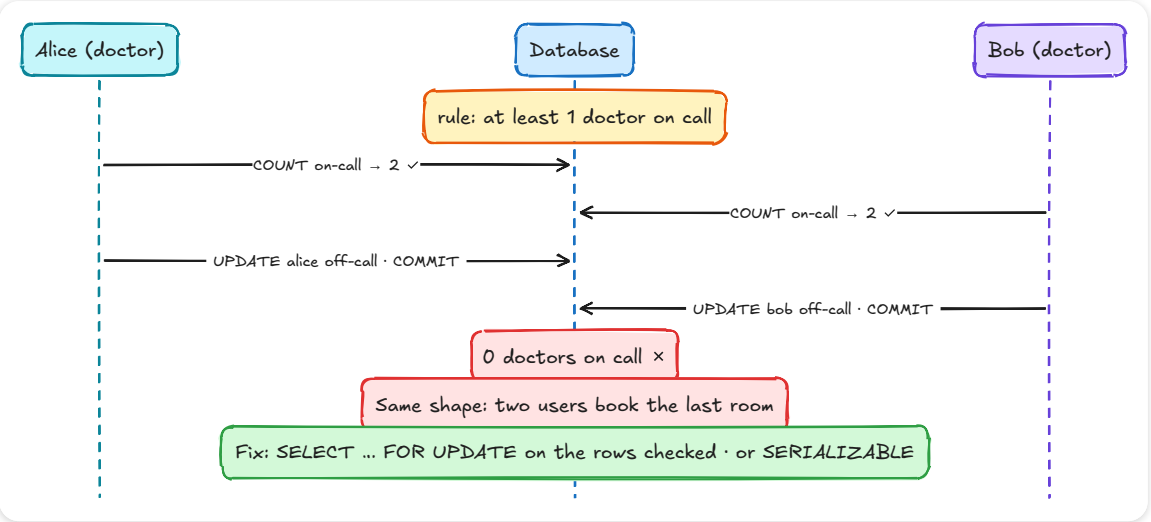
1. Both read the same value.
2. Both write value+1. The second write overwrites the first, so one like is lost.
3. **Atomic operations** are the best fix when possible: the DB does read+write as one step. Redis INCR works the same way.
4. **Pessimistic lock**: SELECT FOR UPDATE makes the second reader wait. Good under high contention.

5. **Optimistic (CAS)**: a version column; if 0 rows are updated, retry. Good under low contention. Postgres Repeatable Read also auto-detects lost updates and aborts.

> **Tip**
>
> **Say this in the interview:** "For the auction, bids use a conditional write, UPDATE ... SET max_bid=? WHERE max_bid < ?, so concurrent bids can't overwrite each other."

Write skew: the double-booking bug

The anomaly that bites interviews



1. Both check a condition across rows and see it is safe.
2. Each updates a **different** row, so there is no write conflict and snapshot isolation allows both. The invariant is broken.
3. This is the pattern of every booking bug: "check availability, then insert booking". If the check finds no rows to lock, it is a **phantom**.
4. Fixes: lock the rows you checked; use SERIALIZABLE; add a unique constraint; or **materialize the conflict** by creating a row per seat or room-night so there is something to lock.

> **Tip**
>
> **Say this in the interview:** "Each seat is a row with status AVAILABLE/HELD/SOLD. A hold is UPDATE seats SET status=HELD WHERE id=? AND status=AVAILABLE. If one row was updated I won it, if zero someone else did. No write skew possible."

Isolation levels: what each prevents

Cheat sheet

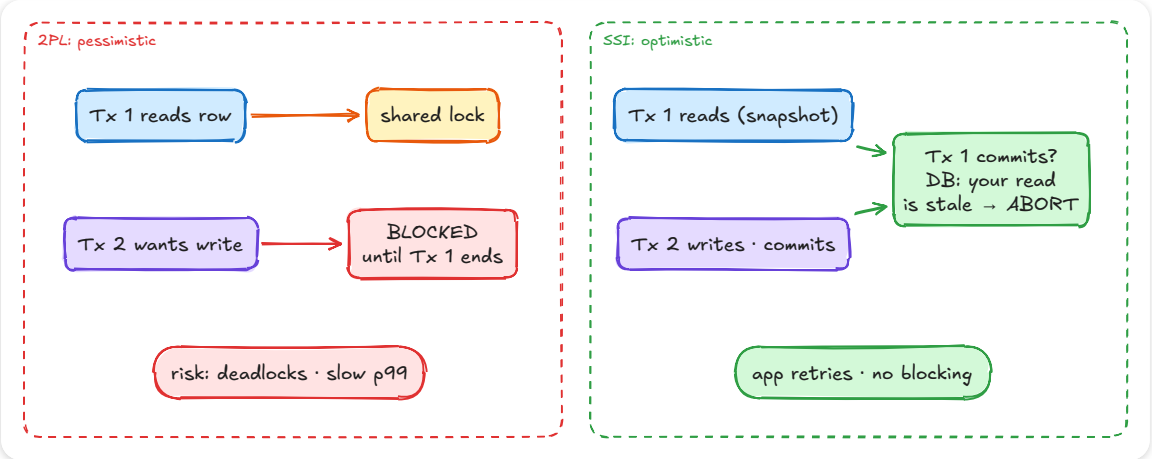
Anomaly	Read Committed	Snapshot / Repeatable Read	Serializable
Dirty read / write	✓ prevented	✓	✓
Read skew (non-repeatable read)	×	✓ prevented	✓
Lost update	× (use atomic op / lock)	✓ Postgres detects; MySQL ×	✓
Write skew	×	×	✓ prevented
Phantoms	×	× (mostly)	✓ prevented
Default in	Postgres, Oracle, SQL Server	MySQL InnoDB	CockroachDB, FoundationDB

Note

Serializable, three ways: Actual serial execution: one thread, stored procedures (VoltDB, Redis Lua). **2PL**: readers block writers and vice versa, with predicate locks; slow, deadlocks. **SSI**: optimistic; detect stale reads at commit and abort (Postgres SERIALIZABLE).

Two-phase locking vs serializable snapshot isolation

Serializability



1. **2PL**: readers hold shared locks and writers need exclusive locks, both held until commit. Correct, but throughput drops and p99 latency is unpredictable under contention.
2. **SSI**: everyone runs on snapshots without blocking. At commit, the DB checks whether anything you read has since changed and, if so, aborts you. Fast at low contention. Under high contention there are many aborts, so retries are needed.

In the interview

Important

Interviewer asks: "Two users book the last room at the same time."
→ Write skew. Use a row per room-night + **conditional UPDATE**, or SELECT FOR UPDATE, or a unique constraint on (room, date).

Important

Interviewer asks: "Like counter at 50k/sec?"
→ Not row locks. Use atomic INCR in Redis, sharded counters, or batch into the DB asynchronously. The lost update is avoided because the increment is atomic.

Important

Interviewer asks: "Optimistic or pessimistic?"
→ Low contention: optimistic version check. High contention on few rows (hot seat map): pessimistic lock or a queue to serialize.

Important

Interviewer asks: "Transaction across 2 services?"
→ No distributed ACID. Use a **saga** with compensations plus the outbox pattern (Ch 9 and 12).

← Ch 6: Partitioning · Ch 8: Trouble with distributed systems →

DDIA Chapter 8: The Trouble with Distributed Systems

> These DDIA pages are study notes in our own words and diagrams, connecting the book to interview problems. They don't replace the book; *Designing Data-Intensive Applications* by Martin Kleppmann is worth reading in full.

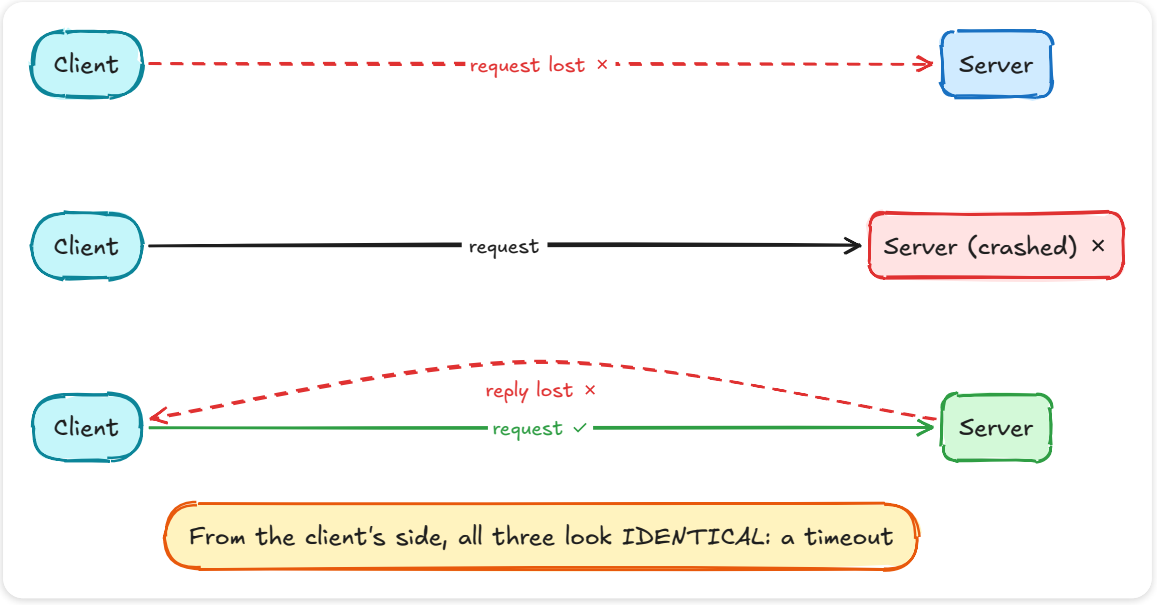
TL;DR

On one machine, things either work or crash. Across machines you get **partial failure**: some parts work, some don't, and you often can't tell which. The three villains are unreliable networks, unreliable clocks, and process pauses.

Key ideas: partial failure · timeouts · clock skew · LWW danger · GC pauses · leases · fencing tokens · majority quorum

The network: you cannot tell why there was no reply

Unreliable networks



1. ① The request was lost in the network (or is still queued in a switch).
2. ② The request arrived, but the server crashed or is in a long GC pause.
3. ③ The server **did the work**, but the reply was lost. A retry runs it twice.

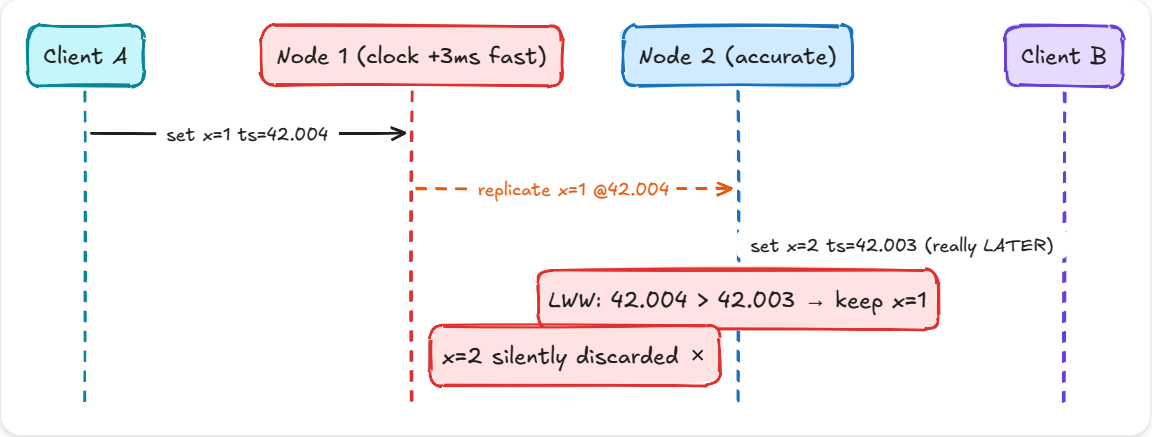
4. The client only knows "no reply before the timeout". That is why every retried operation must be **idempotent**.

> **Tip**
>
> **Say this in the interview:** "Payment calls to the provider can time out after the charge succeeded. Every request carries an idempotency key, so a retry is safe."

> **Note**
>
> **Choosing a timeout:** Too short means healthy nodes are declared dead and failover storms follow. Too long means users wait. Use p99.9 latency × margin, or adaptive (phi-accrual) detectors.

Clocks: last-write-wins can drop the newer write

Unreliable clocks



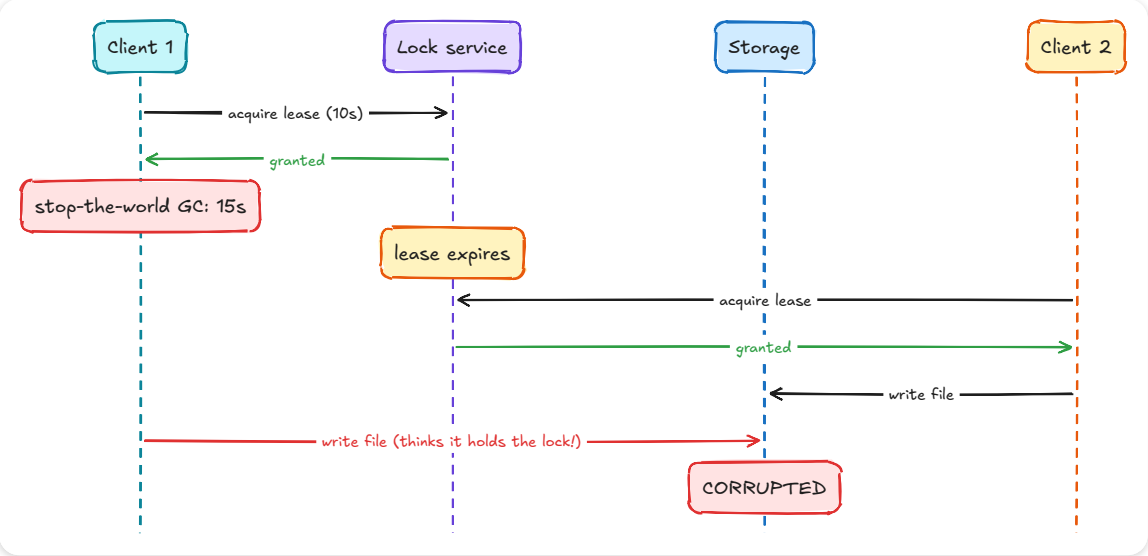
1. Node 1's clock runs a few ms fast. Client A's write gets timestamp 42.004.
2. Client B writes later in real time, but Node 2's accurate clock says 42.003.
3. Last-write-wins compares timestamps, and the causally later write **loses**. No error is raised. NTP accuracy is only tens of ms at best.

> **Note**
>
> **Two kinds of clock: Time-of-day** (System.currentTimeMillis) can jump backwards after an NTP sync. Never use it to measure durations. **Monotonic** (System.nanoTime) only moves forward and is for durations and timeouts.

> **Important**
>
> **Interviewer asks:** Google Spanner uses **TrueTime**: clocks return an interval [earliest, latest], and a commit waits out the uncertainty (~7ms).

Process pauses: the expired-lease bug

GC pauses

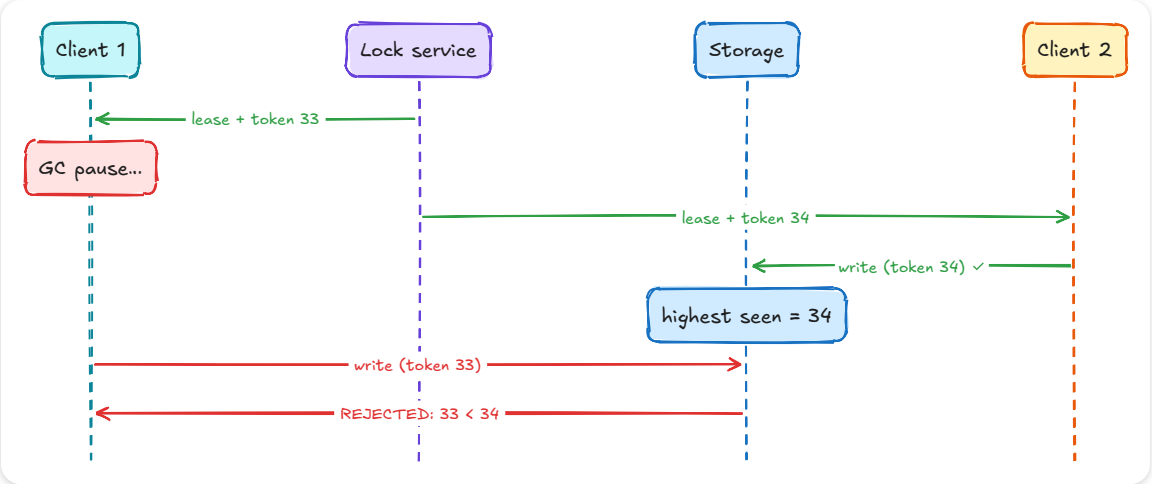


1. Client 1 takes a lease, a lock with a timeout.
2. Then it pauses: a GC pause, VM migration, swapping, or a laptop lid closing. The thread doesn't know it paused.
3. The lease expires and is given to Client 2.
4. Client 2 writes, correctly.
5. Client 1 wakes up, checks nothing, and writes. Two writers think they own the lock.

> **Warning**
>
> **Checking the time does not help:** "Check lease validity before writing" still races, because the pause can happen **between** the check and the write.

Fencing tokens: the fix

Fencing



1. Every lock grant comes with a **monotonically increasing token**. ZooKeeper's zxid or an etcd revision works.
2. The storage remembers the highest token it has seen.
3. A late writer with an old token is rejected. The protection is enforced by the **resource**, not trusted to the client.

> **Tip**
>
> **Say this in the interview:** "A Redis lock alone isn't safe under GC pauses. I'd pair the lock with a fencing token, for example a version column the DB checks: UPDATE ... WHERE version < :token."

> **Note**
>
> **Truth = majority:** A node can't trust its own view ("am I still leader?"). Decisions are made by a **quorum** of nodes, which is the basis of Ch 9.

System models and Byzantine faults

Assumptions

Model	Assumes	Reality
Timing: synchronous	Bounded network delay, pauses, clock drift	Not realistic
Timing: partially synchronous	Usually bounded, sometimes not	**The realistic model**; algorithms must stay safe when bounds break

Model	Assumes	Reality
Timing: asynchronous	No timing assumptions at all	Very restrictive; no timeouts
Node: crash-stop	Crashed nodes never return	Simplifies proofs
Node: crash-recovery	Nodes crash and return; disk survives, RAM doesn't	**Realistic** for datacenters
Node: Byzantine	Nodes may lie or act maliciously	Blockchains, aerospace. Rarely needed inside one company.

> **Note**
>
> **Safety vs liveness: Safety**: nothing bad ever happens (no two leaders). This must always hold. **Liveness**: something good eventually happens (a leader gets elected). This may wait for the network to heal.

In the interview

> **Important**
>
> **Interviewer asks:** "What if the payment provider times out?"
> → Outcome unknown. Retry with the **same idempotency key**, or reconcile with a status query. Never blindly charge again.

> **Important**
>
> **Interviewer asks:** "Distributed lock with Redis?"
> → Fine for efficiency (avoid duplicate work). For correctness add **fencing tokens**, or use the DB's own conditional writes.

> **Important**
>
> **Interviewer asks:** "Order events by timestamp?"
> → Wall clocks skew. Use a sequence from one leader, a Kafka partition offset, or logical clocks (Ch 9).

Important

Interviewer asks: "How do you detect a dead node?"
→ Heartbeats + timeouts tuned to p99.9, and accept false positives. Leader election by majority, with epochs to fence the old leader.

← Ch 7: Transactions · Ch 9: Consistency and consensus →

DDIA Chapter 9: Consistency & Consensus

> These DDIA pages are study notes in our own words and diagrams, connecting the book to interview problems. They don't replace the book; *Designing Data-Intensive Applications* by Martin Kleppmann is worth reading in full.

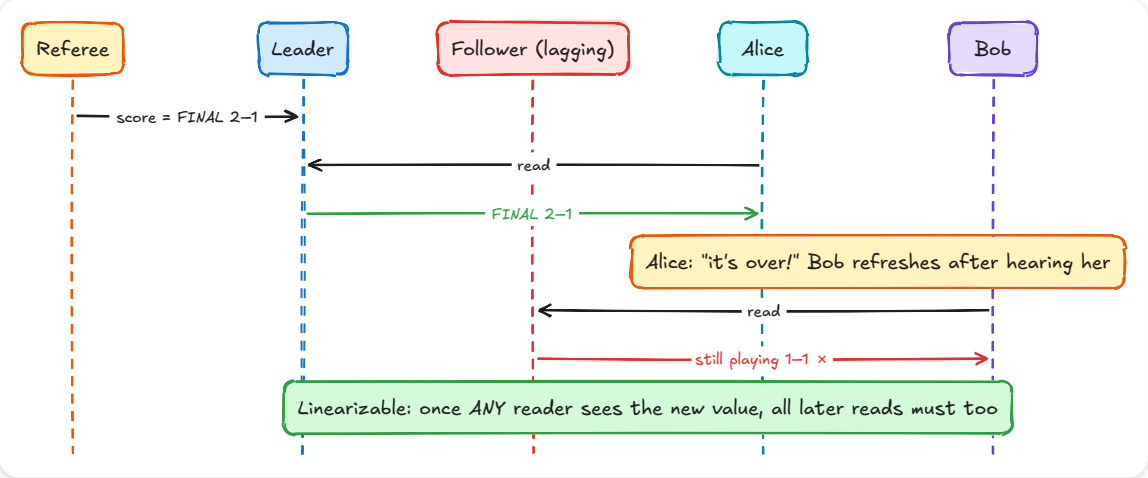
TL;DR

Given everything in Ch 8, how do nodes **agree**? This chapter covers linearizability (behave like one copy), ordering and causality, distributed transactions (2PC), and consensus (Raft, Paxos, ZooKeeper), the tools behind leader election, locks, and uniqueness.

Key ideas: linearizability · CAP · causality · Lamport clocks · total order broadcast · 2PC · Raft · ZooKeeper

Linearizability: behave as if there is one copy

Strongest single-object guarantee



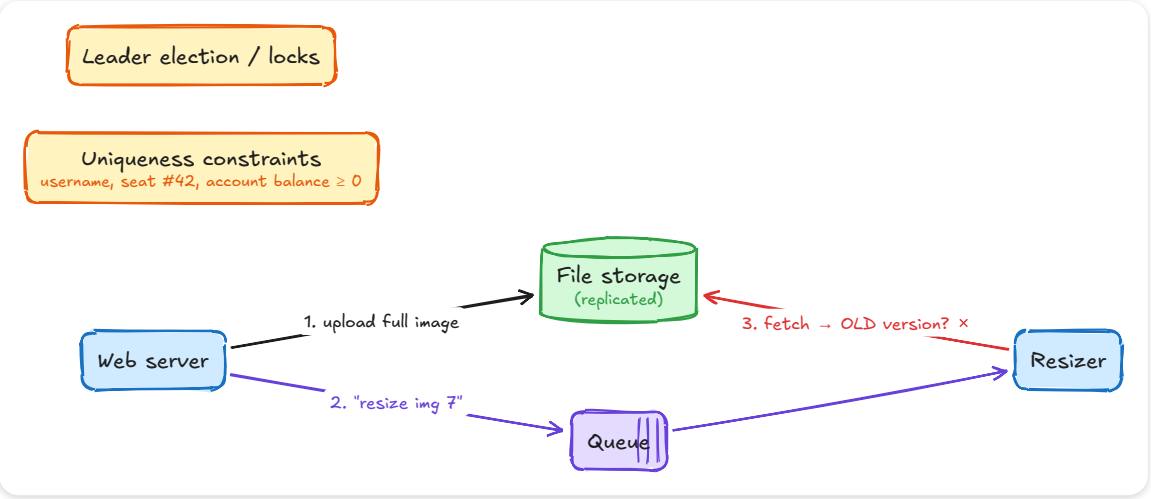
1. The final score is written to the leader.
2. Alice reads it.
3. Bob reads **after** Alice (he heard her shout), from a lagging replica, and sees an older value. That violates linearizability.
4. **Linearizability** means every operation appears to take effect atomically at one point in time, consistent with real-time order. It is a recency guarantee on a single

object.

> **Warning**
>
> **Not the same as serializability: Serializable** is about transactions over many objects (isolation, Ch 7). **Linearizable** is about recency of a single object. "Strict serializability" is both.

When you actually need linearizability

Use cases



1. Leader election and locks need every node to agree on **one** holder, and uniqueness constraints need everyone to agree on who got there first. Both require linearizable storage.
2. **Cross-channel timing**: the web server stores an image, then sends a message through a different channel (the queue).
3. If the resizer reads a lagging storage replica, it resizes the old image. Two channels give a race. The fix is linearizable storage, or putting the version number in the message.

> **Note**
>
> **Everything else:** Feeds, likes, view counts, search results: **eventual consistency is fine**. Saying this explicitly is a strong staff signal.

Which systems are linearizable, and CAP done right

CAP · PACELC

Replication	Linearizable?	Why
Single-leader (reads from leader)	Potentially yes	If reads hit the leader and the leader is really the leader (fencing)
Consensus (Raft / ZooKeeper / etcd)	Yes	Designed for it
Multi-leader	No	Concurrent writes on different leaders
Leaderless (Dynamo / Cassandra)	Probably not	LWW + clock skew, sloppy quorums, timing edge cases

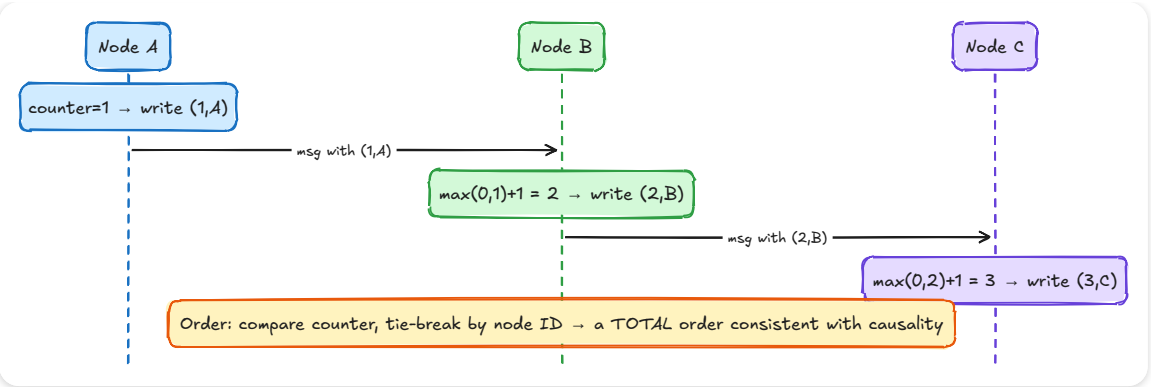
> **Note**
>
> **CAP, correctly stated:** Network **partitions will happen**. When one does, choose: refuse some requests to stay consistent (**CP**), or answer from possibly stale data (**AP**). It is a decision per feature, not per database.

> **Note**
>
> **PACELC: E**lse, when there is no partition, you trade **L**atency vs **C**onsistency. Linearizability is slow even when nothing fails, because each operation waits for a quorum round trip.

> **Tip**
>
> **Say this in the interview:** "Inventory and payments are CP: I'd rather reject than oversell. Hotel search and reviews are AP: stale by a few seconds is fine."

Ordering: Lamport timestamps

Causality without clocks

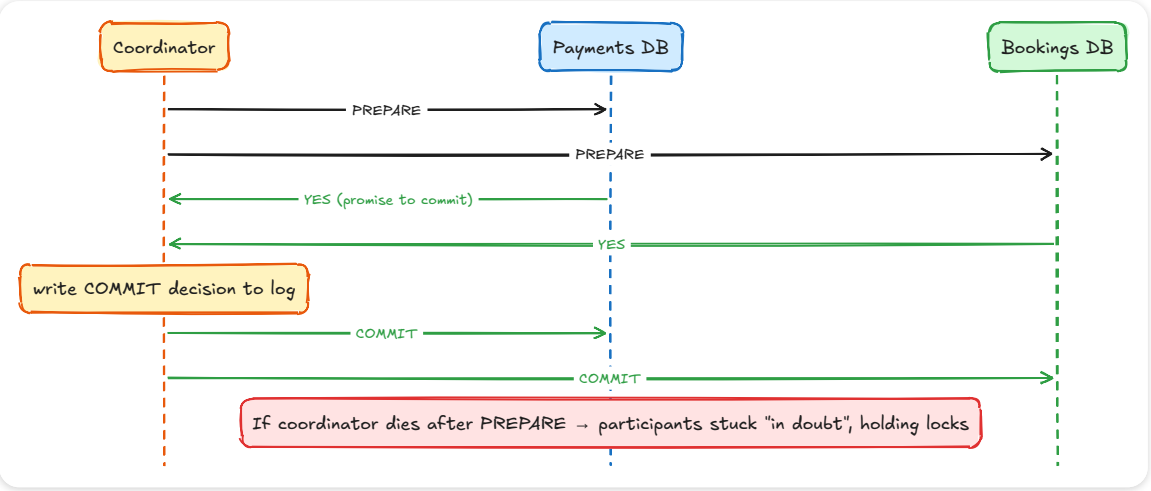


1. Each node keeps a counter and tags every operation with (counter, nodeId).
2. On receiving a message, a node sets counter = max(own, received) + 1. Causality is carried along with the message.
3. So if A happened before C, then A's timestamp is smaller than C's.
4. The limit: the order is only known **after the fact**. To decide "username taken?" right now you must know nobody else is claiming it concurrently, which needs **total order broadcast**, which is consensus.

> **Note**
>
> **Total order broadcast:** Deliver all messages to all nodes in the same order, reliably. This is exactly a **replicated log**: Kafka partitions (within a partition), Raft logs, ZooKeeper's zab.

Two-phase commit (2PC)

Distributed atomic commit



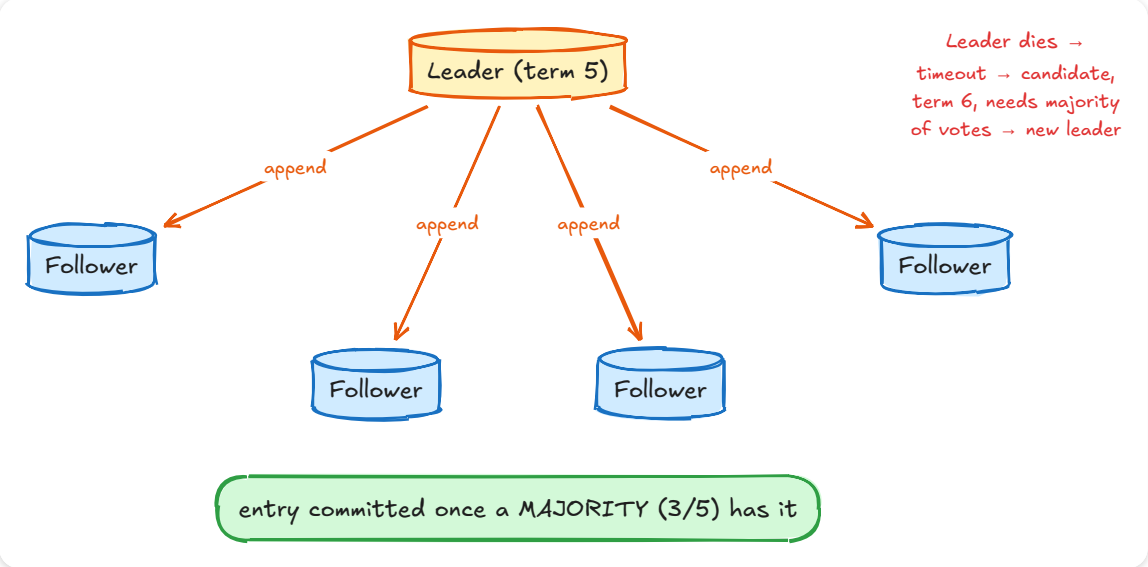
1. Phase 1: ask every participant "can you commit?"

2. A YES is a binding promise: the participant must be able to commit even after a crash, so it holds locks.
3. The coordinator logs its decision (the commit point), then tells everyone.
4. The weakness is that 2PC **blocks**. If the coordinator dies, participants hold locks until it recovers. This is why microservices use **sagas** instead.

> **Tip**
>
> **Say this in the interview:** "I won't use 2PC across the payment and booking services. I'll use a saga: reserve room → charge → confirm, with compensations (release room, refund), driven by events with the outbox pattern."

Consensus: Raft in one picture

Raft / Paxos / Zab

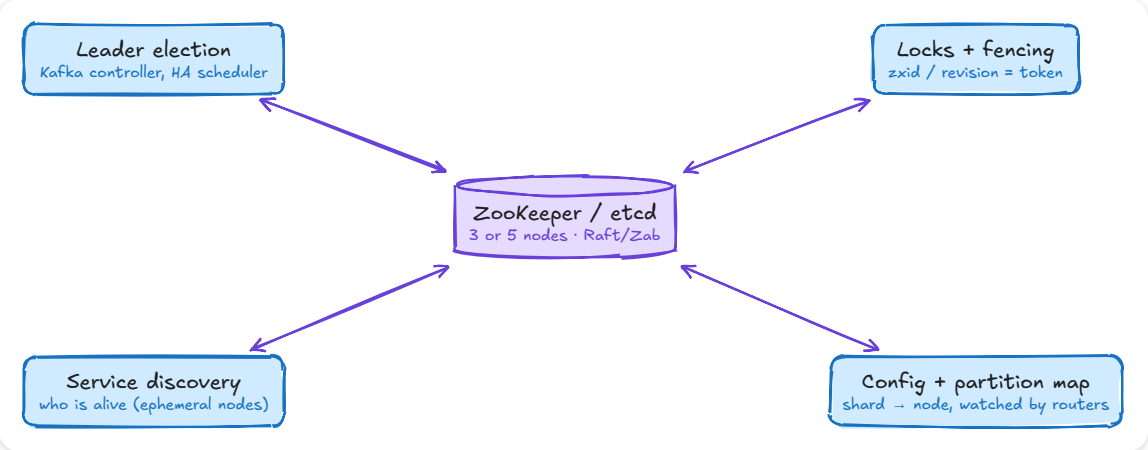


1. A cluster of 5 nodes (tolerates 2 failures). One elected leader per **term** (epoch).
2. The leader appends each command to its log and replicates it to followers.
3. An entry is **committed** once a majority store it. Any future majority overlaps this one, so committed entries are never lost.
4. If the leader goes silent, a follower times out, increments the term, and asks for votes. A majority makes it leader. A higher term **fences** the old leader.

> **Note**
>
> **Sizing:** 2f+1 nodes tolerate f failures: 3 nodes → 1, 5 nodes → 2. Every write needs a majority round trip, so consensus is used for **metadata**, not bulk data.

ZooKeeper / etcd: consensus as a service

Coordination services



1. You almost never implement consensus yourself. You use a small, strongly consistent store.
2. Common uses: elect one active scheduler or controller; distributed locks with fencing tokens.
3. Membership via **ephemeral nodes** that vanish when a session dies; config and shard maps that clients **watch** for changes.

> **Tip**
>
> **Say this in the interview:** "The job scheduler runs active-passive. Leader election goes through etcd with a lease, and the lease revision is the fencing token on job-claim writes."

In the interview

Important

Interviewer asks: "Is this CP or AP?"
→ Answer **per feature**: booking and payment are CP, search and feeds are AP. Mention PACELC latency cost.

Important

Interviewer asks: "Transaction spanning booking + payment services?"
→ Not 2PC (blocking, coupling). Use a **saga** + outbox + idempotent consumers.

Important

Interviewer asks: "Exactly one scheduler must run."
→ Leader election via ZooKeeper or etcd (consensus), lease + fencing token.

Important

Interviewer asks: "Unique usernames across regions?"
→ Needs linearizable compare-and-set: one strongly consistent store (Spanner, CockroachDB, or a single-leader region for usernames).

← Ch 8: Trouble with distributed systems · Ch 10: Batch processing →

DDIA Chapter 10: Batch Processing

> These DDIA pages are study notes in our own words and diagrams, connecting the book to interview problems. They don't replace the book; *Designing Data-Intensive Applications* by Martin Kleppmann is worth reading in full.

TL;DR

Take a big, bounded, **immutable** input, run a pure function over it, and produce a derived output. Because inputs are never modified, a buggy job is fixed by re-running it. That property, being able to recompute from source, runs through all of Part III.

Key ideas: Unix philosophy · MapReduce · HDFS · shuffle · sort-merge join · broadcast join · Spark DAG

The Unix philosophy: small tools, one interface

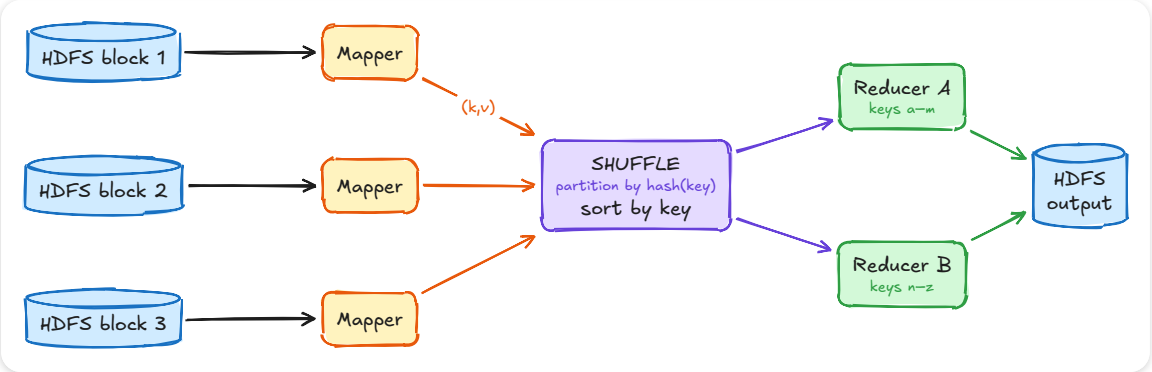
Where batch comes from



1. Read the log and extract the URL field from each line.
2. Sort so identical URLs are adjacent, then count each run. When the data is bigger than memory, sort spills to disk, so this scales past RAM.
3. Sort by count and take the top 5. Six tiny programs, composed through one uniform interface: a stream of lines.
4. **Immutable input**: run it as many times as you like and nothing breaks.
5. **Uniform interface**: every tool reads and writes files, so you can wire anything to anything.
6. **Separation of logic and wiring**: the program doesn't know where its input came from.
7. MapReduce is this same idea spread across thousands of machines, with HDFS as the "file".

MapReduce: map, shuffle, reduce

Distributed batch

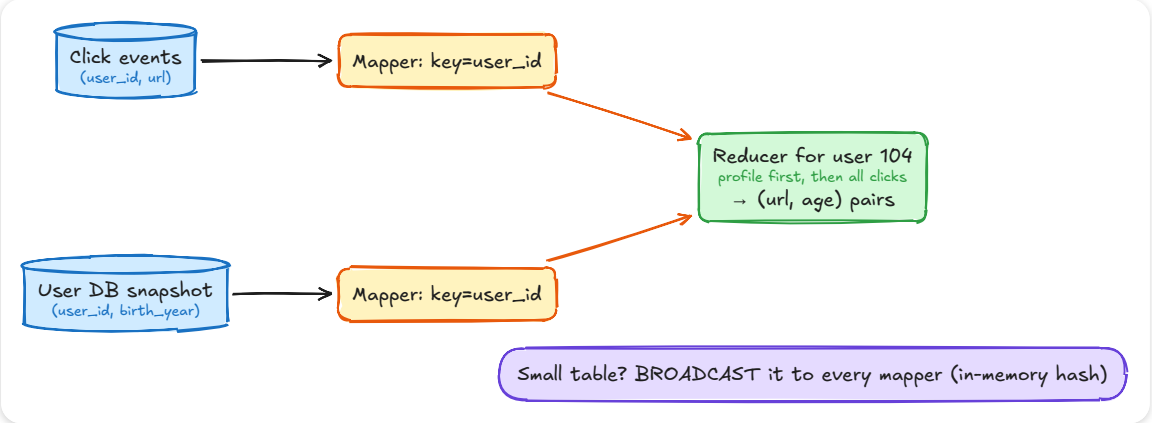


1. Input is split into blocks, replicated across machines (HDFS).
2. The scheduler runs a **mapper** next to each block (move the compute to the data). Mappers emit key-value pairs.
3. The **shuffle** sends every pair with the same key to the same reducer and sorts them. This is the expensive network step.
4. Each **reducer** sees all values for its keys together, aggregates them, and writes output files. If a task fails, just rerun that task, because inputs are immutable.

> **Note**
>
> **Scale:** Mapper output goes to local disk; reducers pull over the network. Jobs at Google and Yahoo processed petabytes daily, and the same pattern now runs in Spark, BigQuery, and Snowflake.

Joins in batch: bring related records together

Sort-merge · broadcast · partitioned

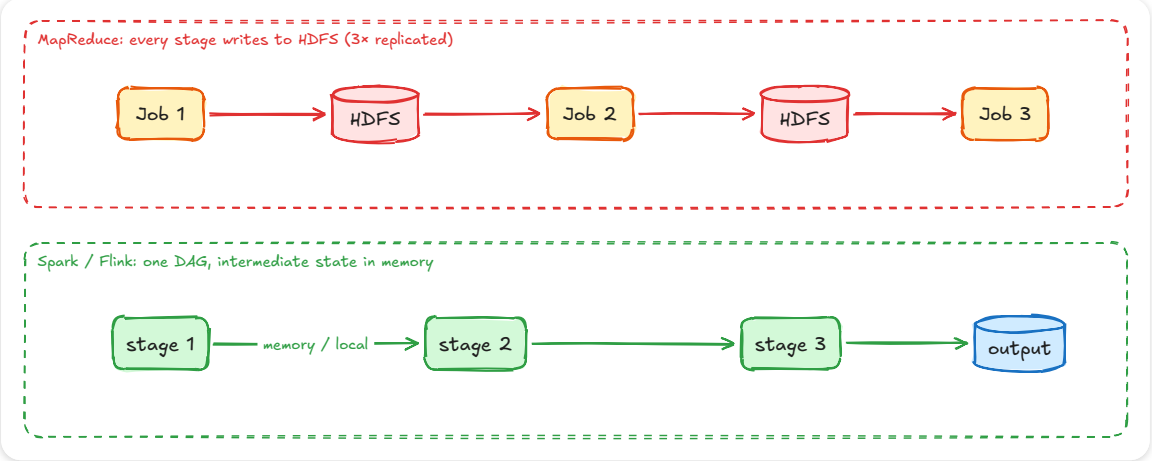


1. Goal: "which URLs are popular with which age group?" Don't query the user DB per event (millions of random lookups). Take a **snapshot** of it.
2. **Sort-merge join**: both mappers emit user_id as the key. The shuffle delivers one user's profile and clicks to the same reducer, adjacent. One linear pass joins them.
3. **Broadcast hash join**: if one side is small (a hotel dimension table), load it into each mapper's memory. No shuffle is needed. **Partitioned hash join**: both sides partitioned the same way, so join partition by partition.

> ⚠ **Warning**
>
> **Skew:** One hot key (a celebrity user) sends millions of records to one reducer. Fixes: skewed-join handling (sample hot keys, spread them across reducers), the same idea as Ch 6.

Beyond MapReduce: dataflow engines and batch outputs

Spark · Flink · outputs



1. Chaining MapReduce jobs **materializes** every intermediate result to replicated HDFS: slow, and each job waits for the whole previous one.
2. Dataflow engines run the whole workflow as one **DAG** and pipeline data between operators. On failure they recompute lost partitions from **lineage**, so typically 10–100× faster.

> **Note**
>
> **What batch jobs produce:** Search indexes (build offline, swap atomically), pre-computed recommendations, ML features, read-only KV stores bulk-loaded into serving. A bad output is rolled back by pointing at yesterday's files.

In the interview

> **Important**
>
> **Interviewer asks:** "Web crawler for 10B pages?"
> → The frontier queue feeds fetchers. Text extraction and dedup run as batch (Spark) over S3. Re-runnable because raw pages are stored immutably.

> **Important**
>
> **Interviewer asks:** "Daily top-K or reconciliation reports?"
> → Nightly batch over the event log in S3 or the warehouse. The streaming path gives fast approximate results and batch gives exact ones (Ch 12, Lambda).

> **Important**
>
> **Interviewer asks:** "How do you rebuild the search index?"
> → Batch-build a new index from the source of truth, then atomically switch the alias. Old index kept for rollback.

> **Tip**
>
> **Say this in the interview:** "Raw events land immutably in S3, partitioned by date. Any derived table can be recomputed from them, which is our safety net for bugs."

← Ch 9: Consistency and consensus · Ch 11: Stream processing →

DDIA Chapter 11: Stream Processing

> These DDIA pages are study notes in our own words and diagrams, connecting the book to interview problems. They don't replace the book; *Designing Data-Intensive Applications* by Martin Kleppmann is worth reading in full.

TL;DR

Batch with the input never ending. Events are processed shortly after they happen. This is Kafka, CDC, the outbox pattern, event sourcing, windowed aggregation, and exactly-once semantics: the backbone of most modern interview designs.

Key ideas: message brokers · Kafka partitions · consumer groups · CDC · outbox · event sourcing · windows · watermarks · exactly-once

Two kinds of message broker

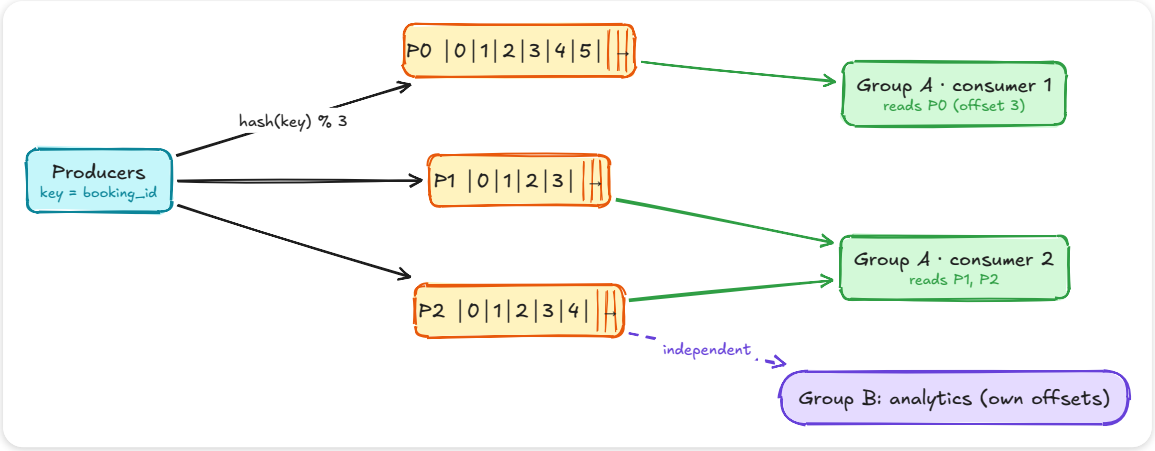
Queue vs log

	Queue (RabbitMQ, SQS, JMS)	**Log (Kafka, Kinesis, Pulsar)**
After consume	Message deleted once acked	Message stays; consumer just advances an offset
Ordering	Lost under parallel consumers + redelivery	Strict order **within a partition**
Parallelism	Any consumer takes any message (load balancing)	One consumer per partition per group
Replay	No: gone after ack	Yes: reset offset, re-read days of history
Slow message	Others continue (per-message ack)	Blocks its partition (head-of-line)
Best for	Task queues: send email, resize image, each job independent	Event streams: ordering matters, multiple consumers, replay, CDC

> **Tip**
>
> **Say this in the interview:** "Image resize jobs go on SQS, since they are independent with per-message retry. Booking events go on Kafka, since order per booking matters and several services consume them."

Kafka: partitions, offsets, consumer groups

Log-based broker

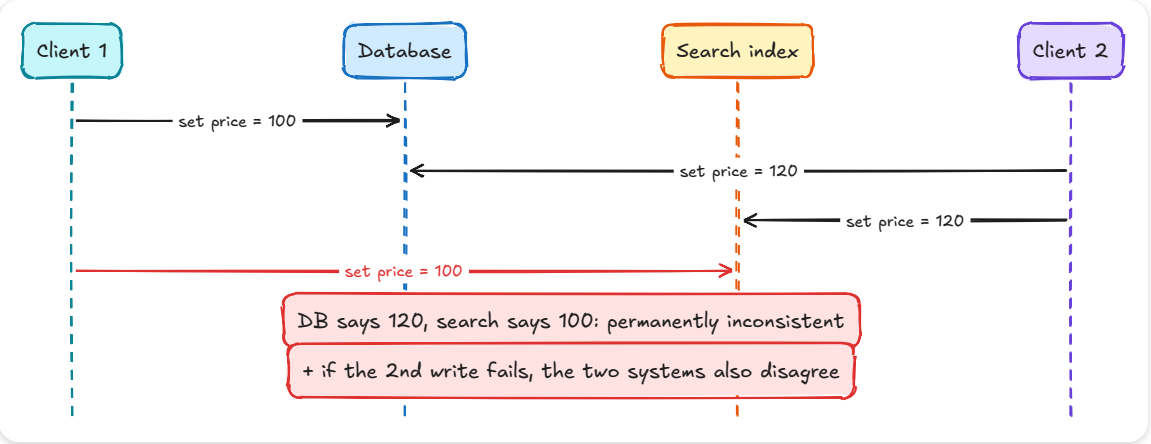


1. A topic is split into **partitions**, each an append-only log. The producer picks a partition by hashing the key, so all events for one booking land in one partition, in order.
2. Within a **consumer group**, each partition is read by exactly one consumer, so maximum parallelism equals the partition count. Consumers commit their **offset**; after a crash they resume from it.
3. Another group reads the same data independently with its own offsets. Adding consumers doesn't cost the producer anything.

> **Note**
>
> **Numbers:** ~5–10k msg/sec per partition is a comfortable planning number; a broker handles ~1M msg/sec. Default retention is 7 days. Choose partitions = target throughput ÷ per-consumer throughput.

The dual-write problem

Why not just write to both?

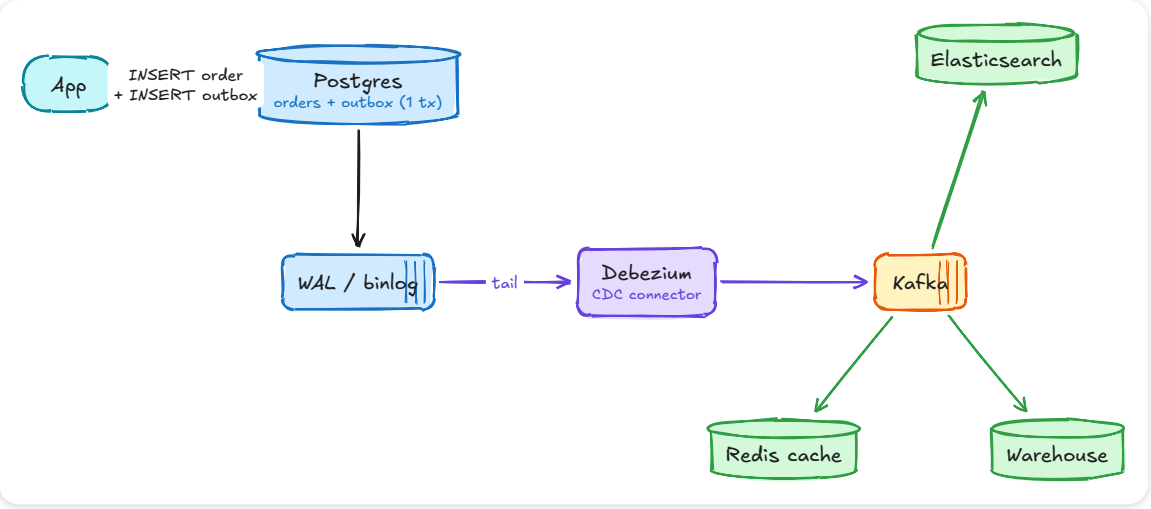


1. Two clients update the same item, and the DB applies them in one order.
2. The search index receives them in a different order (the network reorders them).
3. The two systems now disagree **permanently**, and nothing will repair it.
4. And if the app crashes between the two writes, one system has the change and the other doesn't. Writing to two systems is never atomic.

> **Tip**
>
> **Say this in the interview:** "I won't dual-write to Postgres and Elasticsearch. I'll write only to Postgres and stream changes to Elasticsearch with CDC, so one order, defined by the DB log, reaches every consumer."

Change data capture (CDC) and the outbox pattern

One source of truth → many views

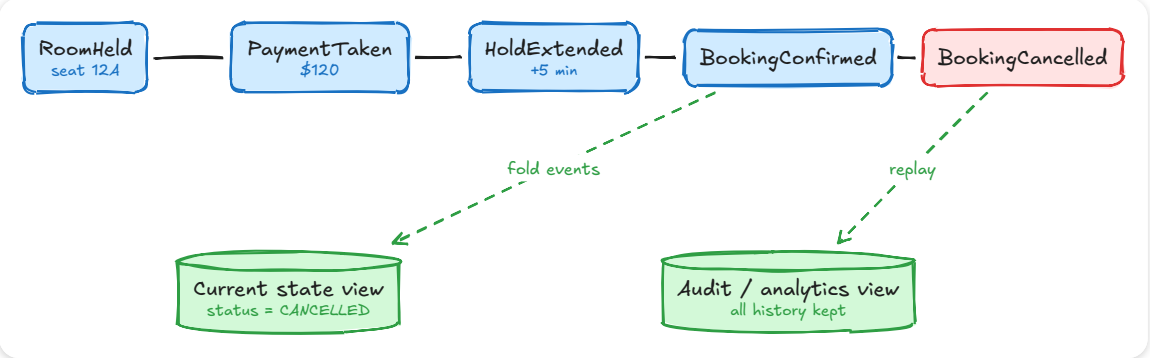


1. **Outbox**: in the **same transaction** as the business write, insert an event row into an outbox table. Either both commit or neither does, so there is no dual-write problem.
2. A CDC connector tails the database's replication log (Ch 5) and publishes each change, including the outbox rows, to Kafka in commit order.
3. Every derived system (search, cache, warehouse) consumes the log and updates itself. Each can be rebuilt by replaying from offset 0, or from a snapshot plus the log.

> **Note**
>
> **Consumers must be idempotent:** The relay can publish an event twice (crash after publish, before marking sent). Consumers dedupe by event ID or use upserts.

Event sourcing: store what happened, derive the state

Events as the source of truth



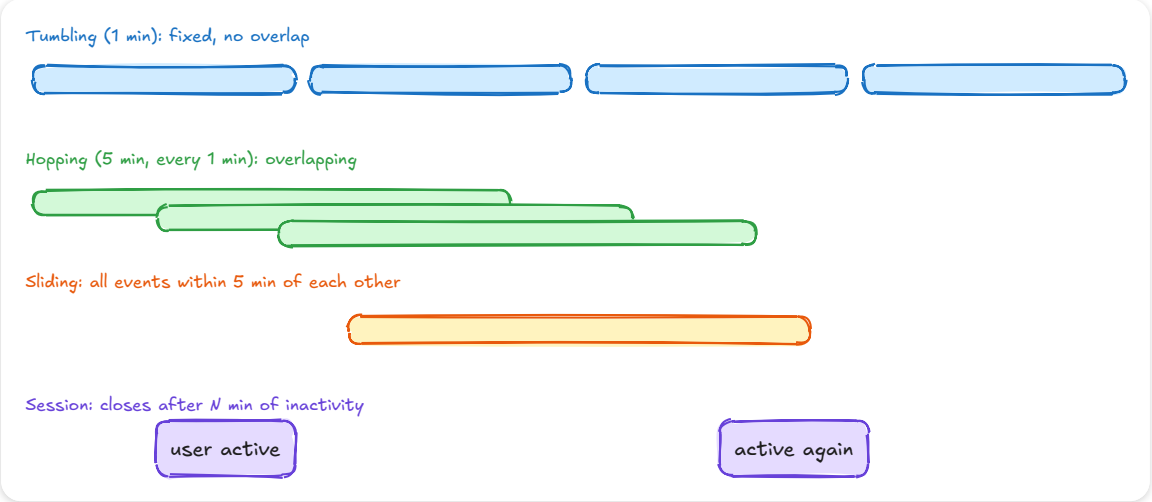
1. Instead of updating a row in place, append immutable **events** that describe what the user did.
2. A cancellation is a new event, not a deletion. History is never lost.
3. State is a **derived view**, a fold over the events. You can build new views later by replaying, and debug by replaying to any point in time.

> **Warning**
>
> **Costs:** Replay gets slow, so you need snapshots. Schema evolution of old events is hard (Ch 4). "Delete my data" (GDPR) conflicts with immutability. Use it for audit-heavy domains (payments, ledgers), not everywhere.

> **Note**
>
> **CDC vs event sourcing:** CDC derives events from DB row changes (low level). Event sourcing makes app-level events the truth (high level, intent preserved).

Time and windows

Event time vs processing time



1. **Tumbling**: "clicks per minute". Each event belongs to exactly one window.
2. **Hopping**: a 5-minute average updated every minute. Windows overlap, so each event belongs to several.
3. **Sliding**: "two failed logins within 5 minutes of each other". The window is defined by the events themselves.
4. **Session**: group a user's activity until a gap, as in web analytics and Strava activities.

> **Warning**
>
> **Event time ≠ processing time:** A phone offline in a tunnel sends events 10 minutes late. Bucket by **event time** (when it happened), not arrival time. A **watermark** says "probably all events up to T have arrived"; later stragglers are dropped or used to correct the result.

Stream joins

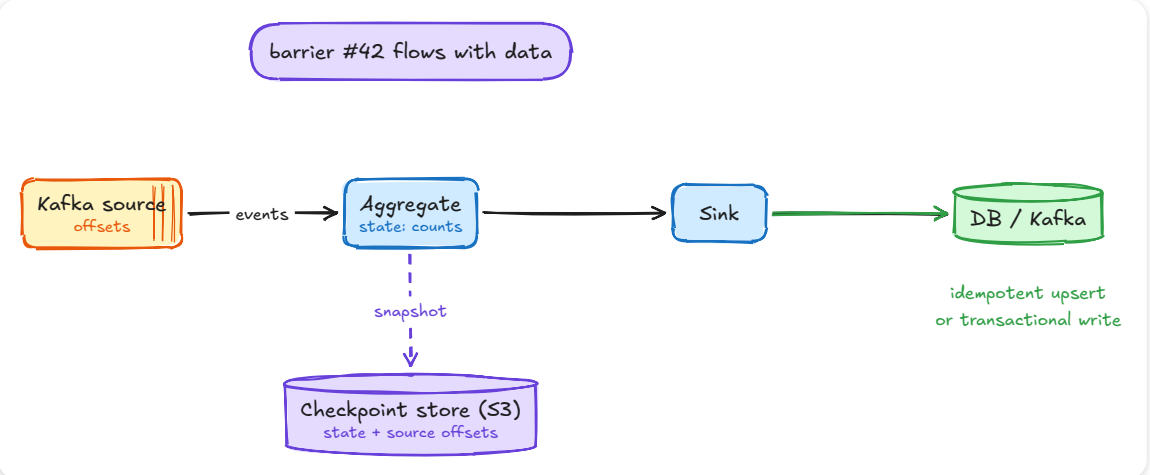
Three kinds

Join	Example	How
Stream–stream (window join)	Search event + click event within 1 hour → click-through rate	Buffer both streams in state, indexed by session ID, for the window length
Stream–table (enrichment)	Click event + user profile → add country and age	Keep a **local copy** of the table in the processor, updated by CDC. No remote DB call per event.
Table–table (materialized view)	Tweets ⋈ follows → each user's timeline cache	Both are changelogs; maintain the joined view incrementally (fan-out on write from Ch 1)

> **Note**
>
> **Time-dependence:** A join with a changing table depends on **when**: tax rate at sale time vs today's rate. To keep a join reproducible, give versioned records an ID and join on that version.

Exactly-once really means effectively-once

Fault tolerance



1. A failure mid-stream means some events were processed but their results weren't saved, or vice versa.
2. **Checkpointing** (Flink): barriers flow through the data. Each operator snapshots its state and the source offsets together. On failure, restore the snapshot and replay from those offsets. Spark does the same with **micro-batches**.

3. Replay means some outputs are sent twice. Make the sink **idempotent** (upsert by key, dedupe by event ID) or **transactional** (Kafka transactions commit offsets + outputs atomically). Duplicates still happen; they just have no effect.

> **Tip**
>
> **Say this in the interview:** "Flink checkpoints every 10s. The sink upserts (ad_id, minute) → count, so replays after a failure overwrite instead of double-counting."

In the interview

> **Important**
>
> **Interviewer asks:** "Ad click aggregation at 10k/sec?"
> → Kafka partitioned by ad_id → Flink 1-min tumbling windows on event time → OLAP store. Raw clicks to S3 for batch reconciliation.

> **Important**
>
> **Interviewer asks:** "Keep search, cache, and DB in sync?"
> → CDC (Debezium) or outbox → Kafka → idempotent consumers. Never dual-write.

> **Important**
>
> **Interviewer asks:** "Kafka or SQS?"
> → Ordering, replay, or many consumers → Kafka. Independent tasks with per-message retry and DLQ → SQS.

> **Important**
>
> **Interviewer asks:** "Exactly-once?"
> → "At-least-once delivery plus idempotent processing equals effectively-once. I make the consumer idempotent with an event ID."

← Ch 10: Batch processing · Ch 12: The future of data systems →

DDIA Chapter 12: The Future of Data Systems

> These DDIA pages are study notes in our own words and diagrams, connecting the book to interview problems. They don't replace the book; *Designing Data-Intensive Applications* by Martin Kleppmann is worth reading in full.

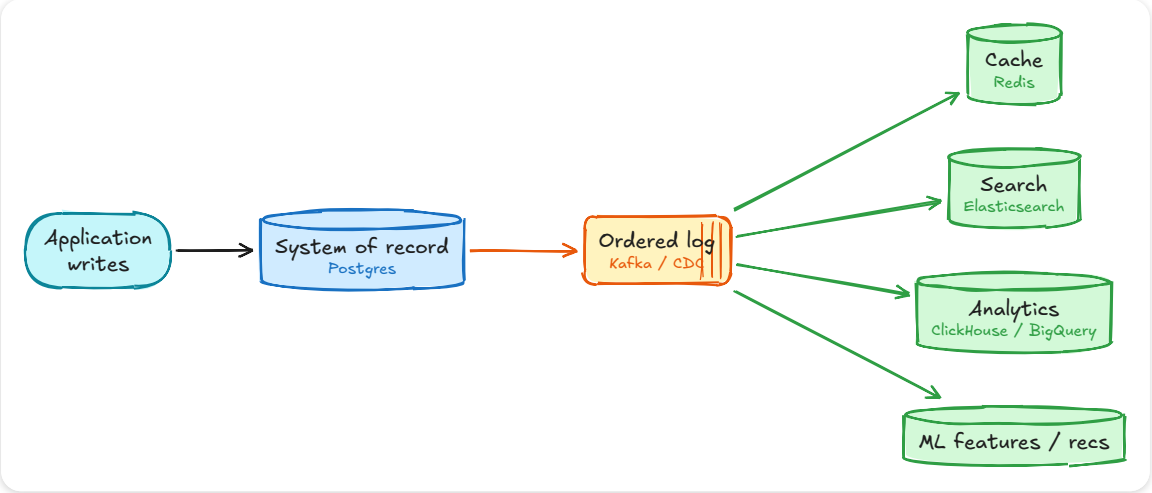
TL;DR

The book's conclusion: there is no one database that does everything. Build systems as a **source of truth plus derived views** connected by logs. Push correctness to the ends with idempotency, and separate "eventually up to date" (timeliness) from "never wrong" (integrity).

Key ideas: derived data · unbundling the DB · Lambda vs Kappa · end-to-end argument · idempotency keys · timeliness vs integrity · audit

Unbundling the database

Data integration



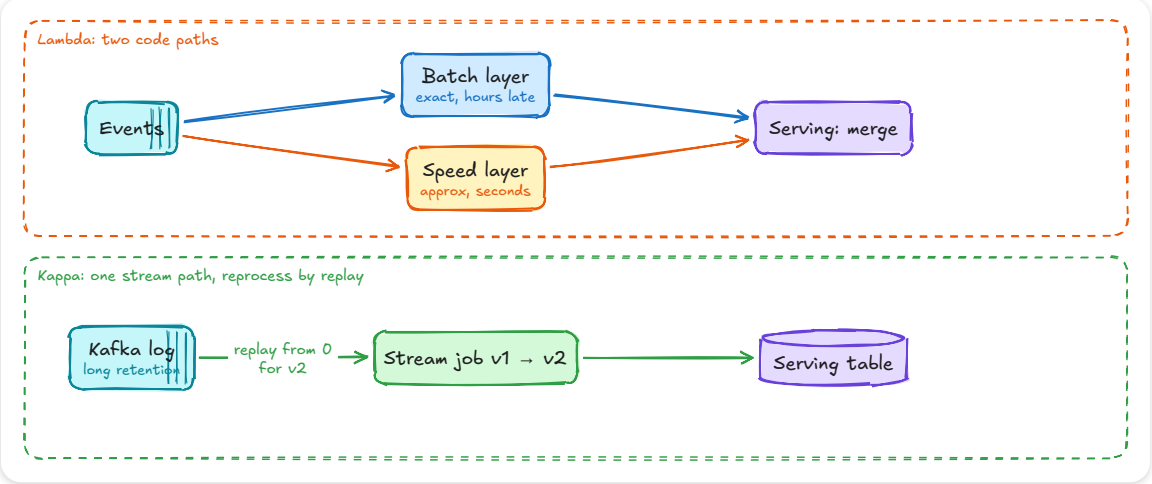
1. One **system of record** accepts writes. Its change log is the single ordered history of what happened.
2. Every other store is a **derived view**, like an index or materialized view, except in separate specialized systems. Any view can be dropped and rebuilt by replaying

the log. Inside one database these are "indexes"; across a company they are "unbundled" into Kafka plus separate stores.

> **Tip**
>
> **Say this in the interview:** "Postgres is the source of truth for bookings. Search, the cache, and analytics are derived views fed from CDC, so they are eventually consistent and rebuildable."

Lambda vs Kappa architecture

Batch + stream

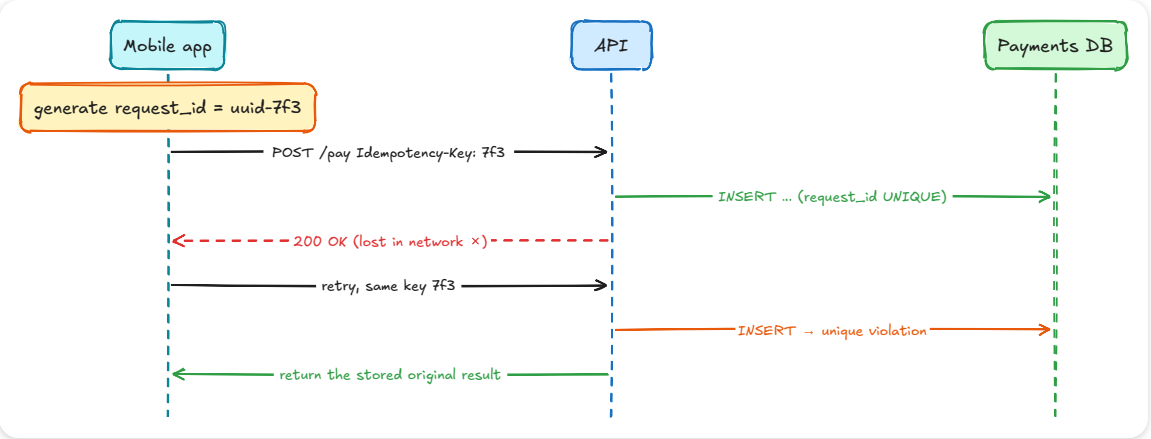


1. **Lambda**: a fast approximate stream path plus a slow exact batch path, merged at query time. Correct, but the same logic is written twice in two frameworks.
2. **Kappa**: one streaming codebase. To fix a bug or change logic, start v2 reading the log from the beginning, let it catch up, and switch reads over. This needs long log retention, such as Kafka tiered storage.

> **Note**
>
> **In interviews:** Top-K and ad clicks often still use a Lambda-like design: Flink for real-time dashboards, nightly batch reconciliation for billing accuracy. Say why: **money needs exact numbers**.

The end-to-end argument: idempotency keys

Correctness at the edges



1. The **client** generates a unique ID for the operation, once, before the first attempt.
2. The server records it with a unique constraint in the same transaction as the effect.
3. The response is lost and the client retries. TCP, load-balancer, and DB-level dedup can't catch this, because it is a new connection and a new request.
4. The duplicate hits the unique constraint, and the server returns the original result. Only an **end-to-end** ID from the origin catches every duplicate.

> **Tip**
>
> **Say this in the interview:** "Every mutating API takes an Idempotency-Key from the client. It is stored with a unique index, results are cached for 24h, and retries are always safe."

Timeliness vs integrity

Two kinds of "consistency"

	Timeliness	**Integrity**
Means	Users see an up-to-date state	No corruption: no lost or double money, no contradictions
If violated	Temporary and self-healing: "wait and refresh"	**Permanent**: needs manual repair
Example	Booking shows in "My trips" 2s late	Room sold twice; card charged twice
Mechanism	Read-your-writes, sync replication	Idempotency, unique constraints, atomic log append, reconciliation

	Timeliness	Integrity
Can relax?	Often yes (eventual consistency)	Almost never

> **Note**
>
> **Apologize vs prevent:** Airlines overbook on purpose and compensate later. Some constraints can be checked **asynchronously** with a compensating action. Decide per invariant how much a violation costs.

> **Note**
>
> **Trust, but verify:** Run **reconciliation and audit** jobs that recompute derived data from source and compare (ledger sums, inventory counts). Bugs and bit-rot happen; detect them.

In the interview

> **Important**
>
> **Interviewer asks:** "How do you guarantee no double charge?"
> → Client idempotency key + unique constraint + the payment provider's idempotency + a daily reconciliation job against provider settlement files.

> **Important**
>
> **Interviewer asks:** "Search shows a stale price."
> → "Search is a derived view. Timeliness can lag seconds; the **checkout price is read from the source of truth** (integrity)."

> **Important**
>
> **Interviewer asks:** "How would you change the aggregation logic?"
> → Kappa: deploy v2 consumer from offset 0 to a new table, compare, switch.

> **Tip**
>
> **Say this in the interview:** "I separate integrity from timeliness: money and inventory are strongly consistent and audited; everything the user browses is a derived, eventually consistent view."

← Ch 11: Stream processing

Low-level design

Low-level design (LLD) interviews ask you to model one component in code: its classes, their responsibilities, and how they behave when many threads use them at once.

The approach

1. **Clarify requirements.** List the operations (`mkdir`, `book`, `log`, `transfer`) and the rules (no double booking, levels filter logs).
2. **Find the entities.** Nouns in the requirements become candidate classes. Keep only the ones with behavior or state.
3. **Assign responsibilities.** Each class should have one reason to change. Put a method on the class that owns the data it needs.
4. **Choose relationships.** Prefer composition ("has a") to inheritance; use interfaces where behavior varies (sinks, strategies).
5. **Handle concurrency.** Ask "what if two threads do this at once?" for every operation that reads then writes. Lock at the right granularity.
6. **Plan for change.** Show where a new requirement plugs in: a new sink, a new seat type, a new storage backend.

> **Tip**
>
> **Say this in the interview:** "I'll start with the entities and their responsibilities, then walk through each operation, and then check every read-then-write for race conditions."

Patterns that come up again and again

Pattern	Use it for	Example here
Composite	Trees where leaves and containers share an interface	File system
Strategy	Interchangeable behavior	Log formatters in the logger

Pattern	Use it for	Example here
Observer	Notify others when something happens	Low-stock alerts in inventory
Producer–consumer	Decouple callers from slow work	Async log dispatch
Fine-grained locking	Concurrency without one global lock	Per-show locks in movie booking
Lock ordering	Avoid deadlocks when taking two locks	Warehouse transfers in inventory

Problems

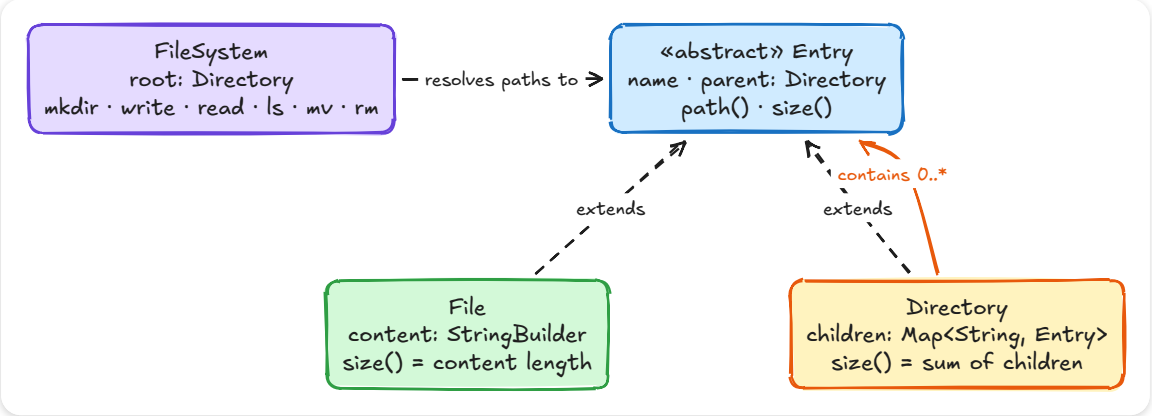
Problem	Main idea
In-memory file system	Composite tree, path resolution, read/write locking
Movie ticket booking	Atomic all-or-nothing seat holds with expiry, locked per show
Logging library	Levels, pluggable sinks and formatters, async dispatch
Inventory management	Per-warehouse locks, deadlock-free transfers, low-stock observers

LLD: In-memory file system

Requirements

- `mkdir(path)` creates directories, including missing parents.
- `write(path, text)` creates or appends to a file; `read(path)` returns its content.
- `ls(path)` lists a directory's children in sorted order (or returns the file name).
- `rm(path)` deletes a file or directory; `mv(src, dst)` moves one.
- Paths are absolute, like `/a/b/c.txt`. Many threads may call these at once.

Design



- **Composite pattern:** `File` and `Directory` both extend `Entry`, so code that walks the tree treats them the same way. `size()` is a leaf value for a file and the sum of children for a directory.
- **Directory children are a map** from name to entry, giving O(1) lookup per path segment. `ls` sorts the keys, or uses a `TreeMap` if listing is frequent.
- **Parent pointers** let any entry compute its full path and let `mv` detach an entry from its old parent cheaply.
- **Concurrency:** reads (`ls`, `read`) far outnumber structural changes, so one `ReentrantReadWriteLock` lets reads run in parallel while writes are exclusive. Finer locks (per directory) are possible but make `mv` across directories much harder to get right; mention it as a later step.

Code

```
import java.util.*;
import java.util.concurrent.locks.ReentrantReadWriteLock;

abstract class Entry {
    String name;
    Directory parent;
    Entry(String name, Directory parent) { this.name = name; this.parent = parent; }
    abstract long size();
    String path() { return parent == null ? "/" : (parent.parent == null ? "" : parent.path()) + "/"
+ name; }
}

class File extends Entry {
    private final StringBuilder content = new StringBuilder();
    File(String name, Directory parent) { super(name, parent); }
    long size() { return content.length(); }
    void append(String text) { content.append(text); }
    String read() { return content.toString(); }
}

class Directory extends Entry {
    final Map<String, Entry> children = new HashMap<>();
    Directory(String name, Directory parent) { super(name, parent); }
    long size() { return children.values().stream().mapToLong(Entry::size).sum(); }
}

public class FileSystem {
    private final Directory root = new Directory("", null);
    private final ReentrantReadWriteLock lock = new ReentrantReadWriteLock();

    private static List<String> parts(String path) {
        if (!path.startsWith("/")) throw new IllegalArgumentException("absolute path required: " +
path);
        List<String> out = new ArrayList<>();
        for (String p : path.split("/")) if (!p.isEmpty()) out.add(p);
        return out;
    }

    /** Walks to the entry at path; returns null if any segment is missing. */
    private Entry find(String path) {
        Entry cur = root;
        for (String p : parts(path)) {
            if (!(cur instanceof Directory d)) return null;
            cur = d.children.get(p);
            if (cur == null) return null;
        }
        return cur;
    }

    /** Walks to the directory at path, creating missing directories. */
    private Directory mkdirs(List<String> segments) {
        Directory cur = root;
        for (String p : segments) {
            Entry next = cur.children.computeIfAbsent(p, n -> new Directory(n, null));
            if (!(next instanceof Directory d)) throw new IllegalStateException(p + " is a file");
            d.parent = cur;
            cur = d;
        }
        return cur;
    }

    public void mkdir(String path) {
        lock.writeLock().lock();
        try { mkdirs(parts(path)); } finally { lock.writeLock().unlock(); }
    }

    public void write(String path, String text) {
        lock.writeLock().lock();
        try {
            List<String> ps = parts(path);
            Directory dir = mkdirs(ps.subList(0, ps.size() - 1));
            String name = ps.get(ps.size() - 1);
            Entry e = dir.children.computeIfAbsent(name, n -> new File(n, dir));
```

```
            if (!(e instanceof File f)) throw new IllegalStateException(path + " is a directory");
            f.append(text);
        } finally { lock.writeLock().unlock(); }
    }

    public String read(String path) {
        lock.readLock().lock();
        try {
            if (!(find(path) instanceof File f)) throw new NoSuchElementException(path);
            return f.read();
        } finally { lock.readLock().unlock(); }
    }

    public List<String> ls(String path) {
        lock.readLock().lock();
        try {
            Entry e = find(path);
            if (e == null) throw new NoSuchElementException(path);
            if (e instanceof File) return List.of(e.name);
            List<String> names = new ArrayList<>(((Directory) e).children.keySet());
            Collections.sort(names);
            return names;
        } finally { lock.readLock().unlock(); }
    }

    public void rm(String path) {
        lock.writeLock().lock();
        try {
            Entry e = find(path);
            if (e == null || e == root) throw new NoSuchElementException(path);
            e.parent.children.remove(e.name);
        } finally { lock.writeLock().unlock(); }
    }

    public void mv(String src, String dst) {
        lock.writeLock().lock();
        try {
            Entry e = find(src);
            if (e == null || e == root) throw new NoSuchElementException(src);
            List<String> ps = parts(dst);
            Directory target = mkdirs(ps.subList(0, ps.size() - 1));
            for (Directory d = target; d != null; d = d.parent)
                if (d == e) throw new IllegalArgumentException("cannot move a directory into
itself");
            String newName = ps.get(ps.size() - 1);
            if (target.children.containsKey(newName)) throw new IllegalStateException(dst + "
exists");
            e.parent.children.remove(e.name);
            e.name = newName;
            e.parent = target;
            target.children.put(newName, e);
        } finally { lock.writeLock().unlock(); }
    }
}
```

Follow-up questions

Question	Answer
How would you support permissions?	Add owner and mode to `Entry`; check them during path resolution.
Very large directories?	`TreeMap` for sorted listing, or paginate `ls` with a cursor.

Question	Answer
More write concurrency?	Lock per directory, taking locks top-down in path order; `mv` locks the common ancestor.
Persist to disk?	Separate the tree from storage behind an interface; write an append-only journal of operations.

Related

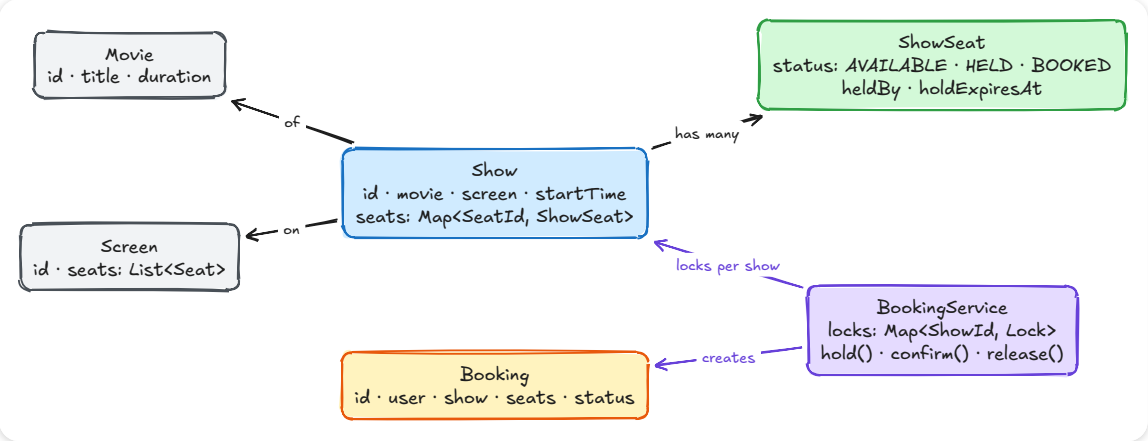
- Low-level design approach

LLD: Movie ticket booking

Requirements

- Browse shows (a movie on a screen at a time) and see which seats are free.
- **Hold** several seats at once for a few minutes while the user pays: all of them or none.
- **Confirm** a hold after payment; **release** it if the user cancels. Expired holds free their seats automatically.
- A seat is never sold twice, even when many users click the same seats at the same moment.

Design



- `ShowSeat` **carries the state** for one seat in one show: `AVAILABLE`, `HELD` (with holder and expiry) or `BOOKED`. The physical `Seat` stays immutable and is shared by all shows on that screen.
- **One lock per show.** Holding seats is a check-then-act over several seats, so it must be atomic. Locking the whole service would serialize every booking in the city; locking one show only serializes users fighting over the same screening, which is exactly where contention is.
- **All-or-nothing holds:** check every requested seat first, and only then mark them all held. If any seat is unavailable, change nothing.
- **Lazy expiry:** a held seat whose expiry has passed counts as available. No background thread is needed for correctness; an optional sweeper just tidies up.
- **Injected clock**, so expiry is testable.

Code

```
import java.time.*;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.locks.ReentrantLock;

enum SeatStatus { AVAILABLE, HELD, BOOKED }

final class ShowSeat {
    final String seatId;
    SeatStatus status = SeatStatus.AVAILABLE;
    String heldBy;
    Instant holdExpiresAt;
    ShowSeat(String seatId) { this.seatId = seatId; }

    boolean isFree(Instant now) {
        return status == SeatStatus.AVAILABLE
            || (status == SeatStatus.HELD && holdExpiresAt.isBefore(now));
    }
}

final class Show {
    final String id;
    final Map<String, ShowSeat> seats = new HashMap<>();
    Show(String id, Collection<String> seatIds) {
        this.id = id;
        seatIds.forEach(s -> seats.put(s, new ShowSeat(s)));
    }
}

record Booking(String id, String userId, String showId, List<String> seatIds, Instant expiresAt) {}

public class BookingService {
    private final Map<String, Show> shows = new ConcurrentHashMap<>();
    private final Map<String, ReentrantLock> locks = new ConcurrentHashMap<>();
    private final Map<String, Booking> holds = new ConcurrentHashMap<>();
    private final Clock clock;
    private final Duration holdTime;

    public BookingService(Clock clock, Duration holdTime) { this.clock = clock; this.holdTime =
holdTime; }

    public void addShow(Show show) { shows.put(show.id, show); locks.put(show.id, new
ReentrantLock()); }

    /** Holds all requested seats or none. Returns the hold, or empty if any seat is taken. */
    public Optional<Booking> hold(String showId, String userId, List<String> seatIds) {
        Show show = shows.get(showId);
        ReentrantLock lock = locks.get(showId);
        lock.lock();
        try {
            Instant now = clock.instant();
            for (String id : seatIds) {
                ShowSeat s = show.seats.get(id);
                if (s == null || !s.isFree(now)) return Optional.empty();   // nothing changed yet
            }
            Instant expires = now.plus(holdTime);
            for (String id : seatIds) {
                ShowSeat s = show.seats.get(id);
                s.status = SeatStatus.HELD; s.heldBy = userId; s.holdExpiresAt = expires;
            }
            Booking b = new Booking(UUID.randomUUID().toString(), userId, showId,
List.copyOf(seatIds), expires);
            holds.put(b.id(), b);
            return Optional.of(b);
        } finally { lock.unlock(); }
    }

    /** Called after payment succeeds. Fails if the hold expired and someone else took a seat. */
    public boolean confirm(String bookingId) {
```

```
        Booking b = holds.get(bookingId);
        if (b == null) return false;
        Show show = shows.get(b.showId());
        ReentrantLock lock = locks.get(b.showId());
        lock.lock();
        try {
            Instant now = clock.instant();
            for (String id : b.seatIds()) {
                ShowSeat s = show.seats.get(id);
                boolean stillMine = s.status == SeatStatus.HELD && b.userId().equals(s.heldBy) &&
!s.holdExpiresAt.isBefore(now);
                if (!stillMine) return false;          // caller refunds the payment
            }
            for (String id : b.seatIds()) show.seats.get(id).status = SeatStatus.BOOKED;
            holds.remove(bookingId);
            return true;
        } finally { lock.unlock(); }
    }

    public void release(String bookingId) {
        Booking b = holds.remove(bookingId);
        if (b == null) return;
        Show show = shows.get(b.showId());
        ReentrantLock lock = locks.get(b.showId());
        lock.lock();
        try {
            for (String id : b.seatIds()) {
                ShowSeat s = show.seats.get(id);
                if (s.status == SeatStatus.HELD && b.userId().equals(s.heldBy)) {
                    s.status = SeatStatus.AVAILABLE; s.heldBy = null; s.holdExpiresAt = null;
                }
            }
        } finally { lock.unlock(); }
    }

    public List<String> freeSeats(String showId) {
        Show show = shows.get(showId);
        ReentrantLock lock = locks.get(showId);
        lock.lock();
        try {
            Instant now = clock.instant();
            return show.seats.values().stream().filter(s -> s.isFree(now)).map(s ->
s.seatId).sorted().toList();
        } finally { lock.unlock(); }
    }
}
```

Why the details matter

Detail	What goes wrong without it
Check all seats before changing any	A user gets seats 1–2 of 3 they asked for, and the others are left half-held.
Lock per show, not global	Every booking in every cinema waits on one lock.
`confirm` re-checks ownership	A hold expires during a slow payment, another user takes the seat, and the first user's confirm overwrites it.
Lazy expiry inside `isFree`	A crashed sweeper thread would leave seats stuck as held forever.

Taking it to a real system

In a distributed service the in-memory lock becomes a **conditional update** in the database, so the database enforces the rule for all servers:

```
UPDATE show_seats SET status = 'HELD', held_by = :user, hold_until = now() + interval '10 minutes'
WHERE show_id = :show AND seat_id = ANY(:seats)
  AND (status = 'AVAILABLE' OR (status = 'HELD' AND hold_until < now()));
-- success only if the number of updated rows equals the number of seats requested; otherwise roll
back
```

See the system design version in Ticketmaster.

Related

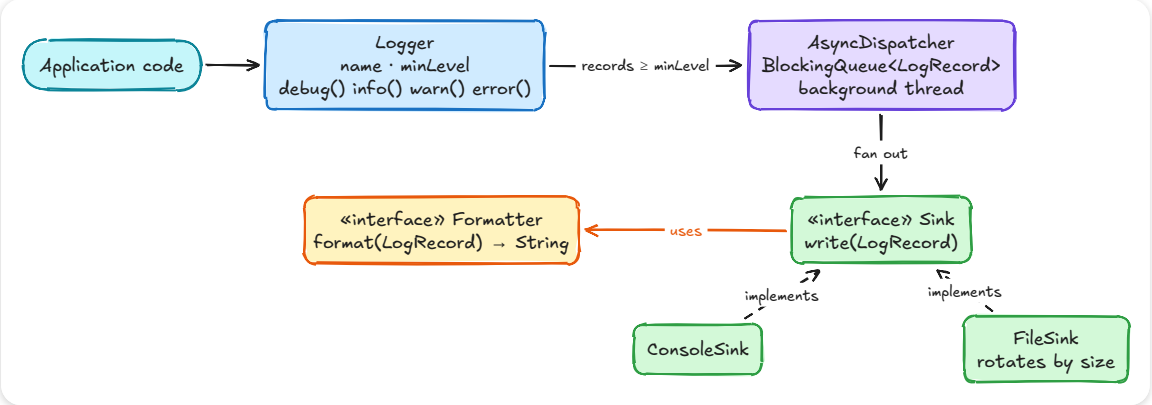
- Dealing with contention
- DDIA Ch 7: Transactions explains write skew, the bug this design prevents.

LLD: Logging library

Requirements

- Log messages at levels `DEBUG < INFO < WARN < ERROR`; each logger has a minimum level.
- Send records to one or more destinations (**sinks**): console, file, later a network service.
- Format records differently per sink (plain text, JSON).
- Logging must not slow the application down, must be safe from many threads, and must flush on shutdown.

Design



- `Sink` **is an interface**, so a new destination is a new class, not an edit to the logger (open/closed principle). Composition, not inheritance: the logger *has* sinks.
- `Formatter` **is a strategy** each sink is given, so the same record can be plain text on the console and JSON in a file.
- **Level filtering happens first**, in the caller's thread, before any work like string formatting is done.
- **Asynchronous dispatch:** the caller puts the record on a bounded queue and returns immediately; one background thread writes to the sinks. Sinks then see one thread and need no locks of their own.
- **Backpressure policy:** when the queue is full, drop `DEBUG` / `INFO` but block briefly for `WARN` / `ERROR`. Losing an error log is worse than a short pause.

Code

```java
import java.io.*;
import java.time.Instant;
import java.util.List;
import java.util.concurrent.*;

enum Level { DEBUG, INFO, WARN, ERROR }

record LogRecord(Instant time, Level level, String logger, String message, Throwable error) {}

interface Formatter { String format(LogRecord r); }

class PlainFormatter implements Formatter {
    public String format(LogRecord r) {
        String base = r.time() + " " + r.level() + " [" + r.logger() + "] " + r.message();
        return r.error() == null ? base : base + " | " + r.error();
    }
}

class JsonFormatter implements Formatter {
    private static String q(String s) { return "\"" + s.replace("\\", "\\\\").replace("\"", "\\\"") +
"\""; }
    public String format(LogRecord r) {
        return "{\"time\":" + q(r.time().toString()) + ",\"level\":" + q(r.level().name())
             + ",\"logger\":" + q(r.logger()) + ",\"message\":" + q(r.message()) + "}";
    }
}

interface Sink extends Closeable { void write(LogRecord r) throws IOException; }

class ConsoleSink implements Sink {
    private final Formatter fmt;
    ConsoleSink(Formatter fmt) { this.fmt = fmt; }
    public void write(LogRecord r) { System.out.println(fmt.format(r)); }
    public void close() { System.out.flush(); }
}

class FileSink implements Sink {
    private final Formatter fmt;
    private final Writer out;
    FileSink(String path, Formatter fmt) throws IOException {
        this.fmt = fmt;
        this.out = new BufferedWriter(new FileWriter(path, true));
    }
    public void write(LogRecord r) throws IOException { out.write(fmt.format(r)); out.write('\n'); }
    public void close() throws IOException { out.close(); }
}

/** One background thread drains a bounded queue into the sinks. */
class AsyncDispatcher implements AutoCloseable {
    private static final LogRecord STOP = new LogRecord(Instant.EPOCH, Level.ERROR, "", "", null);
    private final BlockingQueue<LogRecord> queue = new ArrayBlockingQueue<>(10_000);
    private final List<Sink> sinks;
    private final Thread worker;

    AsyncDispatcher(List<Sink> sinks) {
        this.sinks = List.copyOf(sinks);
        this.worker = new Thread(this::run, "log-dispatcher");
        this.worker.setDaemon(true);
        this.worker.start();
    }

    void submit(LogRecord r) {
        if (r.level().compareTo(Level.WARN) < 0) {
            queue.offer(r);                                  // drop low-priority records if full
        } else {
            try { queue.offer(r, 100, TimeUnit.MILLISECONDS); }  // wait briefly for important ones
            catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        }
    }
```

```
    private void run() {
        try {
            for (LogRecord r = queue.take(); r != STOP; r = queue.take())
                for (Sink s : sinks) {
                    try { s.write(r); } catch (IOException e) { System.err.println("sink failed: " +
e); }
                }
        } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }

    public void close() throws Exception {
        queue.put(STOP);           // records already queued are written first
        worker.join(5_000);
        for (Sink s : sinks) s.close();
    }
}

public class Logger {
    private final String name;
    private final Level minLevel;
    private final AsyncDispatcher dispatcher;

    public Logger(String name, Level minLevel, AsyncDispatcher dispatcher) {
        this.name = name; this.minLevel = minLevel; this.dispatcher = dispatcher;
    }

    public boolean isEnabled(Level level) { return level.compareTo(minLevel) >= 0; }

    public void log(Level level, String message, Throwable error) {
        if (!isEnabled(level)) return;                      // cheap filter in the caller's thread
        dispatcher.submit(new LogRecord(Instant.now(), level, name, message, error));
    }
    public void debug(String m) { log(Level.DEBUG, m, null); }
    public void info(String m)  { log(Level.INFO, m, null); }
    public void warn(String m)  { log(Level.WARN, m, null); }
    public void error(String m, Throwable t) { log(Level.ERROR, m, t); }
}
```

Usage:

```
var dispatcher = new AsyncDispatcher(List.of(
        new ConsoleSink(new PlainFormatter()),
        new FileSink("app.log", new JsonFormatter())));
var log = new Logger("checkout", Level.INFO, dispatcher);
log.info("order placed");
log.debug("ignored: below INFO");
dispatcher.close();   // flush on shutdown
```

Follow-up questions

Question	Answer
A sink is slow (network)	Give it its own queue and thread so it can't delay the others.
Expensive messages at DEBUG	Check `isEnabled(DEBUG)` first, or accept a `Supplier<String>` so the message is built only when needed.
File grows forever	A rolling `FileSink`: rotate when size or date crosses a limit.

Question	Answer
Per-package levels	A `LoggerFactory` holding a level per name prefix, returning cached loggers.

Related

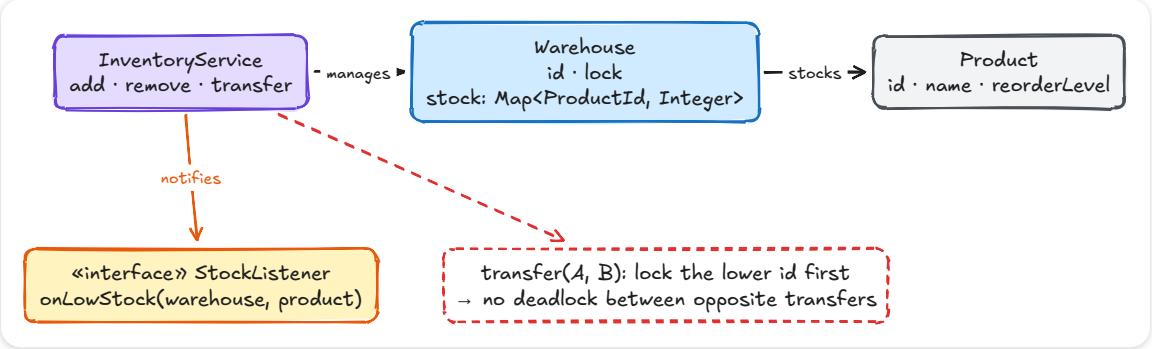
- Low-level design approach
- Long-running tasks: the same producer–consumer idea at system scale.

LLD: Inventory management

Requirements

- Track stock of each product in each warehouse.
- `add` stock (deliveries) and `remove` stock (orders); never let stock go negative.
- `transfer` stock between two warehouses atomically: it leaves one and arrives in the other, or nothing happens.
- Notify interested parties when a product's stock in a warehouse falls below its reorder level.
- Many threads operate at once.

Design



- **One lock per warehouse.** Operations on different warehouses run in parallel; only operations on the same warehouse wait for each other.
- **Remove is check-then-act**, so it happens under the warehouse lock: check stock ≥ quantity, then subtract.
- **Transfer needs two locks.** If one thread transfers A→B while another transfers B→A, each could hold one lock and wait forever for the other. Fix: **always lock the warehouse with the smaller id first.** With one global order, a cycle of waiting is impossible.
- **Observer for low stock.** The service doesn't know who cares (purchasing, alerting). Listeners register themselves; notifications are sent **after** releasing locks, so a slow listener can't block inventory operations.

Code

```
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.locks.ReentrantLock;

record Product(String id, String name, int reorderLevel) {}

interface StockListener { void onLowStock(String warehouseId, Product product, int remaining); }

final class Warehouse {
    final String id;
    final ReentrantLock lock = new ReentrantLock();
    private final Map<String, Integer> stock = new HashMap<>();   // guarded by lock
    Warehouse(String id) { this.id = id; }
    int get(String productId) { return stock.getOrDefault(productId, 0); }
    void set(String productId, int qty) { stock.put(productId, qty); }
}

public class InventoryService {
    private final Map<String, Warehouse> warehouses = new ConcurrentHashMap<>();
    private final Map<String, Product> products = new ConcurrentHashMap<>();
    private final List<StockListener> listeners = new CopyOnWriteArrayList<>();

    public void addWarehouse(String id) { warehouses.putIfAbsent(id, new Warehouse(id)); }
    public void addProduct(Product p) { products.put(p.id(), p); }
    public void subscribe(StockListener l) { listeners.add(l); }

    public void add(String warehouseId, String productId, int qty) {
        if (qty <= 0) throw new IllegalArgumentException("qty must be positive");
        Warehouse w = warehouses.get(warehouseId);
        w.lock.lock();
        try { w.set(productId, w.get(productId) + qty); } finally { w.lock.unlock(); }
    }

    /** Returns false (and changes nothing) if there isn't enough stock. */
    public boolean remove(String warehouseId, String productId, int qty) {
        if (qty <= 0) throw new IllegalArgumentException("qty must be positive");
        Warehouse w = warehouses.get(warehouseId);
        int remaining;
        w.lock.lock();
        try {
            int have = w.get(productId);
            if (have < qty) return false;
            remaining = have - qty;
            w.set(productId, remaining);
        } finally { w.lock.unlock(); }
        notifyIfLow(warehouseId, productId, remaining);    // outside the lock
        return true;
    }

    /** Moves stock atomically. Locks are always taken in warehouse-id order to prevent deadlock. */
    public boolean transfer(String fromId, String toId, String productId, int qty) {
        if (fromId.equals(toId)) throw new IllegalArgumentException("same warehouse");
        if (qty <= 0) throw new IllegalArgumentException("qty must be positive");
        Warehouse from = warehouses.get(fromId), to = warehouses.get(toId);
        Warehouse first = fromId.compareTo(toId) < 0 ? from : to;
        Warehouse second = first == from ? to : from;
        int remaining;
        first.lock.lock();
        try {
            second.lock.lock();
            try {
                int have = from.get(productId);
                if (have < qty) return false;
                remaining = have - qty;
                from.set(productId, remaining);
                to.set(productId, to.get(productId) + qty);
            } finally { second.lock.unlock(); }
        } finally { first.lock.unlock(); }
        notifyIfLow(fromId, productId, remaining);
        return true;
    }
```

```
    public int stock(String warehouseId, String productId) {
        Warehouse w = warehouses.get(warehouseId);
        w.lock.lock();
        try { return w.get(productId); } finally { w.lock.unlock(); }
    }

    private void notifyIfLow(String warehouseId, String productId, int remaining) {
        Product p = products.get(productId);
        if (p != null && remaining < p.reorderLevel())
            for (StockListener l : listeners) l.onLowStock(warehouseId, p, remaining);
    }
}
```

Why lock ordering works

Deadlock needs a cycle: thread 1 holds A and waits for B, while thread 2 holds B and waits for A. If every thread takes locks in the same global order (here, by warehouse id), thread 2 must also take A first, so it waits *before* holding B. No cycle can form.

Follow-up questions

Question	Answer
Reserve stock for an order before payment?	Add a `reserved` count per product: available = stock − reserved; a reservation expires like a seat hold.
One very hot product?	Split its stock into several buckets, each with its own lock, and take from any bucket that has enough.
Distributed across services?	Conditional database updates: `UPDATE stock SET qty = qty - :n WHERE … AND qty >= :n`. A transfer becomes one transaction, or a saga across databases.
Audit trail?	Append every change to a movements table (who, what, when, why) and derive totals from it.

Related

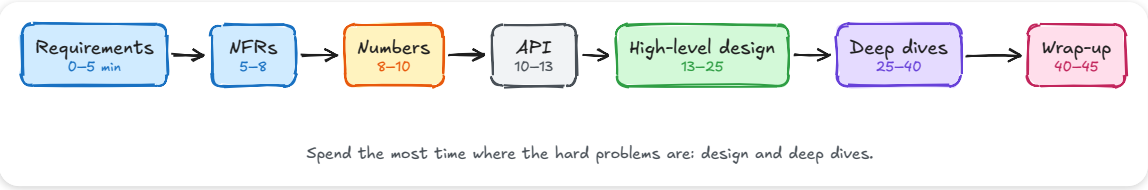
- Dealing with contention
- Local delivery: the system-scale version of "never oversell".

Interview playbook

- The 45-minute delivery framework: what to do in each phase.
- Phrases, signals and traps: what senior answers sound like.
- Quick-fire practice: 18 short questions to answer out loud.

Each problem page also has its own interview script in section 7.

The 45-minute delivery framework



Spend the most time where the hard problems are: design and deep dives.

Time	Phase	Do	Avoid
0–5	**Functional requirements**	3 "users can..." statements. Say what's out of scope.	Listing ten features.
5–8	**Non-functional requirements**	3–5 NFRs **with numbers**. Say where you need consistency vs availability.	"It should be fast and scalable."
8–10	**Estimates**	Only the numbers that change a decision: QPS, storage, fan-out.	Ten minutes of arithmetic.
10–13	**Entities and API**	Core entities; 3–5 endpoints; idempotency keys; cursor pagination.	Perfecting every field.
13–25	**High-level design**	Satisfy each functional requirement with the simplest components. Walk one request end to end.	Adding Kafka or sharding nobody asked for.
25–40	**Deep dives**	Take each NFR, show where the simple design breaks with a number, fix it. Follow the interviewer's lead.	Defending a choice instead of discussing trade-offs.
40–45	**Wrap-up**	Remaining bottlenecks, failure modes, what you'd monitor, what you'd do next.	Silence.

Phase by phase

Requirements (0–8 minutes)

Ask, then write down. Good questions: *Who are the users? What's the most important action? How many users, and how spiky? What must never go wrong?*

> **Tip**
>
> "Let me confirm scope: users can A, B and C. I'll leave D out unless you want it. For NFRs, the key ones are..."

Estimates (8–10 minutes)

Use the method in Numbers and estimation. Finish each number with a conclusion: *"... so one Postgres is enough for writes, but reads need a cache."*

High-level design (13–25 minutes)

Start from the simplest design. Draw the client, the service and the database, then trace one request through it for each functional requirement. Add components only when a requirement needs them.

Deep dives (25–40 minutes)

This is where senior candidates separate themselves. For each NFR:

1. Point to where the current design fails it, with a number.
2. Propose the fix (usually one of the 7 patterns).
3. Name the trade-off you're accepting.
4. Mention a failure mode and how you'd handle it.

If the interviewer steers you somewhere else, follow them. They're telling you what they want to grade.

Wrap-up (40–45 minutes)

> **Tip**
>
> "The biggest remaining risk is X. I'd monitor p99 latency, error rate and queue depth, and I'd alert on Y. With more time I'd look at Z."

Related

- Phrases, signals and traps
- Quick-fire practice

Phrases, signals and traps

What a senior answer sounds like

Signal	Example
Quantify the break	"At 100k reads per second a single Postgres falls over, so the first thing I'd add is a cache. Traffic is skewed, so I expect an 80%+ hit rate."
Name the trade-off	"Fan-out on write makes reads one lookup but costs one write per follower, so I'll switch to pull above ~100k followers."
Choose consistency per feature	"The stock decrement is strongly consistent through a conditional update; the product page is cached and can be a few seconds stale."
Raise failure modes unprompted	"If the payment call times out we don't know whether it charged, so the order stays pending and we reconcile by our idempotency key."
Start simple, on purpose	"One Postgres handles this write rate, so I won't shard. If writes grow 10×, I'd shard by user ID."
Tie choices to requirements	"We need replay and several consumers, so Kafka rather than a queue."

Habits that give away a junior answer

- Listing technologies without saying what problem each solves.
- "We'll use microservices and Kafka" before any numbers.
- Ignoring retries, timeouts and partial failure.
- Sharding at a few hundred requests per second.
- Treating the whole system as either "strongly" or "eventually" consistent.
- Designing in silence, then presenting.

Phrases to avoid, and what to say instead

Instead of	Say
"It's eventually consistent" (about money)	"Payments are strongly consistent; only the dashboard is eventually consistent."
"Just add more servers" (for a hot key)	"More servers don't help one hot key. I'll split the key or cache it locally."
"We'll use a distributed transaction across services"	"I'll use a saga with compensating steps and idempotent handlers."
"Redis lock, done"	"A Redis lock with a TTL plus a fencing check in the database, so a paused process can't write late."
"NoSQL scales better"	"This access pattern is key-based at 50k writes/s, which suits Cassandra. Postgres would need sharding to match."
"We'll cache everything"	"We'll cache the read-heavy, tolerant-of-staleness paths; checkout reads the database."

NFR checklist for the first five minutes

- [ ] Scale: DAU, peak QPS, read:write ratio
- [ ] Latency: which path, which percentile
- [ ] Availability target, and what can degrade
- [ ] Consistency: where stale or duplicate data is unacceptable
- [ ] Durability: can anything ever be lost?
- [ ] Data size and growth
- [ ] Anything special: security, privacy, cost, regions

Related

- The 45-minute delivery framework
- Non-functional requirements

Quick-fire practice

Answer each question out loud in under 30 seconds, then open it to compare.

1. Cache-aside vs write-through?

Cache-aside: app reads cache, on miss reads DB and fills cache; writes update DB then delete key. Write-through writes both synchronously: fresher, slower writes.

2. How do you stop a cache stampede?

Single-flight lock per key (only one request recomputes), jittered TTLs, serve stale while revalidating, pre-warm hot keys.

3. SSE vs WebSocket vs long polling?

SSE: server→client over HTTP with auto-reconnect. WebSocket: bidirectional, stateful. Long polling: fallback. Pick the simplest that fits the traffic direction.

4. Two users book the last seat. What exactly goes wrong and how do you fix it?

Check-then-act race (write skew). An atomic conditional UPDATE ... WHERE status=AVAILABLE, or a lock with TTL; exactly one row update succeeds.

5. Why not 2PC across microservices?

It blocks on coordinator failure and couples availability. Use a saga with compensations, outbox, and idempotent consumers.

6. What is the outbox pattern?

Write the business row and an event row in one local transaction; a relay/CDC publishes the event. No dual-write inconsistency.

7. At-least-once vs exactly-once?

Delivery is at-least-once in practice; make processing idempotent (dedupe by ID, upserts) to get an effectively-once result.

8. When do you shard?

When the calculated write load or data size exceeds one primary (~10–50k TPS, multiple TB) after caching and replicas. Pick the key from the hottest query.

9. Consistent hashing in one sentence?

Nodes and keys on a ring, key → next node clockwise; adding a node moves ~1/N of keys; virtual nodes balance load.

10. Hot key, e.g., a celebrity or viral item?

Split the key with suffixes, cache with local L1, buffer writes and aggregate, or special-case it (hybrid fan-out).

11. Presigned URLs: why?

Clients upload and download directly to object storage; app servers never carry the bytes; multipart allows resumable parallel chunks.

12. Kafka vs SQS?

Kafka: ordered partitions, replay, multiple consumer groups, stream processing. SQS: simple task queue with per-message retry, visibility timeout, DLQ.

13. How do you make an API retry-safe?

Client-generated Idempotency-Key stored with a unique constraint; return the stored result on duplicates.

14. Replication lag bites how?

Read-your-writes failures, non-monotonic reads, causally reordered reads. Fixes: read-from-leader after write, sticky replicas, partition by causal key.

15. Redis lock is enough for correctness?

No. GC pauses or clock issues can leave two holders. Add a fencing token checked by the resource, or use DB conditional writes.

16. CAP in an interview?

Per feature: during a partition choose to reject (CP) or serve stale (AP). Booking and payments CP; feeds and search AP.

17. p99 vs average?

Users feel the tail; with fan-out, tail latency compounds. Set SLOs on p99; use timeouts and hedged requests.

18. LSM vs B-tree?

LSM: sequential writes, compaction, great for write-heavy (Cassandra). B-tree: in-place pages, predictable reads, transactions (Postgres).

Related

- Phrases, signals and traps
- Every problem page ends with deep-dive questions in the same style.

Slide decks

Three interactive decks with the same content as the handbook, for quick review and out-loud practice. Diagrams build up one step at a time. Use → or Space to step, ↓ to skip a slide, **A** to show all steps, **O** for an overview of all slides. They work on phones too: swipe, or tap a diagram.

Deck	Slides	Use it to
DDIA in diagrams	96	Review all 12 chapters of *Designing Data-Intensive Applications*, each ending with its interview angle
Learn deck	81	See the 7 patterns, 9 concepts and 29 problems built step by step
Interview deck	96	Rehearse: requirements and numbers, architecture, then deep-dive questions with hidden answers

Each deck is a single HTML file, so you can download it and use it offline.